

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2023

Huấn luyện viên: Yang Yaoliang, Peng Sijin

China Computer Federation, 2023

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2023

IOI China candidate team papers / National training team 2023 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2023. Tập được tạo bằng LaTeX với lớp văn bản Unicode đọc được; mục lục và các trang thân bài đã kiểm tra đều trích xuất tốt bằng pdftotext. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục và mười tám bài trong tập, đồng thời ghi lại các chủ đề chính để phục vụ bản dịch chi tiết hơn.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2023*. Trang bìa ghi huấn luyện viên là Yang Yaoliang và Peng Sijin. Tập PDF gồm 249 trang và được tạo bằng LaTeX với gói hyperref.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Tổng quan một số phương pháp xử lý tính liên thông và các bài toán liên quan trong lý thuyết đồ thị	Wan Chengzhang
31	Bàn về xây dựng và sinh dữ liệu trong thi tin học	Liu Yiping
50	Bàn về một lớp bài toán có treewidth bị chặn	Wu Chang
63	Bao hàm–loại trừ trên quan hệ đôi một khác nhau	Zhou Ziheng
70	Báo cáo ra đề <i>Nhà ảo thuật</i>	Meng Yuhao
85	Hình học trong OI	Chang Ruinian
104	Bàn về các thuật toán liên quan tới xâu palindrome và ứng dụng	Xu Anyi
113	Bàn về một số ứng dụng của trường hữu hạn trong OI	Qi Langrui
123	Bàn về một lớp bài toán thống kê trên cây	Zhu Yikai
131	Bàn về ứng dụng của ma trận trong thi tin học	Yang Ningyuan
147	Bàn về một số bài toán liên quan tới matching đồ thị hai phía	Ke Yisi
156	Bàn về ứng dụng đặc biệt của DSU trong OI	Wang Xiangwen
167	Bàn về ứng dụng static top tree trên cây và đồ thị nối tiếp–song song tổng quát	Cheng Siyuan
181	Bước đầu về hypercube và các thuật toán liên quan	Guan Yanru
198	Bàn về một số phương pháp phân tích thừa số nguyên tố	Luo Siyuan
209	Một lớp cấu trúc dữ liệu xâu con cơ bản	Xu Tingqiang
220	Xem xét lại một vài bài toán số học cổ điển	Zheng Xuanye
232	Bàn về ứng dụng tính lồi của hàm số trong OI	Guo Yuchong

3 Tổng quan một số phương pháp xử lý tính liên thông và các bài toán liên quan trong lý thuyết đồ thị

Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông
Wan Chengzhang

Tóm tắt

Bài viết này chia thành hai phần: đồ thị vô hướng và đồ thị có hướng. Nội dung thảo luận một số tư tưởng thuật toán về tính liên thông có tính thực dụng trong thi tin học; ngoài ra còn khảo sát thêm một số bài toán lý thuyết đồ thị có liên hệ nhất định với tính liên thông.

3.1 Định nghĩa và quy ước

Với đồ thị vô hướng hoặc có hướng G :

- mặc định ký hiệu V là tập đỉnh của G , E là tập cạnh của G , $n = |V|$ là số đỉnh của G , $m = |E|$ là số cạnh của G ;
- nếu E không có phần tử lặp, ta nói G không có cạnh bội; nếu mọi $(u, u) \notin E$, ta nói G không có khuyên;
- G là đồ thị đơn khi và chỉ khi G không có cạnh bội và không có khuyên;
- ký hiệu $u \rightarrow v$ biểu thị một cạnh (u, v) ;
- ký hiệu $u \longrightarrow v$ biểu thị một đường đi bắt đầu tại u và kết thúc tại v , tức $u = x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k = v$;
- một chu trình là một đường đi dạng $u \rightarrow u$;
- đường đi đơn là đường đi không đi qua đỉnh lặp;
- chu trình đơn là chu trình không đi qua đỉnh lặp, ngoại trừ đỉnh đầu và đỉnh cuối;
- $G' = (V', E')$ là đồ thị con của G khi và chỉ khi $V' \subseteq V$, $E' \subseteq E$;
- đồ thị con cảm sinh theo tập đỉnh V' của G là $G' = (V', E')$, $E' = \{(u, v) \in E \mid u, v \in V'\}$;
- đồ thị con cảm sinh theo tập cạnh E' của G là $G' = (V', E')$, $V' = \{x \in V \mid (\exists y)((x, y) \in E' \vee (y, x) \in E')\}$.

3.2 Đồ thị vô hướng

Tính liên thông trong đồ thị vô hướng là một khái niệm rất dễ hiểu. Nếu tồn tại đường đi $u \rightarrow v$, ta nói u, v liên thông (vẫn dùng ký hiệu $u \rightarrow v$ để biểu thị u, v liên thông). Đây là một quan hệ tương đương. Tiếp theo ta định nghĩa một số khái niệm sẽ dùng về sau.

Định nghĩa 2.1 (tập cắt đỉnh/cạnh). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V$: nếu $S \subseteq V$ thỏa $u, v \notin S$ và trong đồ thị con cảm sinh của G theo $V \setminus S$ ta có $u \not\rightarrow v$, thì S được gọi là một tập cắt đỉnh của u, v ; nếu $T \subseteq E$ thỏa trong $G' = (V, E \setminus T)$ ta có $u \not\rightarrow v$, thì T được gọi là một tập cắt cạnh của u, v .

Định nghĩa 2.2 (độ liên thông đỉnh/cạnh). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V, u \neq v$, nếu mọi tập cắt đỉnh của u, v đều có kích thước không nhỏ hơn s , ta nói u, v là s -liên thông đỉnh; nếu mọi tập cắt cạnh của u, v đều có kích thước không nhỏ hơn t , ta nói u, v là t -liên thông cạnh. Độ liên thông đỉnh/cạnh giữa u, v là giá trị nhỏ nhất của s, t ở trên. Độ liên thông đỉnh/cạnh của G là giá trị nhỏ nhất trên mọi cặp đỉnh.

Ví dụ, 1-liên thông đỉnh và 1-liên thông cạnh là tương đương; cả hai đều tương đương với tính liên thông theo nghĩa thông thường.

Định lý 2.1 (Menger). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V, u \neq v$:

- u, v là k -liên thông cạnh khi và chỉ khi tồn tại k đường đi $u \rightarrow v$ đôi một không chung cạnh;
- u, v là k -liên thông đỉnh khi và chỉ khi tồn tại k đường đi $u \rightarrow v$ đôi một không chung đỉnh ngoài hai đầu mút.

Đây thực chất là một phiên bản đặc biệt của định lý luồng cực đại cắt cực tiểu. Vì chứng minh không liên quan nhiều tới bài viết này nên được lược bỏ. Cần chú ý rằng trong định lý Menger ta không yêu cầu các đường đi phải khác nhau. Đồ thị đầy đủ K_2 gồm hai đỉnh là k -liên thông đỉnh với mọi k , nhưng trong đó chỉ tồn tại một đường đi đơn.

3.2.1 Cây DFS

Trước hết ta xét loại liên thông đơn giản nhất, tức 1-liên thông. Với một đồ thị vô hướng liên thông G , luôn tìm được một đồ thị con không chu trình T chứa tất cả các đỉnh. Ta gọi T như vậy là cây khung của G .

Định nghĩa 2.3 (thứ tự DFS, cây DFS). Trên đồ thị vô hướng liên thông $G = (V, E)$, thực hiện tìm kiếm theo chiều sâu từ đỉnh xuất phát $r \in V$. Nếu đỉnh x là đỉnh thứ p được thăm, gọi p là thứ tự DFS của x , ký hiệu $\text{dfn}_x = p$. Nếu $x \neq r$, nối một cạnh giữa x và đỉnh y lần đầu thăm được x ; đồ thị tạo thành được gọi là một cây DFS gốc r , ký hiệu T .

Đồ thị T ở trên có $n - 1$ cạnh, và có thể chứng minh bằng quy nạp rằng T liên thông, nên T thật sự là một cây khung của G . Thông thường ta xem nó là cây có gốc r ; các cạnh trên T gọi là cạnh cây, các cạnh còn lại gọi là cạnh không thuộc cây.

Cần lưu ý rằng thứ tự DFS và cây DFS tương ứng một-một. Cùng một r có thể có nhiều thứ tự DFS và nhiều cây DFS khác nhau.

Tính chất 2.1.1. Thứ tự DFS của toàn bộ các đỉnh tạo thành một hoán vị của $1, \dots, n$. Với mọi đỉnh x , tập thứ tự DFS của tất cả đỉnh trong cây con gốc x trên cây DFS tạo thành một đoạn liên tiếp bắt đầu từ dfn_x .

Tính chất 2.1.2. Mọi cạnh không thuộc cây đều nối một cặp đỉnh tổ tiên–hậu duệ trên cây DFS. Các cạnh như vậy gọi là cạnh ngược lên tổ tiên.

Chứng minh chỉ cần xét tính chất của quá trình DFS. Tính chất 2.1.2 là tính chất cốt lõi của cây DFS; đây cũng là lý do nhiều bài toán liên quan tới đồ thị liên thông được xét bằng cây DFS.

Ví dụ 2.1.1 (Ehab’s Last Theorem¹). Với đồ thị vô hướng liên thông G có n đỉnh, ít nhất một trong hai mệnh đề sau đúng:

¹CF 1325 F

1. G có một chu trình đơn độ dài không nhỏ hơn $\sqrt{n}$.
2. G có một tập độc lập kích thước không nhỏ hơn $\sqrt{n}$.

Chứng minh. Đặt $p = \lceil \sqrt{n} \rceil$. Ta đưa ra một chứng minh mang tính xây dựng. Trước hết dựng một cây DFS gốc 1 của G , và ký hiệu d_x là độ sâu của x trên cây DFS.

Xét một cạnh không thuộc cây bất kỳ $x \rightarrow y$, trong đó x là hậu duệ. Nếu $d_x - d_y \geq p - 1$, ta đã tìm được một chu trình đơn có độ dài ít nhất p , gồm đường đi trên cây $x \rightarrow y$ cộng với cạnh không thuộc cây $x \rightarrow y$.

Giả sử không tồn tại chu trình như vậy. Khi đó với mọi cạnh không thuộc cây $x \rightarrow y$ ta có $|d_x - d_y| < p - 1$. Ta có thể xây dựng một tập độc lập kích thước ít nhất p bằng quá trình sau:

- Định nghĩa tập đỉnh S , ban đầu rỗng.
- Chọn một đỉnh chưa bị đánh dấu có độ sâu lớn nhất hiện tại, gọi là x . Đặt $S \leftarrow S \cup \{x\}$, rồi đánh dấu các tổ tiên bậc $0, 1, 2, \dots, p - 2$ của x .
- Lặp lại cho tới khi mọi đỉnh đều bị đánh dấu. Khi đó S là tập độc lập cần tìm.

Trong mỗi vòng, nhiều nhất $p - 1$ đỉnh bị đánh dấu, nên $|S| \geq \frac{n}{p-1} > p - 1$. Đồng thời, đỉnh x được chọn chỉ có thể có cạnh tới những đỉnh trong cây con của nó hoặc tới các tổ tiên bậc $0, 1, \dots, p - 2$. Cách chọn x theo độ sâu lớn nhất bảo đảm các đỉnh trong cây con đã bị đánh dấu, và các tổ tiên bậc $0, 1, \dots, p - 2$ cũng bị đánh dấu. Vì vậy ở mọi thời điểm, mọi đỉnh có cạnh nối với một đỉnh trong S đều đã bị đánh dấu, suy ra S thật sự là tập độc lập. Mệnh đề được chứng minh. ■

Với đồ thị vô hướng không liên thông, ta có thể chọn riêng một đỉnh gốc trong mỗi thành phần liên thông và dựng cây DFS, tạo thành một rừng DFS. Các khái niệm như thứ tự DFS vẫn dùng được, và các tính chất trên vẫn áp dụng. Chẳng hạn ví dụ 2.1.1 thật ra cũng đúng với đồ thị không liên thông; chứng minh tương tự trường hợp liên thông.

3.2.2 Song liên thông

Tiếp theo ta nghiên cứu tính 2-liên thông, phức tạp hơn một chút so với 1-liên thông, còn gọi là song liên thông.

Vì song liên thông là nội dung khá quen thuộc trong OI, phần cơ sở ở đây sẽ được trình bày ngắn gọn hơn, nhưng vẫn nêu đầy đủ mọi tính chất cần dùng. Nội dung này cũng được mô tả toàn diện trong tài liệu tham khảo [1].

Song liên thông cạnh. Do song liên thông cạnh trực quan hơn song liên thông đỉnh, ta giới thiệu nó trước.

Một cặp đỉnh song liên thông cạnh chắc chắn là 1-liên thông, nên các thành phần liên thông khác nhau độc lập với nhau. Vì vậy trước tiên xét trường hợp G liên thông. Điều đầu tiên cần nói là song liên thông cạnh (có thể tổng quát lên k -liên thông cạnh bất kỳ) là một quan hệ tương đương trên tập đỉnh V . Nói cách khác, ta có thể phân hoạch V theo quan hệ song liên thông cạnh.

Định nghĩa 2.4 (cạnh cắt). Với đồ thị vô hướng liên thông $G = (V, E)$, nếu $e \in E$ thỏa $G' = (V, E \setminus \{e\})$ không liên thông, thì e được gọi là một cạnh cắt hay cầu. Với đồ thị vô hướng không nhất thiết liên thông, mọi cạnh cắt của một thành phần liên thông bất kỳ đều được gọi là cạnh cắt của đồ thị.

Một mô tả tương đương là: nếu $\{e\}$ là tập cắt cạnh của một cặp đỉnh (u, v) , thì e là cạnh cắt.

Bổ đề 2.2.1. Trong đồ thị vô hướng liên thông G , mọi cạnh cắt đều nằm trên cây DFS. Giả sử một cạnh cắt là $e = x \rightarrow y$, trong đó x là đỉnh con. Sau khi xóa e , đồ thị tách thành hai thành phần liên thông: cây con của x và phần bù của cây con đó.

Bổ đề 2.2.2. Gọi E' là tập các cạnh cắt của đồ thị vô hướng liên thông $G = (V, E)$, và $G' = (V, E \setminus E')$. Khi đó với mọi $u, v \in V$, $u \neq v$, hai đỉnh u, v song liên thông cạnh trong G khi và chỉ khi u, v liên thông trong G' .

Chứng minh. Nếu u, v liên thông trong G' nhưng không song liên thông cạnh, thì tồn tại một cạnh e sao cho sau khi xóa e , u, v không còn liên thông. Khi đó e tất nhiên là cạnh cắt. Tuy nhiên trong G' đã xóa mọi cạnh cắt, u, v vẫn liên thông, mâu thuẫn. Do đó u, v song liên thông cạnh.

Nếu u, v không liên thông trong G' , xét đường đi trên cây $u \rightarrow v$. Trên đường đi này chắc chắn tồn tại một cạnh e là cạnh cắt (nếu không thì u, v đã liên thông). Theo bổ đề 2.2.1, sau khi xóa cạnh này, u, v không nằm trong cùng một thành phần liên thông, nên u, v không song liên thông cạnh. ■

Định nghĩa 2.5 (thành phần song liên thông cạnh). Các đỉnh của đồ thị vô hướng G có thể được phân hoạch thành vài lớp tương đương theo quan hệ song liên thông cạnh. Đồ thị con cảm sinh bởi mỗi lớp tương đương được gọi là một thành phần song liên thông cạnh của G .

Sự tồn tại của thành phần song liên thông cạnh được bảo đảm bởi bổ đề 2.2.2: chỉ cần xóa mọi cạnh cắt của G để được G' , khi đó mọi thành phần liên thông của G' chính là các thành phần song liên thông cạnh của G .

Ta có thể dựa vào bổ đề 2.2.2 để tìm tất cả thành phần song liên thông cạnh; với một số kỹ thuật cấu trúc dữ liệu, độ phức tạp thời gian có thể đạt $O(n + m)$. Tuy nhiên thuật toán Tarjan cũng có cùng độ phức tạp và gọn hơn.

Xét khi nào một cạnh trở thành cạnh cắt. Gọi cạnh cây DFS là $e = (x, y)$, trong đó x là đỉnh con. Khi đó e là cạnh cắt khi và chỉ khi không tồn tại cạnh từ một đỉnh trong cây con của x tới một tổ tiên của x (không tính x). Chú ý rằng mọi cạnh không thuộc cây đều là cạnh ngược lên tổ tiên. Vì vậy ta có thể tính low_x : thứ tự DFS nhỏ nhất của một đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x bằng cách đi qua nhiều nhất một cạnh không thuộc cây. Giá trị này được tính bằng quy hoạch động trên cây:

$$\text{low}_x = \min \left(\text{dfn}_x, \min_{y \in \text{Son}(x)} \text{low}_y, \min_{(x,y) \in E} \text{dfn}_y \right).$$

Những đỉnh thỏa $\text{low}_x = \text{dfn}_x$ là đỉnh nông nhất của một thành phần song liên thông cạnh. Khi trong quá trình DFS phát hiện một đỉnh như vậy, phần chưa bị xóa trong cây con của nó tạo thành một thành phần song liên thông cạnh; sau đó xóa cây con này.

Hình 1: Một đồ thị vô hướng và các thành phần song liên thông cạnh khác nhau của nó.

Song liên thông đỉnh. Song liên thông đỉnh không phải là quan hệ tương đương trên các đỉnh, nhưng ta vẫn có thể bắt đầu nghiên cứu từ cây DFS.

Định nghĩa 2.6 (đỉnh cắt). Với đồ thị vô hướng liên thông $G = (V, E)$, nếu $u \in V$ thỏa đồ thị con cảm sinh của G theo $V \setminus \{u\}$ không liên thông, thì u được gọi là một đỉnh cắt. Với đồ thị vô hướng không nhất thiết liên thông, mọi đỉnh cắt của một thành phần liên thông bất kỳ đều được gọi là đỉnh cắt của đồ thị.

Đỉnh cắt cũng có mô tả tương đương: nếu $\{u\}$ là tập cắt đỉnh của một cặp đỉnh nào đó, thì u là đỉnh cắt.

Nhắc lại mảng low trong thuật toán Tarjan: low_x là thứ tự DFS nhỏ nhất của đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x sau khi đi qua nhiều nhất một cạnh không thuộc cây. Ta có thể dùng nó để tìm đỉnh cắt.

Bổ đề 2.2.3. Trong đồ thị vô hướng liên thông $G = (V, E)$, x là đỉnh cắt khi và chỉ khi ít nhất một trong hai điều kiện sau đúng:

1. x là gốc của cây DFS và có ít nhất hai đỉnh con;
2. x không phải gốc của cây DFS, và tồn tại một đỉnh con y thỏa $\text{low}_y \geq \text{dfn}_x$.

Chứng minh. Điều kiện thứ nhất khá hiển nhiên.

Xét điều kiện thứ hai. Nếu tồn tại đỉnh con y như vậy, thì trong cây con của y không có cạnh ngược tới tổ tiên của x (không tính x). Sau khi xóa x , cây con của y không liên thông với bên ngoài và tự tạo thành một thành phần liên thông, nên x chắc chắn là đỉnh cắt.

Ngược lại, nếu đỉnh không phải gốc x là đỉnh cắt, gọi V' là phần bù của cây con x trong V . Sau khi xóa x , tồn tại ít nhất một đỉnh con y của x không liên thông với V' . Điều này nghĩa là không đỉnh nào trong cây con của y có cạnh ngược tới V' , tức $\text{low}_y \geq \text{dfn}_x$. ■

Định nghĩa 2.7 (thành phần song liên thông đỉnh). Trong đồ thị vô hướng $G = (V, E)$, nếu tập đỉnh $V' \subseteq V$ thỏa đồ thị con cảm sinh G' của V' là song liên thông đỉnh, và với mọi $V' \subset V'' \subseteq V$, đồ thị con cảm sinh của V'' không song liên thông đỉnh, thì G' được gọi là một thành phần song liên thông đỉnh của G .

Thuật toán Tarjan có thể dùng để tìm tất cả thành phần song liên thông đỉnh, theo cách tương tự tìm thành phần song liên thông cạnh: thực hiện quy hoạch động trên cây để tính low_x . Nếu với một cặp cha con (x, y) ta có $\text{low}_y \geq \text{dfn}_x$, thì lấy phần chưa bị xóa trong cây con của y , cộng thêm x , để tạo thành một thành phần song liên thông đỉnh. Sau đó xóa phần chưa bị xóa trong cây con của y . Độ phức tạp thời gian là $O(n + m)$.

Dựa trên bổ đề 2.2.3 và chứng minh của nó, không khó để chứng minh tính đúng đắn của thuật toán Tarjan. Tuy nhiên hiện tại ta chưa nắm nhiều tính chất của thành phần song liên thông đỉnh. Trong mục 2.2.1, ta định nghĩa thành phần song liên thông cạnh dựa trên phân hoạch của V' . Vậy thành phần song liên thông đỉnh có hình thức tương tự hay không? Câu trả lời là có, nhưng ta cần chuyển đổi tương nghiên cứu từ đỉnh sang cạnh. Tiếp theo ta đưa ra một cách mô tả thành phần song liên thông đỉnh từ góc nhìn cạnh. Trong phần còn lại của mục này, mặc định đồ thị không có khuyên.

Bổ đề 2.2.4. Trong đồ thị song liên thông đỉnh G , với hai đỉnh khác nhau bất kỳ u, v , tồn tại một chu trình đơn đi qua u, v .

Chứng minh. Đây là hệ quả trực tiếp của định lý Menger. ■

Bổ đề 2.2.5. Trong đồ thị song liên thông đỉnh G , với một đỉnh bất kỳ u và một cạnh bất kỳ e , tồn tại một chu trình đơn đi qua u, e ; với hai cạnh bất kỳ e_1, e_2 , tồn tại một chu trình đơn đi qua e_1, e_2 .

Chứng minh. Tách một cạnh $x \rightarrow y$ thành hai cạnh $x \rightarrow z$ và $z \rightarrow y$. Hiển nhiên đồ thị mới vẫn song liên thông đỉnh, và lúc này cạnh đã được chuyển thành đỉnh. Áp dụng trực tiếp bổ đề 2.2.4 là đủ. ■

Định nghĩa 2.8. Trong đồ thị vô hướng G , nếu hai cạnh e_1, e_2 cùng nằm trên một chu trình đơn, ta nói chúng đồng chu trình.

Bổ đề 2.2.6. Quan hệ đồng chu trình là một quan hệ tương đương; các cạnh trong mỗi thành phần song liên thông đỉnh tạo thành một lớp tương đương theo nghĩa đồng chu trình.

Chứng minh. Trước hết, hai cạnh bất kỳ trong cùng một thành phần song liên thông đỉnh đều đồng chu trình, theo bổ đề 2.2.5.

Với hai cạnh thuộc hai thành phần song liên thông đỉnh khác nhau $e_1 = (x, y)$, $e_2 = (u, v)$, giả sử tồn tại một chu trình đơn đồng thời chứa e_1, e_2 . Khi đó mọi đỉnh trên chu trình này chắc chắn song liên thông đỉnh, mâu thuẫn với việc e_1, e_2 thuộc hai thành phần song liên thông đỉnh khác nhau. ■

Cần chú ý rằng dù trong định nghĩa thành phần song liên thông đỉnh và trong chứng minh trên, ta chưa chứng minh một sự thật cơ bản: mỗi cạnh chỉ thuộc một thành phần song liên thông đỉnh. Thực ra từ định nghĩa hoặc từ thuật toán Tarjan đều có thể dễ dàng suy ra giao của hai thành phần song liên thông đỉnh bất kỳ có kích thước nhiều nhất 1, nên mỗi cạnh đúng là chỉ nằm trong một thành phần song liên thông đỉnh.

Nhờ bổ đề 2.2.6, ta có thể nhìn thành phần song liên thông đỉnh tương tự thành phần song liên thông cạnh; chỉ khác là thành phần song liên thông cạnh là phân hoạch tập đỉnh, còn thành phần song liên thông đỉnh là phân hoạch tập cạnh.

Cây tròn–vuông. Cây tròn–vuông là một cấu trúc dữ liệu dùng để mô tả cấu trúc các thành phần song liên thông đỉnh của đồ thị vô hướng.

Định nghĩa 2.9 (cây tròn–vuông). Với đồ thị vô hướng liên thông $G = (V, E)$ có ít nhất 2 đỉnh, xây một đồ thị vô hướng mới T . Mỗi đỉnh của T tương ứng với một đỉnh của G hoặc một thành phần song liên thông đỉnh của G . Hai đỉnh của T có cạnh khi và chỉ khi chúng lần lượt biểu diễn một đỉnh x của G và một thành phần song liên thông đỉnh $G' = (V', E')$ của G , đồng thời $x \in V'$. Đồ thị T được gọi là cây tròn–vuông của G . Trong T , đỉnh tương ứng với đỉnh của đồ thị gốc gọi là đỉnh tròn; đỉnh tương ứng với thành phần song liên thông đỉnh của đồ thị gốc gọi là đỉnh vuông.

Hình 2: Một đồ thị vô hướng, các thành phần song liên thông đỉnh khác nhau của nó, và cây tròn–vuông.

Xét một đường đi độ dài k là $u \rightarrow v$. Nó lần lượt đi qua các thành phần song liên thông đỉnh chứa từng cạnh của nó, nên tương ứng với một đường đi độ dài $2k$ từ u tới v trên cây tròn–vuông: ngoài các đỉnh trên đường đi gốc, nó còn lần lượt đi qua các đỉnh vuông tương ứng với các thành phần song liên thông đỉnh chứa từng cạnh. Vì vậy cây tròn–vuông của một đồ thị liên thông là liên thông. Đồng thời, cây tròn–vuông chắc chắn không có chu trình, nếu không chu trình đó có thể co thành một thành phần song liên thông đỉnh. Do đó cây tròn–vuông thật sự là một cây. Với đồ thị vô hướng G không nhất thiết liên thông, có thể định nghĩa “rừng tròn–vuông” bằng cách dựng cây tròn–vuông riêng cho từng thành phần liên thông.

Tính chất 2.2.1. Cây tròn–vuông là đồ thị hai phía; hai phía lần lượt là các đỉnh tròn và các đỉnh vuông.

Tính chất 2.2.2. Một đỉnh là đỉnh cắt khi và chỉ khi đỉnh tròn tương ứng của nó trên cây tròn–vuông có bậc lớn hơn 1. Lá trên cây tròn–vuông (kể cả gốc bậc 1) chắc chắn là đỉnh tròn, và chúng chính là toàn bộ các đỉnh không phải đỉnh cắt.

Tính chất 2.2.3. Trong một đồ thị vô hướng có $n \geq 2$ đỉnh, tổng số đỉnh của tất cả thành phần song liên thông đỉnh không vượt quá $2n - 2$.

Chứng minh. Chọn một đỉnh tròn làm gốc trên cây tròn–vuông. Sau đó, với mỗi đỉnh vuông, chỉ định một đỉnh con của nó. Các đỉnh con này đều là đỉnh tròn và chắc chắn không trùng nhau, nên số đỉnh vuông không vượt quá số đỉnh tròn không phải gốc, tức $n - 1$.

Vì vậy tổng số đỉnh của cây tròn–vuông không vượt quá $2n - 1$, tổng số cạnh không vượt quá $2n - 2$. Tổng kích thước của các thành phần song liên thông đỉnh chính là tổng bậc của các đỉnh vuông, tức tổng số cạnh của cây tròn–vuông. Suy ra kết luận. ■

Tính chất 2.2.4. Với hai đỉnh $u, v \in V$ trong đồ thị vô hướng liên thông G , mọi đường đi đơn giữa u và v chắc chắn đi qua, và chỉ đi qua, tất cả thành phần song liên thông đỉnh tương ứng với các đỉnh vuông trên đường đi giữa chúng trong cây tròn–vuông (cụ thể là đi qua các cạnh nằm trong những thành phần đó); đồng thời chắc chắn đi qua mọi đỉnh tròn trên đường đi đó (tức các đỉnh tương ứng trong đồ thị gốc).

Cây tròn–vuông là một cấu trúc rất quan trọng trong OI và có nhiều ứng dụng. Dưới đây là vài ví dụ.

Ví dụ 2.2.1 (xương rồng). Đồ thị vô hướng liên thông mà mỗi cạnh nằm trên nhiều nhất một chu trình đơn được gọi là xương rồng theo cạnh. Nhiều thao tác vốn làm được trên cây có thể được chuyển sang xương rồng thông qua cây tròn–vuông.

Xét thuật toán Tarjan trên xương rồng. Ta nhận được một cây DFS, và mỗi cạnh không thuộc cây tương ứng với một đường đi trên cây; các đường đi này đôi một không chung cạnh. Mỗi cạnh không thuộc cây tương ứng với một chu trình đơn. Vì vậy quá trình Tarjan trực tiếp giúp ta tìm mọi chu trình, điều này rất tiện. Ví dụ khi cần DP trên xương rồng, ta có thể làm từ dưới lên trên cây tròn–vuông (hoặc trên cây DFS): mỗi khi tìm được một chu trình thì DP trên chu trình, nếu không thì DP trên cây.

Khi cần sửa đổi/truy vấn đường đi trên xương rồng, thông thường phải chuyển đường đi trên xương rồng thành đường đi trên cây tròn–vuông, rồi qua một số thảo luận chuyển bài toán thành bài toán trên cây. Tuy nhiên loại bài này thường khó cài đặt, và hiện không thường gặp trong OI.

Ví dụ 2.2.2 (Địa địa thiết thiết²). Cho đồ thị vô hướng liên thông $G = (V, E)$, các cạnh có hai màu đen và trắng. Hãy tính số cặp đỉnh không thứ tự (x, y) sao cho tồn tại một đường đi đơn $x \rightarrow y$ có cả cạnh đen lẫn cạnh trắng.

Ta chỉ cần đếm các cặp (x, y) không thỏa điều kiện. Có ba trường hợp:

1. mọi đường đi đơn $x \rightarrow y$ chỉ đi qua cạnh đen;
2. mọi đường đi đơn $x \rightarrow y$ chỉ đi qua cạnh trắng;
3. tồn tại riêng một đường đi đơn $x \rightarrow y$ chỉ gồm cạnh đen và một đường đi đơn chỉ gồm cạnh trắng, nhưng không có đường đi đơn nào đồng thời đi qua hai màu cạnh.

Trước hết xét trường hợp 1. Theo tính chất 2.2.4 của cây tròn–vuông, ta biết chỉ cần xét các cạnh trong mọi đỉnh vuông nằm trên đường đi $x \rightarrow y$ của cây tròn–vuông, vì các cạnh khác không thể xuất hiện trên đường đi đơn nào từ x tới y .

Gọi một đỉnh vuông là thuần đen khi và chỉ khi mọi cạnh trong thành phần song liên thông đỉnh tương ứng đều là đen; thuần trắng được định nghĩa tương tự. Ta khẳng định trường hợp 1 xảy ra khi và chỉ khi mọi đỉnh vuông trên đường đi $x \rightarrow y$ của cây tròn–vuông đều thuần đen.

Bổ đề phụ: trong đồ thị song liên thông đỉnh, với hai đỉnh cho trước x, y ($x \neq y$) và một cạnh e , tồn tại một đường đi đơn $x \rightarrow y$ đi qua e .

Chứng minh bổ đề phụ chỉ cần nhắc lại bổ đề 2.2.5. Trước hết tìm một chu trình C chứa x, e , sau đó chọn một đường đi đơn P từ y tới x sao cho tập đỉnh của P và C giao nhau nhiều hơn một đỉnh (chú ý x

²Luogu P8456, SWTR-8

chắc chắn nằm trong giao, nên luôn tìm được P như vậy; nếu không, x là đỉnh cắt). Gọi u là đỉnh gần y nhất trên P trong giao của hai tập đỉnh. Khi đó chọn đường đi trên chu trình từ x tới u có chứa e , ghép với tiền tố $u \rightarrow y$ của P , ta được một đường đi đơn $x \rightarrow y$ đi qua e .

Vì vậy nếu tồn tại một đỉnh vuông trên đường đi không thuần đen, khi đi tới thành phần song liên thông đỉnh đó ta có thể tùy ý đi qua một cạnh trắng trong đó, mâu thuẫn với trường hợp 1. Ngược lại, nếu mọi đỉnh vuông trên đường đi đều thuần đen, hiển nhiên mọi đường đi đơn từ x tới y đều thuần đen. Khẳng định được chứng minh. Trường hợp 2 tương tự trường hợp 1.

Trường hợp 3 nhìn qua phức tạp hơn một chút. Trước hết cần một bổ đề.

Bổ đề phụ: nếu (x, y) thỏa trường hợp 3, thì với hai đường đi đơn bất kỳ P_1, P_2 từ x tới y lần lượt thuần trắng và thuần đen, chúng không chung đỉnh nào ngoài hai đầu mút.

Chứng minh bổ đề không khó: giả sử hai đường đi còn giao nhau ở một đỉnh khác ngoài đầu mút, thì có thể lấy một tiền tố và một hậu tố để tạo thành một đường đi mới; đường đi mới không thuần đen cũng không thuần trắng, mâu thuẫn với mô tả của trường hợp 3.

Bổ đề này cho biết x, y chắc chắn nằm trong cùng một thành phần song liên thông đỉnh, nếu không mọi đường đi giữa chúng đều đi qua một đỉnh cắt nào đó. Ngoài ra, giả sử trong thành phần song liên thông đỉnh chứa x, y còn có một đỉnh $u \neq x, y$ kề với hai cạnh e_1, e_2 lần lượt đen và trắng, thì x, y chắc chắn không thỏa trường hợp 3. Lý do là ta có thể tìm hai đường đi đơn $x \rightarrow y$ lần lượt đi qua e_1, e_2 , và chúng giao nhau tại u , mâu thuẫn với bổ đề.

Từ phân tích trên, trong mỗi thành phần song liên thông đỉnh có nhiều nhất một cặp (x, y) thỏa trường hợp 3; chỉ cần tìm các đỉnh kề đồng thời cạnh đen và cạnh trắng.

Sau khi phân tích ba trường hợp, phần còn lại chỉ là tính số lượng các cặp đỉnh này; phần đó khá đơn giản nên lược bỏ.

Ví dụ 2.2.3 (Tom & Jerry³). Trên một đồ thị vô hướng liên thông G , Tom đuổi Jerry. Mỗi lượt Jerry đi trước, Tom đi sau. Trong mỗi lần đi, Jerry có thể đi qua tùy ý nhiều cạnh nhưng không được đi qua đỉnh Tom đang đứng; Tom mỗi lần chỉ được đi qua nhiều nhất một cạnh. Nếu sau khi đi, Tom đứng tại vị trí của Jerry thì Tom thắng. Có q truy vấn (a, b) : khi ban đầu Tom và Jerry lần lượt đứng tại a, b , hỏi Tom có thể thắng trong thời gian hữu hạn hay không.

Định nghĩa cặp đỉnh (x, y) là tốt khi và chỉ khi mọi cặp đỉnh tròn kề nhau trên đường đi $x \rightarrow y$ của cây tròn–vuông cũng kề nhau trong đồ thị gốc G .

Kết luận chính: Tom thắng khi và chỉ khi ít nhất một trong hai điều kiện sau đúng:

1. tồn tại một đỉnh u sao cho với mọi x , cặp (u, x) là tốt;
2. với mọi đỉnh x trong thành phần liên thông chứa b sau khi xóa a , cặp (a, x) là tốt.

Trước hết chứng minh tính đủ. Hai điều kiện thực ra tương tự nhau; lấy điều kiện 2 làm ví dụ. Mỗi lần Tom chỉ cần tiến một bước về phía Jerry theo đường đi trên cây tròn–vuông (tính khả thi ở đây được điều kiện 2 bảo đảm). Từ tính chất của cây tròn–vuông, Jerry luôn chỉ có thể di chuyển trong một nhánh của đỉnh Tom đang đứng; do đó cuối cùng Tom và Jerry sẽ gặp nhau, Tom thắng.

Xét tính cần. Giả sử cả hai điều kiện đều không thỏa. Khi đó bước đầu tiên Jerry có thể đi tới một đỉnh x sao cho (a, x) không tốt, rồi đứng yên chờ tới khi Tom đi tới một đỉnh y không nằm trên đường đi cây $a \rightarrow x$. Vì điều kiện 1 không đúng, trong toàn đồ thị chắc chắn tồn tại một đỉnh z sao cho (y, z) không tốt. Nếu đường đi cây $z \rightarrow x$ không đi qua x , lúc này Jerry đi thẳng từ x tới z ; nếu không, Jerry có thể đi tới z ở bước ngay trước khi Tom đi tới y . Tóm lại, ta lại trở về tình huống cặp đỉnh gồm vị trí của Tom và vị trí của Jerry không tốt; vì vậy Tom sẽ không bao giờ bắt được Jerry.

³Luogu P7353; bài này do tác giả ra cho bài tập trại tình anh IOI 2021.

Sau khi có kết luận chính, thêm DP trên cây tròn–vuông là giải được bài này. Phần sau không liên quan tới bài viết nên lược bỏ.

Phân rã tai. Phân rã tai là một cách mô tả khác của đồ thị song liên thông, và có thể cung cấp cho ta một góc nhìn khác trong một số bài toán.

Định nghĩa 2.10 (tai, tai mở). Trong đồ thị vô hướng $G = (V, E)$ có một đồ thị con $G' = (V', E')$. Nếu một đường đi đơn hoặc chu trình đơn $P : x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k$ thỏa $x_1, x_k \in V'$ và $x_2, \dots, x_{k-1} \notin V'$, thì P được gọi là một tai của G đối với G' . Nếu P là đường đi đơn, P được gọi là tai mở của G đối với G' .

Định nghĩa 2.11 (phân rã tai, phân rã tai mở). Với đồ thị vô hướng liên thông G , nếu dãy đồ thị liên thông $(G_0, G_1, \dots, G_k)$ thỏa:

1. G_0 là một chu trình đơn (có thể chỉ có một đỉnh), $G_k = G$;
2. G_{i-1} là đồ thị con của G_i ;
3. đặt $G_i = (V_i, E_i)$, khi đó $E_i \setminus E_{i-1}$ tạo thành một tai (tai mở) của G_{i-1} ,

thì $(G_0, G_1, \dots, G_k)$ được gọi là một phân rã tai (phân rã tai mở) của G .

Định lý 2.2. Đồ thị vô hướng liên thông G có phân rã tai khi và chỉ khi G song liên thông cạnh.

Chứng minh. Trước hết chứng minh tính cần. Nếu G có phân rã tai $(G_0, \dots, G_k)$, thì vì G_0 là chu trình đơn, G_0 chắc chắn song liên thông cạnh. Nếu G_i song liên thông cạnh, thì G_{i+1} thu được bằng cách thêm một đường đi vào G_i cũng song liên thông cạnh (vì không cạnh nào có thể là cạnh cắt). Quy nạp suy ra $G = G_k$ song liên thông cạnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ song liên thông cạnh và không có cạnh nào, kết luận hiển nhiên. Nếu không, dựng một cây DFS gốc 1, rồi lấy một phân rã tai của G theo cách sau:

1. Tìm một cạnh không thuộc cây có đầu mút là 1, viết là $1 \rightarrow x$. Cho G_0 là chu trình đơn gồm đường đi trên cây $1 \rightarrow x$ cộng với cạnh này.
2. Giả sử hiện đã sinh được G_i . Nếu tập đỉnh của G_i chưa phải V , tìm một đỉnh x chưa thuộc G_i sao cho cha y của x thuộc G_i . Do song liên thông cạnh, tồn tại một cạnh ngược có hai đầu lần lượt là một tổ tiên u của y và một hậu duệ v của x . Ta chọn đường đi trên cây $y \rightarrow v$ hợp với cạnh $v \rightarrow u$; đây là một tai của G_i . Cho G_{i+1} bằng G_i cộng tai này.
3. Lặp lại cho tới khi tập đỉnh của G_i là V . Nếu khi đó còn cạnh nào của G chưa được thêm vào G_i , thêm từng cạnh riêng lẻ như một tai.

Ở bước thứ hai, ta luôn bảo đảm tập đỉnh của G_i là một khối liên thông trên cây chứa 1, nên luôn tìm được x như vậy. ■

Ví dụ 2.2.4 (Quare⁴). Cho đồ thị vô hướng song liên thông cạnh G có trọng số cạnh. Hãy tìm tổng trọng số nhỏ nhất của một đồ thị con song liên thông cạnh chứa mọi đỉnh.

Xét trực tiếp hình dạng đồ thị song liên thông cạnh là khá khó, nhưng từ góc nhìn phân rã tai thì dễ xử lý hơn. Gọi $f(S)$ là tổng trọng số nhỏ nhất để nối các đỉnh trong tập S thành một đồ thị song liên thông cạnh. Khi đó chuyển trạng thái có dạng $f(S) + E(S, T \setminus S) \rightarrow f(T)$, trong đó $E(S, R)$ là tổng trọng số nhỏ nhất của một tai có hai đầu mút thuộc S và các đỉnh còn lại thuộc R . Có thể tính $E(S, R)$ bằng

⁴SNOI 2013

cách liệt kê đỉnh cuối cùng ngoài hai đầu mút trên tai; độ phức tạp $O(2^n \text{ Poly}(n))$. Tuy nhiên khi tính mảng f , ta cần liệt kê tập con, nên tổng độ phức tạp là $O(3^n \text{ Poly}(n))$.

Có thể tối ưu điều này. Ta không tính $E(S, R)$, mà trực tiếp liệt kê thứ tự các đỉnh trên tai trong quá trình tính mảng f : mỗi lần chỉ thêm một đỉnh ở cuối tai, thay vì thêm cả tai. Cụ thể, đặt $f(S, i, j)$ là giá trị khi đã nối tập đỉnh S thành một đồ thị song liên thông cạnh, hiện đang thêm một tai, tai này đã kéo dài từ một đầu tới i , và quy định đầu còn lại của tai là $j \in S$. Khi chuyển trạng thái chỉ cần liệt kê đỉnh kế tiếp sau i trên tai. Độ phức tạp là $O(2^n \text{ Poly}(n))$.

Định lý 2.3. Đồ thị vô hướng liên thông không khuyên G có ít nhất ba đỉnh có phân rã tai mở khi và chỉ khi G song liên thông đỉnh.

Chứng minh. Chứng minh nhìn chung tương tự định lý 2.2.

Trước hết chứng minh tính cần. Nếu G có phân rã tai mở $(G_0, \dots, G_k)$, thì vì G_0 là chu trình đơn, G_0 chắc chắn song liên thông đỉnh. Nếu G_i song liên thông đỉnh, thì G_{i+1} thu được bằng cách thêm một đường đi vào G_i cũng song liên thông đỉnh. Quy nạp suy ra $G = G_k$ song liên thông đỉnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ song liên thông đỉnh, dựng một cây DFS gốc 1, rồi lấy một phân rã tai mở của G như sau:

1. Tìm một cạnh không thuộc cây có đầu mút là 1, viết là $1 \rightarrow x$. Cho G_0 là chu trình đơn gồm đường đi trên cây $1 \rightarrow x$ cộng với cạnh này.
2. Giả sử hiện đã sinh được G_i . Nếu tập đỉnh của G_i chưa phải V , tìm một đỉnh x chưa thuộc G_i sao cho cha y của x thuộc G_i . Do song liên thông đỉnh, tồn tại một cạnh ngược có hai đầu lần lượt là một tổ tiên u của cha của y và một hậu duệ v của x . Ta chọn đường đi trên cây $y \rightarrow v$ hợp với cạnh $v \rightarrow u$; đây là một tai mở của G_i . Cho G_{i+1} bằng G_i cộng tai mở này.
3. Lặp lại cho tới khi tập đỉnh của G_i là V . Nếu khi đó còn cạnh nào của G chưa được thêm vào G_i , thêm từng cạnh riêng lẻ như một tai mở (ở đây dùng điều kiện không có khuyên).

Ở bước thứ hai, ta luôn bảo đảm tập đỉnh của G_i là một khối liên thông trên cây chứa 1, nên luôn tìm được x như vậy. ■

Ví dụ 2.2.5 (định hướng lưỡng cực). Cho đồ thị vô hướng $G = (V, E)$ và hai đỉnh khác nhau s, t . Bốn mệnh đề sau tương đương:

1. sau khi thêm cạnh vô hướng $s \rightarrow t$, G song liên thông đỉnh;
2. trong cây tròn–vuông của G , mọi đỉnh vuông tạo thành một dây chuyền, và $s \rightarrow t$ là một đường kính của cây tròn–vuông;
3. tồn tại một cách định hướng các cạnh của G để thu được một đồ thị có hướng không chu trình, trong đó s có bậc vào bằng 0, t có bậc ra bằng 0, và mọi đỉnh còn lại đều có bậc vào và bậc ra khác 0;
4. tồn tại một hoán vị các đỉnh $p_1, p_2, \dots, p_n$ sao cho $p_1 = s, p_n = t$, và đồ thị con cảm sinh bởi mọi tiền tố cũng như mọi hậu tố đều liên thông.

Chứng minh. Trước hết chứng minh mệnh đề 1,2 tương đương và mệnh đề 3,4 tương đương; sau đó chứng minh hai cặp mệnh đề này cũng tương đương.

Sự tương đương của mệnh đề 1 và 2 khá hiển nhiên.

Xét mệnh đề 3 và 4. Nếu mệnh đề 3 đúng, lấy một thứ tự tô pô bất kỳ của đồ thị đã định hướng làm hoán vị p trong mệnh đề 4. Theo mô tả của mệnh đề 3, ngoài s , mỗi đỉnh đều có một láng giềng đứng trước nó trong thứ tự tô pô. Vì vậy với mọi p_i , tồn tại một láng giềng trong $p_1, \dots, p_{i-1}$; từ tính liên thông

của đồ thị con cảm sinh bởi $p_1, \dots, p_{i-1}$ suy ra tính liên thông của đồ thị con cảm sinh bởi $p_1, \dots, p_i$. Quy nạp được mọi tiền tố liên thông; mọi hậu tố cũng tương tự. Nếu mệnh đề 4 đúng, định hướng mỗi cạnh từ đỉnh đứng trước trong p tới đỉnh đứng sau trong p , để kiểm tra mệnh đề 3 đúng.

Tiếp theo chứng minh mệnh đề 1 suy ra mệnh đề 3. Sau khi nối s, t , lấy một phân rã tai mở $(G_0, \dots, G_k)$ của G sao cho G_0 chứa cạnh $s \rightarrow t$ (nhắc lại chứng minh định lý 2.3, ta có thể làm được điều này trong quá trình dựng). G_0 là một chu trình đơn, nên hiển nhiên có thể định hướng mọi cạnh ngoài $s \rightarrow t$ từ s tới t . Tiếp tục quy nạp: nếu G_i đã định hướng xong, nay thêm một tai mở có hai đầu u, v để được G_{i+1} . Nếu trong định hướng của G_i , u tới được v , định hướng tai từ u tới v ; ngược lại định hướng tai từ v tới u . Điều này không phá vỡ tính không chu trình, và mọi đỉnh ngoài s, t khi được thêm vào đều có bậc vào và bậc ra bằng 1. Vì vậy định hướng cuối cùng của $G = G_k$ thỏa mệnh đề 3.

Cuối cùng chứng minh mệnh đề 4 suy ra mệnh đề 1. Giả sử mệnh đề 1 không đúng. Khi đó tồn tại một đỉnh cắt u sao cho sau khi xóa u , hai đỉnh s, t nằm trong cùng một thành phần liên thông; do đó chắc chắn còn một thành phần liên thông S không chứa s, t . Gọi x, y lần lượt là đỉnh xuất hiện sớm nhất và muộn nhất trong hoán vị p trong tập $S \cup \{u\}$. Ít nhất một trong hai đỉnh x, y không phải u . Không mất tính tổng quát, giả sử $x \neq u$. Xét đồ thị con cảm sinh bởi tiền tố $p_1, \dots, p_i = x$. Vì u không nằm trong đó, còn s, x đều nằm trong đó, đồ thị này chắc chắn không liên thông, mâu thuẫn với mệnh đề 4. Do đó mệnh đề 4 suy ra mệnh đề 1.

Kết hợp các phần trên, ta chứng minh được bốn mệnh đề tương đương, đồng thời đã đưa ra cách dựng từ mệnh đề 1 tới mệnh đề 3,4. ■

Hình 3: Một đồ thị vô hướng, phân rã tai mở sau khi nối s, t , và định hướng lưỡng cực.

3.2.3 Tập cắt và tương đương theo lát cắt

Không gian cắt và không gian chu trình. Trước đó ta đã định nghĩa tập cắt đỉnh và tập cắt cạnh giữa hai điểm, nhưng tập cắt trong phần này có nghĩa hơi khác: nó được định nghĩa trên một phân hoạch hai phần của tập đỉnh V .

Định nghĩa 2.12 (tập cắt). Trong đồ thị vô hướng $G = (V, E)$, với một phân hoạch hai phần $V = V_1 \sqcup V_2$, định nghĩa tập cắt giữa V_1, V_2 là

$$C(V_1, V_2) = \{e \in E \mid e = (x, y), x \in V_1, y \in V_2\}.$$

Trong bài viết này cũng viết gọn là $C(V_1)$ hoặc $C(V_2)$.

Có thể thấy, nếu G không liên thông thì một tập cắt của G có thể tách thành hợp các tập cắt của từng thành phần liên thông. Vì vậy về cơ bản ta chỉ cần xét tập cắt của đồ thị liên thông. Đồ thị liên thông đơn giản nhất là cây, nên ta bắt đầu từ tập cắt của cây.

Bổ đề 2.3.1. Với cây $T = (V, E)$, mọi tập con cạnh $E' \subseteq E$ đều là tập cắt, và phân hoạch V_1, V_2 tương ứng là duy nhất.

Chứng minh. Với tập con cạnh bất kỳ $E' \subseteq E$, sau khi xóa các cạnh trong E' , cây tách thành vài thành phần liên thông. Gọi tập các thành phần này là $\mathcal{C}$. Các cạnh trong E' nối các phần tử của $\mathcal{C}$ thành một cây; gọi cây có đỉnh là các phần tử của $\mathcal{C}$ này là T' .

Nếu E' thật sự là tập cắt, thì với mọi $c \in \mathcal{C}$, tất cả đỉnh trong c phải nằm cùng một phía của tập cắt; với hai đỉnh kề nhau $c_1, c_2 \in \mathcal{C}$ trên T' , chúng phải nằm ở hai phía khác nhau của tập cắt.

Do đó ta nhận được phân hoạch duy nhất có thể: một phía gồm tất cả các đỉnh trong những thành phần liên thông ứng với đỉnh độ sâu lẻ trên T' , phía còn lại gồm tất cả các đỉnh trong những thành phần

liên thông ứng với đỉnh độ sâu chẵn. Để kiểm tra E' đúng là tập cắt của phân hoạch này, nên mệnh đề được chứng minh. ■

Bổ đề 2.3.2. Nếu đồ thị liên thông $G = (V, E)$ có cây khung $T = (V, E_0)$, thì với mọi $E'_0 \subseteq E_0$, tồn tại duy nhất một tập cắt E' của G sao cho $E' \cap E_0 = E'_0$, và phân hoạch V_1, V_2 tương ứng là duy nhất.

Chứng minh. Theo bổ đề 2.3.1, từ E'_0 ta trực tiếp thu được duy nhất một cặp V_1, V_2 . Với mọi cạnh không thuộc cây e , nếu hai đầu của e đều nằm trong V_1 hoặc đều nằm trong V_2 thì $e \notin E'$; ngược lại $e \in E'$. Như vậy thu được tập cắt duy nhất thỏa điều kiện. ■

Ta còn có thể phân tích thêm những cạnh không thuộc cây nào nằm trong E' . Xét cạnh không thuộc cây e có hai đầu x, y . Nếu đường đi trên cây $x \rightarrow y$ có số lẻ cạnh thuộc E'_0 , thì trong T' ở chứng minh bổ đề 2.3.1, hai thành phần chứa x, y có độ sâu khác tính chẵn lẻ, nghĩa là chúng nằm ở hai phía của tập cắt, nên $e \in E'$. Ngược lại $e \notin E'$. Nói cách khác, E' chứa đúng các cạnh không thuộc cây bằng qua số lẻ cạnh cây thuộc E'_0 .

Định lý 2.4 (không gian cắt). Trong đồ thị vô hướng $G = (V, E)$, xem mỗi tập cạnh như một vector m chiều trên $\mathbb{F}_2$. Khi đó tập tất cả các tập cắt tạo thành một không gian tuyến tính trên $\mathbb{F}_2$, gọi là không gian cắt. Số chiều của nó là $n - c$, trong đó c là số thành phần liên thông của G .

Chứng minh. Trước hết xét trường hợp đồ thị liên thông.

Để chứng minh các tập cắt tạo thành không gian tuyến tính, chỉ cần chứng minh hiệu đối xứng của hai tập cắt bất kỳ vẫn là tập cắt. Giả sử có hai tập cắt C_1, C_2 , và các tập con của chúng trên cây khung lần lượt là E_1, E_2 . Khi đó tập con trên cây khung của $C_1 \oplus C_2$ là $E_1 \oplus E_2$. Xét một cạnh không thuộc cây e . Nếu nó bằng qua số lẻ cạnh trong $E_1 \oplus E_2$, thì nhờ tính chất bảo toàn chẵn lẻ của hiệu đối xứng, ta có đúng một trong hai điều sau: e bằng qua số lẻ cạnh trong E_1 , hoặc e bằng qua số lẻ cạnh trong E_2 . Điều này tương đương $e \in C_1 \oplus C_2$, nên $C_1 \oplus C_2$ đúng là tập cắt.

Để tính số chiều không gian, chỉ cần đưa ra một cơ sở. Theo bổ đề 2.3.2, với mỗi cạnh trên cây khung e , định nghĩa $f(e)$ là tập cạnh gồm e và các cạnh không thuộc cây bằng qua e . Khi đó mọi $f(e)$ tạo thành một cơ sở của không gian cắt, nên số chiều là $n - 1$.

Khi đồ thị không liên thông, các thành phần liên thông độc lập với nhau; không gian cắt của toàn đồ thị bằng tổng trực tiếp các không gian cắt của từng thành phần liên thông. Cộng số chiều lại, không gian cắt của G có số chiều $n - c$. ■

Trong đồ thị vô hướng còn có một không gian tuyến tính khác trên $\mathbb{F}_2$. Vì cấu trúc của nó đơn giản và trực quan hơn, còn quá trình thảo luận tương tự không gian cắt, ta chỉ nêu kết luận mà không chứng minh.

Định lý 2.5 (không gian chu trình). Trong đồ thị vô hướng $G = (V, E)$, xét mọi đồ thị con $G' = (V, E')$ sao cho bậc của mỗi đỉnh đều chẵn. Xem các E' này như vector m chiều trên $\mathbb{F}_2$, chúng tạo thành một không gian tuyến tính trên $\mathbb{F}_2$, gọi là không gian chu trình của G . Số chiều của nó là $m - n + c$, trong đó c là số thành phần liên thông.

Tương tự không gian cắt, một cơ sở của không gian chu trình được dẫn ra bởi từng cạnh không thuộc cây: với mỗi cạnh không thuộc cây $e = (x, y)$, hợp của e với đường đi trên cây $x \rightarrow y$ tạo thành một chu trình; các chu trình này là một cơ sở của không gian chu trình.

Định lý 2.6. Không gian cắt và không gian chu trình của cùng một đồ thị vô hướng là bù trực giao của nhau.

Chứng minh. Dùng cùng một rừng khung, ta chứng minh các phần tử trong cơ sở của không gian cắt

và cơ sở của không gian chu trình tạo từ rừng khung này đôi một trực giao. Xét một cạnh cây e_1 và một cạnh không thuộc cây $e_2 = (x, y)$; gọi các phần tử cơ sở tương ứng của không gian cắt và không gian chu trình là v_1, v_2 . Nếu e_1 không nằm trên đường đi cây $x \rightarrow y$, thì hai tập cạnh ứng với v_1, v_2 không giao nhau. Nếu e_1 nằm trên đường đi cây $x \rightarrow y$, thì giao của hai tập cạnh ứng với v_1, v_2 chỉ gồm e_1, e_2 . Trong cả hai trường hợp đều có $v_1 \cdot v_2 = 0$.

Đồng thời tổng số chiều của hai không gian bằng m , nên chúng là bù trực giao của nhau. ■

Ví dụ 2.3.1 (phủ chu trình⁵). Cho đồ thị vô hướng đơn G . Với mỗi $i \in [1, m]$, hãy tính số phần tử trong không gian chu trình có tập cạnh kích thước i .

Bài này có hai hướng suy nghĩ chính.

Cách 1: một cách làm trực tiếp. Xem mỗi cạnh $e = (x, y)$ như một vector n chiều trên $\mathbb{F}_2$, chỉ có chiều thứ x, y bằng 1. Khi đó bài toán chuyển thành đếm số tập con i phần tử có tổng bằng không.

Cách 2: tìm một rừng khung của G , rồi xét một cơ sở của không gian chu trình dẫn ra bởi rừng. Liệt kê một phần tử là tổng của x phần tử cơ sở, và chỉ giữ lại các chiều ứng với cạnh cây (vì số cạnh không thuộc cây đã được liệt kê là x). Bài toán chuyển thành đếm trong các tập con x phần tử của cơ sở, có bao nhiêu tập cho tổng có số bit 1 bằng $i - x$.

Cả hai phép chuyển đổi đều có thể xử lý trực tiếp bằng FWT, độ phức tạp $O(n2^n + \text{Poly}(m))$.

Trên cơ sở đó, cách 1 có thể dùng định nghĩa của FWT để chuyển FWT thành một DP nén trạng thái. Nếu có thể tính popcount và lowbit trong $O(1)$, độ phức tạp đạt $O(2^n + \text{Poly}(m))$.

Tối ưu cách 2 cần thêm một số kiến thức nền. Vì khá phức tạp nên không trình bày chi tiết ở đây; độc giả quan tâm có thể tham khảo lời giải CF1336E2. Thực chất, cách 2 đang tìm phân bố số bit 1 của mọi phần tử trong một không gian tuyến tính trên $\mathbb{F}_2$. Kết luận là: nếu ta giải được bài toán này trong không gian bù trực giao, thì có thể trả lời cho không gian ban đầu với chi phí đa thức. Vì vậy ta có thể xét bài toán trên bù trực giao của không gian chu trình, tức không gian cắt. Số chiều của không gian cắt là $n - c$, có thể liệt kê trực tiếp. Độ phức tạp cuối cùng là $O(2^{n/w}m + \text{Poly}(m))$ theo cách ghi của nguồn.

Ứng dụng của tập cắt và tương đương theo lát cắt. Dù đã giới thiệu các tính chất của không gian cắt, tới đây ta vẫn chưa biết cách kiểm tra một tập cạnh có phải tập cắt hay không.

Giả sử trong đồ thị có tổng cộng t cạnh không thuộc cây. Ta xem mỗi cạnh là một vector t chiều trong $\mathbb{F}_2$: cạnh không thuộc cây thứ i ứng với vector chỉ có bit thứ i bằng 1, còn vector ứng với một cạnh cây có bit bằng 1 tại đúng các cạnh không thuộc cây bằng qua cạnh cây đó, và các bit khác bằng 0.

Xét cơ sở của không gian cắt thu được từ rừng khung. Mỗi phần tử cơ sở có dạng một cạnh cây hợp với tất cả cạnh không thuộc cây bằng qua cạnh cây đó. Theo định nghĩa trên, tổng các vector t chiều ứng với những cạnh trong mỗi phần tử cơ sở bằng không.

Ta biết không gian cắt là không gian tuyến tính. Kết hợp định nghĩa vector t chiều của cạnh ở trên, không khó thu được: một tập cạnh là tập cắt khi và chỉ khi tổng các vector t chiều ứng với mọi cạnh trong tập đó bằng không.

Khi cài đặt, ta không thể thật sự thao tác với vector t chiều. Có thể ánh xạ mỗi chiều tới một giá trị trong phạm vi 2^{64} , rồi biểu diễn một vector bằng xor của các giá trị ứng với các bit bằng 1. Nói cách khác, gán một trọng số ngẫu nhiên cho mỗi cạnh không thuộc cây; định nghĩa trọng số của một cạnh cây là xor của trọng số mọi cạnh không thuộc cây bằng qua nó. Nếu xor trọng số cạnh của một tập cạnh bằng không, ta tin rằng nó là tập cắt.

⁵Bài kiểm tra chéo đội tuyển quốc gia Trung Quốc IOI 2023.

Ví dụ 2.3.2 (DZY Loves Chinese II⁶). Cho đồ thị vô hướng $G = (V, E)$. Có q truy vấn, mỗi truy vấn cho một tập cạnh và hỏi sau khi xóa tập cạnh đó, đồ thị còn liên thông hay không. Bảo đảm kích thước tập cạnh không vượt quá 15.

Theo phép chuyển đổi trên, ta cần xác định liệu có tồn tại một tập con không rỗng của tập cạnh đã cho có xor trọng số bằng không hay không. Chỉ cần xây một cơ sở tuyến tính là kiểm tra được.

Chú ý rằng buộc kích thước tập cạnh không vượt quá 15 là quan trọng. Vì trọng số cạnh được đặt trong phạm vi 2^{64} , còn bài này phải xét mọi tập con để kiểm tra có tập con xor bằng không hay không, nên xác suất sai cao hơn nhiều so với kiểm tra một tập đơn có phải tập cắt. Vì vậy chỉ xử lý được phạm vi nhỏ. Nếu kích thước tập cạnh đã cho lớn hơn 64, dù thế nào ta cũng sẽ kết luận là không liên thông; điều đó rõ ràng vô lý.

Trọng số (vector) cạnh định nghĩa theo cách trên sẽ chia mọi cạnh thành vài lớp tương đương, mỗi lớp gồm các cạnh có trọng số bằng nhau. Ta gọi quan hệ tương đương này là tương đương theo lát cắt. Quan hệ này có vài dạng định nghĩa khác nhau.

Bổ đề 2.3.3. Trong đồ thị vô hướng G , với hai cạnh e_1, e_2 , ba mệnh đề sau tương đương:

1. e_1, e_2 tương đương theo lát cắt;
2. e_1, e_2 đều là cạnh cắt; hoặc e_1, e_2 nằm trong cùng một thành phần song liên thông cạnh, và sau khi xóa e_1, e_2 , thành phần song liên thông cạnh đó không còn liên thông;
3. với mọi chu trình, chu trình đó hoặc đồng thời chứa e_1, e_2 , hoặc đồng thời không chứa e_1, e_2 .

Chứng minh. Từ định nghĩa trọng số cạnh suy ra ngay mệnh đề 1 và mệnh đề 2 tương đương.

Nếu mệnh đề 2 đúng, giả sử có một chu trình chứa e_1 nhưng không chứa e_2 . Khi đó e_1, e_2 không phải cạnh cắt. Xóa e_2 xong, thành phần song liên thông cạnh vẫn liên thông, và lúc này vẫn còn một chu trình chứa e_1 , nên xóa tiếp e_1 cũng không ảnh hưởng tới tính liên thông. Điều này nói rằng sau khi xóa e_1, e_2 , thành phần song liên thông cạnh vẫn liên thông, mâu thuẫn. Do đó mệnh đề 3 đúng.

Nếu mệnh đề 3 đúng, giả sử sau khi xóa e_1, e_2 không phải cạnh cắt. Điều đó nghĩa là tồn tại một chu trình chứa e_2 nhưng không chứa e_1 , mâu thuẫn. Do đó mệnh đề 2 đúng. Vậy mệnh đề 2 và mệnh đề 3 tương đương. ■

Ví dụ 2.3.3 (Tours⁷). Cho đồ thị vô hướng có chu trình G . Cần tô màu mỗi cạnh bằng một trong k màu sao cho trên mỗi chu trình đơn, số cạnh của từng màu xuất hiện bằng nhau. Hỏi k lớn nhất là bao nhiêu.

Trực giác cho thấy, vì yêu cầu mọi chu trình đơn có màu phân bố đều, nên với mỗi lớp tương đương theo lát cắt trừ cạnh cắt, màu sắc trong lớp đó cũng phải phân bố đều. Rõ ràng điều này dẫn tới một phương án tô màu khả thi: lấy k là ước chung lớn nhất của kích thước mọi lớp tương đương không phải cạnh cắt, rồi tô màu đều trong từng lớp. Theo mệnh đề 3 của bổ đề 2.3.3, ta biết trên mỗi chu trình, màu sắc thật sự được phân bố đều.

Tiếp theo cần chứng minh k như vậy là lớn nhất, tức chứng minh mỗi lớp tương đương đều phải được tô đều. Dựng một cây DFS. Giả sử có t cạnh không thuộc cây, và C_i là chu trình đơn chỉ chứa cạnh không thuộc cây thứ i . Ta chỉ cần chứng minh mỗi tập $D_1 \cap D_2 \cap \dots \cap D_t$, trong đó mỗi D_i là C_i hoặc phần bù của C_i , nếu giao này không rỗng thì là một lớp tương đương theo lát cắt, và đều có thể biểu diễn thành tổ hợp tuyến tính hệ số thực của các phần tử $\{\oplus_{i \in S} C_i\}$ trong không gian chu trình. Khi có kết luận này, ta có thể chứng minh riêng cho từng màu rằng số lượng là đúng.

⁶BZOJ 3569

⁷ICPC 2015 World Final

Trước hết chứng minh giao của vài C_i , tức $\bigcap_{i \in S} C_i$, có thể biểu diễn bằng tổ hợp tuyến tính của các $\bigoplus_{i \in S} C_i$. Thật vậy:

$$\bigcap_{i \in S} C_i = \frac{1}{2^{|S|-1}} \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|} \bigoplus_{i \in T} C_i.$$

Kết luận này có thể suy ra từ tính chất của FWT. Vì bài viết không giới thiệu riêng FWT, và kết luận này không liên quan tới chủ đề chính, ta không chứng minh ở đây.

Tiếp theo, dùng nguyên lý bao hàm–loại trừ để chuyển từ giao của vài C_i sang giao của các D_i : nếu một D_i là phần bù của C_i , viết nó thành toàn tập trừ C_i , từ đó chuyển thành tổ hợp tuyến tính của 2^s giao của các tập C_i , trong đó s là số i sao cho D_i là phần bù của C_i . Mệnh đề được chứng minh.

Do đó đáp án chính là ước chung lớn nhất của kích thước mọi lớp tương đương theo lát cắt không phải cạnh cắt. Dùng phương pháp ngẫu nhiên ở trên, độ phức tạp thời gian dễ dàng đạt $O((n+m) \log n)$.

3.3 Đồ thị có hướng

Trong đồ thị có hướng, quan hệ liên thông hai chiều không nhất thiết tồn tại. Nếu trong đồ thị có hướng tồn tại đường đi $x \rightarrow y$, ta nói x tới được y .

Biến mọi cạnh có hướng trong một đồ thị có hướng thành cạnh vô hướng, ta thu được đồ thị vô hướng gọi là đồ thị nền của đồ thị có hướng ban đầu. Nếu đồ thị nền liên thông, ta nói đồ thị có hướng ban đầu liên thông yếu.

Với đồ thị có hướng không chu trình, định nghĩa thứ tự tô pô của nó: thứ tự tô pô là một hoán vị các đỉnh sao cho nếu x tới được y , thì x phải đứng trước y trong hoán vị.

3.3.1 Bài toán khả đạt

Bài toán khả đạt có thể chia thành hai loại tùy theo yêu cầu là quan hệ khả đạt giữa một cặp/một số cặp đỉnh cho trước, hay quan hệ khả đạt giữa mọi cặp đỉnh. Loại sau gọi là bài toán bao đóng bắc cầu.

Bao đóng bắc cầu tĩnh. Trong toán rời rạc, bao đóng của một quan hệ hai ngôi R là quan hệ nhỏ nhất R' sao cho $R \subseteq R'$ và R' thỏa một tính chất nào đó. Bao đóng bắc cầu là R' nhỏ nhất thỏa tính bắc cầu. Rõ ràng với đồ thị có hướng G , trước hết bổ sung quan hệ cạnh R thành phản xạ (tức bổ sung khuyên), rồi bao đóng bắc cầu của nó chính là quan hệ khả đạt R' .

Thuật toán Floyd là thuật toán bao đóng bắc cầu thường dùng nhất. Đặt quan hệ R'_k là quan hệ khả đạt khi chỉ được đi qua các đỉnh đánh số $1, 2, \dots, k$. Bằng cách liệt kê các đường đi qua k , ta có được công thức truy hồi từ R'_{k-1} tới R'_k . Vì Floyd rất quen thuộc trong OI, phần này không trình bày chi tiết. Độ phức tạp thời gian của Floyd là $O(n^3)$.

Trong đồ thị thưa, Floyd rất kém hiệu quả. Thực ra ta có thể trực tiếp liệt kê đỉnh cuối y , rồi tìm kiếm từ y trên đồ thị đảo để thu được mọi đỉnh x có thể tới y . Làm như vậy cho mọi y sẽ thu được bao đóng bắc cầu với độ phức tạp $O(nm)$.

Thuật toán trên có thể tối ưu bằng bitset, nhưng trước đó cần biến đồ thị thành không chu trình. Dùng nội dung sẽ giới thiệu ở mục 3.2, ta có thể co mỗi thành phần liên thông mạnh thành một đỉnh; khi đó đồ thị gốc trở thành một đồ thị có hướng không chu trình G' .

Đặt $f(x, y)$ biểu thị x có tới được y hay không. Xem $f(x, *)$ như một vector 01 và duy trì bằng bitset. Theo thứ tự tô pô đảo của G' , liệt kê từng đỉnh u . Khi đó $f(u, *)$ được lấy bằng phép or theo bit của mọi $f(v, *)$ với v là hậu duệ trực tiếp của u ; tất nhiên còn thêm khả đạt tới chính nó, $f(u, u) = 1$. Phép toán bit trên vector 01 độ dài n bằng bitset có độ phức tạp $O(n/w)$, nên tổng độ phức tạp là $O(nm/w)$.

Ví dụ về bài toán khả đạt động. Nói chung, bao đóng bắc cầu động là một bài toán tương đối khó; phần tài liệu tham khảo của bài viết có liệt kê các nghiên cứu liên quan. Loại thường gặp hơn là bài toán khả đạt động giữa một số cặp đỉnh cho trước. Với loại bài này, ta thường tiếp tục dùng phương pháp bitset để tính bao đóng bắc cầu, đồng thời kết hợp các kỹ thuật cấu trúc dữ liệu như chia khối truy vấn (hoặc chia khối thao tác).

Ví dụ 3.1.1 (Range Reachability Query⁸). Cho đồ thị có hướng không chu trình $G = (V, E)$, các cạnh có chỉ số. Có q truy vấn, mỗi truy vấn cho $u, v \in V$ và l, r , hỏi nếu chỉ giữ lại các cạnh có chỉ số trong $[l, r]$, thì u có tới được v hay không.

Vì chỉ có q truy vấn, ta có thể mô phỏng phương pháp bitset ở trên, nhưng đổi chiều thứ hai của bitset từ đỉnh thành truy vấn. Cụ thể, đặt $f(x, i)$ là: khi chỉ giữ các cạnh có chỉ số trong $[l_i, r_i]$, đỉnh x có tới được v_i hay không, trong đó l_i, r_i, v_i là các tham số tương ứng của truy vấn thứ i . Khi đó đáp án truy vấn thứ i là $f(u_i, i)$.

Xét cạnh $x \rightarrow y$ có chỉ số c . Chuyển trạng thái tương ứng là: với mọi i thỏa $l_i \leq c \leq r_i$, đặt $f(x, i) \leftarrow f(x, i) \vee f(y, i)$. Theo ngôn ngữ bitset, nếu S_c là tập mọi i thỏa $l_i \leq c \leq r_i$, được biểu diễn bằng vector 01 độ dài q , thì

$$f(x) \leftarrow f(x) \vee (f(y) \wedge S_c).$$

Dùng quét đường thẳng có thể tính mọi S_c , rồi chuyển trạng thái theo thứ tự tô pô đảo. Độ phức tạp thời gian là $O(q(m+n)/w)$, nhưng độ phức tạp bộ nhớ cũng là $O(q(m+n)/w)$, hơi lớn.

Loại bài này có một cách tối ưu bộ nhớ kinh điển: chia khối theo chiều được lưu bằng bitset. Trong bài này, chia khối truy vấn, mỗi khối gồm B truy vấn, và chạy thuật toán trên riêng cho từng khối. Như vậy bộ nhớ được tối ưu thành $O(B(m+n)/w)$, còn tổng thời gian vẫn là $O(q(m+n)/w)$. Tất nhiên độ dài khối B không được quá nhỏ, nếu không hệ số $1/w$ trong thời gian sẽ mất tác dụng.

Ví dụ 3.1.2 (Dynamic Reachability⁹). Cho đồ thị có hướng $G = (V, E)$, mỗi cạnh có màu đen hoặc trắng; ban đầu mọi cạnh đều màu đen. Sau đó có q thao tác, mỗi thao tác đảo màu một cạnh, hoặc cho $u, v \in V$ và hỏi u có thể tới v chỉ bằng cạnh đen hay không.

Vì màu cạnh thay đổi, bao đóng bắc cầu tĩnh trước đó khó áp dụng. Ta xét chia khối truy vấn; trong cùng một khối có nhiều cạnh không đổi màu, thuận lợi cho việc tiếp tục dùng phương pháp bitset.

Gọi độ dài khối là B . Trong một khối, tổng số đỉnh xuất hiện trong mọi sửa đổi và truy vấn nhiều nhất là $2B$; gọi các đỉnh này là đỉnh then chốt, lần lượt là $u_1, \dots, u_k$. Trước hết cần tính ảnh hưởng của các cạnh đen không bao giờ đổi màu trong khối. Gọi $f(i, j)$ biểu thị chỉ xét các cạnh đen không bao giờ đổi màu trong khối, i có tới được j hay không. Chú ý ta thực ra chỉ cần thông tin khi i, j đều là đỉnh then chốt, nên chiều thứ hai của bitset chỉ có kích thước $k = O(B)$ (vì chiều thứ hai chỉ cần ghi $u_1, \dots, u_k$). Độ phức tạp tiền xử lý phần này là $O(B(m+n)/w)$.

Lúc này ta nhận được một đồ thị có hướng G' chỉ chứa k đỉnh then chốt, trong đó có cạnh từ u_i tới u_j khi và chỉ khi $f(u_i, u_j) = 1$. Như vậy quy mô bài toán đã thu nhỏ; mọi ảnh hưởng của các đỉnh và cạnh không then chốt đều được thể hiện trong G' .

Xét cách trả lời truy vấn. Giả sử cần hỏi khả đạt từ u_i tới u_j . Ngoài các cạnh của G' , còn phải xét các cạnh đen mới xuất hiện do sửa đổi trong khối; số cạnh như vậy là $O(B)$. Ta có thể trực tiếp tìm kiếm thô để tìm tập đỉnh then chốt mà u_i tới được. Lấy BFS làm ví dụ: mỗi lần lấy một phần tử khỏi hàng đợi, cần đưa những đỉnh mới có thể thăm từ đỉnh đó vào hàng đợi. Quá trình này rõ ràng có thể tối ưu bằng bitset: chỉ cần duy trì tập mọi đỉnh hiện ở trong hàng đợi hoặc đã ra khỏi hàng đợi, ta nhanh chóng tìm được các đỉnh nên đưa vào hàng đợi. Độ phức tạp là $O(B^2/w)$.

⁸Huấn luyện đa trường HDU năm 2022.

⁹ZJCPC 2022, <https://codeforces.com/gym/103687>.

Tổng độ phức tạp thời gian là $O(q(n+m)/w + qB^2/w)$. Giống bài trước, độ dài khối không thể quá nhỏ, nếu không hệ số $1/w$ của bitset mất tác dụng. Vì vậy có thể lấy $B = w/2 = 32$, khi đó một số nguyên 64 bit đủ mô tả vector 01 có nhiều nhất $2B$ chiều. Trong bài này, lấy B lớn hơn một chút có vẻ chạy nhanh hơn.

Ví dụ 3.1.3 (Đồ thị có hướng không chu trình¹⁰). Cho đồ thị có hướng không chu trình $G = (V, E)$, đỉnh có trọng số, ban đầu mọi trọng số đỉnh bằng 0. Có q thao tác, gồm ba loại:

1. cho u, x , gán trọng số của mọi đỉnh mà u tới được thành x ;
2. cho u, x , lấy min với x trên trọng số của mọi đỉnh mà u tới được;
3. cho u , hỏi trọng số của u .

Bài này thực ra không phải bài toán khả đạt động, nhưng cách xử lý có nét tương tự.

Trước hết tính bao đóng bắc cầu của G với độ phức tạp $O(nm/w)$.

Truy vấn trọng số của u có thể tách thành hai bài toán con: trước hết tìm thao tác gán cuối cùng trước đó ảnh hưởng tới u , sau đó lấy giá trị nhỏ nhất trong các thao tác lấy min ảnh hưởng tới u từ thao tác đó tới thời điểm hiện tại.

Xét bài toán thứ nhất. Chia khối truy vấn với độ dài B . Mỗi khi xử lý xong một khối thao tác, dùng $O(n+m)$ để cập nhật thao tác gán cuối cùng của mỗi đỉnh. Khi truy vấn, ta đã biết đáp án trước khi khối hiện tại bắt đầu; chỉ cần kiểm tra trong khối hiện tại có thao tác gán nào liên quan tới u hay không. Việc này có thể làm bằng cách liệt kê trực tiếp mọi thao tác gán trong khối hiện tại, kiểm tra đỉnh u_i của thao tác đó có tới được u hay không.

Bài toán thứ hai được xử lý gần như giống vậy. Với mỗi u và mỗi khối, duy trì giá trị nhỏ nhất trong mọi thao tác lấy min của khối đó ảnh hưởng tới u . Truy vấn có dạng truy vấn đoạn theo u : với các khối nguyên dùng kết quả tiền xử lý, còn các phần lẻ liệt kê thô từng thao tác.

Độ phức tạp thời gian là $O(nm/w + qB + q(n+m)/B)$. Lấy $B = O(\sqrt{n+m})$, độ phức tạp là $O(nm/w + q\sqrt{n+m})$. Chú ý ở đây ta gặp vấn đề bộ nhớ giống ví dụ 3.1.1, có thể tối ưu tương tự: với mỗi khối thao tác, chiều lưu bằng bitset chỉ giữ những đỉnh được dùng trong các sửa đổi hoặc truy vấn của khối.

3.3.2 Liên thông mạnh

Định nghĩa 3.1 (liên thông mạnh). Trong đồ thị có hướng G , nếu hai đỉnh u, v thỏa u tới được v và v tới được u , ta nói u và v liên thông mạnh. Nếu mọi cặp đỉnh trong G đều liên thông mạnh, ta nói G là đồ thị liên thông mạnh.

Vì quan hệ khả đạt có tính bắc cầu, quan hệ liên thông mạnh cũng có tính bắc cầu, do đó là một quan hệ tương đương. Trong đồ thị có hướng G , ta có thể phân hoạch mọi đỉnh theo quan hệ liên thông mạnh.

Định nghĩa 3.2 (thành phần liên thông mạnh). Các đỉnh của đồ thị có hướng G có thể được phân hoạch thành vài lớp tương đương theo quan hệ liên thông mạnh. Đồ thị con cảm sinh bởi mỗi lớp tương đương được gọi là một thành phần liên thông mạnh của G .

Cây DFS và thuật toán Tarjan. Trong đồ thị có hướng G , nếu đỉnh r tới được mọi đỉnh, ta có thể bắt đầu DFS từ r . Khi đó, tương tự đồ thị vô hướng, ta định nghĩa cây DFS của G .

¹⁰Nguồn bài không truy vết được.

Định nghĩa 3.3 (thứ tự DFS, cây DFS). Trong đồ thị có hướng $G = (V, E)$, bắt đầu tìm kiếm theo chiều sâu từ một đỉnh $r \in V$ tới được mọi đỉnh. Nếu đỉnh x là đỉnh thứ p được thăm, gọi p là thứ tự DFS của x , ký hiệu $\text{dfn}_x = p$. Nếu $x \neq r$, nối một cạnh giữa x và đỉnh y lần đầu thăm được x ; đồ thị tạo thành được gọi là một cây DFS gốc r .

Cây DFS là một cây hướng ra ngoài. Trên cây DFS của đồ thị có hướng có vài tính chất tương tự đồ thị vô hướng.

Tính chất 3.2.1. Thứ tự DFS của mọi đỉnh tạo thành một hoán vị của $1, \dots, n$. Với mọi đỉnh x , tập thứ tự DFS của tất cả đỉnh trong cây con gốc x trên cây DFS tạo thành một đoạn liên tiếp bắt đầu từ dfn_x .

Tính chất 3.2.2. Cạnh không thuộc cây hoặc là cạnh từ hậu duệ tới tổ tiên (cạnh ngược), hoặc là cạnh từ tổ tiên tới hậu duệ (cạnh xuôi), hoặc là cạnh có hai đầu không có quan hệ tổ tiên và đi từ đỉnh có thứ tự DFS lớn tới đỉnh có thứ tự DFS nhỏ (cạnh chéo).

Chứng minh đơn giản, nhưng tính chất 3.2.2 cho thấy cấu trúc cạnh không thuộc cây trong đồ thị có hướng phức tạp hơn đồ thị vô hướng.

Tiếp theo xét cách dùng cây DFS để tìm thành phần liên thông mạnh. Quá trình tương tự tìm thành phần song liên thông: trong quá trình DFS vẫn ghi low_x , là thứ tự DFS nhỏ nhất của đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x bằng cách đi qua nhiều nhất một cạnh không thuộc cây. Khi tìm được đỉnh x thỏa $\text{low}_x = \text{dfn}_x$, ta thu được một thành phần liên thông mạnh có x là đỉnh nông nhất.

Nhưng cách làm trên chưa hoàn toàn đúng, vì tồn tại cạnh chéo. Nếu trong cây con của x có một cạnh chéo trở tới một thành phần liên thông mạnh đã được thống kê trước đó, thì cạnh chéo này thực tế là vô hiệu (vì không giúp x đi tới vị trí nông hơn trên cây DFS). Ta cần bỏ qua các cạnh chéo như vậy, nên cải tiến thuật toán như sau:

Trong quá trình DFS, duy trì một ngăn xếp gồm các đỉnh đã thăm nhưng chưa được phân vào bất kỳ thành phần liên thông mạnh nào. Vẫn tính low_x trong DFS, nhưng chỉ khi đầu cuối của cạnh không thuộc cây còn nằm trong ngăn xếp (tức chưa bị đưa vào thành phần liên thông mạnh nào) thì cạnh đó mới được tính vào low_x . Khi gặp đỉnh $\text{low}_x = \text{dfn}_x$, lấy phần trong cây con của x đang nằm trên đỉnh ngăn xếp (chắc chắn là một đoạn liên tiếp ở đỉnh ngăn xếp) ra, cùng với x tạo thành một thành phần liên thông mạnh.

Chú ý rằng nếu ta chọn tùy ý một điểm xuất phát thì không nhất thiết DFS tới mọi đỉnh. Điều này không ảnh hưởng: nếu còn đỉnh chưa thăm, chọn tùy ý một đỉnh chưa thăm để DFS tiếp, lặp lại cho tới khi mọi đỉnh được thăm.

Đó là thuật toán Tarjan tìm thành phần liên thông mạnh, độ phức tạp thời gian $O(n + m)$. Ta không chứng minh chi tiết tính đúng đắn, nhưng sau khi hiểu thuật toán Tarjan trên đồ thị vô hướng và các tính chất của cây DFS trong đồ thị có hướng, mọi thứ trở nên tự nhiên.

Sau khi tìm tất cả thành phần liên thông mạnh, xem mỗi thành phần như một đỉnh. Cạnh trong đồ thị gốc nếu nằm hoàn toàn trong một thành phần thì bỏ qua, ngược lại xem là cạnh nối hai thành phần. Đồ thị có hướng mới thu được là một đồ thị có hướng không chu trình; quá trình này gọi là co đỉnh.

Hình 4: Một đồ thị có hướng, các thành phần liên thông mạnh khác nhau của nó, và kết quả co đỉnh.

Nếu bản thân G đã liên thông mạnh, thì nó chỉ có một thành phần liên thông mạnh là chính nó. Theo định nghĩa, bắt đầu DFS từ bất kỳ đỉnh nào cũng thăm được mọi đỉnh. Hơn nữa, theo thuật toán Tarjan, trên cây DFS, trong cây con của mọi đỉnh x đều phải tồn tại một cạnh trở tới một đỉnh có thứ tự DFS nhỏ hơn x , dù đó là cạnh ngược hay cạnh chéo.

Ví dụ 3.2.1 (Indiana Jones and the Uniform Cave¹¹). Đây là một bài tương tác. Có một đồ thị có hướng liên thông mạnh G , mỗi đỉnh có bậc ra bằng k . Tại mỗi đỉnh có một viên đá chỉ vào một cạnh ra, đồng thời có một nhãn thuộc $\{0, 1, 2\}$. Ban đầu ở mỗi đỉnh, viên đá chỉ ngẫu nhiên vào một cạnh ra và nhãn bằng 0.

Bạn không biết cấu trúc của G , nhưng cần xuất phát từ một đỉnh nào đó và duyệt qua mọi cạnh của G . Mỗi khi đi theo một cạnh tới một đỉnh, bạn biết nhãn của đỉnh đó, rồi có thể chọn sửa nhãn và hướng chỉ của viên đá, cuối cùng chọn một cạnh ra để rời đi. Tuy nhiên nhãn sau khi sửa chỉ có thể là 1 hoặc 2. Mọi cạnh ra của một đỉnh có thể xem như phân bố đều trên một đường tròn, nên khi mô tả một cạnh ra, bạn chỉ có thể dùng dạng “đếm thuận chiều kim đồng hồ từ cạnh đang được viên đá chỉ tới, cạnh thứ i ”.

Bạn chỉ được phép đi qua $O(nm)$ cạnh; chú ý ở mọi thời điểm bạn đều biết mọi quyết định trước đó của mình.

Ta thử mô phỏng quá trình DFS. Gọi quá trình tìm kiếm cây con của x là $dfs(x)$; ta kỳ vọng khi $dfs(x)$ được phép kết thúc, mọi cạnh ra của x đều đã được thăm.

Trong ba nhãn, 0 biểu thị tự nhiên cho đỉnh chưa thăm; quy định 1 và 2 lần lượt biểu thị đỉnh tương ứng là/không là tổ tiên của x , trong đó x là đỉnh đang thực hiện quá trình dfs hiện tại.

Với quá trình $dfs(x)$, lần lượt liệt kê mọi cạnh ra $x \rightarrow y$ và xét nhãn của y :

- Nhãn của y là 0. Điều này tương ứng y là con của x trên cây DFS. Ta đệ quy thực hiện $dfs(y)$, rồi quay lui.
- Nhãn của y là 1. Điều này nói $x \rightarrow y$ là cạnh ngược. Sau khi đi qua cạnh này, ta muốn quay về x . Để không lạc đường, quy định với đỉnh có nhãn 1, viên đá của nó chỉ vào cạnh nằm trên đường đi từ đỉnh đó tới x trong cây DFS. Như vậy chỉ cần đi theo cạnh được viên đá chỉ là có thể về x . Vấn đề là làm sao biết mình đã về tới x ? Câu trả lời là tạm thời sửa nhãn của x thành 2, khi lần sau thăm một đỉnh nhãn 2 thì chắc chắn đã về tới x .
- Nhãn của y là 2. Điều này nói $x \rightarrow y$ là cạnh xuôi hoặc cạnh chéo. Dù thế nào, ta vẫn muốn quay về x , nhưng lần này không tiện như trường hợp trước. Ta cần đi ngược hướng cây DFS cho tới khi tới một đỉnh nhãn 1, rồi quay về x theo cách ở trên (cách nhận biết đã về tới x hơi khác nhưng cũng đơn giản, nên không nói thêm). Vấn đề là làm sao đi tới một đỉnh nhãn 1. Nhắc lại mảng low trong thuật toán Tarjan: nếu từ đỉnh hiện tại u ta liên tục đi tới low_u , thì do đồ thị liên thông mạnh, chắc chắn có thể quay về đỉnh gốc, tức chắc chắn đi qua đỉnh nhãn 1. Vì vậy với mỗi đỉnh nhãn 2 là u , định nghĩa hướng viên đá của nó là hướng tới low_u . Nếu cạnh tương ứng với low_u là $u \rightarrow low_u$, viên đá chỉ vào cạnh đó; nếu cạnh tương ứng là $v \rightarrow low_u$ với v là hậu duệ của u , viên đá chỉ về hướng cây con chứa v .

Vấn đề cuối cùng: sau khi thực hiện xong $dfs(x)$, ta cần quay lui tới cha của x . Điều này thực ra giống trường hợp cuối cùng vừa nói; chỉ cần trước hết đi tới low_x , rồi đi xuống một đoạn là được.

Khi cài đặt có nhiều chi tiết, nhưng khung thuật toán đại thể đã rõ. Ta cần $O(nm)$ bước để duyệt mọi cạnh.

Phân rẽ tai. Trước hết nhắc lại phân rẽ tai đã định nghĩa ở phần đồ thị vô hướng và mở rộng nó sang đồ thị có hướng.

Định nghĩa 3.4 (tai và phân rẽ tai). Trong đồ thị có hướng $G = (V, E)$ có đồ thị con $G' = (V', E')$. Nếu đường đi đơn hoặc chu trình đơn $P : x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k$ thỏa $x_1, x_k \in V'$ và $x_2, \dots, x_{k-1} \notin V'$, thì P được gọi là một tai của G đối với G' .

¹¹NEERC 2016, <https://codeforces.com/gym/101190>.

Nếu một dãy đồ thị con $(G_0, G_1, \dots, G_k)$ của G thỏa:

1. G_0 là một chu trình đơn (có thể chỉ có một đỉnh), $G_k = G$;
2. G_{i-1} là đồ thị con của G_i ;
3. đặt $G_i = (V_i, E_i)$, khi đó $E_i \setminus E_{i-1}$ tạo thành một tai của G_{i-1} ,

thì $(G_0, G_1, \dots, G_k)$ được gọi là một phân rã tai của G .

Tương tự đồ thị vô hướng, ta có kết luận sau.

Định lý 3.1. Đồ thị có hướng G có phân rã tai khi và chỉ khi G liên thông mạnh.

Chứng minh. Trước hết chứng minh tính cần. Nếu G có phân rã tai $(G_0, \dots, G_k)$, chu trình đơn G_0 là liên thông mạnh. Nếu G_i liên thông mạnh, giả sử tai được thêm vào G_{i+1} là $x \rightarrow y$. Từ tính liên thông mạnh của G_i , y tới được x , nên tồn tại một chu trình chứa tai; do đó mọi đỉnh trên tai liên thông mạnh với x, y , và G_{i+1} cũng liên thông mạnh. Quy nạp suy ra $G = G_k$ liên thông mạnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ liên thông mạnh, ta đưa ra cách dựng một phân rã tai. Trước hết dựng một cây DFS gốc 1 của G , và tính mảng low theo thuật toán Tarjan. Tính liên thông mạnh của G cho biết khi $x \neq 1$ thì $\text{low}_x < \text{dfn}_x$. Sau đó xây $(G_0, \dots, G_k)$ bằng quá trình sau.

Cho G_0 là đồ thị chỉ chứa đỉnh 1. Sau đó liệt kê các đỉnh theo thứ tự DFS tăng dần. Giả sử đỉnh hiện tại là x có thứ tự DFS p ; khi đó mọi đỉnh có thứ tự DFS $1, \dots, p-1$ đã nằm trong G_i hiện tại. Nếu x đã ở trong G_i thì bỏ qua. Ngược lại, theo $\text{low}_x < \text{dfn}_x$, tồn tại một hậu duệ y của x và một cạnh $y \rightarrow z$ sao cho z đã nằm trong G_i . Khi đó lấy đường đi $f_x \rightarrow x \rightarrow y \rightarrow z$ làm một tai thêm vào G_i , trong đó f_x là cha của x ; thu được G_{i+1} . Sau khi liệt kê mọi đỉnh, mọi đỉnh đều đã nằm trong G_i hiện tại; lúc này thêm từng cạnh chưa được thêm vào như một tai riêng lẻ. ■

Ví dụ 3.2.2 (Economic One-way Roads¹²). Cho một đồ thị vô hướng G . Cần gán hướng cho mỗi cạnh; mỗi cạnh có chi phí riêng cho từng hướng. Hãy định hướng sao cho đồ thị có hướng thu được liên thông mạnh và tổng chi phí theo hướng được chọn nhỏ nhất.

Để đối chiếu, ví dụ này thực ra là phiên bản liên thông mạnh của ví dụ 2.2.4.

Gọi $f(S)$ là đáp án ứng với đồ thị con cảm sinh bởi S . Mỗi lần chuyển trạng thái bằng cách thêm một tai vào S . Nếu trực tiếp liệt kê tập đỉnh của tai, độ phức tạp là $O(3^n \text{Poly}(n))$; nếu ngay trong chuyển trạng thái thêm từng đỉnh theo thứ tự trên tai, có thể đạt $O(2^n \text{Poly}(n))$. Vì quá trình cụ thể gần như giống ví dụ 2.2.4, không nhắc lại chi tiết.

Định lý 3.2. Đồ thị vô hướng G có thể được định hướng thành đồ thị liên thông mạnh khi và chỉ khi G song liên thông cạnh.

Chứng minh. Tính cần: nếu G không song liên thông cạnh, thì G không liên thông hoặc có một cạnh cắt. Đồ thị không liên thông hiển nhiên không thể định hướng thành liên thông mạnh; nếu G có cạnh cắt (x, y) , dù định hướng cạnh này thế nào thì hoặc x không tới được y , hoặc y không tới được x , nên không thể có định hướng liên thông mạnh.

Tính đủ: nếu G song liên thông cạnh, nó có phân rã tai. Định hướng G_0 thành chu trình có hướng, rồi định hướng mỗi tai sau đó thành đường đi có hướng hoặc chu trình có hướng. Ta thu được một phân rã tai có hướng; theo định lý 3.1, đồ thị có hướng tương ứng là liên thông mạnh. ■

Thật ra việc dựng định hướng không cần phải tìm phân rã tai phức tạp như vậy. Ta chỉ cần dựng cây DFS của đồ thị song liên thông cạnh, định hướng mọi cạnh cây từ cha tới con, và định hướng mọi cạnh không thuộc cây từ hậu duệ tới tổ tiên. Điều này về cơ bản giống với cách dựng bằng phân rã tai ở trên.

¹²XXI Open Cup, Grand Prix of Korea, <https://codeforces.ml/gym/102759>.

Ví dụ 3.2.3 (trình quản lý App¹³). Một hỗn hợp đồ thị G (có thể đồng thời chứa cạnh có hướng và cạnh vô hướng) có thể được định hướng các cạnh vô hướng thành đồ thị liên thông mạnh khi và chỉ khi hai điều kiện sau cùng đúng:

1. nếu xem mọi cạnh có hướng như cạnh vô hướng, đồ thị song liên thông cạnh;
2. nếu tách mỗi cạnh vô hướng thành một cặp cạnh có hướng ngược nhau, đồ thị liên thông mạnh.

Chứng minh. Tính cần hiển nhiên. Dưới đây chứng minh tính đủ.

Nếu điều kiện 1 và 2 đều đúng, và không có cạnh vô hướng nào, ta không cần định hướng. Nếu còn cạnh vô hướng, lấy tùy ý một cạnh vô hướng (u, v) và chứng minh nó có một cách định hướng sao cho sau khi định hướng, điều kiện 1 và 2 vẫn đúng (thực ra chỉ cần xét điều kiện 2, vì điều kiện 1 không phụ thuộc hướng). Như vậy có thể chứng minh kết luận bằng quy nạp.

Gọi đồ thị hiện tại là G_0 , và đồ thị thu được từ G_0 sau khi xóa cạnh vô hướng (u, v) là G'_0 . Với một đỉnh bất kỳ x , do G_0 liên thông mạnh nên tồn tại một đường đi đơn $x \rightarrow u$. Nếu đường đi này không đi qua cạnh vô hướng (u, v) , thì trong G'_0 , x tới được u ; ngược lại trong G'_0 , x tới được v . Tương tự, trong G'_0 , hoặc u tới được x , hoặc v tới được x .

Trong G'_0 , nếu tồn tại x sao cho u tới được x và x tới được v , thì u tới được v . Tương tự, nếu tồn tại x sao cho v tới được x và x tới được u , thì v tới được u . Nếu không, với mỗi x đều là u, x tới được nhau hoặc v, x tới được nhau. Vì vậy toàn đồ thị có nhiều nhất hai thành phần liên thông mạnh. Giả sử u, v không tới được nhau, thì giữa hai thành phần liên thông mạnh này không có cạnh, nghĩa là đồ thị nền của G'_0 không liên thông, tức (u, v) là cạnh cắt trong G_0 , mâu thuẫn với điều kiện 1. Do đó ít nhất một trong hai quan hệ u tới được v , hoặc v tới được u , là đúng.

Nếu u tới được v , định hướng (u, v) thành $v \rightarrow u$; ngược lại định hướng thành $u \rightarrow v$. Điều này làm cho u, v tới được nhau, do đó toàn đồ thị vẫn liên thông mạnh, đúng như cần chứng minh. ■

Chu trình có hướng. Trong đồ thị vô hướng có không gian chu trình; mục này nghiên cứu cấu trúc chu trình của đồ thị có hướng.

Định nghĩa 3.5 (đồ thị phủ được bằng chu trình). Nếu trong đồ thị có hướng G , bậc vào của mỗi đỉnh bằng bậc ra của nó, ta nói G phủ được bằng chu trình.

Bổ đề 3.2.1. Tập cạnh của một đồ thị có hướng phủ được bằng chu trình có thể biểu diễn thành hợp rời của một số chu trình đơn.

Chứng minh xét bằng quy nạp. Khái niệm đồ thị phủ được bằng chu trình có thể so sánh với các phần tử của không gian chu trình trong đồ thị vô hướng.

Tương tự không gian chu trình, ta muốn dùng cây DFS để thu được một bộ “cơ sở” của các đồ thị con phủ được bằng chu trình; tuy nhiên nó không phải là cơ sở tuyến tính thật sự, nên không phải lúc nào cũng dùng được. Nói chung, ta xét trường hợp cạnh có trọng số (nếu không có quy định đặc biệt về trọng số, xem mỗi cạnh có trọng số 1), và nghiên cứu tính chất của tổng trọng số chu trình trong các cấu trúc như không gian xor hoặc các lớp dư modulo m .

Bổ đề 3.2.2. Trong đồ thị liên thông mạnh G , nếu tồn tại một đường đi $P : x \rightarrow y$ có tổng trọng số S , thì tồn tại một đường đi $y \rightarrow x$ có tổng trọng số S' thỏa $S' \equiv -S \pmod{m}$.

Chứng minh. Nếu $m = 1$ thì kết luận hiển nhiên. Ngược lại, lấy tùy ý một đường đi $Q : y \rightarrow x$ có tổng trọng số T . Xét đường đi ghép $Q + P + Q + P + \dots + Q$, gồm $m - 1$ bản sao của P và m bản

¹³UOJ Round 9, <https://uoj.ac/contest/18>.

sao của Q ; dấu cộng biểu thị ghép đường đi. Đây là một đường đi từ y tới x , và tổng trọng số của nó là $(m-1)S + mT \equiv -S \pmod{m}$. ■

Bổ đề 3.2.3. Trong đồ thị liên thông mạnh G , với mỗi cạnh $x \rightarrow y$ có trọng số z , thêm một cạnh mới $y \rightarrow x$ có trọng số $-z$. Ta thu được một đồ thị mới G' có trọng số cạnh phản đối xứng. Tập tổng trọng số của mọi đồ thị con phủ được bằng chu trình của G và G' là tương đương theo modulo m .

Chứng minh. Đây là hệ quả của bổ đề 3.2.2, vì mỗi chu trình của G' có thể tương ứng với một chu trình của G bằng bổ đề 3.2.2. ■

Vì vậy ta chỉ cần nghiên cứu đồ thị có trọng số phản đối xứng như vậy. Cây DFS của đồ thị này có thể xem như không còn một chiều: ta có thể đi hai chiều trên cây. Qua tính toán đơn giản, tổng trọng số của đường đi trên cây (không nhất thiết đơn) từ x tới y chắc chắn là $\text{dep}_y - \text{dep}_x$, trong đó dep_x là tổng trọng số trên đường đi từ gốc cây DFS tới x .

Bổ đề 3.2.4. Trên đồ thị G có trọng số phản đối xứng, gọi một chu trình đơn chỉ chứa một cạnh không thuộc cây (ở đây cạnh ngược chiều của cạnh cây không tính là cạnh không thuộc cây) là chu trình cơ bản. Khi đó tổng trọng số của mọi đồ thị con phủ được bằng chu trình của G bằng một tổ hợp tuyến tính hệ số nguyên của tổng trọng số một số chu trình cơ bản.

Chứng minh. Giả sử một chu trình lần lượt đi qua các cạnh không thuộc cây $e_1, \dots, e_k$, trong đó e_i có dạng $x_i \rightarrow y_i$ và trọng số z_i . Tổng trọng số chu trình là

$$\sum_{i=1}^k (z_i + \text{dep}_{y_{i+1}} - \text{dep}_{x_i}),$$

với $y_{k+1} = y_1$. Đổi chỗ các hạng chứa y_i trong tổng, ta được

$$\sum_{i=1}^k (z_i + \text{dep}_{y_i} - \text{dep}_{x_i}),$$

mà mỗi hạng đúng là tổng trọng số của một chu trình cơ bản. Kết hợp với tính chất đồ thị con phủ được bằng chu trình có thể tách thành một số chu trình, bổ đề được chứng minh. ■

Định lý 3.3. Trong đồ thị liên thông mạnh G , ước chung lớn nhất của tổng trọng số mọi đồ thị con phủ được bằng chu trình bằng ước chung lớn nhất của mọi $z + \text{dep}_y - \text{dep}_x$, trong đó (x, y, z) chạy qua bộ ba đỉnh đầu, đỉnh cuối và trọng số của mọi cạnh không thuộc cây, còn dep_x là độ dài đường đi trên cây từ gốc tới x .

Chứng minh. Gọi ước chung lớn nhất cần tìm là g . Theo bổ đề 3.2.3 và 3.2.4, ta biết trong modulo g , tập tổng trọng số các đồ thị con phủ được bằng chu trình của G tương đương với tập mọi $z + \text{dep}_y - \text{dep}_x$. Vì vậy nếu mọi phần tử của một trong hai tập đều là bội của g , thì tập còn lại cũng vậy. Điều này chứng minh hai cách tính ước chung lớn nhất là nhất quán. ■

Ở trên ta chỉ thảo luận tính chất theo các lớp dư modulo m . Tuy nhiên chỉ cần bổ đề 3.2.2 vẫn đúng, các tính chất trên có thể mở rộng sang một số cấu trúc khác, chẳng hạn không gian xor, hoặc tổng quát hơn là cấu trúc cộng không nhớ trong hệ cơ số k . Dù vậy, khi xét tính chất modulo m , chỉ cần nắm ước chung lớn nhất là cơ bản có được toàn bộ thông tin cấu trúc; còn trong các cấu trúc khác ở trên có thể cần thêm công cụ như cơ sở tuyến tính.

Ví dụ 3.2.4 (chu kỳ của đồ thị có hướng). Trong đồ thị có hướng $G = (V, E)$, nếu tồn tại một cặp số nguyên p, k sao cho với mọi cặp đỉnh x, y và mọi $t \geq k$, tồn tại một đường đi $x \rightarrow y$ gồm t cạnh khi và chỉ khi tồn tại một đường đi $x \rightarrow y$ gồm $t + p$ cạnh; đồng thời theo khóa thứ nhất là p và khóa thứ hai là k ,

cặp số nguyên nói trên là nhỏ nhất. Khi đó p, k lần lượt được gọi là một chu kỳ và chỉ số lũy thừa hội tụ của G .

Một cách nói tương đương: gọi A là ma trận kề của G , thì với mọi $t \geq k$ có $A^t = A^{t+p}$, trong đó phép nhân ma trận được định nghĩa bằng phép nhân là and theo bit, phép cộng là or theo bit.

Xét chu kỳ của đồ thị liên thông mạnh. Thực ra nó chính là ước chung lớn nhất g của mọi độ dài chu trình trong định lý 3.3, vì khi t đủ lớn, ta có thể đi tùy ý trong đồ thị liên thông mạnh. Theo định lý Bezout, ta có thể ghép ra bội bất kỳ của g bằng tổ hợp tuyến tính hệ số nguyên của độ dài các chu trình. Khi số bước đủ lớn, mọi hệ số đều có thể lấy là số nguyên dương.

Với đồ thị tổng quát, chu kỳ chắc chắn phải là bội của chu kỳ của từng thành phần liên thông mạnh; lấy bội chung nhỏ nhất của chu kỳ mọi thành phần liên thông mạnh là được. Điều này khá dễ hiểu, nên lược bỏ chứng minh nghiêm ngặt.

Với chỉ số lũy thừa hội tụ, không có công thức trực tiếp để tính.

Ví dụ 3.2.5 (Phoenix and Odometers¹⁴). Cho đồ thị có hướng có trọng số cạnh G . Có q truy vấn, mỗi truy vấn cho v, b, m , hỏi có tồn tại một chu trình (không nhất thiết đơn) đi qua v có tổng trọng số S thỏa $S \equiv b \pmod{m}$ hay không.

Đây là một ví dụ của cấu trúc lớp dư modulo m . Rõ ràng chỉ cần xét thành phần liên thông mạnh chứa v . Theo định lý 3.3, tính ước chung lớn nhất g của tổng trọng số mọi chu trình trong đó, rồi kiểm tra $\gcd(g, m) \mid b$ là đủ.

Ví dụ 3.2.6 (Thuật thụ số¹⁵). Cho một đồ thị vô hướng có trọng số G và số nguyên k . Có q truy vấn, mỗi truy vấn cho x, y, v , hỏi có tồn tại một đường đi $x \rightarrow y$ sao cho kết quả cộng không nhớ trong hệ cơ số k của mọi trọng số cạnh bằng v hay không.

Đây là một ví dụ của phép cộng không nhớ trong hệ cơ số k . Khác với trước đó, đây là đồ thị vô hướng, nên cần tách mỗi cạnh vô hướng thành một cặp cạnh có hướng ngược nhau. Vì vậy khi thực hiện thao tác tương tự định lý 3.3, ngoài cạnh không thuộc cây còn phải xét cạnh ngược chiều của cạnh cây; tức với cạnh cây có trọng số w , $2w$ cũng là một phần tử trong cơ sở. Sau đó dùng tất cả tổng trọng số chu trình cơ bản này để dựng một cơ sở tuyến tính hệ cơ số k phục vụ truy vấn.

Một điểm khác trong bài này là mục tiêu không phải chu trình mà là đường đi $x \rightarrow y$. Ta chỉ cần chọn tùy ý một đường đi $x \rightarrow y$, chẳng hạn đường đi trên cây, rồi cộng thêm một tập con các phần tử cơ sở là có thể thu được mọi đường đi từ x tới y . Vì vậy khi truy vấn, lấy tổng trọng số đường đi cây $x \rightarrow y$ làm giá trị ban đầu, rồi kiểm tra trong cơ sở tuyến tính có tồn tại một số phần tử sao cho cộng với giá trị ban đầu thành v hay không.

3.3.3 Đồ thị giải đấu

Đồ thị giải đấu là đồ thị có hướng đơn sao cho với mỗi cặp đỉnh khác nhau u, v , đúng một trong hai cạnh $u \rightarrow v$ và $v \rightarrow u$ tồn tại. Bây giờ ta thảo luận ngắn gọn vài kết luận liên quan tới tính liên thông trong đồ thị giải đấu.

Đường Hamilton.

Định nghĩa 3.6 (đường Hamilton, chu trình Hamilton). Trong đồ thị G , đường đi đi qua mọi đỉnh đúng một lần gọi là đường Hamilton; chu trình đi qua mọi đỉnh đúng một lần (đỉnh đầu và đỉnh cuối tính

¹⁴CF 1515 G

¹⁵CTT 2020; phát biểu bài ở đây không hoàn toàn giống bản gốc.

là một lần) gọi là chu trình Hamilton.

Định lý 3.4. Mọi đồ thị giải đấu đều có đường Hamilton.

Chứng minh. Quy nạp theo số đỉnh. Nếu đồ thị giải đấu $n - 1$ đỉnh có đường Hamilton, thì trong đồ thị giải đấu n đỉnh, giả sử không mất tính tổng quát rằng đường Hamilton của đồ thị con cảm sinh bởi $n - 1$ đỉnh đầu là $1 \rightarrow 2 \rightarrow \dots \rightarrow n - 1$. Khi đó:

1. nếu có cạnh $n \rightarrow 1$, nối n vào đầu đường Hamilton;
2. nếu có cạnh $n - 1 \rightarrow n$, nối n vào cuối đường Hamilton;
3. nếu có cạnh $1 \rightarrow n$ và $n \rightarrow n - 1$, thì chắc chắn tồn tại $x \in [1, n - 2]$ sao cho $x \rightarrow n$ và $n \rightarrow x + 1$ (có thể dùng nhị phân để tìm x như vậy), rồi chèn n vào giữa x và $x + 1$.

Vậy đồ thị giải đấu n đỉnh cũng có đường Hamilton. Theo quy nạp, kết luận đúng. ■

Định lý 3.5. Đồ thị giải đấu liên thông mạnh có chu trình Hamilton.

Chứng minh. Theo định lý 3.4, trước hết dựng một đường Hamilton, giả sử là $1 \rightarrow 2 \rightarrow \dots \rightarrow n$. Tìm t lớn nhất sao cho có cạnh $t \rightarrow 1$. Khi đó với các đỉnh x sau t , đều có cạnh $1 \rightarrow x$, tức $1 \rightarrow t + 1, \dots, 1 \rightarrow n$. Hiện ta có một chu trình $1 \rightarrow 2 \rightarrow \dots \rightarrow t \rightarrow 1$; tiếp theo cố gắng thêm các đỉnh $t + 1, \dots, n$ vào chu trình.

Mô tả lại bài toán: có một chu trình chứa 1, và từ 1 có cạnh tới mọi đỉnh chưa nằm trên chu trình. Ta muốn chứng minh tồn tại một đỉnh hiện chưa nằm trên chu trình có thể được thêm vào để tạo chu trình lớn hơn. Thật vậy, do đồ thị liên thông mạnh, chắc chắn tồn tại một đỉnh x chưa nằm trên chu trình và một đỉnh y trên chu trình sao cho có cạnh $x \rightarrow y$; nếu không, các đỉnh ngoài chu trình không tới được các đỉnh trên chu trình. Kết hợp với $1 \rightarrow x$, ta tìm được một vị trí thích hợp z trên chu trình (nằm giữa 1 và y) sao cho $z \rightarrow x \rightarrow \text{next}_z$. Chèn x vào giữa z và next_z , trong đó next_z là đỉnh kế tiếp của z trên chu trình. ■

Thành phần liên thông mạnh. Sau khi co thành phần liên thông mạnh, đồ thị tổng quát trở thành đồ thị có hướng không chu trình. Với đồ thị giải đấu, ta có kết quả tốt hơn.

Bổ đề 3.3.1. Sau khi co thành phần liên thông mạnh, đồ thị giải đấu tạo thành một dây chuyền.

Chứng minh. Với hai đỉnh x, y thuộc hai thành phần liên thông mạnh khác nhau, do chắc chắn tồn tại $x \rightarrow y$ hoặc $y \rightarrow x$, nên x tới được y hoặc y tới được x . Vì vậy quan hệ khả đạt giữa các thành phần liên thông mạnh tạo thành một thứ tự toàn phần, tức sau khi co đỉnh sẽ tạo thành một dây chuyền. ■

Trong đồ thị giải đấu, để tìm thành phần liên thông mạnh không cần dùng thuật toán Tarjan; có nhiều cách đơn giản hơn.

Trước hết có thể tìm một đường Hamilton của đồ thị giải đấu, giả sử là $1 \rightarrow 2 \rightarrow \dots \rightarrow n$. Khi đó mỗi thành phần liên thông mạnh chắc chắn là một đoạn liên tiếp. Với một cạnh $x \rightarrow y$, nếu $x < y$ thì nó không ảnh hưởng tới tính liên thông (vì trên dây chuyền đã có thể đi từ x tới y); nếu $x > y$, thì $y \rightarrow y + 1 \rightarrow \dots \rightarrow x$ hợp với cạnh này tạo thành một chu trình, tức mọi đỉnh trong $[y, x]$ thuộc cùng một thành phần liên thông mạnh. Hợp nhất tất cả các đoạn $[y, x]$ như vậy, ta nhận được một phân hoạch đoạn của $[1, n]$; các đoạn đó chính là mọi thành phần liên thông mạnh.

Còn một cách đơn giản và thực dụng hơn để tìm thành phần liên thông mạnh của đồ thị giải đấu, dựa vào định lý Landau dưới đây.

Định lý 3.6 (Landau). Giả sử trong đồ thị giải đấu G , mọi đỉnh được sắp theo bậc ra tăng dần thành $p_1, \dots, p_n$, và bậc ra của p_i là d_i . Khi đó với mọi $1 \leq k \leq n$:

$$\sum_{i=1}^k d_i \geq \binom{k}{2}.$$

Mở rộng: mỗi thành phần liên thông mạnh của G là một đoạn liên tiếp trong hoán vị p , và p_k là đầu phải của một thành phần liên thông mạnh khi và chỉ khi

$$\sum_{i=1}^k d_i = \binom{k}{2}.$$

Chứng minh. Chỉ xét đồ thị con cảm sinh bởi k đỉnh đầu, nó có $\binom{k}{2}$ cạnh; các cạnh này đã đóng góp $\binom{k}{2}$ vào $\sum_{i=1}^k d_i$, nên $\sum_{i=1}^k d_i \geq \binom{k}{2}$.

Nếu x, y thuộc hai thành phần liên thông mạnh khác nhau và x tới được y , thì với mọi cạnh ra $y \rightarrow z$ của y , dễ chứng minh chắc chắn tồn tại cạnh $x \rightarrow z$; ngoài ra x còn có thêm cạnh $x \rightarrow y$, nên bậc ra của x lớn hơn bậc ra của y . Điều này nói rằng mỗi thành phần liên thông mạnh đúng là một đoạn liên tiếp trong hoán vị p .

Nếu đẳng thức $\sum_{i=1}^k d_i = \binom{k}{2}$ xảy ra, thì không có cạnh nào từ $p_{k+1}, \dots, p_n$ tới $p_1, \dots, p_k$; do đó các đỉnh từ p_{k+1} trở đi không tới được các đỉnh trước p_k , và hai phía này không thể thuộc cùng một thành phần liên thông mạnh. Ngược lại, nếu $p_i, \dots, p_k$ tạo thành một thành phần liên thông mạnh, cũng suy ra các đỉnh sau p_{k+1} không tới được các đỉnh trước p_k , tức đẳng thức xảy ra. Điều này chứng minh phần mở rộng của định lý. ■

Trong định lý Landau, thay bậc ra bằng bậc vào thì kết luận tương ứng hiển nhiên cũng đúng. Dưới đây dùng mô tả theo bậc vào. Sắp mọi đỉnh theo bậc vào tăng dần; lấy các k thỏa $\sum_{i=1}^k d_i = \binom{k}{2}$ làm đầu phải, mỗi đoạn liên tiếp được chia ra chính là một thành phần liên thông mạnh, và các thành phần liên thông mạnh nằm bên trái có thể tới các thành phần nằm bên phải.

Ví dụ 3.3.1 (bài luyện tập lý thuyết đồ thị cơ sở¹⁶). Cho đồ thị giải đấu G . Với mỗi cạnh, hãy tính số thành phần liên thông mạnh sau khi lật hướng cạnh đó.

Cách 1: dùng đường Hamilton.

Nếu cạnh bị lật nối hai thành phần liên thông mạnh khác nhau, hiển nhiên sau khi lật sẽ hợp nhất thành một mọi thành phần liên thông mạnh nằm giữa hai thành phần này trên đường Hamilton; phần này rất dễ tính.

Nếu cạnh bị lật nằm trong cùng một thành phần liên thông mạnh, tìm một chu trình Hamilton của thành phần đó. Nếu cạnh bị lật không nằm trên chu trình Hamilton, rõ ràng tính liên thông mạnh không bị ảnh hưởng. Ngược lại, sau khi xóa cạnh đó trên chu trình Hamilton, còn lại một đường Hamilton. Như đã nói ở trên, với mọi cạnh $x \rightarrow y$ ngược hướng đường Hamilton, tác dụng của nó là hợp nhất đoạn $[y, x]$ thành một thành phần liên thông mạnh. Lúc này $[y, x]$ là đoạn trên chu trình; dùng một số kỹ thuật cấu trúc dữ liệu để duy trì mọi $[y, x]$, có thể truy vấn trong $O(1)$ tình hình thành phần liên thông mạnh sau khi xóa một cạnh trên chu trình, nhưng cài đặt khá rắc rối.

Cách 2: dùng định lý Landau.

Sắp mọi đỉnh theo bậc vào tăng dần thành $p_1, \dots, p_n$, bậc vào của p_i là d_i . Lật một cạnh $p_x \rightarrow p_y$ sẽ làm bậc vào của p_x tăng 1 và bậc vào của p_y giảm 1. Chỉ xét dãy d_i , phát hiện sửa đổi trên có thể biểu diễn bằng $O(1)$ phép sửa điểm trên d_i . Điều cần trả lời là số lượng k thỏa $\sum_{i=1}^k d_i = \binom{k}{2}$. Vì vậy chỉ cần duy trì trên mỗi đoạn giá trị nhỏ nhất của $\sum_{i=1}^k d_i - \binom{k}{2}$ và số lần đạt nhỏ nhất, là có thể trả lời đơn giản.

Cả hai cách đều có độ phức tạp thời gian $O(n^2)$, nhưng cách 2 gọn hơn.

¹⁶CTT 2020

3.4 Tổng kết

Bài viết đã thảo luận nhiều bài toán và thuật toán liên quan tới tính liên thông của đồ thị, đồng thời cung cấp một số cấu trúc và tư tưởng thường dùng để giải các bài toán này. Trong đồ thị vô hướng, ta lấy cây DFS làm trung tâm, tập trung thảo luận quan hệ song liên thông và một số tính chất, ứng dụng của tập cắt. Trong đồ thị có hướng, ta lấy quan hệ khả đạt và thành phần liên thông mạnh làm trung tâm, rồi thảo luận các tính chất và ứng dụng tương ứng.

Thực ra còn nhiều nội dung bài viết chưa đi sâu nhắc tới, chẳng hạn cắt nhỏ nhất trong đồ thị vô hướng, thành phần tam liên thông, quan hệ thống trị trong đồ thị có hướng, bao đóng bắc cầu động, v.v. Có hai lý do: một là giới hạn dung lượng, hai là các nội dung này đã được thảo luận trong các luận văn đội tuyển các năm trước. Tác giả liệt kê các luận văn tương ứng trong tài liệu tham khảo; độc giả quan tâm có thể đọc thêm.

3.5 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn thầy Jin Jing của Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông đã quan tâm và hướng dẫn.
- Cảm ơn sự giúp đỡ của các bạn học đã trao đổi với tôi về các nội dung liên quan.
- Cảm ơn sự quan tâm và ủng hộ của cha mẹ.

3.6 Tài liệu tham khảo

- [1] Yu Haoxiang, “Bàn lại các thuật toán liên quan tới tính liên thông của đồ thị”, Tập luận văn đội tuyển IOI 2021.
- [2] Chang Ruinian, “Ứng dụng của đại số tuyến tính trong OI”, Tập luận văn đội tuyển IOI 2022.
- [3] Wikipedia, “Ear Decomposition”, https://en.wikipedia.org/wiki/Ear_decomposition.
- [4] Wikipedia, “Bipolar Orientation”, https://en.wikipedia.org/wiki/Bipolar_orientation.
- [5] zx2003, “Bàn sơ về định hướng lưỡng cực và ứng dụng”, <https://www.luogu.com.cn/blog/user19567/yi-dao-tu-lun-ti-ji-ji-yan-sheng>.
- [6] CCF, Niên giám Olympic Tin học Toàn quốc 2020.
- [7] Wang Wentao, “Bàn về một số thuật toán và ứng dụng của bài toán cắt nhỏ nhất trong đồ thị vô hướng”, Tập luận văn đội tuyển IOI 2016.
- [8] Sun Yaofeng, “Nghiên cứu bài toán bao đóng bắc cầu động”, Tập luận văn đội tuyển IOI 2017.
- [9] Chen Sunli, “Bàn về cây thống trị và ứng dụng”, Tập luận văn đội tuyển IOI 2020.

4 Bàn về xây dựng và sinh dữ liệu trong thi tin học

Trường Trung học phổ thông số 1 Weifang, tỉnh Sơn Đông
Liu Yiping

Tóm tắt

Bài viết này giới thiệu các phương pháp sinh ngẫu nhiên và xây dựng nhiều loại dữ liệu như dãy, cây, đồ thị, xâu ngoặc, đồng thời trình bày các lỗi hash thường gặp và cách xây dựng dữ liệu tương ứng.

4.1 Lời nói đầu

Trong thi tin học, ta thường cần kiểm tra tính đúng đắn của chương trình tại máy cục bộ.

Một phương pháp thường dùng là tự xây dựng dữ liệu kiểm thử. Với dữ liệu kích thước nhỏ, kết hợp với một chương trình khác có độ phức tạp cao hơn nhưng bảo đảm đúng, ta có thể kiểm tra tính đúng đắn của chương trình; với dữ liệu kích thước lớn, ta có thể kiểm tra hiệu suất của chương trình. Thí sinh thường gọi cách này là “đối phách”.

Trong đối phách, nếu cường độ dữ liệu sinh ra chưa đủ, có thể khó tìm ra lỗi của chương trình, hoặc để thuật toán có độ phức tạp cao vượt qua dữ liệu kích thước lớn.

Bài viết này sẽ giới thiệu vài cách sinh những loại dữ liệu thường gặp, cũng như cách sinh dữ liệu ngẫu nhiên hơn hoặc có tính nhắm mục tiêu hơn.

4.2 Dãy

4.2.1 Hoán vị

Thông thường có hai cách sinh hoán vị ngẫu nhiên. Chúng nhìn khá giống nhau, nhưng nguyên lý không giống nhau.

Cách thứ nhất như sau.

Thuật toán 1: Hoán vị ngẫu nhiên

Input: n

Output: một hoán vị ngẫu nhiên của 1 tới n

Đặt p là một dãy độ dài n , ban đầu $p_i = i$.

for $i = 1$ **to** n **do**

$j \leftarrow \text{random}(1, i)$

$\text{swap}(p_i, p_j)$

end for

return p

Ở đây $\text{random}(l, r)$ có tác dụng chọn đều một số nguyên trong $[l, r]$.

Bổ đề 1.1.1. Thuật toán 1 có thể sinh đều ngẫu nhiên một hoán vị của 1 tới n .

Chứng minh. Chú ý rằng thuật toán xáo trộn này trước hết thực hiện xáo trộn kích thước $n - 1$ trên $n - 1$ phần tử đầu của p , sau đó đặt $j = \text{random}(1, n)$ và hoán đổi p_n với p_j .

Xét quy nạp theo n .

Mệnh đề hiển nhiên đúng với $n = 1$; xét bước từ $n - 1$ suy ra n .

Sau khi hoán đổi ta có $p_j = n$. Vì j được chọn đều từ 1 tới n , chỉ cần chứng minh

$$p_1, \dots, p_{j-1}, p_{j+1}, \dots, p_n$$

là một hoán vị ngẫu nhiên đều.

Mặt khác, $p_1, \dots, p_{n-1}$ trước khi hoán đổi hiển nhiên song ánh với $p_1, \dots, p_{j-1}, p_{j+1}, \dots, p_n$ sau khi hoán đổi, nên hoán vị thu được là ngẫu nhiên đều. ■

Cách thứ hai như sau.

Thuật toán 2: Hoán vị ngẫu nhiên

Input: n

Output: một hoán vị ngẫu nhiên của 1 tới n

Đặt p là một dãy độ dài n , ban đầu $p_i = i$.

for $i = 1$ **to** n **do**

$j \leftarrow \text{random}(i, n)$

$\text{swap}(p_i, p_j)$

end for

return p

Bổ đề 1.1.2. Thuật toán 2 có thể sinh đều ngẫu nhiên một hoán vị của 1 tới n .

Chứng minh. Thuật toán này có thể xem là trước hết hoán đổi p_1 đều thành một trong các giá trị từ 1 tới n , rồi thực hiện thuật toán kích thước $n - 1$ trên p_2 tới p_n .

Xét quy nạp theo n , tính đúng đắn là hiển nhiên. ■

4.2.2 Dãy không có phần tử lặp

Xét bài toán sau: cần sinh đều ngẫu nhiên một dãy độ dài n , miền giá trị là các số nguyên từ 1 tới m ($m \geq n$), và các phần tử trong dãy không lặp.

Một phương pháp như sau.

Thuật toán 3: Dãy ngẫu nhiên không có phần tử lặp

Input: n

Output: một dãy ngẫu nhiên độ dài n không có phần tử lặp

Đặt a là một dãy độ dài n .

for $i = 1$ **to** n **do**

$a_i \leftarrow \text{random}(1, m)$

while $a_i \in \{a_1, a_2, \dots, a_{i-1}\}$ **do**

$a_i \leftarrow \text{random}(1, m)$

end while

end for

return p

Bổ đề 1.2.1. Thuật toán 3 có thể sinh đều ngẫu nhiên một dãy độ dài n , miền giá trị $[1, m] \cap \mathbb{Z}$, và các phần tử không lặp.

Chứng minh. Hiển nhiên, dãy được sinh ra chắc chắn hợp lệ, thuật toán có thể sinh mọi dãy hợp lệ, và xác suất sinh mọi dãy hợp lệ đều bằng nhau, đều là $1/m^n$. ■

Nhược điểm của phương pháp này là khi m tương đối nhỏ thì quá trình sinh sẽ khá chậm.

Ví dụ, khi $m = n$, số lần gọi random kỳ vọng có thể tính là

$$\sum_{i=1}^n n/i = \Theta(n \log n).$$

Việc kiểm tra $a_i \in \{a_1, a_2, \dots, a_{i-1}\}$ có thể làm bằng bảng hash trong $\Theta(1)$, nên tổng độ phức tạp là $\Theta(n \log n)$.

Nhưng chú ý rằng khi m tương đối nhỏ, có thể dùng một thuật toán khác: sinh một hoán vị ngẫu nhiên của 1 tới m , rồi lấy n phần tử đầu.

Khi $m \leq 2n$, dùng thuật toán thứ hai, độ phức tạp của phần này là $\Theta(n)$.

Khi $m > 2n$, dùng thuật toán thứ nhất; với mỗi i , xác suất mỗi lần chọn được một giá trị chưa xuất hiện là $(m - i + 1)/m \geq 1/2$, nên độ phức tạp kỳ vọng là $\Theta(n)$.

Một phương pháp khác là dùng thuật toán 2: ta chỉ cần cho vòng lặp i chạy tới n , dùng bảng hash lưu các vị trí đã bị thao tác và giá trị của chúng; độ phức tạp có thể đạt $\Theta(n)$.

4.2.3 Dãy có tổng cố định

Xét bài toán sau: cần sinh đều ngẫu nhiên một dãy số nguyên dương độ dài n , $a_1, a_2, \dots, a_n$, yêu cầu

$$\sum_{i=1}^n a_i = m.$$

Đặt $s_i = \sum_{j=1}^i a_j$, và chuyển sang sinh s .

Không khó thấy mọi s thỏa

$$1 \leq s_1 < s_2 < \dots < s_n = m$$

đều tương ứng một-một với a . Vì vậy, sinh đều ngẫu nhiên một a tương đương với sinh đều ngẫu nhiên một s như vậy.

Sinh s như vậy lại tương đương với: chọn đều ngẫu nhiên $n - 1$ số nguyên dương khác nhau trong $[1, m - 1]$. Điều này chuyển về bài toán sinh dãy không có phần tử lặp.

Với trường hợp a_i là số nguyên không âm, có thể cộng 1 vào mọi a_i để chuyển về trường hợp số nguyên dương.

Tổng quát hơn, nếu cận dưới của a_i là b_i , có thể trừ $b_i - 1$ khỏi a_i , từ đó chuyển về trường hợp cận dưới bằng 1.

4.3 Đồ thị

4.3.1 Cây

Cách 1: cha ngẫu nhiên. Một cách sinh cây ngẫu nhiên thường gặp trong thi tin học là: với mỗi $2 \leq i \leq n$, chọn đều ngẫu nhiên một đỉnh trong 1 tới $i - 1$ làm cha của i .

Với một cây có gốc, định nghĩa độ sâu của một đỉnh là số cạnh trên đường đi ngắn nhất từ nó tới gốc; định nghĩa chiều cao của cây là giá trị lớn nhất của độ sâu trên mọi đỉnh.

Bổ đề 2.1.1. Với mỗi $2 \leq i \leq n$, chọn đều ngẫu nhiên một đỉnh trong 1 tới $i - 1$ làm cha của i ; chiều cao kỳ vọng của cây sinh ra là $\Theta(\log n)$.

Bổ đề 2.1.2 (bất đẳng thức Jensen). Với hàm f lồi xuống trên đoạn $[L, R]$, với các số thực bất kỳ $x_1, x_2, \dots, x_n \in [L, R]$ và $a_1, a_2, \dots, a_n$ thỏa $\sum_{i=1}^n a_i = 1$ và $a_i > 0$, ta có

$$f\left(\sum_{i=1}^n a_i x_i\right) \leq \sum_{i=1}^n a_i f(x_i).$$

Chứng minh bổ đề 2.1.2 được lược bỏ; có thể tham khảo [1].

*Chứng minh bổ đề 2.1.1.*¹⁷ Giả sử chiều cao cây là h . Theo bất đẳng thức Jensen, có $2^{E[h]} \leq E[2^h]$. Xét cận trên của $E[2^h]$.

Gọi độ sâu của đỉnh i là d_i , khi đó

$$\begin{aligned} E[2^h] &= E\left[\max_{i=1}^n 2^{d_i}\right] \\ &\leq E\left[\sum_{i=1}^n 2^{d_i}\right] \\ &= \sum_{i=1}^n E[2^{d_i}]. \end{aligned}$$

Đặt $f_i = E[2^{d_i}]$, khi đó $f_1 = 1$, và với mọi $2 \leq i \leq n$ có

$$f_i = \frac{2}{i-1} \sum_{j=1}^{i-1} f_j.$$

Đặt $F_i = \sum_{j=1}^i f_j$, khi đó với mọi $2 \leq i \leq n$ có

$$F_i - F_{i-1} = \frac{2}{i-1} F_{i-1}.$$

Do đó

$$F_i = \frac{i+1}{i-1} F_{i-1} = \prod_{j=2}^i \frac{j+1}{j-1} = \frac{(i+1)i}{2}.$$

Vì vậy

$$2^{E[h]} \leq E[2^h] \leq F_n = \frac{(n+1)n}{2}.$$

Lấy log hai vế được $E[h] = O(\log n)$.

Theo chiều ngược lại,

$$E[h] \geq E\left[\frac{1}{n} \sum_{i=1}^n d_i\right] = \frac{1}{n} \sum_{i=1}^n E[d_i].$$

Đặt $g_i = E[d_i]$, khi đó $g_1 = 0$, và với $2 \leq i \leq n$ có

$$g_i = 1 + \frac{1}{i-1} \sum_{j=1}^{i-1} g_j.$$

Đặt $G_i = \sum_{j=1}^i g_j$, với $2 \leq i \leq n$ có

$$G_i - G_{i-1} = 1 + \frac{1}{i-1} G_{i-1}.$$

Do đó

$$G_i = 1 + \frac{i}{i-1} G_{i-1} = \sum_{j=2}^i \prod_{k=j}^{i-1} \frac{k+1}{k} = \sum_{j=2}^i \frac{i}{j} = \Theta(i \log i).$$

Vì vậy

$$E[h] \geq \frac{1}{n} \sum_{i=1}^n E[d_i] = \frac{1}{n} G_n = \Theta(\log n).$$

Nên $E[h] = \Omega(\log n)$.

Tổng hợp lại, $E[h] = \Theta(\log n)$. ■

¹⁷Một phần tham khảo từ [2].

Chiều cao kỳ vọng của cây là $\Theta(\log n)$; trong nhiều trường hợp, điều này không kiểm thử tốt hiệu suất thuật toán.

Ví dụ, một số thuật toán có độ phức tạp $\Theta(\sum_i \text{siz}_i) = \Theta(\sum_i \text{dep}_i)$, trong đó siz_i và dep_i lần lượt biểu thị kích thước cây con và độ sâu của đỉnh i . Trên loại dữ liệu này, độ phức tạp kỳ vọng là $\Theta(n \log n)$, trong khi độ phức tạp lớn nhất thực tế là $\Theta(n^2)$.

Nếu khi gộp heuristic không chọn con lớn nhất làm con nặng, hoặc trong một số tình huống sửa đổi trên đây chuyển xuất hiện lỗi, đều có thể làm độ phức tạp đạt tới $\Theta(\sum_i \text{dep}_i)$; dùng loại dữ liệu này thường không phát hiện được lỗi.

Cách 2: cha ngẫu nhiên trong một phạm vi nhất định. Một cách sửa đổi thường gặp của cách 1 là giới hạn phạm vi chọn cha ngẫu nhiên.

Ta có thể đặt một ngưỡng B , cha của đỉnh i được chọn ngẫu nhiên trong $[\max(1, i - B), i - 1]$.

Khi dùng phương pháp này, thường cần điều chỉnh kích thước B đúng lúc để sinh những loại dữ liệu khác nhau.

Thông thường B được đặt khá nhỏ; chẳng hạn với dữ liệu $n = 2 \times 10^5$ có thể đặt $B = 10$ hoặc 20, khi đó cây sinh ra khá phù hợp để kiểm thử gộp heuristic.

Cách 3: dãy Prufer. Với một cây vô hướng có nhãn, dãy Prufer của nó được sinh bằng cách sau.

Thuật toán 4: Dãy Prufer

Input: một cây vô hướng có nhãn T gồm n đỉnh

Output: dãy Prufer của T

Đặt a là một dãy độ dài $n - 2$.

for $i = 1$ **to** $n - 2$ **do**

$u \leftarrow$ lá có nhãn nhỏ nhất (đỉnh bậc 1)

$a_i \leftarrow$ nhãn của đỉnh kề với u

 Xóa u và các cạnh kề nó.

end for

return a

Có thể thấy, dãy Prufer của cây vô hướng có nhãn n đỉnh ($n \geq 2$) là một dãy độ dài $n - 2$, miền giá trị $[1, n] \cap \mathbb{Z}$.

Một cây tương ứng duy nhất với một dãy Prufer; vậy một dãy hợp lệ có tương ứng với một cây hay không?

Bổ đề 2.1.3. Dãy Prufer xây dựng một song ánh từ các cây n đỉnh tới các dãy độ dài $n - 2$, miền giá trị $[1, n] \cap \mathbb{Z}$.

Chứng minh. Chỉ cần chứng minh mỗi dãy Prufer đều tồn tại duy nhất một cây T tương ứng.

Xét quy nạp theo n . Trường hợp $n = 2$ đơn giản; xét dùng $n - 1$ suy ra n .

Giả sử dãy Prufer đã cho là $a_1, a_2, \dots, a_{n-2}$.

Không khó thấy số lần một nút xuất hiện trong dãy Prufer bằng bậc của nó trừ 1.

Từ đó có thể xác định toàn bộ lá trong cây; gọi lá có nhãn nhỏ nhất là u , khi đó đỉnh kề với u là a_1 .

Xét dãy Prufer $a_2, \dots, a_{n-2}$, với tập nhãn $\{1, \dots, n\} \setminus \{u\}$. Theo giả thiết quy nạp, tồn tại duy nhất một cây tương ứng với nó; gọi cây này là T' .

Thêm đỉnh u vào T' và nối cạnh với a_1 , nhận được cây T ; hiển nhiên đây là cây duy nhất tương ứng với a . ■

Chúng minh trên đồng thời cho một phương pháp dựng cây tương ứng từ dãy Prufer: dựng từ sau ra trước, mỗi lần thêm một lá.

Nếu muốn chọn đều ngẫu nhiên một cây vô hướng có nhãn trong tất cả các cây như vậy, chỉ cần chọn ngẫu nhiên dãy Prufer của nó. Cách sinh này còn có thể cố định bậc của từng đỉnh.

Cách 4: dây chuyền. Với đỉnh i ($2 \leq i \leq n$), chọn $i - 1$ làm cha của i là có thể sinh một dây chuyền.

Cách 5: cây nhị phân hoàn chỉnh. Với đỉnh i ($2 \leq i \leq n$), chọn $\lfloor i/2 \rfloor$ làm cha của i là có thể sinh một cây nhị phân hoàn chỉnh.

Với cây sinh theo cách này, nếu thực hiện phân rã nặng nhẹ, tổng kích thước các con nhẹ của mọi đỉnh là cấp $\Theta(n \log n)$.

Cách 6: kết hợp dây chuyền và cây nhị phân hoàn chỉnh. Ta có thể lấy hai ngưỡng B_1, B_2 : dùng B_1 đỉnh đầu để dựng cây nhị phân hoàn chỉnh, rồi cứ mỗi B_2 đỉnh phía sau nối thành một dây chuyền và gắn lên một trong B_1 đỉnh đầu.

Thông thường B_1 và B_2 đều được chọn ở cấp $\Theta(\sqrt{n})$.

Một cách chọn thường gặp là $B_1 = \lfloor 2\sqrt{n} \rfloor$, $B_2 = \lfloor \sqrt{n} \rfloor$. Trong tiểu mục 2.1.7 dưới đây ta cũng chọn như vậy.

Dữ liệu sinh theo cách này vừa có dây chuyền tương đối dài, vừa có tổng kích thước các con nhẹ của mọi đỉnh là $\Theta(n \log n)$.

Đối chiếu. Tiếp theo đối chiếu sáu phương pháp sinh.

Trước hết sinh một cây kích thước n , quy định đỉnh 1 là gốc. Sau đó xáo ngẫu nhiên các cạnh kề của từng đỉnh và thực hiện DFS. Cuối cùng thực hiện Q truy vấn; mỗi lần chọn ngẫu nhiên hai đỉnh u, v và truy vấn thông tin trên đường đi từ u tới v .

Quá trình trên được thực hiện T lần, sau đó đo các đại lượng dưới đây và lấy trung bình qua T lần:

- S_{size} biểu thị tổng kích thước cây con của mọi đỉnh. Một phần thuật toán độ phức tạp cao phụ thuộc vào đại lượng này.
- S_{son} biểu thị tổng kích thước các cây con nhẹ của mọi đỉnh sau khi phân rã nặng nhẹ. Nó có thể dùng để đánh giá hiệu suất gộp heuristic.
- S_{random} biểu thị tổng kích thước các cây con nhẹ của mọi đỉnh sau khi mỗi đỉnh chọn ngẫu nhiên một con làm con nặng. Nó có thể dùng để đánh giá hiệu suất của phương pháp gộp heuristic sai.
- S_{len} biểu thị độ dài đường đi trung bình trong Q truy vấn. Một phần thuật toán độ phức tạp cao phụ thuộc vào đại lượng này. Nó cũng có thể dùng để đánh giá hiệu suất của cấu trúc dữ liệu trên dây chuyền.
- S_{jump} biểu thị số cạnh nhẹ trung bình trên đường đi truy vấn sau khi phân rã nặng nhẹ. Nó có thể dùng để đánh giá hiệu suất của phân rã nặng nhẹ.

Lấy $n = 10^5$, $Q = 10^5$, $T = 10^3$, kết quả đo được như sau (giữ ba chữ số thập phân):

	S_{size}	S_{son}	S_{random}	S_{len}	S_{jump}
Cách 1	1212656.229	400679.705	873426.103	20.198	7.720
Cách 2 ($B = 10$)	909341143.306	116601.017	300801584.156	6069.061	2.332
Cách 2 ($B = 20$)	476433950.830	157012.831	166402833.765	3192.626	3.139
Cách 2 ($B = 40$)	244176851.716	198905.253	87602621.886	1664.193	3.972
Cách 3	39602294.635	325800.562	14507184.812	394.728	6.245
Cách 4	5000050000.000	0.000	0.000	33333.811	0.000
Cách 5	1568946.000	715030.000	734861.455	27.184	13.525
Cách 6	16675299.000	396342.000	418815.824	328.686	7.168

Có thể thấy:

- Cách 1: ưu điểm là mã ngắn, S_{son} tương đối lớn. Nhược điểm là S_{random} , S_{len} , S_{jump} đều quá nhỏ, khó kiểm thử các vấn đề độ phức tạp như gộp heuristic và phân rã nặng nhẹ.
- Cách 2: S_{size} , S_{random} , S_{len} lớn, và có xu hướng tăng khi B giảm.
- Cách 3: trong các loại dữ liệu được kiểm thử, các đại lượng có kích thước tương đối trung gian.
- Cách 4: S_{size} , S_{len} rõ ràng lớn; dãy chuyển khá phù hợp để kiểm thử hiệu suất của cấu trúc dữ liệu trên dãy chuyển ở kích thước cực hạn.
- Cách 5: S_{son} , S_{jump} lớn, khá phù hợp để kiểm thử hiệu suất của gộp heuristic và phân rã nặng nhẹ.
- Cách 6: cường độ nằm giữa cách 4 và cách 5.

4.3.2 Đồ thị tùy ý

Sinh đồ thị vô hướng n đỉnh m cạnh, không có cạnh bội và khuyên, chính là chọn m cạnh trong $\binom{n}{2}$ cạnh; có thể dùng kỹ thuật về dãy ở trên.

Nếu muốn sinh đồ thị liên thông, có thể liên tục sinh cho tới khi đồ thị liên thông.

4.3.3 Xương rỗng

Xương rỗng là đồ thị mà mỗi cạnh nằm trong nhiều nhất một chu trình đơn. Trong bài viết này, xương rỗng được phép có cạnh bội nhưng không được có khuyên.

Hiểu trực quan, xương rỗng là nhiều chu trình ghép lại thành hình dạng một cái cây. Cây cũng là một loại xương rỗng, nên xương rỗng thường được xem là mở rộng của cây.

Dưới đây giới thiệu ba cách sinh xương rỗng ngẫu nhiên.

Cách 1. Xét một đồ thị liên thông và thực hiện biến đổi sau trên nó. Dựng một đồ thị mới chứa toàn bộ đỉnh của đồ thị gốc, gọi là đỉnh tròn; với mỗi thành phần song liên thông đỉnh của đồ thị gốc, tạo thêm một đỉnh mới, gọi là đỉnh vuông (đặc biệt, ta xem một thành phần song liên thông đỉnh có thể chỉ chứa hai đỉnh và một cạnh). Nối mỗi đỉnh vuông với các đỉnh mà nó chứa trong đồ thị gốc; đồ thị thu được chắc chắn là một cây, gọi là cây tròn-vuông của đồ thị gốc.

Dễ thấy cây tròn-vuông thỏa các tính chất sau:

- Mỗi cạnh chắc chắn nối một đỉnh tròn với một đỉnh vuông. Nói cách khác, đỉnh tròn và đỉnh vuông tạo thành một tô màu đen-trắng của đồ thị này.
- Mọi đỉnh có bậc không quá 1 đều là đỉnh tròn.

Với xương rỗng, cây tròn-vuông của nó tương đương với việc tạo một đỉnh vuông cho mỗi chu trình.

Nếu muốn sinh ngẫu nhiên một xương rồng, trước hết ta có thể sinh ngẫu nhiên một cây, rồi tô màu đen-trắng cho nó, xem nó là cây tròn-vuông.

Nếu muốn cây tròn-vuông được sinh có n đỉnh, ta có thể sinh trước một cây $2n$ đỉnh, sau khi tô màu đen-trắng thì lấy màu có số lượng ít hơn làm đỉnh tròn; khi đó số đỉnh tròn chắc chắn không quá n .

Vì lá của cây tròn-vuông bắt buộc phải là đỉnh tròn, ta cần xóa mọi đỉnh vuông là lá.

Cách 2. Trước hết sinh ngẫu nhiên một cây, rồi thêm một số cạnh vào cây này để nó trở thành xương rồng.

DFS trên cây này; khi quay lui, với một xác suất nhất định nối tới một tổ tiên ngẫu nhiên, đồng thời đánh dấu các cạnh trên đường đi này, biểu thị rằng chúng đã nằm trên một chu trình. Vấn đề là xác định xác suất đó và tổ tiên được nối.

Ta có thể đặt một ngưỡng P . Khi quay lui ở đỉnh u , có thể xác định cạnh nối tới tổ tiên bằng cách sau.¹⁸

Thuật toán 5: Nối cạnh

```
 $v \leftarrow u$ 
while  $v$  không phải gốc và cạnh cha của  $v$  chưa bị đánh dấu do
   $x \leftarrow$  một số thực ngẫu nhiên trong  $[0, 1]$ 
  if  $x < P$  then
     $v \leftarrow$  cha của  $v$ 
  else
    break
  end if
end while
if  $u \neq v$  then
  Nối cạnh  $(u, v)$ 
  Đánh dấu đường đi từ  $u$  tới  $v$ 
end if
```

Ví dụ, lấy $P = 0.1$ thì các chu trình sinh ra sẽ tương đối nhỏ.

Cách 3. Vẫn xét cách trước hết sinh ngẫu nhiên một cây, rồi thêm cạnh.

Ta lấy một tham số K , sau đó chọn ngẫu nhiên K lần; mỗi lần chọn ngẫu nhiên hai đỉnh u, v và kiểm tra có thể nối một cạnh giữa u, v hay không; nếu có thì nối.

Cách kiểm tra tương tự cách 2: đánh dấu trên mỗi cạnh cây để biểu thị cạnh đó đã nằm trên một chu trình hay chưa. Khi kiểm tra, chỉ cần xét đường đi từ u tới v có cạnh đã đánh dấu hay không.

Nếu muốn bảo đảm độ phức tạp thời gian khi sinh dữ liệu quy mô lớn, cần dùng cấu trúc dữ liệu để duy trì đánh dấu. Nhưng trên thực tế, đi dọc đường đi từng bước và thoát khi gặp đánh dấu cũng có hiệu suất khá cao.

Đối chiếu. Ta kiểm thử T lần; mỗi lần chọn ngẫu nhiên một xương rồng, sau đó đo các đại lượng dưới đây và lấy trung bình qua T lần:

- N biểu thị số nút.
- C biểu thị số chu trình.

¹⁸Tham khảo từ [3].

- L biểu thị độ dài chu trình trung bình.
- σ biểu thị phương sai độ dài chu trình.
- M biểu thị độ dài chu trình lớn nhất.

Lấy $n = 10^5$, $T = 10^3$, phương pháp sinh cây là dãy Prufer ngẫu nhiên đều (tức cách 3 trong 2.1), kết quả đo như sau (giữ ba chữ số thập phân):

	N	C	L	σ	M
Cách 1	99909.960	44791.272	2.819	0.735	8.761
Cách 2 ($P = 0.1$)	100000.000	9889.691	2.110	0.122	5.728
Cách 2 ($P = 0.2$)	100000.000	19072.550	2.243	0.301	7.814
Cách 2 ($P = 0.4$)	100000.000	32611.841	2.567	0.853	11.825
Cách 3 ($K = 10^5$)	100000.000	221.323	35.973	2280.336	479.667

Có thể thấy:

- Cách 1: có thể dùng bậc của từng đỉnh để điều khiển phân bố độ dài chu trình, nhưng sẽ làm N giảm nhẹ. Khi dùng dãy Prufer ngẫu nhiên đều, chu trình khá ngắn, số chu trình nhiều, phương sai nhỏ.
- Cách 2: có thể điều chỉnh độ dài chu trình thông qua P ; khi P tăng, C, L, σ, M đều có xu hướng tăng. Nhìn chung chu trình khá ngắn, số chu trình nhiều, phương sai nhỏ.
- Cách 3: đặc điểm là số chu trình ít, độ dài chu trình lớn, phương sai lớn.

4.4 Xâu ngoặc

Xâu ngoặc là chuỗi được tạo bởi hai ký tự (và).

Với một xâu ngoặc $s = s_1 s_2 \dots s_n$, định nghĩa dãy đường đi $a_0, a_1, a_2, \dots, a_n$ của nó như sau:

$$a_i = \begin{cases} 0 & i = 0, \\ a_{i-1} + 1 & i > 0, s_i = (, \\ a_{i-1} - 1 & i > 0, s_i =). \end{cases}$$

Định nghĩa s là cân bằng khi và chỉ khi $a_n = 0$.

Định nghĩa độ khuyết defect(s) của s là số lượng các i thỏa $\min(a_i, a_{i+1}) < 0$ chia cho 2 (hiển nhiên số lượng đó luôn chẵn).

Định nghĩa s là hợp lệ khi và chỉ khi s cân bằng và độ khuyết bằng 0.

Định nghĩa $D_{n,k}$ là tập các xâu ngoặc cân bằng độ dài n , độ khuyết k .

Định nghĩa s^R biểu thị xâu sau khi đảo ngược s .

Với n chẵn, $D_{n,0}$ chính là tập các xâu ngoặc hợp lệ độ dài n ; theo số Catalan,

$$|D_{n,0}| = \frac{1}{n/2 + 1} \binom{n}{n/2}.$$

Với một xâu ngoặc cân bằng s , định nghĩa phép biến đổi $\Phi(s)$ như sau.¹⁹

- Nếu s rỗng, thì $\Phi(s)$ là xâu rỗng.

¹⁹Tham khảo từ [4].

- Nếu s không rỗng, chắc chắn có thể phân tích nó thành $s = lr$, trong đó l là tiền tố cân bằng không rỗng ngắn nhất của s ; dễ thấy r cũng là một xâu cân bằng.

Có thể thấy $|l|$ chính là i nhỏ nhất thỏa $i > 0, a_i = 0$.

Sau đó xét tính hợp lệ của l :

- Nếu l hợp lệ, thì $\Phi(s) = l\Phi(r)$.
- Nếu không, l chắc chắn có thể biểu diễn thành $l =)t($, và khi đó $\Phi(s) = (\Phi(r))t^R$.

Dễ chứng minh $\Phi(s)$ là một xâu ngoặc hợp lệ, tức $\Phi(s) \in D_{n,0}$.

Bổ đề 3.0.1. Với n chẵn và $0 \leq k \leq n/2$, Φ thiết lập một song ánh từ $D_{n,k}$ tới $D_{n,0}$.

Chứng minh. Trước hết chứng minh đơn ánh.

Xét quy nạp theo n ; mệnh đề hiển nhiên đúng với $n = 0$.

Xét hai xâu $s_1, s_2 \in D_{n,k}$ và $s_1 \neq s_2$; lần lượt phân tích chúng thành $s_1 = l_1 r_1, s_2 = l_2 r_2$, rồi phân loại.

- Trường hợp 1: l_1, l_2 đều hợp lệ.

Khi đó l_1, l_2 lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$. Nếu $l_1 \neq l_2$ thì mệnh đề đúng; nếu không, chắc chắn $r_1 \neq r_2$, theo giả thiết quy nạp có $\Phi(r_1) \neq \Phi(r_2)$, nên mệnh đề đúng.

- Trường hợp 2: l_1, l_2 đều không hợp lệ.

Khi đó $(\Phi(r_1))$ và $(\Phi(r_2))$ lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$; tiếp theo tương tự trường hợp 1, mệnh đề đúng.

- Trường hợp 3: l_1 hợp lệ, l_2 không hợp lệ. Đặt $l_2 =)t_2($.

Khi đó l_1 và $(\Phi(r_2))$ lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$.

Giả sử $\Phi(s_1) = \Phi(s_2)$, khi đó chắc chắn có $\Phi(r_1) = t_2^R$.

Vì l_1 hợp lệ nên $k = \text{defect}(s_1) = \text{defect}(r_1)$; do đó $2k \leq |r_1| = |\Phi(r_1)| = |t_2^R| < |l_2|$, tức $2k < |l_2|$.

Lại vì l_2 là tiền tố cân bằng không rỗng ngắn nhất của s_2 , chắc chắn có $k = \text{defect}(s_2) \geq \text{defect}(l_2)$, tức $2k \geq |l_2|$, mâu thuẫn.

Vì vậy $\Phi(s_1) \neq \Phi(s_2)$, mệnh đề đúng.

- Trường hợp 4: l_1 không hợp lệ, l_2 hợp lệ. Tương tự trường hợp 3.

Sau đó chứng minh toàn ánh.

Vì đã có đơn ánh, nên với mọi $0 \leq k \leq n/2$ có $|D_{n,k}| \leq |D_{n,0}|$. Ta chỉ cần chứng minh $|D_{n,k}| = |D_{n,0}|$.

Giả sử tồn tại $0 \leq k \leq n/2$ thỏa $|D_{n,k}| < |D_{n,0}|$, khi đó

$$\binom{n}{n/2} = \sum_{k=0}^{n/2} |D_{n,k}| < (n/2 + 1)|D_{n,0}| = \binom{n}{n/2},$$

mâu thuẫn.

Tổng hợp lại, với mọi $0 \leq k \leq n/2$, Φ thiết lập một song ánh từ $D_{n,k}$ tới $D_{n,0}$. ■

Có bổ đề 3.0.1, ta có thể chọn ngẫu nhiên một xâu ngoặc hợp lệ rất tiện lợi.

Chỉ cần chọn đều ngẫu nhiên một trong tất cả $\binom{n}{n/2}$ xâu cân bằng, rồi áp dụng Φ lên nó.

4.5 Hash

Hash là thuật toán rất thường dùng trong thi tin học; nó nén thông tin bằng một cách tương đối ngẫu nhiên để tăng tốc việc so sánh thông tin.

Trong tình huống thông thường, muốn làm hash sai thì phải xây dựng dữ liệu nhắm vào mã nguồn cụ thể.

Tuy nhiên có một số phương pháp hash không cần nhắm vào mã nguồn; có thể trực tiếp xây dựng dữ liệu làm chúng sai, còn dữ liệu ngẫu nhiên dùng khi đối phó thường không đủ để phát hiện những vấn đề này.

Dưới đây liệt kê vài thuật toán hash thường gặp, dễ bị xây dựng dữ liệu làm sai.

4.5.1 Nghịch lý sinh nhật

Trong một căn phòng có 23 người. Giả sử một năm luôn có 365 ngày và sinh nhật của mỗi người được chọn độc lập, đều trong 365 ngày này. Hỏi: xác suất tồn tại hai người có cùng sinh nhật là bao nhiêu?

Giả sử một năm có n ngày, trong phòng có m người. Khi đó xác suất sinh nhật của m người này đôi một khác nhau là

$$\prod_{i=1}^m \frac{n-i+1}{n}.$$

Thay $n = 365, m = 23$ vào, xác suất trên xấp xỉ 49%, tức xác suất tồn tại hai người có cùng sinh nhật lớn hơn 50%. Điều này rất trái trực giác, nên ta gọi hiện tượng này là nghịch lý sinh nhật.

Vậy m cần đạt tới bao nhiêu để xác suất tồn tại hai người cùng sinh nhật không nhỏ hơn 50%?

Có thể lập phương trình sau:

$$\prod_{i=1}^m \frac{n-i+1}{n} \leq \frac{1}{2}.$$

Dùng $1+x \leq e^x$ để ước lượng:

$$\prod_{i=1}^m e^{-\frac{i-1}{n}} \leq \frac{1}{2}.$$

Tức là

$$e^{-\frac{m(m-1)}{2n}} \leq \frac{1}{2}.$$

Suy ra $m \approx \sqrt{2n \ln 2}$.

Phương pháp hash thông thường là chọn mô-đun M , nén thông tin cần hash thành một số nguyên trong $[0, M-1]$.

Quá trình này có thể gần đúng xem là chọn ngẫu nhiên một số nguyên trong $[0, M-1]$.

Nếu bài toán cần xử lý có dạng “phán đoán trong n thông tin có trùng lặp hay không”, thì khi n xấp xỉ $\sqrt{2M \ln 2}$, xác suất phán thông tin không trùng thành có trùng sẽ đạt tới $\frac{1}{2}$.

Mô-đun hash đơn thông thường ở cấp 10^9 ; khi n đạt xấp xỉ 3×10^4 , xác suất xuất hiện va chạm khoảng 36%, một xác suất rất lớn.

Trong trường hợp này, nên chọn hai mô-đun nguyên tố cùng nhau M_1, M_2 ; khi đó miền giá trị ngẫu nhiên có thể xem là $M_1 M_2$, làm xác suất va chạm giảm mạnh.

4.5.2 Tràn tự nhiên

Khi làm hash xâu, thông thường ta chọn mô-đun M và cơ số B . Với xâu $s = s_1 s_2 \dots s_n$, giá trị hash của nó được tính là

$$\text{hash}(s) = \left(\sum_{i=1}^n B^{i-1} \text{ord}(s_i) \right) \bmod M.$$

Ở đây $\text{ord}(c)$ biểu thị vị trí của ký tự c trong bảng mã.

Số nguyên không dấu 64 bit trong C++ sẽ tự động lấy mô-đun 2^{64} khi tính toán; nếu chọn $M = 2^{64}$, có thể bớt một phép lấy mô-đun để tăng tốc. Phương pháp hash này gọi là tràn tự nhiên.

Nhưng phương pháp hash này có thể rất dễ bị xây dựng và chặn.

Nếu B là số chẵn, thì $B^{64} \bmod M = 0$, tức trong s nhiều nhất chỉ có 64 vị trí cuối là có hiệu lực, rất dễ xây dựng và chặn. Dưới đây xét trường hợp B là số lẻ.

Xét xâu $s = s_1 s_2 \dots s_{2^k}$ thỏa

$$s_i = \begin{cases} A & \text{popcount}(i-1) \text{ là số chẵn,} \\ B & \text{popcount}(i-1) \text{ là số lẻ.} \end{cases}$$

Trong đó $\text{popcount}(x)$ biểu thị số bit 1 trong biểu diễn nhị phân của x .

Với xâu t , đặt t^F là kết quả thay mọi A trong t bằng B và đồng thời thay mọi B bằng A .

Xét dãy xâu $t_0, t_1, \dots, t_k$ thỏa $t_0 = A$, $t_i = t_{i-1} + t_{i-1}^F$, khi đó $s = t_k$.

Đặt $d = \text{ord}(A) - \text{ord}(B)$, dễ thu được

$$\text{hash}(s) - \text{hash}(s^F) \equiv \sum_{i=1}^{2^k} (-1)^{\text{popcount}(i-1)} B^{i-1} d \pmod{M}.$$

Ta cần chứng minh khi k đủ lớn, giá trị này luôn bằng 0 theo nghĩa mod M .

Đặt

$$f_i = \text{hash}(t_i) - \text{hash}(t_i^F).$$

Dễ thu được

$$f_i = f_{i-1} (1 - B^{2^{i-1}}).$$

Mà $f_0 = d$, nên

$$f_i = d \prod_{j=0}^{i-1} (1 - B^{2^j}).$$

Chú ý rằng

$$1 - B^{2^j} = (1 - B^{2^{j-1}})(1 + B^{2^{j-1}}).$$

Tức $1 - B^{2^{j-1}}$ nhân với một số chẵn. Nếu $2^p \mid (1 - B^{2^{j-1}})$ thì chắc chắn $2^{p+1} \mid (1 - B^{2^j})$.

Do đó f_i chắc chắn chia hết cho $2^1 2^2 \dots 2^i = 2^{\frac{i(i+1)}{2}}$.

Khi $k = 11$, có $2^{64} \mid f_k$, tức $\text{hash}(t_k) = \text{hash}(t_k^F)$.

4.5.3 Hash cây

Hash cây thường được dùng để phán đoán hai cây có gốc có đẳng cấu hay không.

Một phương pháp hash cây tương đối đúng là dùng thứ tự ngoặc và tư tưởng biểu diễn nhỏ nhất, để các cây đẳng cấu nhận được cùng một thứ tự ngoặc.

Cụ thể, giá trị hash của một cây con u có thể được tính đệ quy như sau:

1. Tính giá trị hash của mọi con của u .
2. Sắp xếp chúng theo khóa (giá trị hash, kích thước cây con).
3. Theo thứ tự đã sắp xếp, nối thứ tự ngoặc của mọi con của u lại.
4. Thêm một cặp ngoặc ở ngoài cùng.
5. Dây thu được chính là thứ tự ngoặc của u , và giá trị hash của nó là giá trị hash của u .

Để chứng minh các cây đẳng cấu nhận được cùng giá trị hash.

Việc thêm kích thước cây con vào khóa sắp xếp ở bước thứ hai là để ngăn các cây con khác kích thước nhưng có giá trị hash bằng nhau do nghịch lý sinh nhật làm ảnh hưởng kết quả.

Giả sử số nút của cây là n ; vì cần sắp xếp, độ phức tạp của thuật toán trên là $\Theta(n \log n)$.

Còn có một thuật toán hash độ phức tạp $\Theta(n)$ được biết đến rộng rãi.

Gọi kích thước cây con của đỉnh u là size_u , tập con của nó là son_u .

Ta sinh một dãy $W_1, W_2, \dots, W_n$, thông thường dùng dãy số nguyên tố hoặc dãy ngẫu nhiên.

Sau đó giá trị hash của đỉnh u được tính như sau:

$$h_u = W_{\text{size}_u} \left(1 + \sum_{v \in \text{son}_u} h_v \right).$$

Xét đường đi từ đỉnh u tới gốc; viết lần lượt các giá trị size trên đường đi này, thu được dãy S_u .

Chú ý rằng giá trị hash của toàn bộ cây T chỉ liên quan tới đa tập không thứ tự

$$H(T) = \{S_1, S_2, \dots, S_n\};$$

hai cây có tập này bằng nhau chắc chắn nhận được giá trị hash bằng nhau.

Vấn đề của kiểu hash này là đôi khi chỉ dựa vào $H(T)$ thì không thể xác định duy nhất hình dạng của cây.

Ví dụ, bất kể chọn W như thế nào, hai cây dưới đây chắc chắn có giá trị hash bằng nhau:

$$T_1 : \{(1, 2), (1, 8), (2, 3), (2, 4), (2, 5), (5, 6), (5, 7), (8, 9), (8, 12), (9, 10), (10, 11), (12, 13)\},$$

$$T_2 : \{(1, 2), (1, 8), (2, 3), (2, 4), (4, 9), (9, 10), (10, 11), (8, 5), (8, 12), (5, 6), (5, 7), (12, 13)\}.$$

Thực ra, chỉ cần hoán đổi hai cây con có S bằng nhau là có thể giữ $H(T)$ không đổi. Trong hình gốc, hai cây con của 5 và 9 đã được hoán đổi; các tập cạnh ở trên mô tả đúng hai cây đó.

Bộ dữ liệu này có thể làm sai mọi thuật toán hash chỉ dựa trên $H(T)$.

4.6 Tổng kết

Các kiểu dữ liệu mà bài viết này giới thiệu là những kiểu khá thường gặp trong thi tin học.

Các kiểu dữ liệu như dãy, cây suy sinh đơn giản, nhưng nếu phương pháp không phù hợp thì dễ xây dựng ra dữ liệu có cường độ chưa đủ.

Các kiểu dữ liệu như xương rồng, xoắn ngoặc lại có độ khó nhất định khi sinh.

Còn nếu phương pháp hash không phù hợp, thí sinh có thể vẫn mất điểm dù đã vượt qua dữ liệu ngẫu nhiên.

Trong nhiều trường hợp hơn, thí sinh cần dựa vào tính chất của bài toán để xây dựng dữ liệu có tính nhắm mục tiêu hơn. Hy vọng bài viết này có thể gợi mở suy nghĩ cho thí sinh, giúp hiểu sâu hơn về phương pháp kiểm thử chương trình và xây dựng dữ liệu.

4.7 Lời cảm ơn

- Cảm ơn cha mẹ đã nuôi dưỡng, quan tâm và ủng hộ.
- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn hai huấn luyện viên đội tuyển quốc gia Yang Yaoliang và Peng Sijin đã hướng dẫn.
- Cảm ơn thầy Leng Xuenong đã bồi dưỡng và chỉ dạy.
- Cảm ơn các thầy cô và bạn học đã động viên, giúp đỡ.

4.8 Tài liệu tham khảo

- [1] Wikipedia, “Bất đẳng thức Jensen”, <https://zh.wikipedia.org/wiki/%E7%B0%A1%E6%A3%AE%E4%B8%8D%E7%AD%89%E5%BC%8F>.
- [2] “Nhật ký hoạt động 2020.11.21: chiều cao cây kỳ vọng của cây cha ngẫu nhiên”, https://blog.csdn.net/EI_Captain/article/details/109910307.
- [3] “Chơi với xương rồng”, <https://vfleaking.blog.uoj.ac/blog/89>.
- [4] M.D. Atkinson and J.-R. Sack, “Generating binary trees at random”, <http://www.cs.otago.ac.nz/staffpriv/mike/Papers/RandomGeneration/RandomBinaryTrees.pdf>.

5 Bàn về một lớp bài toán có treewidth bị chặn

Trường Thập Nhất Bắc Kinh
Wu Chang

Tóm tắt

Bài viết giới thiệu các khái niệm liên quan tới treewidth, cách phân rã cây của đồ thị có treewidth bị chặn, và một số thuật toán tốt cho các bài toán kinh điển trên lớp đồ thị này. Nội dung đi từ định nghĩa, tính chất cơ bản và các đồ thị đặc biệt, tới ý nghĩa thuật toán của treewidth bị chặn và phác thảo thuật toán tuyến tính của Bodlaender để tìm phân rã cây khi treewidth là hằng số.

5.1 Mở đầu

Treewidth là một khái niệm quan trọng trong khoa học máy tính. Độ phức tạp của nhiều bài toán trên đồ thị tăng rất nhanh theo treewidth, trong khi nhiều đồ thị phát sinh trong thực tế, chẳng hạn mạng đường, lại thường có treewidth nhỏ. Vì vậy, việc tìm phân rã cây có độ rộng nhỏ và giải các bài toán trên đồ thị có treewidth nhỏ đã trở thành một hướng nghiên cứu được quan tâm.

Trong thi tin học cũng từng xuất hiện các bài toán trên đồ thị đặc biệt có treewidth nhỏ, nhưng khái niệm và thuật toán liên quan chưa phổ biến. Bài viết trình bày các định nghĩa cần dùng, một số quy ước và tính chất cơ bản, phân tích treewidth của vài lớp đồ thị đặc biệt, nhấn mạnh quá trình thu gọn của đồ thị nối tiếp-song song tổng quát, rồi giới thiệu ý nghĩa của treewidth bị chặn và một số thuật toán cổ điển trên lớp đồ thị này.

5.2 Định nghĩa

Định nghĩa 2.1 (phân rã cây). Với đồ thị vô hướng $G = (V, E)$, nếu một cây $T = (I, F)$ và một họ tập đỉnh

$$\{X_i \subseteq V \mid i \in I\}$$

đồng thời thỏa ba điều kiện sau, thì (X, T) được gọi là một phân rã cây của G :

- $\bigcup_{i \in I} X_i = V$;
- với mọi cạnh $(u, v) \in E$, tồn tại $i \in I$ sao cho $u, v \in X_i$;
- với mọi $i, j, k \in I$, nếu j nằm trên đường đi giữa i và k trong T , thì $X_i \cap X_k \subseteq X_j$.

Các tập X_i thường được gọi là các *bag*.

Định nghĩa 2.2 (độ rộng). Độ rộng của phân rã cây (X, T) là $\max_{i \in I} |X_i| - 1$.

Định nghĩa 2.3 (treewidth). Treewidth của G , ký hiệu $\text{tw}(G)$, là độ rộng nhỏ nhất trong tất cả các phân rã cây của G .

Định nghĩa 2.4 (k -tree). Một k -tree bắt đầu từ đồ thị đầy đủ trên $k + 1$ đỉnh. Sau đó lặp lại thao tác thêm một đỉnh mới, sao cho đỉnh mới nối đúng với k đỉnh cũ và k đỉnh này đôi một kề nhau.

Định nghĩa 2.5 (partial k -tree). Một partial k -tree là đồ thị con bất kỳ của một k -tree.

5.3 Quy ước và tính chất cơ bản

Định lý 3.1. $\text{tw}(G) \leq k$ khi và chỉ khi G là một partial k -tree. Chứng minh của định lý này khá dài nên bài viết lược bỏ.

Nếu không nói thêm, các phần sau xét đồ thị G không rỗng, không có cạnh bội và không có khuyên. Với cây T có gốc, lấy tùy ý $r \in I$ làm gốc. Ký hiệu dep_i là độ sâu của nút i trong T , và $\text{subtree}(i)$ là tập nút trong cây con của i .

Với $u \in V$, đặt

$$S_u = \{i \in I \mid u \in X_i\}.$$

Ký hiệu $N_G(u) = \{v \mid (u, v) \in E\}$ và $d_G(u) = |N_G(u)|$. Gọi top_u là nút nông nhất trong S_u , và đặt $\text{lev}_u = \text{dep}_{\text{top}_u}$.

Bổ đề 3.0.1. Với mọi cạnh $(u, v) \in E$, giả sử không mất tổng quát rằng $\text{lev}_u \geq \text{lev}_v$. Khi đó $v \in X_{\text{top}_u}$.

Chứng minh. Vì $S_u \subseteq \text{subtree}(\text{top}_u)$ và $S_u \cap S_v \neq \emptyset$, nếu $\text{top}_u \notin S_v$ thì toàn bộ S_v cũng nằm trong cây con của top_u , mâu thuẫn với $\text{lev}_u \geq \text{lev}_v$.

Bổ đề 3.0.2. Nếu G là partial k -tree và $k < |V|$, thì

$$|E| \leq k|V| - \frac{k(k+1)}{2}.$$

Chứng minh. Lấy đỉnh u có lev lớn nhất trong V . Theo bổ đề trước, $d_G(u) \leq k$. Xóa u rồi quy nạp.

Thu gọn cạnh $(u, v) \in E$ nghĩa là xóa cạnh này, hợp nhất u, v thành một đỉnh mới và để đỉnh mới thừa hưởng các cạnh kề ban đầu.

Bổ đề 3.0.3. Nếu thu gọn cạnh của G được G' , thì $\text{tw}(G') \leq \text{tw}(G)$.

Chứng minh. Giả sử đỉnh mới là w . Với một phân rã cây (X, T) của G , nếu X_i chứa ít nhất một trong hai đỉnh u, v , thay chúng bằng w ; nếu không, giữ nguyên X_i . Họ bag thu được là một phân rã cây của G' và độ rộng không tăng.

5.4 Các đồ thị đặc biệt

5.4.1 Partial 1-tree

Định lý 4.1. Partial 1-tree chính là rừng.

Chứng minh. Nếu G là partial 1-tree thì mỗi thành phần liên thông của nó cũng vậy. Theo bổ đề cạnh, mỗi thành phần liên thông là cây, nên G là rừng.

Ngược lại, nếu $G = (V, E)$ là rừng, ta có thể xây phân rã cây độ rộng 1 như sau. Tập nút I có một nút cho mỗi đỉnh của G và một nút cho mỗi cạnh của G . Với cạnh (u, v) , nối nút đại diện cạnh này với hai nút đại diện u và v , rồi nối thêm tùy ý để T liên thông. Bag của nút đại diện đỉnh u là $\{u\}$, còn bag của nút đại diện cạnh (u, v) là $\{u, v\}$.

5.4.2 Thu gọn partial 2-tree

Xét ba thao tác trên đồ thị G :

- gộp cạnh bội;
- xóa đỉnh bậc 0 hoặc bậc 1;
- thu gọn đỉnh bậc 2: nối hai láng giềng của nó bằng một cạnh rồi xóa nó.

Mỗi lần thực hiện một thao tác gọi là một lần thu gọn.

Bổ đề 4.2.1. Nếu G' thu được từ G bằng một lần thu gọn, thì G là partial 2-tree khi và chỉ khi G' là partial 2-tree.

Hai thao tác đầu hiển nhiên. Với thao tác thứ ba, giả sử xóa u và hai láng giềng là v, w . Nếu G có phân rã cây độ rộng 2, xóa u khỏi các bag chứa nó và thêm v khi cần sẽ cho phân rã của G' . Ngược lại, từ phân rã của G' , lấy một bag chứa v, w , thêm một nút con mới có bag $\{u, v, w\}$, ta được phân rã cây độ rộng 2 của G .

Bổ đề 4.2.2. Nếu $G = (V, E)$ là partial 2-tree thì tồn tại $u \in V$ sao cho $d_G(u) \leq 2$. Chọn đỉnh có lev lớn nhất và dùng Bổ đề 3.0.1.

Một quá trình thu gọn cực đại là quá trình thực hiện liên tiếp các thao tác trên cho tới khi không còn thao tác nào làm được.

Định lý 4.2. Mọi quá trình thu gọn cực đại trên một partial 2-tree đều xóa sạch đồ thị.

Tính chất này gần trùng với khái niệm đồ thị nối tiếp-song song tổng quát; khác biệt là định nghĩa nối tiếp-song song tổng quát không cần xét thao tác xóa đỉnh bậc 0. Nói cách khác, đồ thị nối tiếp-song song tổng quát là partial 2-tree liên thông.

5.4.3 Cây nối tiếp-song song

Quá trình thu gọn trên có thể được mô tả bằng một cây ba nhánh có gốc. Mỗi lá đại diện cho một đỉnh hoặc cạnh ban đầu. Tại mọi thời điểm, mỗi đỉnh hoặc cạnh hiện có trong đồ thị ứng với một nút hiện tại trên cây này. Mỗi lần thu gọn tạo một nút mới x :

- gộp cạnh bội: hai nút hiện tại của hai cạnh bội trở thành con của x , và cạnh mới hiện tại là x ;
- xóa đỉnh bậc 1: nếu xóa u kề v , thì các nút hiện tại của u , v và (u, v) trở thành con của x , còn v hiện tại là x ;
- thu gọn đỉnh bậc 2: nếu xóa u kề v, w , thì các nút hiện tại của u , (u, v) và (u, w) trở thành con của x , còn cạnh mới hiện tại là x .

5.4.4 Quy hoạch động dựa trên thu gọn

Nhiều bài khó trên đồ thị tổng quát có thể giải tốt trên partial 2-tree nhờ DP theo quá trình thu gọn. Ví dụ, xét bài SNOI2020 *Cây khung*: cho đồ thị vô hướng liên thông n đỉnh, m cạnh; biết rằng xóa một cạnh sẽ được cactus; cần đếm số cây khung modulo 998244353.

Có thể chứng minh đồ thị thỏa điều kiện là đồ thị nối tiếp-song song tổng quát. Với một cạnh hiện tại $e = (u, v)$, ghi f_e là số phương án trong phần đã thu gọn thành e sao cho u, v liên thông, và g_e là số phương án sao cho u, v không liên thông; các đỉnh bị xóa trong phần đó vẫn phải liên thông với ít nhất một trong u, v .

Với xóa đỉnh bậc 1, cạnh kề của đỉnh đó bắt buộc nằm trong cây khung, nên nhân đáp án với f_e . Với gộp hai cạnh bội e_1, e_2 thành e_0 , hai cạnh không thể đồng thời liên thông:

$$f_{e_0} = f_{e_1}g_{e_2} + f_{e_2}g_{e_1}, \quad g_{e_0} = g_{e_1}g_{e_2}.$$

Với thu gọn đỉnh bậc 2, hai cạnh kề không thể đồng thời không liên thông:

$$f_{e_0} = f_{e_1}f_{e_2}, \quad g_{e_0} = f_{e_1}g_{e_2} + f_{e_2}g_{e_1}.$$

Độ phức tạp là $O(n + m)$; bản chất thuật toán là DP từ dưới lên trên cây nối tiếp-song song.

5.4.5 Một số đồ thị khác

Đồ thị đầy đủ K_n có treewidth $n - 1$. Đồ thị phẳng có treewidth cỡ căn bậc hai. Cactus và đồ thị ngoài phẳng có treewidth không quá 2. Halin graph và Apollonian network có treewidth không quá 3. Với đồ thị chordal, treewidth bằng kích thước clique lớn nhất trừ 1 và có thể tính trong thời gian đa thức. Một định nghĩa tương đương khác của treewidth là: trong các đồ thị chordal chứa đồ thị gốc, lấy kích thước clique lớn nhất nhỏ nhất rồi trừ 1.

5.5 Treewidth bị chặn

5.5.1 Tìm phân rã cây độ rộng nhỏ

Tính treewidth của đồ thị tổng quát đã được chứng minh là NP-complete. Dù vậy, có nhiều thuật toán tham số theo treewidth:

- với đồ thị n đỉnh có treewidth k , có thuật toán tìm phân rã cây độ rộng k trong thời gian $n^{k+O(1)}$;
- có thuật toán thời gian $2^{\tilde{O}(k^3)}n$;
- có thuật toán cho phân rã độ rộng không quá $5k + 4$ trong thời gian $2^{O(k)}n$;
- có thuật toán xấp xỉ độ rộng $O(k\sqrt{\log k})$ trong thời gian đa thức.

Điểm đáng chú ý là nhiều thuật toán có độ phức tạp tăng rất nhanh theo k , nhưng tuyến tính hoặc gần tuyến tính theo n khi k được xem là hằng số.

5.5.2 Ý nghĩa của treewidth bị chặn

Với mọi hằng số k , việc kiểm tra một đồ thị có phân rã cây độ rộng k hay không, và dựng nó nếu có, có thể làm tuyến tính theo số đỉnh. Ngoài ra, khi giới hạn đồ thị đầu vào có treewidth không vượt quá một hằng số, nhiều bài toán NP có thể giải trong thời gian đa thức. Với một lớp lớn các bài dễ trên cây, ta có thể làm DP trên phân rã cây để đạt độ phức tạp tốt.

Định lý 5.5 (Courcelle). Nếu một bài toán đồ thị biểu diễn được bằng logic bậc hai đơn nguyên, thì trên đồ thị treewidth bị chặn, bài toán đó giải được trong thời gian tuyến tính.

Ví dụ gồm tô màu đồ thị, chu trình Hamilton, clique lớn nhất và tập độc lập lớn nhất. Phần sau minh họa bằng bài tập độc lập lớn nhất.

5.5.3 Phân rã cây hoàn hảo

DP trên phân rã cây tổng quát thường khá rườm rà. Nếu phân rã cây có thêm cấu trúc, DP sẽ dễ viết hơn.

Định nghĩa 5.1 (phân rã cây hoàn hảo). Một phân rã cây (X, T) của $G = (V, E)$ gọi là hoàn hảo nếu T là cây nhị phân có gốc và mỗi nút $i \in I$ thuộc đúng một trong bốn loại sau:

- lá: không có con và $|X_i| = 1$;
- nút đưa vào: có một con j và $X_i = X_j \cup \{v\}$;
- nút quên: có một con j và $X_i = X_j \cup \{v\}$;
- nút hợp: có hai con j, k và $X_i = X_j = X_k$.

Định lý 5.6. Cho một phân rã cây độ rộng bị chặn của G , có thuật toán tuyến tính biến nó thành một phân rã cây hoàn hảo cùng độ rộng.

Ý tưởng chuyển đổi như sau. Nếu một nút lá có bag chứa hơn một đỉnh, thêm nút con và mỗi lần bỏ một đỉnh khỏi bag. Nếu một nút có một con nhưng hai bag khác nhau quá nhiều, chèn các nút trung gian để mỗi bước chỉ đưa vào hoặc quên một đỉnh. Nếu một nút có hai con nhưng bag của con chưa bằng bag của nó, chèn nút trung gian có bag bằng bag của cha. Nếu một nút có nhiều hơn hai con, tách dần thành cây nhị phân bằng các nút trung gian.

5.5.4 Tập độc lập lớn nhất

Cho đồ thị vô hướng $G = (V, E)$, một tập $I \subseteq V$ gọi là độc lập nếu mọi $u, v \in I$ đều không có cạnh nối. Biết một phân rã cây hoàn hảo độ rộng w là hằng số, ta giải tập độc lập lớn nhất bằng DP từ dưới lên.

Với mỗi nút i và mỗi $S \subseteq X_i$, đặt $\text{dp}_{i,S}$ là kích thước tập độc lập lớn nhất trong phần cây con của i , với điều kiện các đỉnh được chọn trong bag X_i đúng là S . Nếu S không độc lập thì trạng thái vô hiệu.

Các chuyển tiếp:

- nút lá: $\text{dp}_{i,\emptyset} = 0$ và $\text{dp}_{i,X_i} = 1$;
- nút đưa vào, với đỉnh đưa vào là v và con j :

$$\text{dp}_{i,S} = \text{dp}_{j,S \setminus \{v\}} + [v \in S];$$

- nút quên, với đỉnh bị quên là v và con j :

$$\text{dp}_{i,S} = \max(\text{dp}_{j,S}, \text{dp}_{j,S \cup \{v\}});$$

- nút hợp, với hai con j, k :

$$\text{dp}_{i,S} = \text{dp}_{j,S} + \text{dp}_{k,S} - |S|.$$

Độ phức tạp là $O(2^w w^2 n)$.

5.5.5 Đường đi ngắn nhất

Xét đồ thị vô hướng không trọng số và nhiều truy vấn đường đi ngắn nhất giữa s, t . Với phân rã cây (X, T) , gọi m là LCA của top_s và top_t trong T .

Định lý 5.7. Nếu xóa mọi đỉnh trong X_m khỏi G , thì s, t không còn liên thông.

Do đó mọi đường đi ngắn nhất từ s tới t phải đi qua ít nhất một đỉnh trong X_m , và

$$\text{dist}(s, t) = \min_{i \in X_m} \text{dist}(s, i) + \text{dist}(i, t).$$

Nếu cây phân rã có chiều cao h và độ rộng w , ta có thể BFS để tiền xử lý khoảng cách cho các cặp có quan hệ tổ tiên trong $O(hn)$, rồi trả lời mỗi truy vấn trong $O(w)$.

5.6 Thuật toán phân rã cây của Bodlaender

Phần này phác thảo thuật toán tuyến tính của Bodlaender năm 1996 cho đồ thị có treewidth bị chặn. Thuật toán đầy đủ khá dài; bài viết chỉ nêu các ý tưởng và sản phẩm phụ quan trọng.

Bổ đề 6.1.1. Với hai hằng số bị chặn k, l , nếu đã có phân rã cây độ rộng l của G , thì có thể kiểm tra $\text{tw}(G) \leq k$ hay không và nếu có thì dựng phân rã cây độ rộng không quá k , trong thời gian tuyến tính.

Dựa trên bổ đề này, cần chứng minh rằng với hằng số k , ta có thể hoặc kết luận $\text{tw}(G) > k$, hoặc tuyến tính thu nhỏ đồ thị theo một tỉ lệ cố định rồi đệ quy. Thuật toán xét hai trường hợp:

1. Nếu có đủ nhiều “đỉnh xanh”, cực đại matching của G đủ lớn; thu gọn các cạnh trong matching rồi đệ quy.
2. Nếu không, sau khi thêm một số cạnh không làm đổi treewidth, sẽ có đủ nhiều “đỉnh I-simplex”; xóa các đỉnh này rồi đệ quy.

5.6.1 Chuẩn bị

Bổ đề 6.2.1. Với một phân rã cây (X, T) của $G = (V, E)$:

1. nếu $W \subseteq V$ tạo thành clique trong G , thì tồn tại $i \in I$ sao cho $W \subseteq X_i$;
2. nếu $W_1, W_2 \subseteq V$ tạo thành đồ thị hai phía đầy đủ, thì tồn tại $i \in I$ sao cho $W_1 \subseteq X_i$ hoặc $W_2 \subseteq X_i$.

Bổ đề 6.2.2. Nếu tồn tại bag chứa cả u và v , thì (X, T) cũng là phân rã cây của đồ thị $G + (u, v)$.

Bổ đề 6.2.3. Nếu $|N_G(u) \cap N_G(v)| > k$, thì

$$\text{tw}(G) \leq k \iff \text{tw}(G + (u, v)) \leq k.$$

Lý do là nếu hai đỉnh có quá nhiều láng giềng chung, trong mọi phân rã cây độ rộng k hoặc phải có bag chứa u, v , hoặc tạo ra clique quá lớn.

Định nghĩa 6.2.1 (phân rã cây đều). Một phân rã cây (X, T) của $G = (V, E)$ gọi là đều nếu với mọi $i \in I$, $|X_i| = k + 1$, và với mọi cạnh cây $(i, j) \in F$, $|X_i \cap X_j| = k$.

Từ một phân rã cây độ rộng bị chặn, có thể biến đổi tuyến tính thành phân rã cây đều cùng độ rộng.

5.6.2 Đỉnh xanh

Chọn hai hằng số $0 < c_1, c_2 < 1$. Đặt

$$d = \max \left(k^2 + 4k + 4, \left\lceil \frac{2k}{c_1} \right\rceil \right).$$

Đỉnh có bậc không quá d gọi là đỉnh bậc thấp; đỉnh có bậc lớn hơn d gọi là đỉnh bậc cao. Một đỉnh bậc thấp kề với ít nhất một đỉnh bậc thấp khác gọi là đỉnh xanh.

Bổ đề 6.3.1. Số đỉnh bậc cao nhỏ hơn $c_1|V|$. Điều này suy từ cận số cạnh của partial k -tree.

Bổ đề 6.3.2. Nếu G có n_f đỉnh xanh, thì mọi cực đại matching của G chứa ít nhất $n_f/(2d)$ cạnh.

Chứng minh. Với một cực đại matching M , mỗi đỉnh xanh hoặc đã là đầu mút của cạnh trong M , hoặc kề với một đỉnh xanh nằm trên M ; nếu không, matching chưa cực đại. Gán đóng góp của đỉnh xanh chưa ghép cho một đỉnh xanh đã ghép kề nó. Mỗi đỉnh như vậy nhận nhiều nhất d đóng góp, nên M có ít nhất n_f/d đỉnh xanh và ít nhất $n_f/(2d)$ cạnh.

Thu gọn mọi cạnh trong một matching M được G' . Thu gọn cạnh không tăng treewidth; ngược lại, từ phân rã cây độ rộng k của G' có thể phục hồi phân rã cây độ rộng $2k + 1$ của G bằng cách thay mỗi đỉnh thu gọn bằng hai đầu mút cũ. Sau đó dùng Bổ đề 6.1.1 để giảm độ rộng về k . Vì vậy khi có đủ nhiều đỉnh xanh, ta thu gọn matching và đệ quy.

5.6.3 Đỉnh I-simplex

Định nghĩa 6.4.1 (đồ thị bổ sung). Với mọi cặp u, v có ít nhất $k + 1$ láng giềng chung bậc thấp, thêm cạnh (u, v) . Đồ thị thu được gọi là đồ thị bổ sung G' . Theo Bổ đề 6.2.3, $\text{tw}(G) \leq k$ khi và chỉ khi $\text{tw}(G') \leq k$.

Định nghĩa 6.4.2. Một đỉnh v gọi là I-simplex nếu trong đồ thị bổ sung, các láng giềng của nó tạo thành clique, và trong đồ thị gốc nó là đỉnh bậc thấp nhưng không phải đỉnh xanh.

Định nghĩa 6.4.3. Với một phân rã cây (X, T) của G , một đỉnh bậc thấp không xanh v gọi là đỉnh đỏ nếu tồn tại $i \in I$ sao cho $N_G(v) \subseteq X_i$. Đỉnh đỏ chỉ dùng trong phân tích, thuật toán không cần tính chúng trực tiếp.

Bài viết chứng minh một chuỗi bổ đề: trong phân rã cây đều, mỗi lá của T và mỗi đường dài đủ lớn gồm các nút bậc 2 đều bảo đảm tồn tại một đỉnh xanh hoặc đỏ xuất hiện cục bộ; mọi cây có nhiều lá hoặc nhiều đường như vậy; số tập quan trọng tạo bởi các đỉnh bậc cao không vượt quá số đỉnh bậc cao. Từ đó suy ra nếu đỉnh xanh không đủ nhiều, thì phải có đủ nhiều đỉnh I-simplex.

Cụ thể, nếu $G = (V, E)$ có $\text{tw}(G) \leq k$, có n_f đỉnh xanh và n_h đỉnh bậc cao, thì số đỉnh I-simplex ít nhất là

$$\frac{|V|}{2k^2 + 6k + 8} - 1 - n_f - \frac{1}{2}k^2(k + 1)n_h.$$

Bổ đề 6.4.5. Giả sử xóa mọi đỉnh I-simplex khỏi đồ thị bổ sung được G' , và (X, T) là một phân rã cây của G' . Với mỗi đỉnh I-simplex bị xóa v , tồn tại bag X_i sao cho $N_G(v) \subseteq X_i$.

Do đó từ phân rã cây của đồ thị đã xóa các đỉnh I-simplex, ta phục hồi nhanh phân rã cây cùng độ rộng của đồ thị ban đầu bằng cách thêm cho mỗi v một bag $\{v\} \cup N_G(v)$ gắn vào bag chứa $N_G(v)$.

5.6.4 Luồng thuật toán

Bài toán cần giải là: cho đồ thị vô hướng $G = (V, E)$ và hằng số k , hãy dựng phân rã cây độ rộng không quá k , hoặc báo rằng $\text{tw}(G) > k$. Nếu $|V|$ nhỏ hơn một hằng số đủ lớn, dùng tìm kiếm trực tiếp.

Trước hết kiểm tra

$$|E| \leq k|V| - \frac{1}{2}k(k + 1).$$

Nếu không thỏa, chắc chắn vô nghiệm.

Nếu số đỉnh xanh ít nhất $|V|/(4k^2 + 12k + 16)$:

- tìm một cực đại matching M ;
- thu gọn các cạnh trong M để được G' ;
- đệ quy trên G' , nếu vô nghiệm thì G cũng vô nghiệm;
- phục hồi phân rã cây độ rộng $2k + 1$ của G ;
- dùng Bổ đề 6.1.1 để giảm về độ rộng k .

Ngược lại:

- dựng đồ thị bổ sung G' ;
- nếu tồn tại đỉnh I-simplex có bậc lớn hơn k , báo vô nghiệm;
- lấy tập S gồm mọi đỉnh I-simplex; nếu $|S| < c_2|V|$, báo vô nghiệm;
- đệ quy trên đồ thị xóa S ;
- với mỗi $v \in S$, thêm một bag $\{v\} \cup N_G(v)$ nối vào bag chứa $N_G(v)$.

Trong trường hợp thứ nhất, ta xóa được ít nhất một tỉ lệ cố định số đỉnh nhờ matching. Trong trường hợp thứ hai, ta xóa được ít nhất $c_2|V|$ đỉnh. Vì mỗi lần đệ quy đều thu nhỏ kích thước xuống còn nhiều nhất c_4n với $c_4 < 1$, độ phức tạp thỏa $T(n) = T(c_4n) + O(n)$, tức là tuyến tính.

5.7 Tổng kết

Bài viết xuất phát từ một số đồ thị đặc biệt, phân tích đặc trưng và tính chất của treewidth, giới thiệu ý nghĩa của treewidth bị chặn trong việc giải các bài toán kinh điển, rồi trình bày tư tưởng cốt lõi của một thuật toán phân rã cây mạnh. Nội dung chỉ mang tính bán phổ biến kiến thức; các liên hệ rộng hơn và những phương pháp phân rã mạnh hơn vẫn còn chờ độc giả tiếp tục tìm hiểu. Tác giả hy vọng treewidth sẽ được ứng dụng nhiều hơn trong thi tin học.

5.8 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Wang Xingming của Trường Thập Nhất Bắc Kinh đã quan tâm và chỉ dẫn; cảm ơn gia đình đã ủng hộ; và cảm ơn các bạn đã trao đổi, thảo luận cùng tác giả.

5.9 Tài liệu tham khảo

- [1] Stefan Arnborg, Derek G. Corneil, and Andrzej Proskurowski. *Complexity of finding embeddings in a k -tree*. SIAM Journal on Algebraic and Discrete Methods, 8:277–284, 1987.
- [2] Hans L. Bodlaender. *A linear-time algorithm for finding tree-decompositions of small treewidth*. SIAM Journal on Computing, 25(6):1305–1317, 1996.
- [3] Hans L. Bodlaender, Pal Grønås Drange, Markus S. Dregi, Fedor V. Fomin, Daniel Lokshtanov, and Michał Pilipczuk. *An $O(c^k n)$ 5-approximation algorithm for treewidth*. FOCS, 2013.
- [4] Uriel Feige, Mohammad Taghi Hajiaghayi, and James R. Lee. *Improved approximation algorithms for minimum-weight vertex separators*. STOC, 2005.
- [5] Bruno Courcelle. *The monadic second-order logic of graphs I: recognizable sets of finite graphs*. Information and Computation, 85(1):12–75, 1990.
- [6] Hans L. Bodlaender and Ton Kloks. *Better algorithms for the pathwidth and treewidth of graphs*. ICALP, 1991.
- [7] Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, and Qing Zhu. *When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks*. SIGMOD, 2018.
- [8] Wikipedia. *Treewidth*.

6 Bao hàm–loại trừ trên quan hệ đôi một khác nhau

THPT trực thuộc Đại học Sư phạm Hoa Nam
Zhou Ziheng

Tóm tắt

Bài viết đưa ra một cách dùng nguyên lý bao hàm–loại trừ khi cần xử lý điều kiện “các phần tử đôi một khác nhau”. Ý tưởng là trước hết bỏ điều kiện khác nhau bằng cách xét các bộ có thứ tự, sau đó bao hàm–loại trừ trên những cặp bị ép bằng nhau. Qua một số bài toán đếm theo modulo và xor, bài viết minh họa cách biến ràng buộc khác nhau thành một tổng trên các đồ thị có số thành phần liên thông nhất định.

6.1 Mở đầu

Trong các bài toán đếm của OI, ta thường gặp điều kiện các phần tử được chọn phải đôi một khác nhau. Thông thường điều kiện này dẫn tới nhiều phân tích và biến đổi chi tiết. Bài viết giới thiệu một cách tiếp cận dựa trên bao hàm–loại trừ: ép một số cặp vị trí bằng nhau, xem các quan hệ ép bằng nhau như cạnh của một đồ thị, rồi đếm theo các thành phần liên thông của đồ thị đó.

Ký hiệu $a \equiv b \pmod{n}$ nghĩa là tồn tại số nguyên k sao cho $a + kn = b$. Ký hiệu $\binom{m}{n}$ là số cách chọn n vật từ m vật, không xét thứ tự:

$$\binom{m}{n} = \frac{m!}{n!(m-n)!} \quad (0 \leq n \leq m).$$

6.2 Bài toán dẫn nhập

6.2.1 Bài toán

Cho số nguyên tố lẻ p và số nguyên dương k . Hỏi trong tập

$$\{1, 2, \dots, kp\}$$

có bao nhiêu tập con kích thước p sao cho tổng các phần tử chia hết cho p ?

Bài toán này có một lời giải trực tiếp đơn giản; phần này dùng nó để giới thiệu cách bao hàm–loại trừ trên quan hệ khác nhau.

6.2.2 Chuyển sang bộ có thứ tự

Xét bài toán tương đương theo bộ có thứ tự: đếm số bộ số nguyên dương $(x_1, x_2, \dots, x_p)$ thỏa:

- $1 \leq x_i \leq kp$;
- với mọi $i \neq j$, $x_i \neq x_j$;
- $\sum_{i=1}^p x_i \equiv 0 \pmod{p}$.

Mỗi tập con p phần tử tương ứng đúng $p!$ hoán vị, nên đáp án của bài toán bộ có thứ tự bằng $p!$ lần đáp án gốc.

Ta áp dụng bao hàm–loại trừ lên điều kiện $x_i \neq x_j$. Với mỗi cặp (i, j) , nếu chọn cặp đó trong bao hàm–loại trừ, ta ép $x_i = x_j$. Xem các cặp bị ép bằng nhau là các cạnh của đồ thị đầy đủ K_p . Gọi E_p là tập cạnh của K_p ; với $E \subseteq E_p$, ký hiệu $V(E)$ là số bộ thỏa điều kiện miền giá trị và điều kiện tổng, sau khi ép mọi cạnh trong E có hai đầu bằng nhau. Khi đó đáp án bộ có thứ tự là

$$\sum_{E \subseteq E_p} (-1)^{|E|} V(E).$$

6.2.3 Tính $V(E)$ và nâng chiều

Gọi $c(E)$ là số thành phần liên thông của đồ thị tạo bởi E . Khi đó

$$V(E) = \begin{cases} kp, & c(E) = 1, \\ (kp)^{c(E)-1} k, & c(E) > 1. \end{cases}$$

Nếu $c(E) = 1$, mọi biến đều bằng nhau và có k cách chọn. Nếu $c(E) > 1$, kích thước mỗi thành phần liên thông đều nguyên tố cùng nhau với p . Chọn tùy ý giá trị của mọi thành phần trừ một thành phần; thành phần còn lại bị ràng buộc vào đúng một lớp dư modulo p , nên có k cách chọn trong $1, \dots, kp$.

Trực tiếp thay công thức trên vào vẫn khó tính. Ta xét rộng hơn: với $t \in \{0, 1, \dots, p-1\}$, đặt $V(E, t)$ là số bộ sau khi ép bằng nhau bởi E và có tổng $\equiv t \pmod{p}$. Khi đó

$$V(E, t) = \begin{cases} kp, & c(E) = 1, t = 0, \\ 0, & c(E) = 1, t > 0, \\ (kp)^{c(E)-1} k, & c(E) > 1. \end{cases}$$

Gọi a_t là đáp án của bài toán bộ có thứ tự với tổng dư t . Ta có

$$a_t = \sum_{E \subseteq E_p} (-1)^{|E|} V(E, t).$$

Dễ thấy $a_1 = a_2 = \dots = a_{p-1}$, và

$$a_0 - a_1 = kp \sum_{\substack{E \subseteq E_p \\ c(E)=1}} (-1)^{|E|}.$$

Mặt khác,

$$a_0 + a_1 + \dots + a_{p-1} = \binom{kp}{p} p!.$$

Vì vậy chỉ cần tính

$$f_p = \sum_{\substack{E \subseteq E_p \\ c(E)=1}} (-1)^{|E|}.$$

6.2.4 Tính f_n

Đặt

$$g_n = \sum_{E \subseteq E_n} (-1)^{|E|}.$$

Ta tính f_n bằng cách xét phần bù: nếu đồ thị không liên thông, liệt kê kích thước i của thành phần liên thông chứa đỉnh 1. Khi đó

$$f_n = g_n - \sum_{i=1}^{n-1} \binom{n-1}{i-1} f_i g_{n-i}.$$

Ở đây ta chọn $i-1$ đỉnh đi cùng đỉnh 1, phần đó phải liên thông và phần còn lại nối cạnh tùy ý.

Vì $g_1 = 1$ và $g_i = 0$ với $i \geq 2$, suy ra

$$f_n = -(n-1)f_{n-1}.$$

Do đó

$$f_n = (-1)^{n-1} (n-1)!.$$

Thay vào bài toán ban đầu, do p lẻ nên $(-1)^{p-1} = 1$, ta thu được

$$a_0 = \frac{\binom{kp}{p} p! + kp(p-1)(p-1)!}{p}.$$

Đáp án cho tập con không thứ tự là

$$\frac{a_0}{p!} = \frac{\binom{kp}{p} + k(p-1)}{p}.$$

Từ cách làm trên cũng có thể suy ra: với mọi số nguyên c không chia hết cho p , trong các tập con kích thước c của $\{1, 2, \dots, kp\}$, số tập có tổng thuộc từng lớp dư modulo p là bằng nhau.

6.2.5 So sánh với cách làm khác

Cách giải đơn giản hơn là xếp toàn bộ pk số thành bảng p hàng, k cột; ô hàng i , cột j chứa số $(j-1)k+i$. Với mỗi tập con p phần tử không nằm trọn trong một cột, lấy cột nhỏ nhất có phần tử được chọn rồi lần lượt dịch các phần tử trong cột đó xuống $1, 2, \dots, p-1$ bước theo vòng. Nhóm p tập thu được có tổng lần lượt phủ đủ các lớp dư modulo p . Chỉ còn k tập nằm trọn trong một cột, và vì p lẻ nên tổng của chúng chia hết cho p .

Cách này ngắn và ít tính toán, nhưng kém mở rộng. Phần sau cho thấy phương pháp bao hàm–loại trừ trên quan hệ khác nhau có thể xử lý nhiều biến thể hơn.

6.3 Các ứng dụng khác

6.3.1 Bài toán xor

Cho số nguyên dương n, k . Hỏi tập

$$\{0, 1, \dots, 2^k - 1\}$$

có bao nhiêu tập con kích thước n có xor bằng 0?

Đặt $m = 2^k$. Tương tự phần trước, trước hết xét bộ có thứ tự, rồi bao hàm–loại trừ trên đồ thị ép bằng nhau. Với $E \subseteq E_n$, ký hiệu $V(E, t)$ là số bộ có xor bằng t . Khi đó

$$V(E, t) = \begin{cases} m^{c(E)}, & \text{mọi thành phần liên thông của } E \text{ có kích thước chẵn, } t = 0, \\ 0, & \text{mọi thành phần liên thông của } E \text{ có kích thước chẵn, } t > 0, \\ m^{c(E)-1}, & \text{tồn tại thành phần liên thông kích thước lẻ.} \end{cases}$$

Gọi a_t là đáp án bộ có thứ tự với xor bằng t . Ta có $a_1 = \dots = a_{m-1}$ và

$$a_0 + a_1 + \dots + a_{m-1} = \binom{m}{n} n!.$$

Vì vậy chỉ cần tính $a_0 - a_1$:

$$a_0 - a_1 = \sum_{\substack{E \subseteq E_n \\ \text{mọi thành phần liên thông có kích thước chẵn}}} (-1)^{|E|} m^{c(E)}.$$

Gọi giá trị này là h_n .

Liệt kê kích thước i của thành phần chứa đỉnh 1. Chỉ các i chẵn đóng góp, và với $f_i = (-1)^{i-1}(i-1)!$, ta có

$$h_n = \sum_{\substack{2 \leq i \leq n \\ 2|i}} m \binom{n-1}{i-1} f_i h_{n-i} = (n-1)! \sum_{\substack{2 \leq i \leq n \\ 2|i}} (-m) \frac{h_{n-i}}{(n-i)!}.$$

Đặt hàm sinh mũ

$$h(x) = \sum_{i \geq 0} \frac{h_i}{i!} x^i.$$

Từ truy hồi suy ra

$$h' = -m(x + x^3 + x^5 + \dots)h = -\frac{m}{2} \left(\frac{1}{1-x} - \frac{1}{1+x} \right) h.$$

Giải phương trình vi phân và dùng $h_0 = 1$:

$$h(x) = (1-x^2)^{m/2}.$$

Do đó

$$h_n = \begin{cases} (-1)^{n/2} \binom{m/2}{n/2} n!, & n \equiv 0 \pmod{2}, \\ 0, & n \equiv 1 \pmod{2}. \end{cases}$$

Thay $h_n = a_0 - a_1$ và tổng các a_i vào là tính được đáp án. Với cách làm truyền thống, công thức này không dễ suy ra.

6.3.2 Mở rộng kết luận của phần trước

Cho số nguyên tố lẻ p và số nguyên dương k, n . Hỏi trong $\{1, 2, \dots, kp\}$ có bao nhiêu tập con kích thước n có tổng chia hết cho p ?

Bằng cách xếp bảng ở trên có thể thấy: nếu $p \nmid n$, số tập thuộc từng lớp dư theo tổng là bằng nhau; nếu $p \mid n$, số tập có tổng dư 0 nhiều hơn mỗi lớp dư khác đúng $\binom{k}{n/p}$. Ta kiểm chứng kết luận này bằng bao hàm–loại trừ.

Đặt $m = kp$. Vì n có thể lớn hơn p , với $E \subseteq E_n$ ta có

$$V(E, t) = \begin{cases} m^{c(E)}, & \text{mọi thành phần liên thông có kích thước là bội của } p, t = 0, \\ 0, & \text{mọi thành phần liên thông có kích thước là bội của } p, t > 0, \\ m^{c(E)-1}, & \text{tồn tại thành phần liên thông có kích thước không chia hết cho } p. \end{cases}$$

Tiếp tục đặt $h_n = a_0 - a_1$. Tương tự phần xor:

$$h_n = \sum_{\substack{p \leq i \leq n \\ p \nmid i}} (-1)^{i-1} m(i-1)! \binom{n-1}{i-1} h_{n-i} = (n-1)! \sum_{\substack{p \leq i \leq n \\ p \nmid i}} (-1)^{i-1} m \frac{h_{n-i}}{(n-i)!}.$$

Với hàm sinh mũ $h(x) = \sum_{i \geq 0} h_i x^i / i!$, ta nhận được

$$h' = m(x^{p-1} - x^{2p-1} + x^{3p-1} - x^{4p-1} + \dots)h = m \frac{x^{p-1}}{1+x^p} h.$$

Suy ra

$$h(x) = (1+x^p)^{m/p}.$$

Vì thế

$$h_n = \begin{cases} \binom{m/p}{n/p} n!, & p \mid n, \\ 0, & p \nmid n. \end{cases}$$

Kết quả này đúng với kết luận thu được từ cách xếp bảng.

6.4 Kết luận

Qua các bài toán trên, bài viết phác họa cách dùng bao hàm–loại trừ cho điều kiện đôi một khác nhau. Nội dung chỉ gồm vài ví dụ đơn giản, nhưng hy vọng đủ để gợi mở thêm các ứng dụng khác của ý tưởng này.

6.5 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang đã hướng dẫn; cảm ơn các thầy Liang Zexian, He Zelin và Huang Binggang đã bồi dưỡng, chỉ dạy; và cảm ơn cha mẹ đã thấu hiểu, ủng hộ.

7 Báo cáo ra đề Nhà ảo thuật

Trường Trung học Chư Ky, Chiết Giang
Meng Yuhao

Tóm tắt

Nhà ảo thuật là một bài do tác giả ra cho đợt thi thử nội bộ của đội tuyển quốc gia Trung Quốc IOI 2023. Bài yêu cầu thí sinh sắp xếp một hoán vị bằng các thao tác đảo đoạn có độ dài cố định, đồng thời số thao tác càng ít càng tốt. Đây là một bài xây dựng. Bài viết trình bày điều kiện cần và đủ để có lời giải, đưa ra vài cách xây dựng và phân tích số thao tác của từng cách. Qua quá trình ra đề và giải bài, tác giả cũng tổng kết một hướng suy nghĩ thường gặp cho các bài xây dựng trong thi tin học.

7.1 Đề bài

7.1.1 Mô tả

King là một nhà ảo thuật. Anh có một chồng n lá bài úp mặt xuống. Mỗi lá được gán một số hiệu khác nhau từ 1 đến n ; ban đầu lá thứ i tính từ trên xuống có số hiệu p_i . King muốn sau một số thao tác, lá thứ i từ trên xuống có số hiệu đúng bằng i .

Để động tác trông tự nhiên, mỗi thao tác của King có thể mô hình hóa như sau:

1. nhấc một chồng bài từ trên cùng đặt lên bàn, gọi là chồng A ; chồng này có thể rỗng;
2. từ phần còn lại, chia lần lượt m lá bài úp mặt xuống lên bàn, thu được chồng B ;
3. đặt chồng B trở lại đỉnh chồng bài;
4. đặt chồng A trở lại đỉnh chồng bài.

Hãy giúp King đưa ra một phương án thao tác, hoặc kết luận rằng không tồn tại phương án như vậy.

7.1.2 Dữ liệu vào

Dòng đầu gồm ba số nguyên T, n, m , lần lượt là số hiệu gói kiểm thử, số lá bài và số lá được chia trong mỗi thao tác. Dòng thứ hai gồm n số nguyên $p_1, p_2, \dots, p_n$, biểu diễn số hiệu các lá từ trên xuống.

7.1.3 Dữ liệu ra

Nếu không có phương án, in -1 . Ngược lại, dòng đầu in một số nguyên không âm k là số thao tác. Trong k dòng tiếp theo, dòng thứ i in một số nguyên không âm c_i với $0 \leq c_i \leq n - m$, là số lá trong chồng A của thao tác thứ i .

7.1.4 Ví dụ

Ví dụ thứ nhất có $n = 8, m = 4$ và hoán vị

6, 2, 5, 1, 7, 4, 3, 8.

Một phương án hợp lệ dùng bốn thao tác với các giá trị

0, 2, 3, 1.

Ví dụ thứ hai có $n = 6, m = 5$ và hoán vị

$$3, 4, 1, 5, 6, 2,$$

kết quả là -1 .

7.1.5 Chấm điểm và giới hạn

Mỗi gói kiểm thử có các tham số $\lim_1, \dots, \lim_5$. Nếu một test vô nghiệm mà thí sinh in ra có nghiệm thì được 0 điểm; nếu kết luận vô nghiệm đúng thì được trọn điểm của gói. Nếu test có nghiệm, lời giải sai hoặc in vô nghiệm được 0 điểm; ngược lại, với số thao tác là k , điểm nhận được là

$$\frac{S}{5} \sum_{i=1}^5 [k \leq \lim_i].$$

Điểm của một gói là điểm nhỏ nhất trên các test trong gói đó.

Với toàn bộ dữ liệu, $1 \leq m \leq n \leq 1000$, và p là một hoán vị của $1, \dots, n$. Với những dữ liệu có nghiệm, hoán vị được sinh ngẫu nhiên trong tập hoán vị có nghiệm. Các gói lẻ bảo đảm m lẻ, các gói chẵn bảo đảm m chẵn. Giới hạn thời gian là 1 giây, riêng các gói 1, 2, 11, 12 là 2 giây; giới hạn bộ nhớ là 512 MB.

7.2 Ký hiệu

Ta dùng các ký hiệu sau:

- $\text{rev}_m(l)$: đảo đoạn độ dài m bắt đầu tại vị trí l ;
- $\text{lshift}_{m,k}(l)$: xoay trái k bước đoạn độ dài m bắt đầu tại l ;
- $\text{rshift}_{m,k}(l)$: xoay phải k bước đoạn độ dài m bắt đầu tại l ;
- $\text{shift3}(x_0, x_1, x_2)$: đồng thời với mọi $0 \leq i \leq 2$, đưa giá trị ở vị trí x_i sang vị trí $x_{(i+1) \bmod 3}$.

Ký hiệu $A \Rightarrow B$ nghĩa là các thao tác trong A có thể xây dựng các thao tác trong B ; $A \Leftrightarrow B$ nghĩa là hai chiều đều đúng.

Với hoán vị $\pi = \{p_1, p_2, \dots, p_n\}$, $\pi(i) = p_i$; π^{-1} là hoán vị ngược, và $\pi \circ g$ là hoán vị sau khi thực hiện thao tác g .

7.3 Lời giải

7.3.1 Rút gọn bài toán

Bản chất của bài là: cho một hoán vị π và độ dài thao tác m , hãy dùng càng ít thao tác rev_m càng tốt để sắp xếp hoán vị.

7.3.2 Trường hợp $n = m$

Khi $n = m$, chỉ có hai hoán vị hợp lệ:

$$\{1, 2, \dots, n\} \quad \text{và} \quad \{n, n-1, \dots, 1\}.$$

Hoán vị đầu không cần thao tác, hoán vị sau chỉ cần thực hiện $\text{rev}_m(1)$.

7.3.3 Trường hợp $n = m + 1$

Ta có

$$\text{rev}_m \Rightarrow \text{lshift}_{m+1,2} \Rightarrow \text{lshift}_{m+1,2k},$$

và tương tự cho xoay phải. Cụ thể, hai thao tác $\text{rev}_m(l+1)$ rồi $\text{rev}_m(l)$ tạo ra $\text{lshift}_{m+1,2}(l)$; đổi thứ tự hai lần đảo sẽ tạo ra $\text{rshift}_{m+1,2}(l)$.

Ngoài ra,

$$\text{lshift}_{m,2k} \Leftrightarrow \text{rshift}_{m,2k}.$$

Nếu m chẵn thì chỉ cần dùng quan hệ giữa xoay trái và xoay phải trên một vòng; nếu m lẻ, xoay chẵn bước sinh được các xoay cần thiết.

Vì hai lần đảo cùng một vị trí liên tiếp là vô nghĩa, khi $n = m + 1$, hoán vị cuối cùng chỉ có thể là hoán vị ban đầu sau một lần $\text{lshift}_{m+1,2k}$, rồi có hoặc không có thêm một rev_m . Do đó chỉ cần liệt kê các khả năng này để xét nghiệm và chọn phương án ngắn.

7.3.4 $n \geq m + 2$ và m chẵn

Khi m chẵn,

$$\text{lshift}_{m+1,2k} \Leftrightarrow \text{lshift}_{m+1,k}.$$

Trước hết có thể đưa lần lượt các số $m+1, \dots, n$ về đúng chỗ từ lớn xuống nhỏ. Giả sử các vị trí $i+1, \dots, n$ đã đúng và đặt $x = \pi^{-1}(i)$. Nếu $x+m-1 \leq i$, thực hiện $\text{rev}_m(x)$ để đẩy i sang phải rồi lặp lại. Nếu không, dùng một thao tác xoay phải $\text{rshift}_{m,i-x}(x)$ để đưa i về vị trí i . Cách này có kỳ vọng khoảng

$$\frac{n^2}{4(m-1)} + \frac{(n-m)m}{2}$$

thao tác đảo.

Khi $i > 2m$, có thể cải tiến: nếu x và i khác tính chẵn lẻ, thực hiện $\text{rev}_m(i-m+1)$ để đổi chẵn lẻ rồi xét lại. Nếu cùng chẵn lẻ, đặt

$$l = i - m + 1 - \frac{i-x}{2}.$$

Nếu $l > 0$, thực hiện $\text{rev}_m(l)$ rồi $\text{rev}_m(i-m+1)$ để hoàn tất; nếu không thì quay lại cách xoay phải. Kỳ vọng giảm xuống

$$\frac{n^2}{4(m-1)} + \frac{m \min(n-m, m)}{2}.$$

Mấu chốt tiếp theo là xây dựng thao tác ba vòng:

$$\{\text{lshift}_{m+1,2}, \text{rshift}_{m+1,2}\} \Rightarrow \text{shift3}(1, m, m+2).$$

Thao tác này dùng hai bước: trước hết xoay phải đoạn bắt đầu tại 2, rồi xoay trái đoạn bắt đầu tại 1.

Hơn nữa, từ các xoay $\text{lshift}_{m+1,k}$ có thể tạo ra mọi $\text{shift3}(x_0, x_1, x_2)$ với ba vị trí phân biệt trong $1, \dots, m+2$. Ý tưởng là dùng các phép xoay đưa ba giá trị cần hoán chuyển về các vị trí chuẩn $1, m, m+2$, thực hiện thao tác chuẩn, rồi đảo ngược chuỗi chuẩn bị. Trường hợp $m = 2$ có thể kiểm chứng riêng bằng xây dựng trực tiếp hoặc bằng máy.

Nếu $m \equiv 0 \pmod{4}$, mỗi rev_m tương đương một số chẵn phép đổi chỗ, nên tính chẵn lẻ của số nghịch thế không đổi; do đó điều kiện cần là số nghịch thế ban đầu chẵn. Điều kiện này cũng đủ: đưa $m+3, \dots, n$ về đúng chỗ, rồi dùng các thao tác shift3 để đưa $1, \dots, m$ về đúng chỗ. Hai vị trí cuối còn lại khi đó cũng đúng do chẵn lẻ nghịch thế.

Nếu $m \equiv 2 \pmod{4}$, mỗi lần đảo đổi chẵn lẻ nghịch thế. Sau khi đưa $m+3, \dots, n$ về đúng chỗ, nếu nghịch thế còn lẻ thì làm thêm một lần đảo bất kỳ để đổi chẵn lẻ, rồi dùng cùng cách với shift3. Vì vậy trong trường hợp này mọi hoán vị đều có nghiệm. Thuật toán trực tiếp từ chứng minh có chi phí kỳ vọng không quá

$$\frac{n^2}{4(m-1)} + \frac{m \min(n-m, m)}{2} + 10m^2$$

nếu bỏ qua các hạng tuyến tính; kết hợp với tìm kiếm rộng cho các kích thước nhỏ đã đủ đạt phần lớn điểm của nhóm m chẵn.

Có thể giảm mạnh phần $10m^2$. Ta không cần một shift3 tùy ý đầy đủ; chỉ cần các dạng như shift3(1, $m+1$, $m+2$) hoặc shift3(1, 2, $m+2$), vốn dựng được bằng ít thao tác hơn. Khi đang đưa $1, \dots, m$ về đúng chỗ, các số đã đúng luôn nằm trong một đoạn liên tục; vì thế luôn chọn được một trong hai thao tác ba vòng mà không phá đoạn này. Với cải tiến này, số thao tác xấu nhất khoảng $3m^2$, kỳ vọng khoảng $2m^2$.

Cách tốt hơn nữa là không trả các số đã đúng về vị trí cũ sau mỗi lần chuẩn bị, mà chỉ duy trì chúng thành một đoạn liên tục ở cuối dãy. Khi cần đưa i vào cuối đoạn đã đúng, nếu $\pi^{-1}(i) = 1$ thì dùng vài xoay để đặt i sát đoạn đó rồi đưa cả đoạn về cuối; nếu $\pi^{-1}(i) > 1$, trước hết xoay để đưa i tới vị trí $m+2$, sau đó xoay để đặt đoạn đã đúng ngay trước i . Trước thuật toán này, ta vẫn bảo đảm chẵn lẻ nghịch thế phù hợp. Sau đó chỉ cần đưa $3, \dots, m+2$ về đúng chỗ; hai số 1, 2 còn lại tự đúng.

Phân tích số thao tác cho bước này như sau. Nếu $\pi^{-1}(i) = 1$ và $i < m+2$, cần khoảng $m+4$ thao tác; nếu $i = m+2$ thì khoảng $3m$. Nếu $\pi^{-1}(i) > 1$ và là vị trí lẻ thì cần $m+2$ thao tác; nếu là vị trí chẵn thì cần m thao tác. Với dữ liệu ngẫu nhiên, trường hợp đầu xuất hiện kỳ vọng khoảng $\ln m$ lần. Dùng ước lượng phân phối nhị thức, khi $m = 998$, xác suất phần này vượt $998 \cdot 998 + 550 \cdot 2$ rất nhỏ, và vượt $998 \cdot 998 + 600 \cdot 2$ gần như bằng không. Thuật toán này vượt qua toàn bộ các gói m chẵn, kỳ vọng đạt 40 điểm.

7.3.5 $n \geq m+2$ và m lẻ

Khi m lẻ, tính chẵn lẻ của vị trí chứa mỗi số không đổi. Vì vậy điều kiện cần là với mọi i , $\pi(i)$ và i cùng chẵn lẻ. Phần dưới chỉ xét các hoán vị thỏa điều kiện này.

Ta tách dãy thành hai hàng:

$$\begin{array}{cccc} p_1 & p_3 & p_5 & \cdots \\ p_2 & p_4 & p_6 & \cdots \end{array}$$

Một thao tác rev_m đảo một vùng hình thang cân trên hai hàng; các thao tác xoay chẵn bước tương ứng với việc xoay hai phần của một vùng hình bình hành.

Trước hết xét $n = m+2$, $m > 3$ lẻ. Nếu sau một số thao tác phần dưới giữ nguyên, thì chẵn lẻ số nghịch thế của phần trên cũng giữ nguyên. Lý do là chọn hai số kề nhau trong phần dưới; một lần đảo luôn làm đổi quan hệ trước-sau của chúng, nên để phần dưới trở về ban đầu thì số lần đảo phải chẵn. Tùy $m \bmod 8$, một lần đảo hoặc không đổi, hoặc đổi đồng thời, hoặc đổi đúng một trong hai hàng; trong mọi trường hợp, khi phần dưới giữ nguyên thì chẵn lẻ phần trên cũng giữ.

Do đó với $n = m+2$, điều kiện có nghiệm gồm hai phần: các số chẵn kề nhau trong phần dưới phải tạo đúng vòng kề cần thiết; sau khi khôi phục phần dưới bằng một chuỗi thao tác bất kỳ, phần trên phải có số nghịch thế chẵn. Việc khôi phục phần dưới tương tự trường hợp $n = m+1$, còn phần trên có thể dùng thuật toán ba vòng đã nêu, trong khi giữ phần dưới không đổi.

Với $n \geq m+3$, ta vẫn dùng phương pháp đưa $m+1, \dots, n$ về đúng chỗ, vì ràng buộc chẵn lẻ vị trí khiến các xoay chẵn bước là đủ.

Điều kiện tồn tại lời giải theo $m \bmod 8$ là:

- nếu $m \equiv 1 \pmod{8}$, số nghịch thế của cả hai hàng đều phải chẵn;
- nếu $m \equiv 5 \pmod{8}$, hai hàng phải có cùng chẵn lẻ nghịch thế;
- nếu $m \equiv 3 \pmod{4}$, mọi hoán vị thỏa điều kiện cùng chẵn lẻ vị trí đều có thể sắp xếp.

Chúng minh đi theo invariant của nghịch thế hai hàng. Sau khi đưa $m + 4, \dots, n$ về đúng chỗ, ta có thể dùng tối đa vài lần đảo để chỉnh chẵn lẻ cho hai hàng, rồi khôi phục từng hàng bằng thao tác ba vòng. Trực tiếp làm như vậy đã cho một thuật toán đúng, nhưng số thao tác còn lớn.

Thiết lập quan hệ ghép cặp. Để giảm chi phí, thay vì giữ một hàng cố định trong khi khôi phục hàng kia, ta cố gắng sắp xếp trước thứ tự hai hàng sao cho khi một hàng được khôi phục thì hàng còn lại cũng khôi phục theo.

Giả sử $n = m + 2$. Với mỗi $1 \leq i \leq (m + 1)/2$, ghép cặp $2i - 1$ với $2i$, tức duy trì chúng kề nhau. Đặt các cặp đã ghép ở cuối dãy, trừ vị trí $m + 2$. Muốn ghép $p_2 - 1$ với p_2 , đặt $a = p_2 - 1$. Nếu $\pi^{-1}(a) = m + 2$, một $\text{lshift}_{m+1,2}(2)$ là đủ. Ngược lại, dùng $\text{lshift}_{m+1,\pi^{-1}(a)-1}(1)$ để đưa a tới vị trí 1, rồi dùng một xoay bắt đầu tại 2 để đưa p_2 về vị trí cũ, cuối cùng dùng $\text{lshift}_{m+1,2}(1)$ để đưa cặp mới ra cuối.

Khi chỉ còn một cặp chưa ghép, quy trình trên không áp dụng trực tiếp. Tuy nhiên, trong toàn bộ quá trình, chẵn lẻ nghịch thế của hai hàng thay đổi đồng thời; sau khi tất cả các cặp đã ghép, hai hàng có cùng chẵn lẻ. Vì vậy chỉ cần điều chỉnh hoán vị ban đầu để hai hàng có cùng chẵn lẻ nghịch thế. Trường hợp $n = m + 3$ cũng dùng được: xử lý $m + 2$ vị trí đầu, khi đó hai vị trí cuối tự khớp.

Sau khi thiết lập ghép cặp, coi mỗi cặp $(2i - 1, 2i)$ như một khối và dùng thuật toán tốt của trường hợp m chẵn để khôi phục các khối.

Tóm lại, thuật toán cho m lẻ gồm:

1. kiểm tra mỗi số có cùng chẵn lẻ với vị trí của nó hay không;
2. xử lý riêng trường hợp $n = m + 2$;
3. dựa vào $m \bmod 8$ và chẵn lẻ nghịch thế hai hàng để xét có nghiệm;
4. đưa $m + 4, \dots, n$ về đúng vị trí;
5. chỉnh hoán vị ban đầu để hai hàng có nghịch thế chẵn;
6. thiết lập các cặp ghép;
7. dùng thuật toán của mục trước để khôi phục toàn bộ hoán vị.

Kỳ vọng số thao tác để đưa $m + 4, \dots, n$ về đúng chỗ xấp xỉ

$$\frac{n^2}{4(m-1)} + \frac{m \min(n-m, m)}{4}.$$

Nếu đặt $z = (m - 1)/2$, tổng kỳ vọng cho quá trình ghép cặp và khôi phục cuối cùng xấp xỉ

$$8 \sum_{i=1}^z \frac{1}{i} \sum_{x=0}^{i-1} \min(x, z+1-x) \approx \left(\frac{3}{4} - \frac{\ln 2}{2} \right) m^2 \approx 0.403m^2.$$

So với thuật toán m chẵn, số thao tác dao động hơn nhưng giới hạn cũng nổi lỏng hơn. Có thể thực hiện thêm vài thao tác ngẫu nhiên ở đầu, chạy thuật toán nhiều lần rồi chọn phương án ngắn nhất.

7.4 Quá trình ra đề

7.4.1 Giải bài toán

Ban đầu tác giả không có nhiều ý tưởng. Tác giả dùng máy tính đếm số hoán vị có nghiệm với các giá trị n, m nhỏ, rồi lần lượt loại những hoán vị không thỏa các điều kiện cần để đoán điều kiện cần và đủ.

Sau đó tác giả phân tích từ những trường hợp có ít nghiệm. Trường hợp $n = m + 1$ cho ra các thao tác $\text{lshift}_{m+1,2}$ và $\text{rshift}_{m+1,2}$. Với $n \geq m + 2$ và $m \equiv 0 \pmod{4}$, điều kiện là số nghịch thế chẵn, gợi ý việc xây dựng thao tác shift_3 . Khi đó có thể dùng tìm kiếm rộng ở kích thước nhỏ để tìm mẫu thao tác có thể mở rộng cho m lớn.

Sau khi có một thuật toán, việc tiếp theo là quan sát cách hiện thực và tìm nút thắt để giảm số thao tác. Khi đã có thuật toán khá tốt cho m chẵn, tác giả chuyển sang m lẻ, vẫn bắt đầu từ trường hợp ít nghiệm $n = m + 2$ và mở rộng các kết luận của trường hợp m chẵn.

7.4.2 Hoàn thiện đề

Sau khi có lời giải, cách trình bày đề cũng cần cân nhắc. Nếu là bài truyền thống, số thao tác cuối cùng phụ thuộc mạnh vào n, m và hình dạng hoán vị, nên khó chia điểm. Nếu là bài nộp đáp án, vấn đề chia điểm dễ hơn, nhưng bài có nhiều trường hợp đặc biệt và nhiều tình huống vô nghiệm, trong khi số test của loại bài này thường ít, khó bao phủ đầy đủ.

Cuối cùng, tác giả kết hợp hai cách: cho phạm vi dữ liệu rõ ràng, đặt tham số chấm điểm cho số thao tác, và dùng gói test ràng buộc.

7.5 Tổng kết và triển vọng

Bài xây dựng là dạng thường gặp trong các kỳ thi tin học gần đây, nhưng không có một khuôn mẫu lời giải cố định và có thể chạm tới nhiều lĩnh vực khác nhau. Bài viết trình bày quá trình ra đề và lời giải của một bài xây dựng như vậy.

Với các bài yêu cầu biến đổi trạng thái ban đầu thành trạng thái mục tiêu bằng một số thao tác, ta có thể dùng máy tính đếm hoặc liệt kê trạng thái có nghiệm, từ đó đoán điều kiện cần và đủ. Sau đó nên bắt đầu từ các trường hợp đơn giản, dùng thao tác đã có để xây dựng thao tác mới hữu dụng hơn, hoặc suy ngược từ điều kiện tồn tại lời giải để biết cần xây dựng thao tác gì. Máy tính vẫn có thể hỗ trợ trong các bước tính toán, xây dựng và liệt kê.

Tác giả hy vọng *Nhà ảo thuật* và hướng suy nghĩ trong bài viết giúp người đọc xử lý các bài xây dựng tốt hơn, đồng thời gợi mở thêm nhiều đề bài xây dựng mới lạ.

7.6 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn cha mẹ đã quan tâm, ủng hộ; cảm ơn các thầy Yuan Rongle, Chen Heli và Dong Yehua đã bồi dưỡng, chỉ dạy; cảm ơn các bạn Chen Weitao và Lou Yuchen đã giúp đỡ trong quá trình ra đề; cảm ơn Lou Yuchen đã đọc và góp ý cho bài viết.

7.7 Tài liệu tham khảo

1. Wikipedia contributors. Binomial distribution, 2022.
2. Wikipedia contributors. Beta function, 2022.

8 Hình học trong OI

Trường Trung học Ba Thục, Trùng Khánh
Chang Ruinian

Tóm tắt

Các bài toán hình học thường đi kèm trực giác trực quan, còn trực giác trong OI thường gắn với tối ưu tổ hợp. Vì vậy hình học không phải mảng quá phổ biến trong OI, trong khi ở những kỳ thi như ACM, do yêu cầu cân bằng đề, thường vẫn có một bài hình học tính toán chất lượng. Tác giả cho rằng hình học nên được xem như một loại bài quan trọng trong OI, bởi không gian vật lý vốn liên tục và nhiều bài toán tối ưu trong đời sống có thể nhìn dưới góc độ hình học.

Bài viết thử tổng kết ở mức nhập môn một số vấn đề hình học trong OI. Phần đầu nói về hình học tính toán, mở rộng các khái niệm tam giác hóa, ứng dụng của nó và phép nghịch đảo tròn. Phần thứ hai trình bày các đối tượng hình học liên quan đến cấu trúc dữ liệu. Phần thứ ba giới thiệu hình học hữu hạn, phần trừu tượng nhất của bài viết.

8.1 Hình học tính toán

8.1.1 Giao nửa mặt phẳng

Một cách hiện thực tương đối chính xác cho giao nửa mặt phẳng dẫn tự nhiên tới đối ngẫu điểm–đường. Giả sử các đường thẳng biên không vuông góc trục x , nên mỗi nửa mặt phẳng có dạng $y \leq kx - b$ hoặc $y \geq kx - b$. Trong mặt phẳng mới tọa độ a, b , đường thẳng

$$l : y = kx - b$$

được biến thành điểm $l^* = (k, b)$, còn điểm $P = (x, y)$ được biến thành đường thẳng

$$P^* : b = xa - y.$$

Đây là phép đối ngẫu điểm–đường. Nó bảo toàn quan hệ liên thuộc: điểm nằm trên đường thì đối ngẫu của đường nằm trên đối ngẫu của điểm; ba điểm thẳng hàng biến thành ba đường đồng quy; điểm ở dưới đường biến thành đường đối ngẫu ở trên điểm đối ngẫu; và phép đối ngẫu hai lần trả về đối tượng ban đầu.

Với các nửa mặt phẳng $y \leq kx - b$, bài toán giao ban đầu chuyển thành bài toán xét các đường nằm phía trên tập điểm đối ngẫu, tức được mô tả bởi bao lồi trên của các điểm đối ngẫu. Tương tự, các nửa mặt phẳng $y \geq kx - b$ tương ứng với bao lồi dưới. Sau khi dựng hai bao, giao có rỗng hay không tương đương với việc tồn tại một đường tách hai bao, nên chỉ cần xét hai bao có cắt nhau không.

Nếu gặp đường thẳng thẳng đứng, trong nhiều bài dữ liệu là điểm nguyên, ta có thể xoay toàn bộ mặt phẳng một góc khó trùng với đường cũ, chẳng hạn nhân các điểm dạng số phức Gaussian với $1 + \sqrt{3}i$, rồi đưa $\sqrt{3}$ vào cấu trúc tính toán.

8.1.2 Lưới hóa mặt phẳng

Lưới hóa là một ý tưởng đơn giản nhưng rất hữu dụng. Ví dụ, với bài toán tìm cặp điểm gần nhất trong tập S , ta có thể dùng phương pháp tăng ngẫu nhiên: trộn ngẫu nhiên các điểm, thêm lần lượt, và nếu khoảng cách tốt nhất hiện tại là d , chia mặt phẳng thành các ô vuông cạnh d . Mỗi ô chỉ chứa $O(1)$ điểm, vì nếu không đáp án đã nhỏ hơn. Khi thêm điểm (x, y) ở ô (a, b) , chỉ cần xét các điểm trong những ô lân cận $(a \pm 1, b \pm 1)$. Nếu đáp án giảm, dựng lại bảng băm của lưới. Xác suất lần thêm thứ i làm giảm đáp án là $O(1/i)$, nên kỳ vọng tổng thời gian là $O(n)$.

Điểm đáng chú ý là ta thay hình tròn khó biểu diễn bằng ô vuông: vẫn chia phủ được toàn mặt phẳng, vẫn giữ một tính chất hình học đủ mạnh, và từ đó dùng cấu trúc dữ liệu để duy trì. Cùng tư tưởng này áp dụng cho tam giác chu vi nhỏ nhất, bài phủ điểm bằng các hình vuông đồng cạnh, hoặc các bài kiểu tìm bán kính nhỏ nhất để một đường tròn phủ đủ K điểm trong một đoạn chỉ số dài L . Với bài cuối, nhị phân bán kính r , chọn cạnh ô $d = \frac{\sqrt{2}}{2}r$, xét chín ô lân cận và làm quét góc trên tập ứng viên nhỏ, thu được độ phức tạp dạng $O(-\log \varepsilon \cdot nK^2 \log K)$.

8.1.3 Tam giác hóa Delaunay

Ở đây tam giác hóa chủ yếu chỉ tam giác hóa Delaunay. Một tam giác hóa thông thường là đồ thị phẳng trên tập điểm đã cho, mọi mặt hữu hạn đều là tam giác. Giả sử không có ba điểm thẳng hàng, không có bốn điểm đồng viên. Tam giác hóa Delaunay được đặc trưng bởi tính chất đường tròn rỗng: đường tròn ngoại tiếp mỗi mặt tam giác không chứa điểm nào khác. Một cạnh xuất hiện trong tam giác hóa khi và chỉ khi có một đường tròn rỗng nhận cạnh đó làm dây.

Các hệ quả quan trọng gồm: mỗi điểm nối cạnh với một điểm gần nhất; góc nhỏ nhất của các tam giác là lớn nhất theo thứ tự từ điển trong mọi tam giác hóa; và bán kính ngoại tiếp lớn nhất của các mặt là nhỏ nhất.

Cách dựng dễ nhớ là tăng ngẫu nhiên. Bắt đầu bằng một tam giác rất lớn chứa mọi điểm. Thêm điểm P vào mặt tam giác chứa nó, nối P với ba đỉnh để chia mặt. Nếu xuất hiện cạnh vi phạm tính chất đường tròn rỗng, thực hiện phép lật cạnh: trong tứ giác gồm hai tam giác kề, xóa đường chéo cũ và thêm đường chéo kia. Lặp cho đến khi mọi cạnh hợp lệ. Dây góc tăng theo thứ tự từ điển nên quá trình dừng.

Để định vị động điểm nằm trong mặt nào, có thể duy trì trong mỗi mặt danh sách các điểm chưa thêm nằm bên trong; khi lật cạnh hoặc chia mặt thì phân phối lại các điểm này. Phân tích ngẫu nhiên cho thấy số cạnh bị lật kỳ vọng mỗi lần thêm là $O(1)$, còn tổng chi phí bảo trì danh sách mặt là $O(n \log n)$. Đây cũng là bậc tối ưu: bằng cách đặt các điểm trên cung tròn đơn vị, bài toán tam giác hóa có thể mã hóa bài toán sắp xếp.

Một ứng dụng kinh điển là cây khung nhỏ nhất Euclid: mọi cạnh của cây khung nhỏ nhất đều nằm trong tam giác hóa Delaunay, nên chỉ cần dựng tam giác hóa rồi chạy MST trên số cạnh tuyến tính. Một ứng dụng khác là các bài tối ưu bán kính hình tròn hoặc matching hình học, nơi tính chất đường tròn rỗng giúp chứng minh số cặp cần xét là tuyến tính.

Delaunay còn là đối ngẫu của biểu đồ Voronoi. Mỗi mặt Voronoi gồm các điểm có cùng điểm gần nhất trong tập ban đầu. Đi dọc cạnh Voronoi nghĩa là đi theo tuyến cách xa điểm gần nhất nhất có thể; vì thế nó xuất hiện trong các bài tìm đường từ S tới T sao cho khoảng cách tới điểm nguy hiểm gần nhất là lớn nhất, hoặc các bài tính kỳ vọng khoảng cách gần nhất bằng cách tích phân trên các ô Voronoi.

8.1.4 Nghịch đảo tròn

Nghịch đảo tròn là một phép biến đổi hình học hữu ích. Chọn tâm O và sau khi co giãn giả sử bán kính nghịch đảo là 1. Điểm P biến thành P^* sao cho O, P, P^* thẳng hàng và

$$|OP| \cdot |OP^*| = 1.$$

Đường thẳng và đường tròn được nghịch đảo bằng cách nghịch đảo mọi điểm trên chúng. Các tính chất cần nhớ là: đường thẳng không qua tâm nghịch đảo và đường tròn qua tâm nghịch đảo biến đổi qua lại; đường tròn không qua tâm biến thành đường tròn không qua tâm khác; quan hệ tiếp xúc, cắt nhau, rời nhau được bảo toàn.

Vì phép tính trên đối tượng hình học thường cơ học, khi đề có nhiều đường tròn phức tạp ta có thể thử nghịch đảo trực tiếp. Ví dụ, cho n đường tròn đôi một không cắt nhau và không qua gốc, cần tìm một

đường tròn qua gốc cắt nhiều đường tròn nhất. Sau nghịch đảo, bài toán trở thành tìm một đường thẳng cắt nhiều đường tròn nhất. Đường thẳng tối ưu có thể giả sử tiếp xúc với một đường tròn; liệt kê đường tròn tiếp xúc, mỗi đường tròn còn lại tạo một khoảng góc đóng góp, rồi quét góc.

8.1.5 Bao lồi ba chiều

Trong hình học tính toán ba chiều, ta chủ yếu dựa vào vector. Tích có hướng cho pháp tuyến của mặt xác định bởi ba điểm có thứ tự; tích vô hướng với pháp tuyến cho biết một điểm nằm ở mặt trước hay sau của mặt đó. Với bao lồi ba chiều, khi thêm điểm P , nếu P ở trong mọi mặt thì không đổi; các mặt nhìn thấy từ P bị xóa, còn các cạnh biên giữa mặt nhìn thấy và không nhìn thấy sẽ cùng P tạo mặt mới. Duyệt các mặt mỗi lần thêm cho độ phức tạp $O(n^2)$.

Nhiều bài hai chiều cũng có thể nâng lên ba chiều. Biến đổi

$$(x, y) \mapsto (x, y, x^2 + y^2)$$

đưa các điểm lên một paraboloid. Đường tròn $x^2 + y^2 + ax + by + c = 0$ tương ứng với giao của paraboloid và một mặt phẳng. Vì vậy việc kiểm tra một điểm nằm trong đường tròn ngoại tiếp ABC trở thành kiểm tra điểm nâng lên nằm phía nào của mặt phẳng $A'B'C'$, có thể tính bằng định thức. Các mặt hướng xuống của bao lồi sau khi nâng chính là các tam giác Delaunay; các mặt hướng lên tương ứng với loại đường tròn đối ngẫu khác.

8.2 Cấu trúc dữ liệu phát sinh từ hình học

8.2.1 Định vị điểm

Với bài toán định vị điểm động trong một phân hoạch phẳng, ta chỉ cần biết tia thẳng đứng hướng lên từ điểm hỏi gặp cạnh nào đầu tiên. Chia các đoạn theo tọa độ x bằng cây đoạn; một đoạn phủ trọn một nút nhưng không phủ trọn nút cha sẽ được lưu tại nút đó, theo thứ tự tung độ. Truy vấn một điểm đi qua $O(\log n)$ nút và tìm tiền nhiệm theo tung độ, cho thời gian $O(\log^2 n)$ nếu mỗi nút dùng cây cân bằng. Thêm hoặc xóa đoạn cũng cập nhật $O(\log n)$ nút.

Nếu bài toán tĩnh, quét theo x và dùng cây cân bằng khả persistent để lưu thứ tự theo y ở từng thời điểm sẽ đưa truy vấn về một logarithm. Đây cũng là một trong những nguồn gốc lịch sử của cấu trúc dữ liệu persistent. Với các cạnh ngang/dọc động, thay persistent bằng cấu trúc retroactive cho biến thể động.

8.2.2 Truy vấn trực giao

Truy vấn trực giao hỏi các điểm trong hộp

$$[l_1, r_1] \times [l_2, r_2] \times \cdots \times [l_d, r_d].$$

Cây đoạn d chiều cho truy vấn $O(\log^d n)$. Với $d = 2$, nếu ở mỗi nút theo chiều x lưu mảng các điểm đã sắp theo y , ta được cấu trúc kiểu range tree. Nhờ fractional cascading, chỉ cần nhị phân ở gốc; kết quả tìm kiếm được nhảy sang các con trong $O(1)$. Tổng quát lên d chiều, có thể đạt $O(\log^{d-1} n)$ mỗi truy vấn với $O(n \log^{d-1} n)$ tiền xử lý và bộ nhớ.

Trong phiên bản động, cây đoạn ngoài cần thay bằng cây cân bằng theo trọng lượng. Ở mỗi mức, các con trở fractional cascading phải được bảo trì khi chèn; có thể dùng chia tách heuristic và phân tích kiểu *dsu on tree* để đạt khẩu hao $O(\log^d n)$ cho cập nhật, còn truy vấn giữ nguyên.

Với $d \geq 3$, có những biến thể đổi bộ nhớ lấy thời gian truy vấn. Chẳng hạn, trong không gian ba chiều, sau khi biến các truy vấn hộp thành vài truy vấn dạng hai phía bằng phân rã kiểu “cat tree”, phần

lỗi là truy vấn $[l_1, r_1] \times (-\infty, r_2] \times (-\infty, r_3]$. Trên mỗi nút của cây đoạn theo chiều thứ nhất, bài toán hai chiều còn lại có thể hình dung bằng các tia đứng từ điểm dữ liệu và tia ngang từ điểm hỏi. Dựng các ô do các tia này chia mặt phẳng, rồi dùng fractional cascading để đi qua các ô bị tia hỏi cắt; chi phí truy vấn là $O(\log n)$ cộng số điểm trả về, còn tiền xử lý và bộ nhớ tăng lên khoảng $O(n \log^3 n)$ do các lớp phân rã ban đầu.

8.2.3 Separator

Một định lý cơ bản cho đồ thị phẳng nói rằng tập đỉnh của đồ thị phẳng G có thể chia thành A, B, C sao cho không có cạnh giữa A và B ,

$$|A|, |B| \leq \frac{2n}{3}, \quad |C| \leq 2\sqrt{2n}.$$

Với một embedding đã biết, có thể chứng minh xây dựng tuyến tính: trước hết bổ sung đồ thị thành tam giác hóa, lấy cây BFS với các lớp $L(0), L(1), \dots$. Chọn các lớp mỏng j, k bao quanh điểm giữa số đỉnh. Nếu các vùng theo độ sâu đã cân bằng, lấy $C = L(j) \cup L(k)$. Nếu vùng giữa còn quá lớn, co một biên và tìm một chu trình nhỏ trong đồ thị con để tách tiếp. Chu trình này sinh từ một cạnh không thuộc cây trong cây sinh, và dùng định lý đường cong Jordan để chứng minh nó tách được hai phần đủ cân bằng.

Hằng số và tỷ lệ có thể cải tiến; ví dụ có dạng $|A|, |B| \leq n/2$ với separator cỡ $O(\sqrt{n})$, và hằng số $2\sqrt{2}$ còn có các cải tiến sâu hơn. Separator kết hợp tự nhiên với chia để trị trên đồ thị phẳng. Ví dụ, để đếm tập độc lập của đồ thị phẳng, liệt kê trạng thái trên C rồi đệ quy trên các thành phần tách ra, thu được thời gian $2^{O(\sqrt{n})}$.

Với đồ thị có genus g , cũng có separator:

$$|C| \leq 6\sqrt{gn} + 2\sqrt{2n} + 1,$$

sau khi bỏ C , thành phần liên thông lớn nhất có không quá $2n/3$ đỉnh. Ý tưởng là tìm một số lớp BFS mỏng, co và cắt các cạnh không thuộc cây sinh để loại bỏ các quai của mặt, đưa phần còn lại về đồ thị phẳng rồi áp dụng định lý phẳng.

Ngoài các thuật toán dựa trên embedding, còn có hướng nội tại bằng phổ. Với đồ thị G , đặt Laplacian

$$L(G) = D(G) - A(G),$$

trong đó D là ma trận bậc và A là ma trận kề. L đối xứng nửa xác định dương và có trị riêng 0; trị riêng dương nhỏ nhất thường gọi là độ liên thông đại số, vector riêng tương ứng là vector Fiedler. Thuật toán chia phổ sắp đỉnh theo vector này rồi cắt theo trung vị. Nhờ bất đẳng thức Fiedler và Courant–Fischer, với đồ thị bậc bị chặn trên bề mặt genus g , tồn tại lát cắt có tỷ lệ khoảng $O(\sqrt{g/n})$. Đây là cách đơn giản để lấy điểm trong phòng thí nghiệm khi không muốn hiện thực separator hình học đầy đủ.

8.3 Hình học hữu hạn

Hình học hữu hạn chỉ giữ lại quan hệ liên thuộc: điểm có nằm trên đường hay không, hai đường có cắt nhau hay không. Một mặt phẳng xạ ảnh hữu hạn thỏa các tiên đề: qua hai điểm có đúng một đường; hai đường có đúng một giao điểm; mỗi đường có ít nhất ba điểm. Fano plane là ví dụ nhỏ nhất. Nếu một mặt phẳng xạ ảnh bậc n có $n + 1$ điểm trên mỗi đường, thì nó có

$$n^2 + n + 1$$

điểm và cũng bằng ấy đường.

Mặt phẳng affine thay tiên đề hai đường luôn cắt bằng tiên đề song song: qua một điểm ngoài đường l có đúng một đường song song với l . Từ affine plane có thể thêm các điểm vô cực và đường vô cực để được projective plane; ngược lại, xóa một đường khỏi projective plane được affine plane. Một affine plane bậc n có n^2 điểm và $n^2 + n$ đường.

Cách xây dựng đại số chuẩn là dùng trường hữu hạn F_q . Trong không gian vector $n + 1$ chiều trên F_q , hình học xạ ảnh $PG(n, q)$ có các điểm, đường, mặt lần lượt là các không gian con 1, 2, 3 chiều; nói chung, k -flat tương ứng với không gian con $k + 1$ chiều. Số k -flat được cho bởi hệ số Gaussian

$$\binom{n}{k}_q = \prod_{i=0}^{k-1} \frac{q^n - q^i}{q^k - q^i}.$$

Một ngôn ngữ tổ hợp liên quan là thiết kế khối. Một $t - (v, k, \lambda)$ design là tập S có v phần tử và họ khối $\mathcal{F}$ gồm b tập con, mỗi khối có kích thước k , mỗi phần tử xuất hiện trong r khối, và mọi t phần tử phân biệt cùng nằm trong đúng λ khối. Khi đó

$$bk = vr, \quad \lambda(v - 1) = r(k - 1).$$

Steiner triple system là $2 - (n, 3, 1)$ design; điểm và đường của projective plane cũng tạo ra một design đặc biệt.

Những cấu trúc này cho cách nghĩ về nhiều bài xây dựng. Ví dụ, để chọn một tập con lớn của lưới $n \times n$ không chứa bốn đỉnh của một hình chữ nhật, ta có thể coi mỗi hàng là một đường trên tập cột; điều kiện không có hình chữ nhật nghĩa là hai đường giao nhau nhiều nhất một điểm. Với $n = q^2 + q + 1$, dùng projective plane bậc q cho nghiệm tối ưu tiệm cận.

Trong một bài cần thiết kế gần tối ưu kiểu $3 - (529, k, 1)$ hoặc $3 - (625, k, 1)$, các số $529 = 23^2$ và $625 = 5^4$ gợi ý cấu trúc hữu hạn. Mobius plane có tiên đề: qua ba điểm có đúng một vòng; với một vòng C , một điểm P trên C và điểm Q ngoài C , có đúng một vòng qua Q tiếp xúc C tại P ; mỗi vòng có ít nhất ba điểm. Hệ điểm-vòng của Mobius plane tạo 3-design, cụ thể có dạng $3 - (n^2 + 1, n + 1, 1)$. Có thể dựng nó từ một affine plane bằng cách thêm điểm vô cực, lấy các đường cộng điểm vô cực và các đường cong bậc hai phù hợp làm các vòng.

Một ví dụ khác: trong đồ thị hai phía có $n = 343 = 7^3$ đỉnh mỗi phía, muốn thêm nhiều cạnh nhưng không có $K_{3,3}$. Coi lân cận của mỗi đỉnh trái là một tập con của phía phải; điều kiện trở thành mọi bộ ba điểm được chứa trong không quá hai tập. Số 7^3 gợi ý không gian ba chiều hữu hạn và các mặt cầu/bề mặt đại số có tham số thích hợp. Tương tự, bài chọn tập con của $[1, n]$ sao cho các xor đôi một khác nhau gợi ý trường $GF(2^k)$ và các đường cong đại số có cỡ $\sqrt{n}$.

Điểm chung là khi dùng hình học hữu hạn cho bài xây dựng, trước hết quan sát số lượng, sau đó tìm quan hệ hình học/đại số phù hợp, rồi chọn cấu trúc hữu hạn để hiện thực. Phần này chỉ là nhập môn; phía sau còn có nhóm cơ bản của hình học hữu hạn, buildings, thiết kế khối và siêu đồ thị.

8.4 Tổng kết

Bài viết nhấn mạnh năm nhóm ý tưởng: đối ngẫu, lưới hóa, tam giác hóa, separator và hình học hữu hạn. Tác giả hy vọng người đọc nắm được trực giác hình học phía sau chúng, biết kết nối trực giác đó với các bài toán khác trong OI, và từ đó giải được nhiều bài thú vị hơn.

8.5 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn nhóm trao đổi của các cựu Oler đã gợi ý chủ đề hình học; cảm ơn Tao Liyu, Tang Shaoyuan và Xu Tingqiang đã

đọc bản thảo, đặc biệt Tang Shaoxuan đã đưa ra nhiều góp ý; cảm ơn Jiang Cheng đã thảo luận về ứng dụng của định lý separator; cảm ơn thầy Huang Xinjun của Trường Trung học Ba Thục, Trùng Khánh đã quan tâm, hướng dẫn; và cảm ơn gia đình đã ủng hộ.

8.6 Tài liệu tham khảo

1. Richard J. Lipton and Robert Endre Tarjan. A separator theorem for planar graphs. *SIAM Journal on Applied Mathematics*, 36(2):177–189, 1979.
2. John R. Gilbert, Joan P. Hutchinson and Robert Endre Tarjan. A separator theorem for graphs of bounded genus. *Journal of Algorithms*, 1984.
3. Jonathan A. Kelner. Spectral partitioning, eigenvalue bounds, and circle packings for graphs of bounded genus. *SIAM Journal on Computing*, 35(4):882–902, 2006.
4. Delaunay triangulation, Wikipedia.
5. *Handbook of Combinatorics*, Volume 1.
6. Lovasz. *Discrete Mathematics*.
7. Zhan Xingzhi. *Matrix Theory*.

9 Bàn về các thuật toán liên quan tới chuỗi palindrome và ứng dụng

Trường Trung học số 2 Hàng Châu
Xu Anyi

Tóm tắt

Các bài toán liên quan tới chuỗi palindrome là một lớp bài thường gặp trong thi tin học. Bài viết giới thiệu một số thuật toán thông dụng cho palindrome: hash chuỗi, thuật toán Manacher, cây palindrome và cách duy trì mọi hậu tố palindrome dưới dạng các cấp số cộng. Ngoài ra, bài viết trình bày ứng dụng mở rộng của việc duy trì thông tin mọi hậu tố palindrome bằng cấp số cộng.

9.1 Mở đầu

Palindrome có nhiều tính chất đẹp, nhưng số thuật toán xử lý trực tiếp bài toán palindrome không quá nhiều. Tác giả hệ thống lại các thuật toán và ví dụ liên quan, đồng thời giới thiệu một tính chất khi nghiên cứu palindrome: mọi hậu tố palindrome có thể được duy trì bằng các cấp số cộng, hỗ trợ hợp nhất thông tin và giải một số bài toán chuỗi có cập nhật động.

9.2 Ký hiệu và định nghĩa

Gọi S là một chuỗi. Ký hiệu $|S|$ là độ dài của S , $S[i]$ là ký tự thứ i , và $S[l \dots r]$ là chuỗi con từ vị trí l đến r ; nếu $l > r$ thì đó là chuỗi rỗng. Ký hiệu Σ là kích thước bảng chữ cái.

Một chuỗi S là palindrome nếu

$$\forall 1 \leq i \leq |S|, \quad S[i] = S[|S| + 1 - i].$$

Với palindrome con $S[l \dots r]$, tâm palindrome là $(l + r)/2$. Bán kính palindrome tại vị trí i là giá trị lớn nhất x sao cho $S[i - x + 1 \dots i + x - 1]$ là palindrome.

9.3 Các thuật toán thường dùng

9.3.1 Hash xâu

Một hàm ánh xạ xâu sang số nguyên được gọi là hàm hash. Nếu hai xâu S, T có $f(S) \neq f(T)$ thì chắc chắn $S \neq T$; nếu $f(S) = f(T)$, ta thường coi như $S = T$, trừ khi xảy ra va chạm hash. Chọn hàm hash phù hợp giúp xác suất va chạm rất nhỏ, và cho phép kiểm tra hai xâu bằng nhau trong $O(1)$.

Một lựa chọn phổ biến là

$$f(S) = \left(\sum_{i=1}^{|S|} B^{|S|-i} \text{val}(S[i]) \right) \bmod P,$$

trong đó val ánh xạ các ký tự khác nhau vào các số khác nhau trong $[1, \Sigma]$, P là số nguyên tố lớn hơn Σ , còn B là số nguyên dương nằm giữa Σ và P . Với prefix hash, có thể tính nhanh

$$f(S[l \dots r]) = (f(S[1 \dots r]) - f(S[1 \dots l-1])B^{r-l+1}) \bmod P.$$

Về xác suất đúng, nếu hai xâu khác nhau S, T vẫn có cùng hash, đặt

$$h(B) = f(S) - f(T) = \sum_{i=1}^l (s[i] - t[i])B^{l-i}, \quad l = \max(|S|, |T|).$$

Đây là đa thức không đồng nhất bậc không quá $l-1$. Nếu B được chọn đều trong $[0, P)$, xác suất va chạm không vượt quá $(l-1)/P$. Trong thực tế có thể dùng nhiều cặp (P, B) để giảm xác suất lỗi.

9.3.2 Thuật toán Manacher

Manacher tính bán kính palindrome tại mọi vị trí trong thời gian tuyến tính. Để tránh phân biệt độ dài chẵn/lẻ và xử lý biên, chèn một ký tự đặc biệt giữa mọi cặp ký tự, rồi thêm hai ký tự chặn khác nhau ở đầu và cuối. Ví dụ aabbacab biến thành \$#a#b#b#a#c#a#b#@\$. Trên xâu mới S , cần tính $R[i]$, bán kính palindrome tại i .

Thuật toán duyệt i từ trái sang phải, duy trì r là biên phải xa nhất của palindrome đã biết và pos là tâm tương ứng. Khi tính $R[i]$:

- nếu $r < i$, ta có cận dưới $R[i] \geq 1$;
- nếu $r \geq i$, do đối xứng qua pos ,

$$R[i] \geq \min(R[2pos - i], r - i + 1).$$

Từ cận dưới đó, mở rộng $R[i]$ cho tới khi hai ký tự đối xứng không còn bằng nhau, rồi cập nhật r, pos . Mỗi lần mở rộng làm r tăng, nên tổng số lần mở rộng là $O(n)$.

9.3.3 Cây palindrome

Cây palindrome của xâu S gồm hai gốc: gốc lẻ và gốc chẵn. Mỗi đỉnh ngoài gốc biểu diễn một palindrome khác nhau; xâu của một đỉnh thu được bằng cách thêm ký tự ghi trên cạnh vào hai đầu xâu của cha. Gốc chẵn biểu diễn xâu rỗng; gốc lẻ biểu diễn một xâu giả độ dài -1 , giúp xử lý biên. Mỗi đỉnh u có con trở thất bại $fail_u$ trở tới đỉnh biểu diễn hậu tố palindrome dài nhất thật sự của xâu tại u .

Cây được dựng tăng dần. Giả sử đã có cây của S , thêm ký tự c vào cuối. Trong Sc , palindrome mới không xuất hiện trong S có nhiều nhất một xâu, chính là hậu tố palindrome dài nhất. Thật vậy, mọi

palindrome mới đều chứa ký tự c nên là hậu tố palindrome; nếu không phải dài nhất, nó sẽ xuất hiện đối xứng bên trong hậu tố dài nhất và đã có trong S .

Do đó ta đi theo chuỗi *fail* từ hậu tố palindrome dài nhất hiện tại cho tới khi tìm được đỉnh có thể kẹp thêm ký tự c hai bên. Nếu cạnh ký tự c chưa tồn tại, tạo đỉnh mới và thiết lập cha cùng con trở thất bại của nó bằng cách tiếp tục đi theo *fail* để tìm palindrome kẹp được c tiếp theo.

Thêm ký tự ở đầu xâu cũng tương tự: vì mỗi đỉnh biểu diễn palindrome, con trở thất bại vừa là hậu tố palindrome dài nhất vừa tương ứng với tiền tố palindrome dài nhất theo hướng ngược, và mỗi lần thêm đầu cũng chỉ sinh nhiều nhất một palindrome bản chất mới.

9.3.4 Duy trì mọi hậu tố palindrome bằng cấp số cộng

Một tính chất quan trọng là mọi hậu tố palindrome của S , nếu sắp theo độ dài, có thể chia thành $O(\log |S|)$ đoạn cấp số cộng. Dựa vào đó, ta duy trì tập hậu tố palindrome ở dạng các cấp số cộng.

Khi thêm ký tự c vào cuối S , các hậu tố palindrome mới của $S' = Sc$ gồm:

1. palindrome một ký tự c ;
2. các hậu tố palindrome cũ được mở rộng bằng ký tự c ở hai đầu.

Với một cấp số cộng cực đại của các hậu tố cũ, viết các xâu tương ứng thành

$$A, BA, BBA, B^3A, \dots, B^kA.$$

Nếu ký tự cuối của B là c , thì toàn bộ cấp số cộng trừ hạng B^kA còn xuất hiện trong hậu tố palindrome mới; nếu không, các hạng đó không xuất hiện. Hạng cuối chỉ cần kiểm tra ký tự bên trái nó có phải c hay không. Sau đó gộp các cấp số cộng liền kề có cùng công sai để giữ số đoạn là $O(\log |S|)$.

9.4 Phân tích ví dụ

9.4.1 APIO 2014 Palindrome

Bài toán: cho xâu S , tìm palindrome có giá trị lớn nhất bằng số lần xuất hiện nhân với độ dài; $|S| \leq 3 \cdot 10^5$.

Dựng cây palindrome của S . Mỗi đỉnh ứng với đúng một palindrome bản chất khác nhau. Để đếm số lần xuất hiện, với mỗi vị trí i , tìm đỉnh ứng với hậu tố palindrome dài nhất kết thúc tại i rồi cộng một trên đường đi theo *fail* về gốc. Có thể tối ưu bằng tổng tiền tố trên cây thất bại. Cuối cùng duyệt mọi đỉnh và cập nhật đáp án bằng độ dài nhân số lần xuất hiện.

9.4.2 Codeforces 1738H Palindrome Addicts

Bài toán cần duy trì xâu S ban đầu rỗng với hai thao tác:

- push c : thêm ký tự c vào cuối;
- pop: xóa ký tự đầu, bảo đảm xâu không rỗng.

Sau mỗi thao tác, hỏi số palindrome bản chất khác nhau trong S ; $1 \leq Q \leq 10^6$.

Mỗi thao tác làm đáp án đổi không quá 1. Với push, chỉ cần tìm hậu tố palindrome dài nhất và kiểm tra nó đã xuất hiện trong xâu trước đó hay chưa. Với pop, làm tương tự cho tiền tố palindrome dài nhất và kiểm tra sau khi xóa. Có thể xử lý offline, mỗi lần thêm cuối và dùng dạng cấp số cộng để duy trì mọi hậu tố palindrome, từ đó tìm hậu tố dài nhất; tiền tố làm tương tự. Hash của xâu được đưa vào bảng băm để kiểm tra xuất hiện nhanh.

Cũng có thể không dùng cấp số cộng, mà nhảy bội trên chuỗi *fail* của cây palindrome để tìm tiền tố/hậu tố palindrome dài nhất. Một cách đơn giản hơn nữa là duy trì trực tiếp hậu tố palindrome dài nhất hiện tại: khi thêm ký tự ở cuối, độ dài của nó tăng không quá 1; nếu không còn palindrome thì giảm dần cho tới khi tìm được hậu tố mới.

9.4.3 Internal Longest Palindrome Queries

Bài toán: cho xâu S độ dài n , có Q truy vấn, mỗi truy vấn cho đoạn $[l, r]$ và hỏi palindrome dài nhất trong $S[l \dots r]$; $1 \leq n, Q \leq 2 \cdot 10^5$.

Trước hết cần tìm tiền tố palindrome dài nhất $S[l \dots i]$ và hậu tố palindrome dài nhất $S[j \dots r]$ của đoạn. Nếu một palindrome trong đoạn có tâm nằm bên trái $(l + i)/2$, độ dài của nó không thể vượt tiền tố palindrome dài nhất. Tương tự, mọi palindrome có tâm trong khoảng

$$\left[\frac{l+i}{2}, \frac{j+r}{2} \right]$$

đều nằm hoàn toàn trong $[l, r]$; nếu không, nó sẽ tạo tiền tố hoặc hậu tố palindrome dài hơn, mâu thuẫn.

Vì vậy, sau khi biết tiền tố và hậu tố palindrome dài nhất, chỉ cần truy vấn giá trị lớn nhất của bán kính Manacher $R[x]$ trên đoạn tâm tương ứng. Tiền tố/hậu tố dài nhất có thể tìm offline bằng cây palindrome và nhảy bội, hoặc dùng DSU tuyến tính trên cây. Truy vấn cực đại trên $R[x]$ là RMQ kinh điển: dùng sparse table cho $O(n \log n)$ tiền xử lý, $O(1)$ truy vấn, hoặc chia khối cỡ $\log n$ kết hợp sparse table ngoài và đơn điệu stack bitmask trong khối để đạt $O(n)$ tiền xử lý, $O(1)$ truy vấn.

9.4.4 Bài Palindrome trong thi thử đội tuyển 2022

Bài toán: cho xâu chữ thường S , có Q thao tác:

1. đổi S_i thành ký tự c ;
2. hỏi $S[l \dots r]$ có bao nhiêu hậu tố palindrome.

Với $1 \leq |S|, Q \leq 2 \cdot 10^5$.

Dựng cây đoạn. Mỗi nút $[l, r]$ duy trì mọi tiền tố palindrome và hậu tố palindrome của $S[l \dots r]$ dưới dạng các cấp số cộng. Nếu có thể hợp nhất nhanh thông tin hai con, thao tác đổi ký tự là cập nhật điểm, còn truy vấn đoạn là hợp nhất các nút được chọn.

Giả sử con trái là xâu S , con phải là T , cần tính hậu tố palindrome của ST . Các hậu tố mới có bốn loại:

1. không đi qua S , kế thừa trực tiếp từ T ;
2. đi qua S và tâm nằm trong T ;
3. đi qua S và tâm nằm giữa S và T ;
4. đi qua S và tâm nằm trong S .

Loại thứ hai sinh từ các tiền tố palindrome của T . Với một cấp số cộng cực đại

$$A, AB, ABB, AB^3, \dots, AB^k$$

và $T = AB^k C$ trong đó B không phải tiền tố của C , nếu C là tiền tố của B thì một hậu tố của cấp số cộng này đóng góp thành một cấp số cộng mới; độ dài phần đóng góp phụ thuộc vào S và có thể tìm bằng nhị phân. Nếu C không phải tiền tố của B , nhiều nhất một hạng đóng góp. Loại thứ ba nhiều nhất tạo một hậu tố palindrome mới, kiểm tra trực tiếp. Loại thứ tư tương tự bài toán thêm ký tự cuối, chỉ khác là đầu thêm đảo của T và cuối thêm cả xâu T .

Trong quá trình hợp nhất cần kiểm tra nhanh hai xâu con có bằng nhau hay không; có thể dùng hash xâu theo khối để cập nhật $O(\sqrt{|S|})$ và truy vấn $O(1)$. Thời gian hợp nhất là $O(\log(|S| + |T|))$, vì các cấp số cộng lồng nhau làm tổng số hạng cần xét chỉ logarithmic. Tổng độ phức tạp là

$$O(n \log n + Q(\log^2 n + \sqrt{n})).$$

9.5 Kết luận

Cây palindrome rất mạnh vì mỗi đỉnh ứng với đúng một palindrome bản chất khác nhau. Với các bài có cập nhật, có thể thử kết hợp cây đoạn với biểu diễn mọi tiền tố/hậu tố palindrome bằng cấp số cộng; tính chất hợp nhất này là điều cây palindrome thông thường không trực tiếp cung cấp. Khi một bài có nhiều hướng giải, nên cân nhắc cả độ khó hiện thực để chọn phương án phù hợp.

9.6 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn các huấn luyện viên đội tuyển quốc gia Peng Sijin và Yang Yaoliang đã hướng dẫn; cảm ơn cha mẹ, nhà trường, thầy Li Jian và các bạn học đã giúp đỡ; cảm ơn Wang Xiangwen đã trao đổi, gợi ý và đọc bản thảo; cảm ơn những tài liệu được tham khảo và cảm ơn độc giả đã dành thời gian đọc bài viết.

9.7 Tài liệu tham khảo

1. OI Wiki. Cây palindrome. <https://oi-wiki.org/string/pam/>.
2. Weng Wentao. Cây palindrome và ứng dụng. Tập luận văn đội tuyển quốc gia IOI 2017.
3. Chen Sunli. Thuật toán liên quan tới truy vấn chu kỳ xâu con và ứng dụng. Tập luận văn đội tuyển quốc gia IOI 2019.
4. Kazuki Mitani, Takuya Mieno, Kazuhisa Seto and Takashi Horiyama. Internal Longest Palindrome Queries in Optimal Time. arXiv:2210.02000v2, 2022.

10 Bàn về một số ứng dụng của trường hữu hạn trong OI

Trường Trung học Thực nghiệm Uy Hải
Qì Langrui

Tóm tắt

Bài viết giới thiệu các khái niệm cơ bản về trường hữu hạn, rồi trình bày một số ứng dụng của trường hữu hạn có đặc trưng bằng 2 trong OI, gồm bài toán đường đi đơn dài nhất và bài toán *Graph Motif*.

10.1 Khái niệm cơ bản

10.1.1 Trường

Một trường là một tập F có hai phép toán cộng và nhân, thỏa các tính chất: đóng dưới hai phép toán; giao hoán và kết hợp cho cộng, nhân; nhân phân phối đối với cộng; tồn tại phần tử đơn vị cộng 0 và nhân 1; mọi phần tử có nghịch đảo cộng; mọi phần tử khác 0 có nghịch đảo nhân. Nói cách khác, với mọi $a \in F$ tồn tại $b \in F$ sao cho $a + b = 0$, và với mọi $a \in F \setminus \{0\}$ tồn tại $b \in F$ sao cho $a \cdot b = 1$.

10.1.2 Trường hữu hạn

Nếu số phần tử của một trường là hữu hạn, ta gọi đó là trường hữu hạn; số phần tử này gọi là bậc của trường. Bậc của trường hữu hạn luôn có dạng p^n , với p là số nguyên tố. Ta ký hiệu trường này là F_{p^n} , và gọi p là đặc trưng của trường.

10.1.3 Các phép toán trong F_p

Các phép toán trong F_p gần giống số học thông thường, chỉ khác là lấy kết quả modulo p . Phép chia tương đương với nhân nghịch đảo modulo p : số a^{-1} thỏa

$$a \cdot a^{-1} \equiv 1 \pmod{p}.$$

10.1.4 Các phép toán trong F_{p^n}

Có thể xem phần tử của F_{p^n} là các đa thức trên F_p có bậc nhỏ hơn n . Chọn một đa thức bất khả quy R bậc n trên F_p làm mô-đun. Cộng và trừ chỉ cần lấy từng hệ số modulo p ; nhân xong thì lấy phần dư theo đa thức R . Chia tương đương với nhân nghịch đảo modulo R : đa thức A^{-1} thỏa

$$A \cdot A^{-1} \equiv 1 \pmod{R}.$$

10.2 Trường hữu hạn đặc trưng 2

Khi $p = 2$, với mọi phần tử a ta có

$$a + a = 0.$$

Tính chất này rất hữu ích với các điều kiện “không được lặp”. Ta có thể xây dựng đa thức trên trường hữu hạn đặc trưng 2 sao cho các cấu hình có phần tử lặp xuất hiện thành từng cặp và tự triệt tiêu. Khi đó bài toán quyết định thường được đưa về kiểm tra một đa thức có bằng 0 hay không.

Ngoài ra, nhiều phép toán trong F_{2^n} có thể hiện thực nhanh bằng bit. Khi n nhỏ, cộng và trừ chính là xor nhị phân; nhân cũng có thể thực hiện bằng các phép xor và dịch bit trong $O(n)$.

Trong phần dưới, “đa thức trên trường đặc trưng 2” đôi khi được gọi ngắn là “đa thức”.

10.3 Bài toán đường đi đơn dài nhất

10.3.1 Mô tả

Cho một đồ thị đơn, có thể có hướng hoặc vô hướng. Cần tìm một đường đi dài nhất không đi qua cùng một đỉnh hai lần. Giả sử đáp án k nhỏ hơn nhiều so với số đỉnh n .

Một cách trực tiếp là quy hoạch động trạng thái tập các đỉnh đã đi, có độ phức tạp dạng $O(\binom{n}{k} \text{poly}(n))$. Bài toán chứa Hamilton path nên là NP-complete, nhưng vì k nhỏ, ta muốn đạt dạng $O(2^k \text{poly}(n))$. Ta liệt kê k từ nhỏ tới lớn và xét bài toán quyết định: có tồn tại đường đi đơn gồm k đỉnh hay không. Vì $\sum_{i=1}^k O(2^i) = O(2^k)$, phép chuyển này không đổi bậc độ phức tạp.

10.3.2 Xây dựng đa thức

Với một đường đi $v_1, v_2, \dots, v_k$, để các đường đi khác nhau không triệt tiêu lẫn nhau, trước hết gán cho nó nhân tử

$$\prod_{i=1}^{k-1} X_{v_i, v_{i+1}},$$

trong đó X là ma trận $n \times n$, mỗi phần tử được chọn ngẫu nhiên đều trong F_{2^v} .

Vấn đề là đường đi có thể lặp đỉnh. Ta thêm một ma trận Y kích thước $n \times k$, các phần tử cũng chọn ngẫu nhiên trong F_{2^v} , và nhân thêm

$$\sum_{p \in \text{permutation}(k)} \prod_{i=1}^k Y_{v_i, p_i},$$

trong đó $\text{permutation}(k)$ là tập các hoán vị của $1, \dots, k$. Nếu các v_i phân biệt, các tích ứng với các hoán vị thường khác nhau và tổng khác 0 với xác suất cao. Nếu có $v_i = v_j$, ghép mỗi hoán vị P với hoán vị Q thu được bằng cách đổi P_i, P_j . Hai tích bằng nhau nên tổng của cặp bằng 0, và toàn bộ đóng góp bị triệt tiêu.

Đa thức ứng với một đường đi là

$$\prod_{i=1}^{k-1} X_{v_i, v_{i+1}} \sum_{p \in \text{permutation}(k)} \prod_{i=1}^k Y_{v_i, p_i}.$$

Cộng biểu thức này trên mọi walk độ dài k ; kết quả khác 0 với xác suất cao khi và chỉ khi tồn tại đường đi đơn k đỉnh.

10.3.3 Tính giá trị đa thức

Cần tính

$$\sum_{v_1, \dots, v_k \in \text{Path}(G)} \prod_{i=1}^{k-1} X_{v_i, v_{i+1}} \sum_{p \in \text{permutation}(k)} \prod_{i=1}^k Y_{v_i, p_i}.$$

Một cách là quy hoạch động trạng thái tập. Gọi $f(pos, S)$ là tổng đóng góp của các walk bắt đầu tại pos khi các giá trị của hoán vị đã dùng là tập S . Chuyển bằng cách chọn cạnh đầu tiên và giá trị điền tiếp trong hoán vị. Phần DP có $O(2^k km)$ chuyển; nếu tính cả nhân trên trường bằng bit, thời gian là $O(2^k kmv)$, bộ nhớ $O(2^k n)$.

Để giảm bộ nhớ về đa thức theo n , dùng bao hàm–loại trừ cho điều kiện p là hoán vị:

$$\sum_{S \subseteq \{1, \dots, k\}} \sum_{p_i \in S} \prod_{i=1}^k Y_{v_i, p_i} = \sum_{S \subseteq \{1, \dots, k\}} \prod_{i=1}^k \sum_{j \in S} Y_{v_i, j}.$$

Dấu (-1) biến mất vì trong đặc trưng 2, $-a = a$. Với mỗi tập S , tính trước $\sum_{j \in S} Y_{i, j}$ cho từng đỉnh i , rồi chạy DP theo độ dài walk trong $O(mkv)$. Tổng thời gian vẫn cùng bậc, nhưng bộ nhớ chỉ còn đa thức theo n .

10.3.4 Xác suất đúng và xuất phương án

Nếu không có đường đi đơn k đỉnh, đa thức chắc chắn bằng 0. Nếu có, đa thức không đồng nhất có tổng bậc $2k - 1$. Theo bổ đề Schwartz–Zippel, khi chọn các biến ngẫu nhiên từ trường kích thước 2^v , xác suất đánh giá nhầm bằng 0 không vượt quá

$$\frac{2k - 1}{2^v}.$$

Với 2^v đủ lớn so với k , xác suất lỗi trong OI có thể xem là rất nhỏ.

Để xuất một đường đi, trước hết thuật toán cho ta đa thức bắt đầu từ từng đỉnh. Chọn một đỉnh có đa thức hợp lệ khác 0 làm đầu, xóa đỉnh đó, giảm k đi 1, và ở vòng sau chỉ cho phép chọn các đỉnh là đích của cạnh đi ra từ đỉnh vừa chọn. Lặp k vòng, các đỉnh đã chọn ghép lại thành một đường đi hợp lệ. Độ phức tạp không đổi bậc vì mỗi vòng k giảm một và 2^k giảm một nửa.

Trong đồ thị vô hướng, còn có thể tối ưu tới $O(1.66^k \text{poly}(n))$; chi tiết có thể xem tài liệu tham khảo.

10.4 Bài toán Maximum Graph Motif

10.4.1 Mô tả

Cho đồ thị vô hướng $G = (V, E)$, mỗi đỉnh có một màu. Cho một đa tập màu M , trong đó màu i xuất hiện m_i lần. Cần chọn một tập con đỉnh sao cho đồ thị cảm sinh trên tập này liên thông, và nếu c_i là số đỉnh màu i được chọn thì $c_i \leq m_i$. Mục tiêu là tối đa hóa số đỉnh chọn. Giả sử $|M|$ nhỏ hơn nhiều so với n .

Tương tự phần trước, ta chuyển thành bài toán quyết định theo kích thước k : có tồn tại tập k đỉnh hợp lệ hay không. Tập đỉnh cảm sinh liên thông khi và chỉ khi tồn tại một cây phủ đúng các đỉnh đó, nên ta có thể tìm một cây có gốc thỏa điều kiện màu.

10.4.2 Xây dựng đa thức

Để các cây có gốc khác nhau không triệt tiêu ngoài ý muốn, gán cho mỗi cạnh có hướng $u \rightarrow v$ một biến ngẫu nhiên $X_{u,v} \in F_{2^v}$. Đóng góp cạnh của cây T là

$$\prod_{e=(u,v) \in T_E} X_{u,v}.$$

Điều kiện $c_i \leq m_i$ được biến thành: với mỗi đỉnh màu i , gán một số hiệu từ 1 đến m_i , và các đỉnh cùng màu phải nhận số hiệu khác nhau. Gọi số hiệu của đỉnh i là a_i , đưa vào ma trận ngẫu nhiên Y với đóng góp Y_{i,a_i} .

Để ép các cặp (màu, số hiệu) không lặp, dùng thêm ma trận ngẫu nhiên Z , trong đó chỉ số hàng là cặp (col_i, a_i) và chỉ số cột là số p_i trong một hoán vị của $1, \dots, |T_V|$. Đóng góp đầy đủ của cây T là

$$\prod_{e=(u,v) \in T_E} X_{u,v} \sum_a \prod_{i \in T_V} Y_{i,a_i} \sum_p \prod_{i \in T_V} Z_{(\text{col}_i, a_i), p_i}.$$

Nếu hai đỉnh có cùng màu và cùng số hiệu, các hoán vị ghép cặp như ở bài đường đi đơn sẽ triệt tiêu; nếu không, đóng góp khác 0 với xác suất cao.

10.4.3 Tính giá trị đa thức

Tiếp tục dùng bao hàm–loại trừ cho phần hoán vị. Với một tập $S \subseteq \{1, \dots, |T_V|\}$ cố định, biểu thức còn lại có thể viết thành

$$\prod_{e=(u,v) \in T_E} X_{u,v} \prod_{i \in T_V} W_i,$$

trong đó

$$W_i = \sum_{a_i=1}^{m_{\text{col}_i}} Y_{i,a_i} \sum_{p_i \in S} Z_{(\text{col}_i, a_i), p_i}.$$

Đặt $dp_{pos, size}$ là tổng đóng góp của mọi cây có gốc pos và số đỉnh là $size$. Khi chuyển, liệt kê các cạnh ra khỏi pos , gán cây con từ đỉnh đầu kia vào cây hiện tại và liệt kê kích thước cây con. Có vẻ chuyển này chưa cắm cây con đúng lại đỉnh đã có, nhưng nếu cấu trúc sinh ra có hai bản sao của cùng một đỉnh, đổi số hiệu của hai bản sao tạo cặp đóng góp bằng nhau và triệt tiêu trong đặc trưng 2. Do đó các cấu hình không hợp lệ không ảnh hưởng đến tổng.

Tổng độ phức tạp là

$$O(2^{|M|} |M|^2 m v).$$

Chứng minh xác suất đúng giống phần đường đi đơn: nếu đa thức không đồng nhất, bổ đề Schwartz–Zippel cho xác suất triệt tiêu giả rất nhỏ khi $2^v \gg |M|$.

10.4.4 Xuất phương án

Sau khi tìm được đáp án k và một gốc hợp lệ $root$, xét lần lượt các cạnh có thể thêm. Gọi x_i là tổng các giá trị $d_{P_{root,k}}$ trên mọi tập S sau khi thử cho phép dùng tới cạnh thứ i , và đặt $x_0 = 0$. Chọn i nhỏ nhất sao cho

$$x_{i-1} = 0, \quad x_i \neq 0.$$

Khi đó tồn tại một phương án hợp lệ chứa đỉnh mà cạnh thứ i trở tới. Co đỉnh đó với gốc, giữ màu của gốc, hợp nhất các cạnh, xóa một bản sao màu của đỉnh đó khỏi M , giảm k , rồi đệ quy cho tới khi $k = 1$. Độ phức tạp vẫn là $O(2^{|M|}|M|^2mv)$.

10.5 Kết luận

Bài viết giới thiệu ngắn gọn các tính chất của trường hữu hạn và một vài ứng dụng của trường đặc trưng 2 trong OI. Ứng dụng của trường hữu hạn, đặc biệt trong các thuật toán tham số cho bài toán NP-complete, còn rộng hơn nhiều so với các ví dụ ở đây. Hy vọng bài viết giúp thêm nhiều bạn đọc tiếp cận và đào sâu chủ đề này.

10.6 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang đã hướng dẫn; cảm ơn Tang Shaoxuan và Yu Yue đã gợi ý và đọc bản thảo; cảm ơn hội Tin học Sơn Đông cùng các huấn luyện viên Trường Trung học Thực nghiệm Uy Hải đã chỉ dạy; và cảm ơn gia đình, bạn bè đã ủng hộ.

10.7 Tài liệu tham khảo

1. Lukasz Kowalik. Algebraic techniques in parameterized algorithms, Part II: Polynomials over finite fields of characteristic two, 2014.
2. Andreas Bjorklund, Petteri Kaski and Lukasz Kowalik. Probably Optimal Graph Motifs, 2013.
3. Li Chao, Ruan Wei, Zhang Xiang and Zhang Long. Mathematical principles of computer algebra systems, 2009.
4. Wikipedia. Finite field arithmetic.
5. Wikipedia. Schwartz–Zippel lemma.

11 Bàn về một lớp bài toán thống kê trên cây

Trường Trung học Chư Ky, Chiết Giang
Zhu Yikai

Tóm tắt

Bài viết xuất phát từ một bài tập do tác giả ra cho đội tuyển quốc gia Trung Quốc IOI 2023, từ đó dẫn ra một lớp bài toán thống kê trên cây. Tác giả giới thiệu một thuật toán có tính tổng quát nhất định cho lớp bài toán này, rồi đưa ra vài bài ví dụ và lần lượt dùng thuật toán đó để giải.

11.1 Dẫn nhập

Bài toán trên cây luôn là một chủ đề quen thuộc trong các kỳ thi thuật toán, và các dạng bài liên quan cũng rất phong phú. Bài viết này giới thiệu một trong số các dạng đó, kèm ví dụ và lời giải, với hy vọng giúp người đọc có thêm góc nhìn khi gặp những bài thống kê trên cây tương tự.

Nếu không nói khác, trong các bài toán dưới đây ta luôn giả sử kích thước cây và số truy vấn, nếu có nhiều truy vấn, đều là $O(n)$.

11.2 Kiến thức chuẩn bị

11.2.1 Phân rã nặng nhẹ

Phân rã đường trên cây dùng để chia cây thành một số đường, từ đó thuận tiện duy trì thông tin trên các đường đi. Phân rã nặng nhẹ là một dạng quen thuộc của kỹ thuật này.

Định nghĩa 2.1.1 (con nặng). Trong các con của một đỉnh, đỉnh con có cây con lớn nhất được gọi là con nặng. Nếu có nhiều con cùng có cây con lớn nhất, chọn tùy ý một đỉnh. Nếu đỉnh không có con thì không có con nặng.

Định nghĩa 2.1.2 (con nhẹ). Các con không phải con nặng đều được gọi là con nhẹ.

Định nghĩa 2.1.3 (cạnh nặng). Cạnh nối một đỉnh với con nặng của nó là cạnh nặng.

Định nghĩa 2.1.4 (cạnh nhẹ). Cạnh nối một đỉnh với con nhẹ của nó là cạnh nhẹ.

Định nghĩa 2.1.5 (đường nặng). Một số cạnh nặng nối tiếp đầu cuối với nhau tạo thành một đường nặng.

Nếu coi cả đỉnh đơn lẻ là một đường nặng, toàn bộ cây được phân hoạch thành các đường nặng. Trong hình minh họa của bản gốc, cạnh nét đứt là cạnh nhẹ, cạnh nét liền là cạnh nặng, và đầu mỗi đường nặng được đánh dấu bằng vòng tròn rỗng.

11.2.2 Tính chất của phân rã nặng nhẹ

Tính chất 2.2.1. Mọi đường đi trên cây có thể tách thành $O(\log n)$ đường nặng.

Chứng minh. Khi đi xuống qua một cạnh nhẹ, kích thước cây con hiện tại ít nhất giảm một nửa. Vì vậy, với một đường đi bất kỳ trên cây, nếu tách nó thành các đoạn đi từ LCA của hai đầu mút xuống hai phía bằng cách nhảy qua các đường nặng, mỗi phía chỉ nhảy nhiều nhất $O(\log n)$ lần. Do đó mỗi đường đi trên cây có thể tách thành không quá $O(\log n)$ đường nặng.

Tính chất 2.2.2. Tổng kích thước cây con của tất cả các con nhẹ của tất cả các đỉnh là $O(n \log n)$.

Chứng minh. Mỗi đỉnh đóng góp đúng bằng số cạnh nhẹ trên đường từ nó tới gốc. Theo tính chất trước, số lượng này là $O(\log n)$. Cộng trên n đỉnh ta được $O(n \log n)$.

11.3 Bài toán

11.3.1 Mô tả

Cho một cây có n đỉnh. Ký hiệu $x \rightarrow y$ là đường đi đơn từ x tới y trên cây, và khoảng cách $\text{dis}(x, y)$ là số cạnh trên đường đi đó. Khoảng cách từ một đỉnh u tới đường đi đơn $x \rightarrow y$ được định nghĩa là

$$\min_{v \in x \rightarrow y} \text{dis}(u, v).$$

Có q truy vấn. Truy vấn thứ i cho ba số nguyên x_i, y_i, d_i , thỏa $1 \leq x_i, y_i \leq n, 0 \leq d_i < n$. Cần tính số đỉnh có khoảng cách tới đường đi $x_i \rightarrow y_i$ không vượt quá d_i .

11.3.2 Giới hạn

Với toàn bộ dữ liệu, $1 \leq n, q \leq 2 \cdot 10^5$. Giới hạn thời gian là 3 giây, bộ nhớ 1024 MB.

11.4 Phân tích bài toán

11.4.1 Biến đổi ban đầu

Giả sử cây được gốc hóa tại 1. Gọi z_i là LCA của x_i và y_i . Ta tách bài toán theo hai đoạn $z_i \rightarrow x_i$ và $z_i \rightarrow y_i$.

Trước hết tính số đỉnh có khoảng cách tới z_i không vượt quá d_i . Đây là bài toán kinh điển, có thể giải bằng phân trị điểm trong $O(n \log n)$.

Phần còn lại trên đoạn $x_i \rightarrow z_i$ cần thống kê

$$\sum_{u \in z'_i \rightarrow x_i} f_{u, d_i},$$

trong đó z'_i là đỉnh thứ hai trên đường từ z_i tới x_i , còn $f_{u, j}$ là số đỉnh trong cây con của u có khoảng cách tới u bằng j . Đoạn $z_i \rightarrow y_i$ xử lý tương tự.

Vì vậy vấn đề trở thành tính nhanh các tổng dạng

$$\sum_{u \in z'_i \rightarrow x_i} f_{u, d_i}.$$

Một cách xử lý quen thuộc là lấy hiệu theo đường từ gốc:

$$\sum_{u \in 1 \rightarrow x_i} f_{u, d_i} - \sum_{u \in 1 \rightarrow z_i} f_{u, d_i}.$$

11.4.2 Một lời giải hơi kém hơn

Ta cần tính nhanh $\sum_{u \in 1 \rightarrow x} f_{u, d_i}$. Để trình bày thuận tiện, thay vào đó ta tính số đỉnh có khoảng cách tới đường $1 \rightarrow x$ không vượt quá d . Do đang dùng hiệu, phần thừa phát sinh trong biến đổi này sẽ tự triệt tiêu, nên đáp án cuối vẫn đúng.

Trước hết tính kích thước đường $1 \rightarrow x$. Sau đó chỉ cần xét đóng góp của các cây con khác của từng đỉnh trên đường này.

Dùng phân rã nặng nhẹ. Trước hết tính đóng góp của cây con con nặng của mỗi đỉnh đối với các truy vấn nằm trong các cây con khác. Phần này tạo ra $O(n \log n)$ truy vấn dạng: trong một cây con, có bao

nhiều đỉnh có khoảng cách tới gốc cây con không vượt quá d . Có thể dùng phân rã đường dài để giải trong $O(n \log n)$.

Còn lại là các cây con của các con nhẹ trên đường từ x về gốc, trừ đi cây con chứa x . Trong hình của bản gốc, các cây con cần tính được tô xanh, còn phần đã tính ở bước đầu được tô vàng.

Theo tính chất của phân rã nặng nhẹ, tổng kích thước cây con của các con nhẹ là $O(n \log n)$. Ta chèn một bản sao của truy vấn x vào mỗi đỉnh được tô đen trong hình, tức các đỉnh gặp khi đi từ x lên trên mà bước vừa đi lên đến từ con nặng. Sau đó xử lý riêng từng đường nặng; bản chất vẫn là một bài toán đếm điểm hai chiều. Cuối cùng cần trừ đóng góp bị tính lặp của các cây con được tô đỏ trong hình.

Độ phức tạp là $O(n \log^2 n)$, đủ để qua bài này.

11.4.3 Lời giải cuối

Nút thắt của lời giải trên nằm ở việc tính đóng góp của các cây con con nhẹ. Sau đây là cách cải thiện.

Với mỗi đường nặng, quét từ trên xuống dưới. Khi tới một đỉnh, lần lượt thêm tất cả các cây con con nhẹ của đỉnh đó, đồng thời duy trì một mảng tổng tiền tố. Nếu cây con con nhẹ đang thêm có độ sâu h , ta cập nhật h vị trí đầu của mảng tổng tiền tố: vị trí i được cộng số đỉnh trong cây con đó có độ sâu không vượt quá i .

Khi gặp truy vấn, trả lời trực tiếp bằng $O(1)$ trên mảng tổng tiền tố. Cách làm này có thời gian $O(n \log n)$, nhưng còn thiếu một phần đóng góp.

Phân tích thêm một chút, nếu $d_i = d$, phần thiếu chính là tổng kích thước của các cây con nhẹ đã được quét tới và có độ sâu nhỏ hơn d .

Để xử lý phần này, nhóm các cây con theo độ sâu. Cụ thể, cây con có độ sâu nằm trong $[2^i, 2^{i+1})$ được đưa vào nhóm i . Với từng nhóm và với toàn bộ nhóm, ta đều duy trì mảng tổng tiền tố. Với mảng tổng tiền tố của toàn bộ nhóm, mỗi lần thêm một cây con nhẹ tốn $O(\log n)$, và mỗi đỉnh chỉ được thêm một lần, nên phần này tốn $O(n \log n)$. Với mảng trong nhóm i , mỗi lần thêm tốn $O(2^i)$. Nếu tổng kích thước các cây con trong nhóm này là A , chi phí cập nhật không vượt quá $A \cdot 2^i / 2^i = A$. Tổng các A là $O(n \log n)$, nên phần này cũng tốn $O(n \log n)$.

Mỗi truy vấn chỉ cần đọc hai mảng tiền tố trong $O(1)$. Do đó bài toán được giải trong $O(n \log n)$.

11.5 Tổng kết bài toán

Sau một số biến đổi, bài toán ban đầu trở thành bài toán đếm số cặp gồm một đỉnh và một truy vấn, trong đó hai đối tượng này thỏa một số ràng buộc liên quan tới LCA.

Với dạng bài này, ta thường chia đóng góp thành hai phần:

- sau phân rã nặng nhẹ, các cây con con nặng của những đỉnh nằm trên đường từ đỉnh truy vấn tới gốc;
- sau phân rã nặng nhẹ, các cây con con nhẹ của những đỉnh nằm trên đường từ đỉnh truy vấn tới gốc, đồng thời trừ đi các cây con nhẹ chứa chính đỉnh truy vấn.

Nếu cả cập nhật và truy vấn đều làm được trong $O(1)$, độ phức tạp của thuật toán là $O(n \log n)$.

11.6 Ví dụ

11.6.1 Ví dụ 1: Codeforces 1098F Zh-function

Cho một chuỗi S . Ký hiệu $S[l \dots r]$ là chuỗi con từ ký tự thứ l tới ký tự thứ r . Định nghĩa

$$F(S) = \sum_{i=1}^{|S|} \text{lcp}(S, S[i \dots |S|]),$$

trong đó $\text{lcp}(S, T)$ là độ dài tiền tố chung dài nhất của hai chuỗi S, T . Cho chuỗi T dài n , có q truy vấn, mỗi truy vấn cho l, r và hỏi giá trị $F(T[l \dots r])$.

Với toàn bộ dữ liệu, $1 \leq n, q \leq 2 \cdot 10^5$. Giới hạn thời gian là 6 giây, bộ nhớ 512 MB.

Dựng cây hậu tố của chuỗi đảo của T . Khi đó lcp của hai hậu tố i, j chính là độ dài tương ứng với LCA của hai đỉnh trên cây hậu tố. Suy ra

$$\text{lcp}(T[l \dots r], T[i \dots r]) = \min(r - i + 1, \text{len}_{\text{lca}(i, l)}).$$

Công thức này rất giống lớp bài toán vừa trình bày.

Cần xét hai phần đóng góp. Phần thứ nhất là đóng góp của cây con con nặng của mỗi đỉnh đối với các truy vấn nằm trong những cây con nhẹ khác. Giả sử đỉnh hiện tại đại diện cho độ dài len . Nếu $i \in [l, r - \text{len})$, đóng góp là len , nên chỉ cần đếm số phần tử trong cây con có vị trí thuộc đoạn đó. Nếu $i \in [r - \text{len}, r]$, đóng góp là $r - i + 1$, nên cần thống kê cả tổng và số lượng trong đoạn. Cả hai đều là bài toán đếm điểm hai chiều đơn giản, có thể giải bằng gộp cây đoạn, thời gian $O(n \log^2 n)$.

Phần thứ hai là đóng góp của cây con con nhẹ đối với truy vấn trong cây con con nặng. Tương tự, phần này biến thành một bài toán thứ tự bộ phận ba chiều: một chiều yêu cầu vị trí nằm trong $[l, r]$, một chiều là độ sâu trên đường nặng, và chiều còn lại đến từ phép lấy min trong LCP. Với bài toán ba chiều, sắp xếp theo một chiều, rồi dùng CDQ chia để trị kết hợp Fenwick tree cho hai chiều còn lại. Phần này có độ phức tạp $O(n \log^3 n)$ với hằng số nhỏ.

Cũng có thể thay phân rã nặng nhẹ bằng cây nhị phân cân bằng toàn cục để bỏ một thừa số $\log$ của phần thứ hai, nhưng thực nghiệm không tốt bằng thuật toán trên nên bài viết gốc không trình bày chi tiết.

11.6.2 Ví dụ 2: Luogu P4482 [BJWC2018] Border bốn cách giải

Cho một chuỗi S dài n , có q truy vấn. Mỗi truy vấn cho hai số dương l, r và hỏi border của $S[l \dots r]$. Border của một chuỗi S là số lớn nhất i thỏa $i < |S|$ và

$$S[1 \dots i] = S[|S| - i + 1 \dots |S|].$$

Với toàn bộ dữ liệu, $1 \leq n, q \leq 2 \cdot 10^5$. Giới hạn thời gian là 5 giây, bộ nhớ 512 MB.

Dựng cây hậu tố cho S . Bài toán tương đương tìm i lớn nhất thỏa

$$i < r, \quad i - \text{len}_{\text{lca}(i, r)} + 1 \leq l.$$

Nếu giá trị tìm được nhỏ hơn l , tức là không tồn tại border.

Vấn tách thành hai phần. Phần thứ nhất tương đương truy vấn giá trị lớn nhất trong cây con có vị trí trên chuỗi gốc thuộc $[1, \min(r - 1, l + \text{len} - 1)]$. Phần này vẫn có thể xử lý bằng gộp cây đoạn trong $O(n \log^2 n)$.

Phần thứ hai cần thỏa hai ràng buộc: $i < r$ và $i - \text{len}_{\text{lca}(i, r)} + 1 \leq l$. Ta duy trì một cây đoạn; mỗi lần chèn giá trị $i - \text{len}_{\text{lca}(i, r)} + 1$ vào vị trí i . Mỗi nút của cây đoạn lưu giá trị nhỏ nhất trong đoạn hiện tại, nhờ đó khi truy vấn có thể nhị phân trên cây đoạn. Cần chú ý thông tin của bài này không có tính giảm, vì vậy phải xử lý riêng một chút. Phần này cũng có độ phức tạp $O(n \log^2 n)$.

11.7 Kết luận

Qua các ví dụ trên có thể thấy thuật toán này có tính tổng quát khi xử lý một lớp bài toán thống kê trên cây. Hy vọng bài viết đóng vai trò gợi mở và đem lại thêm cảm hứng cho các bài toán cây liên quan trong OI.

11.8 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang đã hướng dẫn; cảm ơn Dong Yehua, Chen Heli và Yuan Rongle đã bồi dưỡng, chỉ dạy; cảm ơn cha mẹ đã thấu hiểu và ủng hộ.

11.9 Tài liệu tham khảo

1. OI Wiki. Heavy-light decomposition.
2. Codeforces Round 530 Tutorial.

12 Bàn về ứng dụng của ma trận trong thi tin học

Trường Trung học số 7 Thành Đô, Tứ Xuyên
Yang Ningyuan

Tóm tắt

Ma trận có ứng dụng rất rộng trong tin học. Bài viết xuất phát từ khái niệm và tính chất cơ bản của ma trận, đưa ra các thuật toán tốt cho những công cụ như bổ đề LGV và đa thức đặc trưng, đồng thời thảo luận một số bài thi liên quan. Bài viết cũng giới thiệu phần bù đại số, cách tính hiệu quả của nó và ứng dụng trong định lý ma trận-cây. Cuối cùng, tác giả trình bày một số phương pháp tính nghiệm số của trị riêng như thuật toán Jacobi và lặp QR.

12.1 Dẫn nhập

Ma trận là một phần quan trọng của đại số tuyến tính. Các công cụ liên quan như định thức, phần bù đại số, đa thức đặc trưng, trị riêng và vector riêng thường mang lại góc nhìn bản chất hơn cho nhiều bài toán thi thuật toán.

Bài viết này hệ thống lại một số ứng dụng của ma trận trong OI. Phần 2 quy ước ký hiệu; phần 3 nói về định thức và bổ đề LGV; phần 4 trình bày phần bù đại số, ma trận phụ hợp và định lý ma trận-cây; phần 5 bàn về truy hồi tuyến tính của ma trận; phần 6 giới thiệu trị riêng, đa thức đặc trưng và ứng dụng; phần 7 thảo luận một số cách tính trị riêng.

12.2 Quy ước ký hiệu

Một bảng số gồm $n \times m$ phần tử xếp thành n hàng và m cột được gọi là ma trận $n \times m$. Với ma trận A , phần tử ở hàng i , cột j được ký hiệu là $a_{i,j}$. Khi $n = m$, ta gọi A là ma trận vuông cấp n .

Các phép toán cơ bản. Với hai ma trận A, B cùng kích thước, phép cộng và trừ được định nghĩa theo từng phần tử:

$$(A + B)_{i,j} = a_{i,j} + b_{i,j}, \quad (A - B)_{i,j} = a_{i,j} - b_{i,j}.$$

Nếu A là ma trận $n \times s$, B là ma trận $s \times m$, thì $C = AB$ là ma trận $n \times m$ với

$$c_{i,j} = \sum_{k=1}^s a_{i,k} b_{k,j}.$$

Nhân một ma trận với số x nghĩa là nhân mọi phần tử với x . Ma trận chuyển vị A^T của ma trận vuông A thỏa $(A^T)_{i,j} = a_{j,i}$.

12.3 Định thức

12.3.1 Định nghĩa và cách tính

Định nghĩa. Với ma trận vuông cấp n là A , định thức được định nghĩa bởi

$$|A| = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i,\sigma_i},$$

trong đó S_n là tập mọi hoán vị cấp n , còn $\text{sgn}(\sigma)$ bằng (-1) mũ số nghịch thế của σ .

Tính trực tiếp theo định nghĩa có độ phức tạp $O(n!)$, quá chậm. Ta dùng các phép biến đổi sơ cấp của ma trận:

1. cộng vào hàng i một bội k của hàng j , với $i \neq j$;
2. cộng vào cột i một bội k của cột j , với $i \neq j$;
3. nhân hàng i với $k \neq 0$;
4. đổi chỗ hai hàng i, j .

Hai phép đầu không đổi định thức, phép thứ ba nhân định thức với k , còn phép thứ tư đổi dấu định thức. Vì vậy có thể dùng khử Gauss để đưa ma trận về dạng tam giác và tính định thức trong $O(n^3)$. Nhìn theo ma trận biến đổi, quá trình này tương đương nhân trái hoặc nhân phải bởi các ma trận sơ cấp.

12.3.2 Một số tính chất

Ta có

$$|A| = |A^T|.$$

Lý do là phép lấy nghịch đảo hoán vị không đổi số nghịch thế; định thức của ma trận chuyển vị chính là cách nhìn này trên các cột.

Ngoài ra,

$$|AB| = |A||B|.$$

Khử A, B về các ma trận tam giác trên A', B' ; khi đó định thức của tích chỉ còn là tích các phần tử trên đường chéo, nên công thức trên suy ra ngay.

12.3.3 Bổ đề LGV

Xét đồ thị có hướng không chu trình G với trọng số trên cạnh. Cho tập điểm bắt đầu $a_1, \dots, a_n$ và tập điểm kết thúc $b_1, \dots, b_n$. Gọi $e(a, b)$ là tổng trọng số của mọi đường đi từ a tới b , trong đó trọng số đường đi là tích trọng số các cạnh. Đặt $M_{i,j} = e(a_i, b_j)$.

Bổ đề Lindstrom–Gessel–Viennot nói rằng $|M|$ bằng tổng có dấu của mọi bộ n đường đi đôi một không giao nhau, trong đó đường thứ i đi từ a_i tới b_{σ_i} , và dấu là $\text{sgn}(\sigma)$.

Ý tưởng chứng minh là ghép cặp các bộ đường đi có giao nhau. Chọn cặp đường đi đầu tiên giao nhau và đổi phần đuôi của chúng sau điểm giao đầu tiên. Trọng số không đổi nhưng dấu hoán vị đổi, nên các bộ có giao nhau triệt tiêu từng cặp. Chỉ còn lại các bộ không giao nhau.

12.3.4 Ví dụ LGV

Cho n, m, k, r, c, V . Cần đếm số ma trận $n \times m$ thỏa $1 \leq a_{i,j} \leq k$, các hàng và cột đều không giảm, và $a_{r,c} = V$, lấy modulo 998244353. Ràng buộc: $n, m \leq 200, k \leq 100$.

Bài toán tương đương chia ma trận thành đúng k lớp, lớp có thể rỗng, và yêu cầu ô $a_{r,c}$ thuộc lớp thứ V . Biên của mỗi lớp là một đường đi chỉ đi sang phải hoặc đi lên, các đường biên không được cắt nhau. Sau khi tịnh tiến đường thứ i sang phải $i - 1$ ô và xuống dưới $i - 1$ ô, điều kiện trở thành các đường đi không giao nhau, nên có thể dùng LGV.

Để ép điều kiện $a_{r,c} = V$, gán trọng số cho mỗi đường đi: trọng số 1 nếu đường không đi qua bên trái ô $a_{r,c}$, và trọng số x nếu có đi qua. Đáp án là hệ số $[x^{V-1}]|A|$. Vì $|A|$ là đa thức bậc không quá n , thay n giá trị rồi nội suy Lagrange cho lời giải $O(n^3k)$.

12.4 Phần bù đại số và ma trận phụ hợp

12.4.1 Định nghĩa và tính chất

Với ma trận vuông cấp n là A , phần bù $M_{i,j}$ của phần tử $a_{i,j}$ là định thức của ma trận thu được sau khi xóa hàng i và cột j . Phần bù đại số được định nghĩa là

$$A_{i,j} = (-1)^{i+j} M_{i,j}.$$

Khai triển Laplace theo hàng:

$$|A| = \sum_j a_{i,j} A_{i,j}.$$

Khai triển theo cột cũng tương tự, vì có thể chuyển sang ma trận A^T .

Ma trận phụ hợp A^* là chuyển vị của ma trận phần bù đại số, tức phần tử (i,j) của A^* là $A_{j,i}$. Khi $|A| \neq 0$, ta có

$$AA^* = |A|I.$$

Từ đây có thể tính A^* trong $O(n^3)$. Nếu $\text{rank}(A) = n$, thì $A^* = |A|A^{-1}$. Nếu $\text{rank}(A) \leq n - 2$, mọi phần bù đều bằng 0. Nếu $\text{rank}(A) = n - 1$, tìm vector cột $\vec{p}$ và vector hàng $\vec{q}$ khác không sao cho $A\vec{p} = 0$ và $\vec{q}A = 0$. Với $q_r \neq 0, p_c \neq 0$, ta có quan hệ

$$A_{i,j} = \frac{q_i p_j}{q_r p_c} A_{r,c}.$$

Chứng minh dùng các biến đổi sơ cấp trên các ma trận con còn lại sau khi xóa hàng và cột.

12.4.2 Định lý ma trận-cây

Với đồ thị vô hướng G , số cây khung bằng định thức của ma trận bậc còn lại sau khi lấy ma trận bậc trừ ma trận kề rồi xóa cùng một hàng và cột.

Với đồ thị có hướng, số cây khung hướng ra gốc i bằng định thức của ma trận thu được từ ma trận bậc vào trừ ma trận cạnh, sau khi xóa hàng i , cột i . Tương tự, số cây hướng vào gốc dùng ma trận bậc ra trừ ma trận cạnh.

Ví dụ: cho đồ thị có hướng n đỉnh, với mỗi đỉnh i cần tính số cây hướng ra gốc i . Với $n \leq 500$, bài toán chính là tính mọi phần bù chéo $A_{i,i}$; dùng phương pháp tính ma trận phụ hợp ở trên đạt $O(n^3)$.

12.5 Truy hồi tuyến tính của ma trận

12.5.1 Bài toán dẫn nhập

Cho ma trận vuông cấp n là A , cần tính A^k modulo $10^9 + 7$, với $n \leq 100$ và $k \leq 10^{10000}$. Lũy thừa ma trận trực tiếp tốn $O(n^3 \log k)$, vẫn quá chậm khi số mũ rất lớn.

Đa thức đặc trưng. Đa thức đặc trưng của A là

$$p_A(\lambda) = |A - \lambda I|.$$

Theo định lý Cayley–Hamilton,

$$p_A(A) = a_0 + a_1 A + \dots + a_n A^n = 0.$$

Nếu biết p_A và các A^i với $i \leq n$, ta biểu diễn A^k thành tổ hợp tuyến tính của các lũy thừa đó. Hệ số chính là $x^k \bmod p_A(x)$, có thể tính bằng lũy thừa đa thức trong $O(n^2 \log k)$.

Duy trì đa thức trong khử Gauss. Thay đường chéo của A bằng $a_{i,i} - x$, rồi khử Gauss trên các đa thức modulo x^{n+1} . Tích các phần tử đường chéo cho đa thức đặc trưng. Do phải nhân đa thức, độ phức tạp là $O(n^5)$.

Nội suy Lagrange. Thay $\lambda = 0, 1, \dots, n$, tính $|A - \lambda I|$, rồi nội suy từ các điểm giá trị để thu được p_A . Độ phức tạp $O(n^4)$.

Đưa về ma trận Hessenberg. Nếu biến đổi đồng dạng

$$A' = P_1 P_2 \dots P_k A P_k^{-1} \dots P_2^{-1} P_1^{-1},$$

thì $|A - \lambda I| = |A' - \lambda I|$. Dùng các phép biến đổi hàng/cột tương ứng để đưa A về dạng Hessenberg, tức $a_{i,j} = 0$ với mọi $i > j + 1$. Định thức của ma trận Hessenberg có thể tính bằng DP $O(n^2)$. Kết hợp thay điểm và nội suy cho thuật toán $O(n^3)$ để tìm đa thức đặc trưng.

Berlekamp–Massey. Thuật toán Berlekamp–Massey tìm truy hồi ngắn nhất của một dãy trong $O(n^2)$. Áp dụng trực tiếp cho ma trận tốn thêm chi phí cộng ma trận, thành $O(n^4)$. Có thể chọn ngẫu nhiên vector hàng a và vector cột b , rồi xét dãy $aA^k b$. Khi đó nút thắt là nhân vector với ma trận, tổng $O(n^3)$. Theo Schwartz–Zippel, xác suất tìm sai truy hồi rất nhỏ; truy hồi ngắn nhất thu được tương ứng với số trị riêng khác nhau của ma trận.

12.5.2 Ví dụ truy hồi

Cho đồ thị có hướng n đỉnh, m cạnh, cạnh có trọng số. Trọng số đỉnh ban đầu là a_i , và mỗi thời điểm cập nhật

$$a'_u = \sum_{(v,u) \in E} a_v w(v,u).$$

Hỏi trọng số mọi đỉnh sau k thời điểm, modulo $10^9 + 7$, với $n \leq 5000$, $k \leq 10^{18}$.

Khử Hessenberg $O(n^3)$ là không chấp nhận được. Dùng Berlekamp–Massey: đề bài đã cho vector cột a , chỉ cần chọn ngẫu nhiên vector hàng b . Tính trước $n + 1$ trạng thái đầu, nhân với b để lấy dãy vô hướng, tìm truy hồi, rồi dùng lũy thừa đa thức để tính tổ hợp tuyến tính cần thiết.

12.6 Trị riêng và vector riêng

Nếu tồn tại vector khác không v sao cho

$$Av = \lambda v,$$

thì λ là trị riêng, v là vector riêng. Tương đương $(A - \lambda I)v = 0$, nên $|A - \lambda I| = 0$; tức λ là nghiệm của đa thức đặc trưng.

12.6.1 Chéo hóa ma trận

Nếu ma trận vuông cấp n có n trị riêng phân biệt, nó có n vector riêng độc lập tuyến tính. Ghép các vector riêng thành ma trận V , đặt B là ma trận đường chéo chứa các λ_i . Khi đó

$$AV = VB, \quad V^{-1}AV = B.$$

Vì

$$A^k = P(P^{-1}AP)^kP^{-1},$$

chéo hóa biến bài toán tính lũy thừa ma trận thành nhân ma trận và lũy thừa các phần tử đường chéo.

12.6.2 Ví dụ 1: CF1540E Tasty Dishes

Có n đầu bếp đánh số $1, \dots, n$, năng lực của đầu bếp i là i . Ban đầu món của đầu bếp i có độ ngon a_i , $|a_i| \leq i$. Mỗi đầu bếp có danh sách những đầu bếp được phép sao chép, và chỉ có thể sao chép từ người có kỹ năng cao hơn. Mỗi ngày, họ có thể nhân độ ngon của mình với năng lực, rồi sau đó chọn cộng thêm độ ngon của các đầu bếp trong danh sách. Mọi người đều tối đa hóa độ ngon của mình. Truy vấn gồm hỏi tổng độ ngon đoạn $l \dots r$ sau k ngày, hoặc tăng a_i thêm $x > 0$.

Với $1 \leq n \leq 300$, $1 \leq q \leq 2 \cdot 10^5$, $k \leq 10^3$, lời giải xét riêng đóng góp của từng đầu bếp. Gọi d_i là thời điểm sớm nhất mà đóng góp của người i trở nên dương. Thời điểm này chỉ phụ thuộc vào tập các $a_i > 0$ và thay đổi nhiều nhất $O(n)$ lần.

Nếu T là ma trận chuyển, $\vec{e}_i$ là vector đơn vị, cần tính

$$\sum_{d_i \leq k} T^{k-d_i} \vec{e}_i a_i + \sum_{d_i > k} \vec{e}_i a_i.$$

Tách $\vec{e}_i$ thành tổ hợp tuyến tính của các vector riêng của T , tiền xử lý $O(n^3)$, rồi dùng n Fenwick tree để duy trì hệ số theo từng trị riêng. Độ phức tạp cuối là $O(n^3 + qn \log n)$.

12.6.3 Ví dụ 2: ARC133F Random Transition

Cho biến ngẫu nhiên x . Với $1 \leq i \leq n$, $x = i$ với xác suất $a_i \cdot 10^{-9}$. Mỗi thời điểm, x giảm 1 với xác suất x/n và tăng 1 với xác suất $(n-x)/n$. Hỏi xác suất cuối cùng $x = i$ modulo 998244353, với $n \leq 10^5$.

Viết ma trận chuyển cấp $n+1$. Trị riêng thứ $i+1$ theo thứ tự tăng là

$$\lambda_i = \frac{2i}{n} - 1.$$

Vector riêng tương ứng là hệ số của đa thức

$$(1+x)^i(1-x)^{n-i},$$

tức $(v_i)_j = [x^j](1+x)^i(1-x)^{n-i}$. Nếu ghép các v_i thành P , thì cột thứ $i+1$ của P^{-1} là $n^{-2}v_{n-i}$ theo cách chuẩn hóa trong bài gốc.

Với $b = Pa$,

$$b_i = \sum_{j=0}^n a_j [x^i] (1+x)^j (1-x)^{n-j}.$$

Vì vậy cần tính

$$\sum_{i=0}^n a_i (1+x)^i (1-x)^{n-i}.$$

Có thể dùng chia để trị NTT $O(n \log^2 n)$, nhưng biến đổi $t = 1+x$ cho phép tách các hệ số tổ hợp và dùng NTT để đạt $O(n \log n)$. Tính $P^{-1}a$ cũng tương tự; phần giữa chỉ cần n lần lũy thừa nhanh, nên tổng độ phức tạp $O(n \log n)$.

12.6.4 Tiểu kết

Hai ví dụ trên dùng trị riêng và vector riêng để biến lũy thừa ma trận khó tính thành lũy thừa các phần tử trên đường chéo. Trong nhiều bài, nếu chưa nhìn ra tính chất sinh hàm, việc lập ma trận chuyển rồi tính thử vector riêng có thể gợi ra quy luật.

12.7 Tính trị riêng

Các phần trước chủ yếu xét trường hợp trị riêng có thể suy ra trực tiếp. Phần này giới thiệu ba phương pháp nghiệm số và một phương pháp trong modulo số nguyên tố.

12.7.1 Phương pháp lũy thừa

Với ma trận A cấp n , nếu trị riêng có trị tuyệt đối lớn nhất là duy nhất, có thể tìm trị riêng đó và vector riêng tương ứng bằng lặp lũy thừa. Chọn vector cột khác không x , rồi lặp:

1. tính $x' = Ax$;
2. chuẩn hóa x' để độ dài bằng 1;
3. gán $x = x'$.

Sau đủ nhiều bước, x hội tụ về vector riêng ứng với trị riêng có trị tuyệt đối lớn nhất. Nếu phân tích $x = \sum_i p_i v_i$, sau k lần lặp ta có $\sum_i \lambda_i^k p_i v_i$; các thành phần nhỏ hơn bị áp đảo bởi $|\lambda|_{\max}^k$. Đảo ngược quá trình có thể tìm trị riêng có trị tuyệt đối nhỏ nhất.

12.7.2 Phương pháp Jacobi

Với ma trận thực đối xứng $A = A^T$, dùng các phép quay Givens $R(p, q, \theta)$. Ma trận này giữ nguyên các tọa độ ngoài hai chỉ số p, q , còn trên mặt phẳng (p, q) là phép quay bởi góc θ .

Đặt $B = R(p, q, \theta)AR^T(p, q, \theta)$. Chọn θ sao cho

$$\tan 2\theta = \frac{2a_{p,q}}{a_{q,q} - a_{p,p}},$$

khi đó $b_{p,q} = b_{q,p} = 0$. Đồng thời tổng bình phương mọi phần tử không đổi, nên tổng bình phương đường chéo tăng thêm $2a_{p,q}^2$.

Mỗi lần chọn phần tử ngoài đường chéo có trị tuyệt đối lớn nhất để triệt tiêu. Sau đủ nhiều lần, phần ngoài đường chéo tiến gần 0; các phần tử đường chéo xấp xỉ trị riêng, còn tích các ma trận quay đã dùng cho vector riêng. Nếu $T(A)$ là tổng bình phương phần tử ngoài đường chéo, số lần lặp cỡ $O(n^2(\log T(A) - \log \varepsilon))$. Với cấu trúc dữ liệu phù hợp, độ phức tạp có thể đạt khoảng $O(n^3(\log T(A) - \log \varepsilon))$.

12.7.3 Phân rã QR và biến đổi Householder

Mọi ma trận thực không suy biến A đều có phân rã QR:

$$A = QR,$$

trong đó Q là ma trận trực giao, R là ma trận tam giác trên. Nếu yêu cầu đường chéo của R dương, phân rã là duy nhất.

Biến đổi Householder dùng để tính QR. Với cột đầu x của A , đặt $e_1 = (1, 0, \dots, 0)^T$, lấy

$$v = x - |x|e_1, \quad \omega = \frac{v}{|v|}, \quad Q = I - 2\omega\omega^T.$$

Khi đó QA có cột đầu chỉ còn phần tử hàng đầu khác 0. Lặp đệ quy như khử Gauss để thu được $R = Q_n \cdots Q_2 Q_1 A$, và $A = Q_1^T Q_2^T \cdots Q_n^T R$.

12.7.4 Lặp QR

Với ma trận không suy biến A , phân rã $A_1 = Q_1 R_1$, rồi lặp

$$A_{k+1} = R_k Q_k = Q_k^T A_k Q_k = Q_{k+1} R_{k+1}.$$

Nếu trị riêng của A có trị tuyệt đối đôi một khác nhau, A_k hội tụ về một ma trận tam giác trên T . Vì T đồng dạng với A , các phần tử đường chéo của T chính là trị riêng. Sau đó có thể giải hệ tuyến tính bằng khử Gauss để tìm vector riêng. Bài gốc không chứng minh hội tụ do giới hạn độ dài.

12.7.5 Tìm nghiệm đa thức

Trên trường thực, tìm nghiệm đa thức đặc trưng thường kém ổn định số, nên ít dùng trực tiếp. Trong modulo số nguyên tố thì cách này hữu ích. Một cách phân tích đa thức modulo p : chọn ngẫu nhiên dịch chuyển d , xét

$$\gcd(F(x+d), x^{(p-1)/2} - 1).$$

Mỗi lần kỳ vọng tách được khoảng một nửa số nghiệm của F ; sau đó đệ quy trên hai phần. Tính đa thức đặc trưng bằng thuật toán Hessenberg ở phần trước, tách toàn bộ nghiệm để có trị riêng, rồi chạy n lần khử Gauss để lấy vector riêng.

12.8 Kết luận

Bài viết khảo sát ứng dụng của định thức, phương trình đặc trưng, trị riêng và vector riêng trong thi tin học. Quy hoạch động có liên hệ rất chặt với ma trận: nhiều DP có thể tối ưu bằng lũy thừa ma trận. Với các bài đếm khó tối ưu, thử viết ma trận chuyển, tìm vector riêng và quan sát bảng giá trị đôi khi đem lại quy luật quan trọng.

Các thuật toán nghiệm số cho trị riêng và vector riêng còn nhiều hướng khác; do giới hạn của bài viết, tác giả chỉ giới thiệu một phần và khuyến khích người đọc tự nghiên cứu thêm.

12.9 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn cha mẹ đã quan tâm và ủng hộ; cảm ơn thầy Lin Hong, thầy Lin Yang và thầy Ye Shifu đã bồi dưỡng; cảm ơn tất cả những người đã đồng hành trên con đường OI.

12.10 Tài liệu tham khảo

1. Characteristic polynomial, Wikipedia.
2. Berlekamp–Massey algorithm, Wikipedia.
3. Eigenvalues and eigenvectors, Wikipedia.
4. QR decomposition, Wikipedia.
5. Householder transformation, Wikipedia.
6. Iterative eigenvalue method based on QR decomposition.
7. Jacobi eigenvalue algorithm, Wikipedia.
8. Computational methods for nonsymmetric eigenvalue problems.
9. Classical Jacobi method for matrix eigenvalues.

13 Bàn về một số bài toán liên quan tới matching đồ thị hai phía

Trường THPT số 2 trực thuộc Đại học Sư phạm Hoa Đông
Ke Yisi

Tóm tắt

Matching trên đồ thị hai phía là một lớp bài toán thường gặp trong thi tin học. Bài viết trước hết tổng kết một số dạng matching hai phía đặc biệt, sau đó nghiên cứu các bài toán tối ưu trên tập có thể ghép được.

13.1 Dẫn nhập

Nhiều bài toán thi thuật toán ẩn chứa cấu trúc matching. Bài viết này tập trung vào vài lớp bài toán liên quan: định nghĩa cơ bản về matching hai phía, một số dạng bài đặc biệt, tính chất matroid của tập có thể ghép được, và một thuật toán tìm tập có thể ghép cực đại nhỏ nhất theo thứ tự từ điển.

13.2 Kiến thức chuẩn bị

Một đồ thị vô hướng $G = (V, E)$ là đồ thị hai phía nếu tập đỉnh V có thể chia thành V_1, V_2 , $V_1 \cap V_2 = \emptyset$, $V_1 \cup V_2 = V$, và mọi cạnh đều nối một đỉnh của V_1 với một đỉnh của V_2 .

Một matching M của đồ thị hai phía là tập con của E sao cho không có hai cạnh nào trong M chung đầu mút. Kích thước matching là $|M|$. Nếu tồn tại matching phủ toàn bộ V_1 , ta nói có matching hoàn hảo của V_1 . Nếu $|M|$ lớn nhất trong mọi matching, M là matching cực đại.

Với $S \subseteq V_1$, ký hiệu

$$N(S) = \{v \in V_2 \mid \exists u \in S, (u, v) \in E\}.$$

Theo định lý Hall, đồ thị hai phía $G = (V_1, V_2, E)$ có matching hoàn hảo của V_1 khi và chỉ khi với mọi $S \subseteq V_1$, $|S| \leq |N(S)|$.

Thuật toán kinh điển để tìm matching cực đại là thuật toán Hungary: lần lượt thêm từng $u \in V_1$ và tìm đường tăng. Độ phức tạp $O(|V||E|)$. Một điểm quan trọng trong cách nhìn này là khi một đỉnh đã vào matching, nó sẽ luôn nằm trong matching sau các lần tăng tiếp theo.

13.3 Các bài toán matching liên quan tới đoạn

Xét lớp bài toán cho một số đoạn $[l_i, r_i]$ và một số điểm x_i . Những bài toán này thường giải được bằng tham lam hoặc định lý Hall.

13.3.1 Bài toán 3.1

Cho n đoạn $[l_i, r_i]$ và m điểm x_i . Đoạn $[l_i, r_i]$ ghép được với điểm x_j khi và chỉ khi $x_j \in [l_i, r_i]$. Cần tìm matching cực đại.

Thuật toán tham lam cổ điển: sắp xếp các đoạn theo đầu phải tăng dần. Lần lượt xét từng đoạn; nếu còn điểm chưa ghép nằm trong đoạn, chọn điểm có tọa độ nhỏ nhất. Nếu đoạn $[l, r]$ có thể ghép với $x_i < x_j$, thì mọi đoạn còn lại ghép được với x_i cũng ghép được với x_j , nên dùng x_i cho đoạn hiện tại không làm xấu lời giải.

13.3.2 Bài toán 3.2

Cho n đoạn $[l_i, r_i]$ và m đoạn $[x_i, y_i]$. Đoạn thứ nhất ghép được với đoạn thứ hai khi $[x_j, y_j] \subseteq [l_i, r_i]$. Cần tìm matching cực đại.

Tương tự, sắp xếp $[l_i, r_i]$ theo đầu phải tăng dần. Với mỗi đoạn, chọn một đoạn chưa ghép nằm trong nó có đầu trái nhỏ nhất. Chứng minh đúng giống bài toán điểm-đoạn.

Ví dụ: Clique. Cho n cung trên vòng tròn, dựng đồ thị giao nhau của các cung và hỏi clique lớn nhất, với $n \leq 3000$. Nếu bỏ các cung đi qua vị trí 1 khỏi một clique bất kỳ, các cung còn lại có một điểm chung. Liệt kê điểm chung i , chia các cung thành bốn loại: đi qua cả 1 và i , chỉ qua 1, chỉ qua i , và không qua điểm nào. Loại đầu chắc chắn thuộc clique, loại cuối không thuộc clique trong trường hợp đang xét. Giữ hai loại giữa, xét đồ thị bù của chúng; đồ thị bù là hai phía, và clique lớn nhất trở thành tổng số đỉnh trừ matching cực đại. Sau khi lấy đối xứng một phía của các cung, bài toán quy về Bài toán 3.2. Liệt kê $O(n)$ điểm chung cho độ phức tạp $O(n^2 \log n)$.

13.3.3 Bài toán 3.3

Cho n đoạn $[l_i, r_i]$ và $n - 1$ điểm x_i . Có q truy vấn độc lập, mỗi truy vấn thêm một điểm y và hỏi có matching hoàn hảo hay không.

Dùng tham lam của Bài toán 3.1. Sắp xếp đoạn theo đầu phải, quét để tìm đoạn đầu tiên chưa có điểm ghép được. Khi đó y phải không vượt quá đầu phải của đoạn này. Bỏ qua đoạn đó và quét tiếp; nếu lại có đoạn không ghép được, thì không có y nào hợp lệ. Làm tương tự theo chiều ngược lại để thu được cận dưới. Gọi hai cận là L, R .

Ta chứng minh mọi $y \in [L, R]$ đều hợp lệ. Với n đoạn và n điểm, tồn tại matching hoàn hảo khi và chỉ khi với mọi đoạn $[L_0, R_0]$,

$$\#\{i \mid [l_i, r_i] \subseteq [L_0, R_0]\} \leq \#\{i \mid x_i \in [L_0, R_0]\}.$$

Sự cần thiết hiển nhiên; sự đủ suy ra từ Hall, vì hợp của các đoạn có thể tách thành các đoạn liên tục. Đặt

$$F(l, r) = \#\{i \mid [l_i, r_i] \subseteq [l, r]\} - \#\{i \mid x_i \in [l, r]\}.$$

Do hiện có matching kích thước $n - 1$, không có $F(l, r) > 1$. Những đoạn có $F(l, r) = 1$ buộc điểm mới phải nằm trong giao của chúng, chính là khoảng $[L, R]$.

Ví dụ: *Permutation for Burenka*. Hai mảng cùng độ dài được gọi là tương tự nếu trong mọi đoạn con, vị trí của phần tử lớn nhất đều giống nhau. Cho hoán vị p và mảng a có k vị trí chưa biết. Cho tập S gồm $k - 1$ số; mỗi truy vấn cho d , hỏi có thể điền các số của $S \cup \{d\}$ vào các vị trí trống để a tương tự p hay không. Điều kiện tương tự tương đương hai mảng có cùng cây Cartesian, vì vậy xác định được khoảng giá trị cho từng vị trí trống. Bài toán trở thành matching hoàn hảo giữa k đoạn và k điểm, dùng lời giải của Bài toán 3.3.

13.3.4 Bài toán 3.4

Trong trường hợp đặc biệt của Bài toán 3.1 với mọi $r_i = +\infty$, tức mỗi đoạn chỉ có ràng buộc trái, có cách kiểm tra matching hoàn hảo gọn hơn. Duy trì mảng trên miền giá trị: với mỗi l_i , cộng 1 trên $[l_i, +\infty)$; với mỗi điểm x_i , trừ 1 trên $[x_i, +\infty)$. Có matching hoàn hảo khi và chỉ khi mọi vị trí của mảng đều không âm. Điều này suy ra trực tiếp từ Hall.

Ví dụ 1: *Examination*. Cho hai mảng A_i, B_i , cần chọn một số vị trí và hoán vị lại các A_i ở những vị trí đó sao cho mọi $A_i \geq B_i$. Hỏi số vị trí chọn ít nhất. Mọi vị trí $A_i < B_i$ chắc chắn phải chọn. Chọn thêm vị trí j tương ứng với cộng 1 trên đoạn $[B_j, A_j)$ của mảng kiểm tra. Bài toán thành chọn ít nhất các đoạn để mọi vị trí không âm. Tham lam: mỗi lần lấy vị trí âm ngoài cùng bên trái, chọn trong các đoạn còn lại một đoạn chứa nó và có đầu phải lớn nhất. Lặp nhiều nhất n lần, độ phức tạp $O(n \log n)$.

Ví dụ 2: *Two Faced Cards*. Có n thẻ, thẻ i có mặt trước A_i , mặt sau B_i , và có $n + 1$ số C_i . Mỗi truy vấn thêm một thẻ (D, E) . Cần lật một số thẻ sao cho $n + 1$ thẻ ghép được với $n + 1$ số, với điều kiện số ở mặt trước không vượt quá số được ghép, và hỏi số lần lật ít nhất.

Nếu ban đầu quy ước không lật, lật thẻ i tương ứng với cộng 1 trên $[B_i, A_i)$, còn thẻ mới mặt trước x tương ứng với cộng 1 trên $[x, +\infty)$. Bài toán trở thành: với từng x , chọn ít nhất các đoạn để $[0, x)$ đều không âm và $[x, +\infty)$ đều không nhỏ hơn -1 . Có thể giải bằng tham lam hai pha: trước hết xử lý các vị trí nhỏ hơn -1 từ phải sang trái, chọn đoạn có đầu trái nhỏ nhất; sau đó xử lý các vị trí âm từ trái sang phải, chọn đoạn có đầu phải lớn nhất và duy trì đáp án theo x .

13.4 Tính chất matroid của tập có thể ghép

Một matroid $M = (S, \mathcal{I})$ gồm tập hữu hạn S và họ tập độc lập $\mathcal{I}$ thỏa:

- tính di truyền: nếu $I \in \mathcal{I}, J \subset I$, thì $J \in \mathcal{I}$;
- tính trao đổi: nếu $I, J \in \mathcal{I}, |J| > |I|$, thì tồn tại $z \in J \setminus I$ sao cho $I \cup \{z\} \in \mathcal{I}$.

Trong đồ thị hai phía $G = (V_1, V_2, E)$, một tập $S \subseteq V_1$ được gọi là tập có thể ghép nếu có matching phủ mọi đỉnh của S . Gọi $\mathcal{I}$ là họ các tập có thể ghép trên V_1 . Khi đó $(V_1, \mathcal{I})$ là một matroid.

Tính di truyền là hiển nhiên. Với tính trao đổi, lấy $S_1, S_2 \in \mathcal{I}, |S_1| < |S_2|$, và các matching tương ứng M, N . Xét hiệu đối xứng của M, N . Các thành phần liên thông chỉ là đỉnh cô lập, chu trình chẵn với cạnh xen kẽ thuộc $M \setminus N$ và $N \setminus M$, hoặc đường đi xen kẽ. Vì $|N| > |M|$, tồn tại một đường đi bắt đầu và kết thúc bằng cạnh thuộc $N \setminus M$; đây là đường tăng của S_1 , nên có thể thêm một đỉnh $x \in S_2 \setminus S_1$.

Vì vậy các kết quả của matroid, như tham lam cho tập độc lập trọng số lớn nhất và định lý đối cơ sở, có thể áp dụng cho tập có thể ghép.

Ví dụ: quản lý lịch trình. Có q thao tác: thêm một nhiệm vụ có hạn chót t_i , lợi ích p_i , hoặc xóa một nhiệm vụ đã có. Nếu hoàn thành nhiệm vụ không muộn hơn ngày t_i , ta nhận p_i . Mỗi nhiệm vụ tốn một ngày, mỗi ngày làm nhiều nhất một nhiệm vụ. Sau mỗi thao tác hỏi lợi ích lớn nhất.

Với tập nhiệm vụ cố định, đây là bài toán tập độc lập trọng số lớn nhất trong matroid, nên thêm nhiệm vụ theo thứ tự lợi ích giảm dần. Khi thêm một nhiệm vụ, nếu tập vẫn độc lập thì giữ nó; nếu không, xuất hiện đúng một vòng và ta bỏ phần tử có trọng số nhỏ nhất trong vòng. Khi xóa, chỉ cần thêm lại phần tử có trọng số lớn nhất trong phần còn lại mà vẫn có thể thêm được, và nhiều nhất chỉ có một phần tử như vậy. Việc kiểm tra độc lập giống Bài toán 3.4: duy trì mảng trên miền thời gian. Để tìm vòng, lấy vị trí âm ngoài cùng bên trái p , khi đó các nhiệm vụ có $t_i \leq p$ nằm trong vòng. Để tìm phần tử có thể thêm, lấy vị trí bằng 0 ngoài cùng bên phải p , các nhiệm vụ có $t_i > p$ có thể thêm. Dùng cây đoạn, độ phức tạp $O(q \log T)$.

13.5 Tập có thể ghép cực đại nhỏ nhất theo thứ tự từ điển

Với hai tập A, B cùng kích thước, nói A nhỏ hơn B theo thứ tự từ điển nếu $A \neq B$ và phần tử nhỏ nhất của hiệu đối xứng $A \triangle B$ nằm trong A . Với đồ thị hai phía tổng quát, nếu tăng các đỉnh của V_1 theo thứ tự chỉ số tăng dần, ta thu được tập có thể ghép cực đại nhỏ nhất theo thứ tự từ điển. Phần này trình bày cách làm tốt hơn khi có thuật toán tìm phủ đỉnh nhỏ nhất nhanh.

Gọi tập này là matching tối ưu. Một tập $S \subseteq V$ là phủ đỉnh nếu mọi cạnh đều có ít nhất một đầu mút trong S . Nếu có thể tính phủ đỉnh nhỏ nhất của đồ thị hai phía $G = (V_1, V_2, E)$ trong $T(|V_1|, |V_2|)$, thì có thể tính tập tối ưu trên V_1 trong

$$T(|V_1|, |V_2|) \log |V_1|.$$

Định nghĩa thủ tục $\text{solve}(S_1, S_2, S_3)$: giải đồ thị hai phía $(S_1 \cup S_2, S_3)$, trong đó mọi phần tử của S_2 có chỉ số nhỏ hơn mọi phần tử của S_1 , và S_2 chắc chắn thuộc lời giải tối ưu. Bài toán ban đầu là $\text{solve}(V_1, \emptyset, V_2)$.

Chia S_1 thành hai nửa theo chỉ số: nửa nhỏ X và nửa lớn Y . Tìm phủ đỉnh nhỏ nhất C của đồ thị $(S_2 \cup X, S_3)$. Đặt

$$B_l = (S_2 \cup X) \cap C, \quad A_l = (S_2 \cup X) \setminus B_l, \quad A_r = S_3 \cap C, \quad B_r = S_3 \setminus A_r.$$

Các tính chất cần dùng:

- Đồ thị (B_l, B_r) có matching cực đại kích thước $|B_l|$. Trong matching cực đại tương ứng với phủ đỉnh nhỏ nhất, mỗi cạnh matching có đúng một đầu trong phủ đỉnh, nên mọi đỉnh của B_l ghép sang B_r .
- Matching tối ưu của $(B_l \cup Y, B_r)$ chắc chắn chứa B_l , vì các đỉnh của Y có chỉ số lớn hơn.
- Matching tối ưu của $(S_2 \cup X, S_3)$ là hợp của matching tối ưu trên (A_l, A_r) và (B_l, B_r) . Không có cạnh giữa A_l và B_r , còn cạnh giữa B_l và A_r không thể là cạnh matching trong cấu trúc tối ưu này.
- Matching tối ưu của $(S_1 \cup S_2, S_3)$ là hợp của matching tối ưu trên (A_l, A_r) và $(B_l \cup Y, B_r)$. Khi lần lượt tăng các đỉnh của Y , đường tăng không thể đi vào vùng (A_l, A_r) rồi tìm được đỉnh chưa ghép ở đó.

Do đó ta đệ quy thành

$$\text{solve}(A_l \cap S_1, A_l \cap S_2, A_r) \cup \text{solve}(Y, B_l, B_r).$$

Chọn X, Y có kích thước gần bằng nhau thì kích thước phần cần chia đôi giảm một nửa ở mỗi mức; mỗi đỉnh đi vào đúng một nhánh, nên độ phức tạp là $T(|V_1|, |V_2|) \log |V_1|$. Bài gốc viết thủ tục này dưới dạng giả mã; bản dịch giữ lại mô tả thuật toán thay vì chép lại khối mã giả.

Ví dụ: *Insects*. Cho n điểm đen (a_i, b_i) trên mặt phẳng, rồi lần lượt thêm m điểm trắng (x_i, y_i) . Điểm đen i ghép được với điểm trắng j khi và chỉ khi $a_i \leq x_j, b_i \leq y_j$. Sau mỗi lần thêm điểm trắng, hỏi matching cực đại, với mọi tọa độ và $n, m \leq 10^5$.

Cách trực tiếp là tăng theo từng điểm trắng; thực chất đang tìm tập điểm trắng có thể ghép cực đại nhỏ nhất theo thứ tự từ điển. Quy trình tìm matching cực đại tương tự Bài toán 3.2. Để tìm phủ đỉnh nhỏ nhất, dùng cấu trúc kinh điển từ matching cực đại và tối ưu hóa đồ thị bằng cây đoạn để đạt $O(n \log n)$ cho mỗi mức. Tổng độ phức tạp là $O(n \log^2 n)$.

Ý nghĩa của thuật toán trong phần này là: với một số đồ thị hai phía đặc biệt, ta thường có thể tăng theo một thứ tự đặc biệt để tính matching nhanh; thuật toán trên giúp tính kết quả tăng theo thứ tự bất kỳ với chi phí vẫn khá tốt.

13.6 Kết luận

Bài viết tổng kết ứng dụng của tham lam và định lý Hall trong vài dạng matching hai phía đặc biệt, giới thiệu một số kết quả tối ưu trên tập có thể ghép, và đưa ra thuật toán tìm tập có thể ghép cực đại nhỏ nhất theo thứ tự từ điển. Tác giả cũng nhấn mạnh rằng matching hai phía còn nhiều hướng đáng nghiên cứu.

13.7 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn huấn luyện viên Jin Jing đã hướng dẫn; cảm ơn cha mẹ đã nuôi dưỡng và dạy dỗ; cảm ơn Ke Yijing đã giúp đỡ bài viết; cảm ơn tất cả bạn học và thầy cô đã hỗ trợ.

13.8 Tài liệu tham khảo

1. Wikipedia. Hungarian algorithm.
2. Wikipedia. Hall's marriage theorem.
3. Yang Qianlan. Bàn về một số mở rộng của matroid và ứng dụng, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2018.
4. OI Wiki. Minimum vertex cover in bipartite graphs and Konig's theorem.

14 Bàn về ứng dụng đặc biệt của DSU trong OI

Trường Trung học số 2 Hàng Châu
Wang Xiangwen

Tóm tắt

DSU là một thuật toán mộc mạc nhưng hiệu quả, có phạm vi ứng dụng rộng. Bài viết giới thiệu ba mô hình DSU thường dùng và dùng nhiều ví dụ để minh họa ứng dụng của ba mô hình này.

14.1 Dẫn nhập

DSU là thuật toán mà người học tin học thường gặp rất sớm, nhưng nhiều người dừng lại ở dạng cơ bản nhất. Thực tế, DSU có khả năng mở rộng mạnh; trong không ít bài toán, nó cho lời giải ngắn gọn và nhanh hơn cách kinh điển. Bài viết này hệ thống lại các ứng dụng ít gặp hơn của DSU trong OI.

14.2 Giới thiệu DSU

DSU là cấu trúc dữ liệu quản lý phần tử thuộc tập nào. Nó thường được hiện thực bằng một rừng, mỗi cây biểu diễn một tập và các đỉnh trong cây là phần tử của tập đó. Hai thao tác cơ bản là:

1. $\text{Union}(x, y)$: hợp nhất hai tập chứa x và y ;
2. $\text{Find}(x)$: tìm đại diện của tập chứa x .

Chỉ cần duy trì cha fa của mỗi đỉnh. Find nhảy cha tới gốc cây, còn Union gắn một gốc vào gốc còn lại. Cách mặc định có thể tốn $O(nq)$.

14.2.1 Hợp nhất theo kích thước

Độ phức tạp xấu của Find đến từ chiều cao cây. Ta duy trì thêm kích thước cây con $size$, và khi hợp nhất, gắn gốc của cây nhỏ hơn vào gốc của cây lớn hơn. Khi một đỉnh không còn là gốc, kích thước tập mà nó thuộc về ít nhất tăng gấp đôi, nên chiều cao là $O(\log n)$. Tổng thời gian $O(q \log n)$.

14.2.2 Nén đường đi

Sau một lần $\text{Find}(u)$, ta đặt cha của mọi đỉnh trên đường từ u tới gốc thành chính gốc. Nếu cây ban đầu là một dây, sau khi tìm ở lá thì nó gần như trở thành một cây hình sao. Với $n \leq q$, DSU chỉ nén đường đi có độ phức tạp xấu nhất $O(q \log_{1+q/n} n)$. Bài gốc không trình bày chứng minh chi tiết.

14.2.3 Hợp nhất theo kích thước kết hợp nén đường đi

Hai tối ưu trên có thể dùng đồng thời. Khi đó độ phức tạp là

$$O(q\alpha(q, n)).$$

Tác giả định nghĩa hàm $A(i, j)$:

$$A(i, j) = \begin{cases} 2j, & i = 1, \\ A(i - 1, 2), & i \geq 2, j = 1, \\ A(i - 1, A(i, j - 1)), & i, j \geq 2. \end{cases}$$

Rồi

$$\alpha(q, n) = \min\{i \geq 1 \mid A(i, \lfloor n/q \rfloor) > \log n\}.$$

Trong thực tế có thể coi $\alpha(n)$ là hằng số nhỏ hơn 5.

14.3 Cách làm tuyến tính cho DSU trên cây

14.3.1 Định nghĩa

Cho một cây có gốc, hỗ trợ hai thao tác:

1. $\text{Union}(x, fa_x)$: đánh dấu cạnh giữa x và cha của x là đã hợp nhất;
2. $\text{Find}(x)$: từ x , đi lên theo các cạnh đã hợp nhất tới đỉnh có độ sâu nhỏ nhất có thể.

Dùng DSU thông thường cho $O(q\alpha(n))$. Sau đây là cách tuyến tính.

14.3.2 Lời giải

Lấy ngưỡng $B = \lfloor \log n \rfloor$ và chia cây thành các khối liên thông sao cho mỗi cạnh thuộc đúng một khối. Trong mỗi khối, đánh số đỉnh theo độ sâu tăng dần. Dùng một số nguyên B bit để lưu tập đỉnh trong khối chưa hợp nhất với cha; thao tác hợp nhất trong khối chỉ sửa một bit. Với mỗi đỉnh, lưu thêm tập bit của đường từ nó tới gốc khối.

Find trong khối trở thành: lấy giao giữa đường từ đỉnh tới gốc khối và tập đỉnh chưa hợp nhất với cha, rồi tìm đỉnh sâu nhất trong giao. Vì $B = \lfloor \log n \rfloor$, số bit không vượt $O(n)$, có thể tiền xử lý highbit.

Ngoài khối, co mỗi khối thành gốc khối và dùng hợp nhất theo kích thước. Số đỉnh sau co là $O(n/B)$; tổng chi phí hợp nhất ngoài khối là

$$O\left(\frac{n}{B} \log \frac{n}{B}\right) = O(n),$$

và truy vấn ngoài khối $O(1)$. Một truy vấn thực hiện tìm trong khối; nếu đã tới gốc khối thì tìm ngoài khối rồi tìm trong khối lần nữa. Tổng thời gian $O(n + q)$.

14.3.3 Ứng dụng

Ví dụ 1. Cho dải giấy gồm n ô, ô i có màu a_i . Mỗi giây có thể tô lại một ô hoặc đi sang ô kề nếu hai ô trước và sau khi đi cùng màu. Có q truy vấn độc lập hỏi thời gian ngắn nhất đi từ l tới r , với $n, q \leq 10^6$, $l \leq r$.

Cách thô là mỗi lần tô ô kế tiếp thành màu hiện tại rồi đi, tốn $2(r - l)$. Nếu có $i < j$, $a_i = a_j$, thì đi từ i tới j có thể tiết kiệm một lần tô. Ta muốn chọn nhiều cặp (i, j) với các đoạn $[i, j]$ không giao nhau. Với mỗi i , tính f_{a_i} nhỏ nhất sao cho đoạn $[i, f_{a_i}]$ có hai màu giống nhau; nếu không có thì đặt $f_{a_i} = n + 1$. Truy vấn trở thành lặp $l = f_{a_l}$ tới khi $f_{a_l} > r$; số lần lặp là lượng thời gian tiết kiệm.

Nối i với f_{a_i} tạo cây gốc $n + 1$. Xử lý offline theo r tăng dần: khi tới r , hợp nhất r với các con của nó. Khi đó Find(l) cho điểm cuối sau các bước nhảy, hiệu độ sâu là số lần tiết kiệm. Sắp xếp truy vấn tuyến tính theo r , và DSU trên cây cũng tuyến tính, nên tổng $O(n + q)$.

Ví dụ 2. Định nghĩa trọng số của một dãy là $\max(\text{số cực đại tiền tố, số cực đại hậu tố})$. Cho một hoán vị vòng độ dài n , cần chia thành k đoạn không rỗng, mỗi số thuộc đúng một đoạn, để tổng trọng số lớn nhất, với $k \leq n \leq 6 \cdot 10^5$, $k \leq 30$.

Có thể cắt vòng thành dãy sao cho còn một hậu tố không đưa vào k đoạn. Đặt $dp_{i,j}$ là giá trị tốt nhất khi i số đầu được chia thành j đoạn. Khi chuyển từ $dp_{*,j-1}$ sang $dp_{*,j}$, dùng hai mảng phụ A, B ứng với việc đoạn cuối lấy đóng góp từ cực đại tiền tố hoặc hậu tố.

Giữa A_t và A_{t-1} , nếu có thay đổi thì p_t trở thành cực đại tiền tố, nên chỉ cần cộng 1 trên một hậu tố. Với B_t , trước hết cộng toàn cục 1, rồi dùng ngăn xếp đơn điệu để trừ những cực đại hậu tố cũ bị p_t loại. Cấu trúc cần hỗ trợ cộng hậu tố, giảm tiền tố thông qua giảm toàn cục rồi cộng hậu tố, chèn hậu tố và lấy cực đại toàn cục.

Ta duy trì một ngăn xếp đơn điệu và mảng sai phân bằng danh sách liên kết. DSU duy trì vị trí đầu tiên sau mỗi điểm còn nằm trong ngăn xếp; khi xóa một phần tử chỉ hợp nhất nó với phần tử kế tiếp. Đây là DSU trên dãy, nên tuyến tính. Mỗi lớp DP xử lý trong $O(n)$, tổng $O(nk)$.

Ví dụ 3: LCA. Cho cây có gốc n đỉnh và m truy vấn LCA, $n, m \leq 5 \cdot 10^5$. Gắn mỗi truy vấn (u, v) vào đỉnh có thứ tự DFS nhỏ hơn. Khi DFS tới u , LCA là điểm đầu tiên trên đường từ v đi lên đã được DFS. Có thể xem đây là bài toán xóa cạnh rồi hỏi đi lên theo cạnh chưa xóa tới đâu. Xử lý ngược thứ tự DFS biến xóa cạnh thành thêm cạnh, và dùng DSU trên cây để đạt $O(n + q)$.

Ví dụ 4. Cho dãy dài n , hỗ trợ đặt $a_x = y$ và hỏi cực đại trên đoạn $[l, r]$. Số lần sửa không quá $n\sqrt{n}$, số lần hỏi không quá n , $a_i \leq n$, yêu cầu $O(n\sqrt{n})$.

Chia dãy thành khối với $B = \sqrt{n}$. Phân rã xử lý trực tiếp; còn lại cần $O(n\sqrt{n})$ lần sửa điểm và $O(n\sqrt{n})$ lần hỏi cực đại toàn khối. Với mỗi khối độc lập, chuyển từng giá trị thành các đoạn thời gian nó xuất hiện. Xử lý giá trị từ lớn xuống nhỏ: mỗi lần phủ các thời điểm chưa bị phủ trong đoạn bằng giá trị hiện tại. Dùng DSU trên dãy để tìm thời điểm chưa phủ kế tiếp, nên tuyến tính. Vì miền giá trị nhỏ, có thể sắp xếp offline trước và bỏ thừa số log.

14.4 DSU có trọng số

14.4.1 Định nghĩa và lời giải

Ban đầu có n phần tử, mỗi phần tử là một tập riêng, và giá trị phần tử có phép hợp có tính kết hợp. DSU có trọng số hỗ trợ:

1. cộng một giá trị vào toàn bộ phần tử của một tập;
2. hợp nhất hai tập;
3. hỏi giá trị của một phần tử.

Với mỗi nút, duy trì một tag cộng trên cây con. Khi hợp nhất hai tập, không thể luôn gán trực tiếp một gốc vào gốc khác vì gốc kia có thể có tag cây con. Một cách tổng quát là tạo một nút mới và đặt hai gốc cũ làm con của nó.

Cấu trúc này vẫn hỗ trợ nén đường đi: trong quá trình nén, đặt giá trị của các nút đã đi qua thành giá trị thực của chúng. Độ phức tạp như DSU chỉ nén đường đi, tức $O(q \log_{q/n+1} n)$. Nếu thông tin có phép trừ, vẫn có thể hợp nhất theo kích thước: khi gán x vào y , trừ tag của y khỏi tag của x để triệt ảnh hưởng của y . Khi đó độ phức tạp $O(q\alpha(n))$.

14.4.2 Ứng dụng

Ví dụ 1: chuỗi thức ăn. Có ba loài A, B, C , A ăn B , B ăn C , C ăn A . Cho n con vật và m thông tin dạng x, y cùng loài hoặc x ăn y . Nếu mâu thuẫn với thông tin trước đó thì là thông tin giả. Hỏi số thông tin giả, với $n \leq 5 \cdot 10^4$, $m \leq 10^5$.

Mã hóa ba loài bằng 0, 1, 2 modulo 3. Thông tin cùng loài là $v_x = v_y$, còn x ăn y là $v_x + 1 = v_y$. Mỗi phương trình là một cạnh; trong mỗi thành phần liên thông chỉ cần quan hệ tương đối. Nếu cạnh mới nối hai đỉnh cùng thành phần, kiểm tra hiệu hiện tại có thỏa không. Nếu khác thành phần, cộng toàn bộ một thành phần thêm một giá trị để phương trình đúng. Đây là DSU có trọng số với thông tin có phép trừ, độ phức tạp $O(q\alpha(n))$.

Ví dụ 2: tổng đoạn con lớn nhất trên đường cây. Cho cây n đỉnh, mỗi đỉnh có trọng số w_i . Mỗi truy vấn hỏi trên đường từ u tới v , dãy trọng số tạo thành có tổng đoạn con lớn nhất là bao nhiêu, với $n, q \leq 10^6$.

Thông tin tổng đoạn con lớn nhất có thể hợp nhất bằng bộ bốn (lmx, rmx, sum, ans) : tổng tiền tố lớn nhất, tổng hậu tố lớn nhất, tổng toàn dãy và tổng đoạn con lớn nhất. Phép hợp có tính kết hợp nhưng không có phép trừ.

Tính LCA của mỗi truy vấn, tách đường thành hai đường thẳng lên/xuống và xử lý offline theo độ sâu LCA giảm dần. Khi xét đỉnh u , thêm bộ bốn của u vào toàn bộ cây con của u , rồi hợp nhất u với các con. Dùng DSU có trọng số thay cho cây đoạn để giảm hằng số; truy vấn chỉ cần Find. Vì không có phép trừ nên không hợp nhất theo kích thước, độ phức tạp là dạng DSU chỉ nén đường đi, không gian tuyến tính.

14.5 DSU có thể hoàn tác

14.5.1 Định nghĩa và lời giải

DSU có thể hoàn tác hỗ trợ:

1. Union(x, y);
2. Find(x);
3. Back(): hoàn tác thao tác chưa hoàn tác gần nhất.

Vì nén đường đi là phân tích khấu hao, khó hoàn tác gọn. Ta dùng hợp nhất theo kích thước, vốn bảo đảm $O(\log n)$ mỗi thao tác. Mỗi lần Union gắn gốc x vào gốc y , đẩy x vào ngăn xếp. Back lấy đỉnh trên cùng, cắt nó khỏi cha và trừ kích thước của nó khỏi kích thước cha. Mỗi thao tác $O(\log n)$.

14.5.2 Ứng dụng

Ví dụ 1: quay lại trạng thái cũ. Ban đầu có n tập, tập i chỉ chứa i . Có q thao tác: hợp nhất hai tập, quay lại trạng thái sau thao tác thứ k , hoặc hỏi hai phần tử có cùng tập không. Với $n \leq 10^5$, $q \leq 2 \cdot 10^5$.

Có thể dùng cây đoạn bền vững để duy trì DSU, nhưng $O(m \log^2 n)$ hơi lớn. Vì bài không bắt buộc online, dựng cây thao tác: cha của thao tác i là $i - 1$ nếu thao tác là hợp nhất hoặc hỏi, còn là k nếu thao tác quay lại k . DFS trên cây này; khi vào một nút thì thực hiện cạnh/hỏi tương ứng, khi rời nút thì hoàn tác thao tác đó. Độ phức tạp $O(m \log n)$.

Ví dụ 2: liên thông động. Duy trì đồ thị vô hướng đơn n đỉnh, có q thao tác thêm cạnh, xóa cạnh, hỏi hai đỉnh có liên thông không, với $n \leq 5000$, $q \leq 5 \cdot 10^5$.

Xử lý từng cạnh thành các khoảng thời gian nó tồn tại, rồi chèn mỗi khoảng vào các nút tương ứng của cây đoạn thời gian. DFS trên cây đoạn: khi vào nút, thêm tất cả cạnh của nút vào DSU hoàn tác; khi rời nút thì hoàn tác; tại lá trả lời truy vấn. Mỗi cạnh được chèn vào $O(\log q)$ nút, mỗi lần thêm cạnh tốn $O(\log n)$, tổng thời gian $O(q \log q \log n)$.

14.6 Kết luận

Bài viết giới thiệu ba mô hình DSU: DSU tuyến tính trên cây, DSU có trọng số và DSU có thể hoàn tác, kèm nhiều ví dụ ứng dụng trong OI. Ứng dụng của DSU chắc chắn không dừng ở đây; tác giả hy vọng bài viết gợi thêm hướng nghiên cứu cho người đọc.

14.7 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn huấn luyện viên đội tuyển quốc gia Peng Sijin và Yang Yaoliang đã hướng dẫn; cảm ơn cha mẹ đã bồi dưỡng, giáo dục; cảm ơn nhà trường, thầy Li Jian và các bạn học đã giúp đỡ; cảm ơn Xu Anyi, Ye Zichuan, Zhou Xin, Li Xinlong và Hu Yang đã trao đổi, gợi ý; cảm ơn Li Yichen, Xu Anyi, Ye Zichuan và thầy Li Jian đã đọc bản thảo.

14.8 Tài liệu tham khảo

1. Robert E. Tarjan and Jan van Leeuwen. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2):245–281, 1984.

2. Zhou Xin. Bàn về một lớp thuật toán xây dựng chia khối trên cây và ứng dụng, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2021.
3. OI Wiki contributors. Disjoint Set Union, OI Wiki, 2022.
4. Harold N. Gabow and Robert E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30:209–221, 1985.

15 Bàn về ứng dụng static top tree trên cây và đồ thị nối tiếp-song song tổng quát

Trường Ngoại ngữ Nam Kinh
Cheng Siyuan

Tóm tắt

Bài viết giới thiệu khái niệm static Top Tree; trình bày ba ứng dụng trên bài toán cây: duy trì thông tin bằng thao tác pushup trên Top Tree, chia để trị trên các đỉnh của Top Tree, và dùng Top Tree để xây dựng phân khối cây. Sau đó bài viết mở rộng hai ứng dụng đầu sang đồ thị nối tiếp-song song tổng quát, và đề xuất một cách xây dựng phân khối cho lớp đồ thị này dựa trên Top Tree.

15.1 Dẫn nhập

Trong thi tin học có nhiều bài toán tĩnh trên cây có thể giải bằng static Top Tree. Nếu mở rộng Top Tree sang đồ thị nối tiếp-song song tổng quát, ta cũng có thể xử lý một số bài toán trên lớp đồ thị này.

Ở các thuật toán trước đây dùng Top Tree cho đồ thị nối tiếp-song song tổng quát, cách trực tiếp pushup để duy trì thông tin thường khó xử lý những thông tin phức tạp không ghép được một cách đơn giản. Cách chia để trị theo đỉnh của Top Tree lại phải xét các cạnh nằm ngoài cây con, nên khó hỗ trợ sửa đổi thông tin. Tác giả khảo sát một hướng dùng phân khối đồ thị nối tiếp-song song tổng quát để khắc phục các khó khăn đó.

15.2 Khái niệm cụm và các phép toán

15.2.1 Khái niệm cụm

Với cây vô hướng T , một cụm trên T là bộ ba

$$C = (V, E, B),$$

trong đó (V, E) là một đồ thị con liên thông của T , $B \subseteq V$, $|B| \leq 2$, và với mọi $x \in V$, nếu tồn tại $y \notin V$ sao cho $(x, y) \in E(T)$, thì $x \in B$. Các phần tử của B được gọi là *điểm biên* của cụm. Ký hiệu $V(C)$, $E(C)$ lần lượt là tập đỉnh và tập cạnh của C , còn $B(C)$ là tập điểm biên.

Ưu điểm của định nghĩa này là khi thực hiện phép toán trên cụm, ta chỉ cần quan tâm đến các điểm trong B . Vì $|B| \leq 2$, một cụm có thể được nhìn trực giác như một cạnh đã được co gọn. Trong một số bối cảnh, các cạnh kề một đỉnh có thứ tự vòng; khi đó quanh mỗi điểm biên chỉ cho phép lấy một đoạn liên tiếp các cạnh vào cụm, nhưng bài viết không xét biến thể này.

15.2.2 Phép compress

Với hai cụm C, D trên cây vô hướng T , nếu

$$B(C) \cap B(D) = \{v\},$$

thì kết quả của phép *compress* là

$$(V(C) \cup V(D), E(C) \cup E(D), (B(C) - \{v\}) \cup (B(D) - \{v\})).$$

Phép toán chỉ hợp lệ khi hai cụm có đúng một điểm biên chung và kết quả vẫn là một cụm trên T .

15.2.3 Phép rake

Với hai cụm C, D trên cây vô hướng T , nếu

$$B(C) \cap B(D) = \{x\},$$

thì kết quả của phép *rake* là

$$(V(C) \cup V(D), E(C) \cup E(D), B(C)).$$

Phép toán cũng chỉ hợp lệ khi hai cụm có đúng một điểm biên chung và kết quả vẫn là một cụm. Hình trong bản gốc minh họa trực quan rằng *compress* ghép hai cụm theo kiểu nối tiếp, còn *rake* ghép một cụm phụ vào cụm chính qua điểm biên chung.

15.3 Khái niệm Top Tree và cách dựng tĩnh

15.3.1 Khái niệm Top Tree

Ta đưa vào khái niệm cụm để biểu diễn cả cây T bằng một biểu thức gồm các phép toán trên cụm. Nói cách khác, ta thực hiện một dãy phép *compress* và *rake* để cuối cùng thu được một cụm C thỏa $E(C) = E(T)$.

Tại mỗi thời điểm trong quá trình tính toán, với mỗi cụm đang tồn tại, nối một cạnh giữa hai điểm biên của cụm đó; đồ thị thu được là *cây co*. Cây co ban đầu được tạo bởi tất cả các cụm một cạnh

$$(\{u, v\}, \{(u, v)\}, \{u, v\}) \quad ((u, v) \in E(T)).$$

Vì mỗi cụm ban đầu chỉ chứa một cạnh, thông tin trên chúng rất dễ duy trì.

Bổ đề. Tồn tại một dãy phép toán đưa cây co ban đầu tới cây co cuối cùng.

Chứng minh. Chọn tùy ý một gốc. Mỗi lần xét một lá x của cây co hiện tại và cụm C ứng với cạnh nối x với cha y . Nếu x có anh em thì rake C vào một cụm anh em; nếu không, *compress* C với cụm nối y với cha z của y . Lặp lại thao tác này thì cuối cùng thu được cụm chứa toàn bộ cây.

Do đó cây co ban đầu và cuối cùng đã được xác định. Với một quá trình biến đổi từ cây co ban đầu sang cây co cuối cùng, ta có thể mô tả nó bằng một cây biểu thức; cây này được gọi là Top Tree. Trong hình gốc, nút vuông biểu thị phép *compress*, nút tròn biểu thị phép *rake*.

Cách dựng trong chứng minh trên cho phép dựng Top Tree cho mọi cây, và còn có nhiều thuật toán dễ cài đặt tương tự. Điểm yếu là chúng không bảo đảm độ sâu. Trong khi đó, một số ứng dụng bên dưới phụ thuộc trực tiếp vào độ sâu của Top Tree, nên ta muốn dựng một Top Tree có độ sâu $\Theta(\log |V|)$.

15.3.2 Phân rã nặng nhẹ

Trước hết xét bài toán sau. Top Tree là cây biểu thức nhị phân, nên nếu cần *compress* hoặc *rake* lần lượt k cụm, ta phải dựng một cây đoạn hoặc cây đoạn tổng quát trên k cụm. Nếu dùng cây đoạn cân bằng thông

thường, độ sâu là $\Theta(\log k)$. Hai loại cây sinh ra theo cách này được gọi lần lượt là cây compress và cây rake.

Chọn tùy ý một gốc cho cây ban đầu rồi thực hiện phân rã nặng nhẹ. Với một chuỗi nặng, trước hết đệ quy xử lý mọi con nhẹ. Sau đó, với mỗi đỉnh trên chuỗi nặng, nếu nó có con nhẹ thì rake toàn bộ cụm của các con nhẹ vào cụm chứa con nặng. Cuối cùng compress toàn bộ các cụm trên chuỗi nặng cùng với cha của đỉnh đầu chuỗi, nếu cha đó tồn tại. Như vậy cả một chuỗi nặng được co thành một cụm.

Nếu mỗi lần ghép dùng cây đoạn cân bằng thông thường, chi phí độ sâu cho một chuỗi nặng là $\Theta(\log |V|)$. Vì trên đường từ một đỉnh tới gốc có nhiều nhất $\Theta(\log |V|)$ chuỗi nặng, Top Tree thu được có độ sâu $\Theta(\log^2 |V|)$.

15.3.3 Cây nhị phân cân bằng toàn cục

Cách dựng cây đoạn bằng việc luôn lấy trung điểm là lãng phí. Nó làm cho mọi cụm tham gia phép toán có độ sâu gần bằng nhau trong cây đoạn, nhưng độ sâu cây con Top Tree phía dưới chúng lại khác nhau. Cụm có cây con nông hơn thực ra có thể được đặt sâu hơn trong cây đoạn.

Thuật toán mới là: với dãy cụm cần dựng cây đoạn tổng quát, tính kích thước cây con Top Tree của từng cụm, rồi chọn vị trí sao cho tổng kích thước phía trái là lớn nhất nhưng không vượt quá một nửa tổng kích thước. Chia tại vị trí đó và đệ quy cho hai nửa. Nếu nửa trái rỗng, đặt nó chỉ gồm cụm ngoài cùng bên trái.

Bổ đề. Với một cây kích thước n , Top Tree dựng bằng thuật toán trên có độ sâu $\Theta(\log n)$.

Chứng minh. Xét một nút x trên Top Tree và ký hiệu kích thước cây con của nó là sz_x . Nếu trong các cháu của x tồn tại một nút không nằm cùng cây compress hoặc cây rake với x , thì do mỗi đường từ một đỉnh của cây gốc tới gốc đi qua nhiều nhất $\Theta(\log n)$ chuỗi nặng, số nút loại này trên một đường trong Top Tree cũng chỉ là $\Theta(\log n)$.

Ngược lại, mọi cháu của x đều nằm trong cùng cây compress hoặc rake với x . Hình trong bản gốc mô tả cách chia cây con của x : lá màu đỏ là lá vượt qua mốc $sz_x/2$, và các phần còn lại có tổng kích thước $sz_x/2$. Vì con trái của con phải của x nằm trong cùng cây compress hoặc rake, kích thước cây con của nó không vượt quá một nửa kích thước con phải. Kết hợp với cách chọn điểm chia, các cây con cháu liên quan đều có kích thước không quá $sz_x/2$. Do đó sau mỗi hai tầng loại này, kích thước giảm ít nhất một nửa, nên đóng góp độ sâu cũng là $\Theta(\log n)$.

Hai phần cộng lại cho độ sâu $\Theta(\log n)$. Cây đoạn tổng quát dựng bằng thuật toán này thường được gọi là cây nhị phân cân bằng toàn cục. Với một số bài toán, phép rake có tính chất tốt nên không cần dựng cây rake riêng.

15.4 Ứng dụng của Top Tree trên cây

15.4.1 Duy trì thông tin bằng pushup trực tiếp

Dựa trên cấu trúc cụm, ta có thể duy trì các thông tin bán nhóm phụ thuộc vào trạng thái của điểm biên. Ví dụ, một lớp bài toán quy hoạch động trên cây có thể viết dưới dạng: mỗi đỉnh nhận một trạng thái, còn mỗi cạnh đóng góp một giá trị tùy theo trạng thái của hai đầu nút. Các bài toán điển hình gồm tập độc lập trọng số lớn nhất trên cây, hoặc chọn một số hằng đỉnh trên cây để tính giá trị.

Với các bài toán này, trong mỗi cụm ta ghi giá trị DP cho từng cặp trạng thái của hai điểm biên. Khi compress, liệt kê trạng thái của điểm biên chung; khi rake, cộng các giá trị có cùng trạng thái. Vì Top Tree có độ sâu $\Theta(\log |V|)$, nếu sửa trọng số một cạnh hoặc một đỉnh thì chỉ cần pushup lại $\Theta(\log |V|)$ nút.

Ví dụ: NOIP 2018, Bảo vệ vương quốc. Cho cây vô hướng T gồm n đỉnh có trọng số. Mỗi truy vấn cố định trạng thái của hai đỉnh a, b lần lượt là x, y , rồi hỏi phủ đỉnh trọng số nhỏ nhất của T . Các truy vấn độc lập, $n, q \leq 10^5$.

Nếu hỗ trợ sửa trọng số đỉnh, bài toán cố định trạng thái có thể quy về đặt trọng số của một trạng thái thành dương vô cùng hoặc âm vô cùng. Dựng Top Tree của T ; với mỗi cụm c , ghi

$$f_{0/1,0/1}$$

là chi phí nhỏ nhất của các đỉnh không phải điểm biên khi hai điểm biên nhận bốn tổ hợp trạng thái. Compress thì liệt kê trạng thái điểm giao, rake thì cộng chi phí cùng trạng thái. Khi sửa một trọng số, tìm các nút Top Tree bị ảnh hưởng rồi pushup lại đường lên gốc. Độ phức tạp $\Theta(n + m \log n)$.

Ví dụ: NOI 2022, Đếm điểm trong lân cận trên cây. Cho cây T gồm n đỉnh. Được phép tiền xử lý một số cụm bằng compress và rake. Sau đó có q truy vấn online, mỗi truy vấn cho x, d , yêu cầu dùng không quá một phép toán cụm để thu được cụm gồm mọi đỉnh cách x không quá d . Giới hạn $n \leq 10^5$, $q \leq 2 \cdot 10^6$, số phép toán tiền xử lý không vượt quá $3 \cdot 10^7$.

Dựng Top Tree của T . Với truy vấn x, d , xét một cạnh kề x tùy ý và tìm trong các tổ tiên trên Top Tree cụm nông nhất chứa hoàn toàn mọi đỉnh cách x không quá d . Gọi cụm này là C . Khi đó d không vượt quá đường kính của cụm cha của C , và đáp án có thể biểu diễn bằng cách lấy C compress với cụm gồm các đỉnh bên ngoài C nhưng cách một trong hai điểm biên của C không quá một ngưỡng tương ứng.

Với mỗi cụm C , duy trì $g_{0/1,d}$, trong đó chỉ số 0/1 chọn một điểm biên, còn d là khoảng cách được xét bên ngoài C . Trước hết pushup để duy trì thông tin $f_{0/1,d}$ bên trong cụm, rồi dùng kỹ thuật đổi gốc DP để suy ra g . Để mỗi truy vấn chỉ cần một phép toán, tiền xử lý thêm kết quả compress giữa C và từng phần tử g tương ứng. Tổng số phép toán tiền xử lý là cùng bậc với tổng kích thước cây con của các nút Top Tree, tức $\Theta(n \log n)$.

15.4.2 Chia để trị theo đỉnh của Top Tree

Ta muốn chia để trị trên Top Tree để giải các bài toán thống kê đường đi. Cần định nghĩa cây co tương ứng với một đồ thị con liên thông T' của Top Tree. Lấy nút nông nhất của T' ; xóa khỏi cây gốc mọi cạnh không nằm trong cây con của nút đó, rồi lần lượt thực hiện các phép toán ứng với các nút thuộc $T - T'$ không còn nối với gốc. Cây co thu được là cây co của đồ thị con T' .

Với định nghĩa này, thuật toán chia để trị như sau. Ở mỗi bước đang có một đồ thị con liên thông T của Top Tree; chọn một cạnh (x, fa_x) sao cho hai phần có số đỉnh gần nhau nhất. Chia T thành subtree_x và $T - \text{subtree}_x + \{x\}$. Trước hết thống kê đóng góp của các đường đi trong cây co của T mà hai đầu mút là điểm biên hoặc không nằm trong cùng một đồ thị con, rồi đệ quy hai phía. Nếu mọi đỉnh của cây co hiện tại đều từng được dùng làm điểm biên trong các bước trước thì dừng.

Sau k tầng đệ quy, cây co của một đồ thị con có nhiều nhất k cạnh không thuộc cây gốc; tổng số lần xuất hiện của các cạnh này là $\Theta(|V| \log |V|)$. Mỗi cạnh gốc không đệ quy sang cả hai phía, nên tổng số lần xuất hiện của các cạnh gốc cũng là $\Theta(|V| \log |V|)$. Do đó tổng kích thước các cây co trong toàn bộ chia để trị là $\Theta(|V| \log |V|)$.

15.4.3 Xây dựng phân khối cây

Top Tree cũng cho một cách dựng phân khối cây với tính chất mạnh. Với cây kích thước n , chọn mọi nút trên Top Tree có kích thước cây con không quá $\sqrt{n}$, nhưng cha của nó có kích thước cây con lớn hơn $\sqrt{n}$. Các cụm tương ứng tạo thành một phân hoạch cụm của cây gốc.

Mỗi cụm có kích thước $\Theta(\sqrt{n})$, số cụm là $\Theta(\sqrt{n})$. Do đó độ sâu và đường kính của từng cụm cũng ở mức $\Theta(\sqrt{n})$. Hơn nữa, với mỗi đỉnh không phải điểm biên, cụm kế tiếp đi xuống trong cây con là duy nhất, nên việc mô tả thông tin cây con thường thuận tiện. Trong cài đặt thực tế có những thuật toán dựng phân khối cây ngắn hơn; bài gốc chỉ nêu hướng này và dẫn tới tài liệu tham khảo liên quan.

15.5 Đồ thị tổng quát hơn

Định nghĩa cụm ở trên thực ra không dùng quá nhiều tính chất riêng của cây. Ta thử mở rộng nó sang đồ thị liên thông. Vì đồ thị không còn bảo đảm vô chu trình, cần thêm một phép toán mới gọi là *twist*.

15.5.1 Phép twist

Với hai cụm tổng quát C, D trên đồ thị vô hướng liên thông G , nếu

$$B(C) = B(D),$$

thì kết quả của phép *twist* là

$$(V(C) \cup V(D), E(C) \cup E(D), B(C)).$$

Twist cũng có tính kết hợp. Trên đồ thị co, twist chính là việc chồng hai cạnh song song thành một cụm.

15.5.2 Đồ thị co được: đồ thị nối tiếp-song song tổng quát

Ngay cả khi có đủ compress, rake và twist, vẫn có đồ thị không thể co thành một cụm, ví dụ K_4 . Tổng quát hơn, nếu một đồ thị chứa đồ thị con đồng phôi với K_4 , nó cũng không co được bằng ba phép toán trên.

Thực ra, không chứa đồ thị con đồng phôi với K_4 vừa là điều kiện cần vừa là điều kiện đủ để co được. Các đồ thị liên thông không chứa đồ thị con đồng phôi với K_4 được gọi là đồ thị nối tiếp-song song tổng quát. Chứng minh kết luận này khá phức tạp; bài gốc dẫn tới tài liệu tham khảo. Vì ở mỗi thời điểm các cụm không còn tạo thành cây, từ đây ta gọi cấu trúc co là đồ thị co thay vì cây co.

Có thể dựng Top Tree của đồ thị nối tiếp-song song tổng quát bằng cách lặp:

- nếu trong đồ thị co có đỉnh bậc 1, gọi nó là v và đỉnh kề là u , thì rake cụm ứng với $\{u, v\}$ vào một cụm bất kỳ khác kề u ;
- nếu có đỉnh bậc 2, gọi nó là v và hai đỉnh kề là u, w , thì compress hai cụm ứng với $\{u, v\}$ và $\{v, w\}$;
- nếu có cạnh song song, twist các cụm tương ứng.

Theo kết luận trên, quá trình này cuối cùng co toàn bộ đồ thị thành một cụm và không bị kẹt giữa chừng.

Đáng tiếc là Top Tree dựng theo cách này có thể có độ sâu $\Theta(|V|)$. Hơn nữa, trên đồ thị nối tiếp-song song tổng quát, có thể xây dựng ví dụ khiến mọi cách dựng Top Tree đều có độ sâu $\Theta(|V|)$. Hình trong bản gốc đưa ra một đồ thị như vậy.

Mục tiêu còn lại là mở rộng ba ứng dụng của Top Tree ở phần trước sang đồ thị nối tiếp-song song tổng quát.

15.5.3 Top Tree của Top Tree

Trước hết xét việc pushup trực tiếp trên Top Tree. Nếu Top Tree có độ sâu $\Theta(|V|)$, sửa một phần tử có thể tốn tuyến tính.

Quan sát rằng dựng Top Tree rồi pushup trên đó thực chất là biến một bài toán pushup trên cây bất kỳ thành bài toán pushup trên Top Tree, với chi phí một lần sửa bằng độ sâu của Top Tree. Nay Top Tree đầu tiên có độ sâu $\Theta(|V|)$; nếu xem chính nó như một cây thường và dựng thêm một Top Tree thứ hai trên nó, bài toán pushup trên Top Tree sâu tuyến tính được chuyển thành pushup trên Top Tree mới có độ sâu $\Theta(\log |V|)$.

Cần xét lượng thông tin phải lưu. Giả sử mỗi đỉnh đồ thị gốc có k trạng thái. Vì mỗi cụm có hai điểm biên, mỗi nút trên Top Tree đầu tiên có k^2 trạng thái. Một nút trên Top Tree thứ hai có hai điểm biên là hai cụm của Top Tree đầu, nên cần k^4 trạng thái. Trong các bài toán thực tế, k thường là hằng số. Như vậy ta vẫn xử lý được một lần sửa bằng $\Theta(\log |V|)$ phép toán bán nhóm. Vì Top Tree đầu tiên là cây nhị phân, khi dựng Top Tree của nó không cần dựng cây rake.

Ví dụ: Công viên, tập huấn đội tuyển 2019 Day 3. Cho một đồ thị nối tiếp-song song tổng quát G gồm n đỉnh. Mỗi đỉnh có thể tô đen hoặc trắng; hai trọng số của đỉnh là lợi ích khi tô mỗi màu. Mỗi cạnh có hai trọng số, tương ứng lợi ích khi hai đầu cùng màu hoặc khác màu. Có q lần sửa trọng số đỉnh hoặc cạnh, sau mỗi lần hỏi lợi ích lớn nhất khi tô màu tùy ý. Các lần sửa không độc lập, $n, q \leq 10^5$.

Dựng Top Tree T của G . Với mỗi cụm C , đặt

$$f_{0/1,0/1}$$

là lợi ích lớn nhất của các cạnh và các đỉnh không phải điểm biên trong cụm khi hai điểm biên nhận từng tổ hợp màu. Dựng tiếp Top Tree T' của T . Trên mỗi cụm C của T' , duy trì

$$g_{0/1,0/1,0/1,0/1},$$

ứng với trạng thái của bốn điểm: hai điểm biên trong đồ thị gốc của mỗi cụm biên thuộc T . Pushup trực tiếp duy trì g . Khi sửa, tìm nút tương ứng trên T , rồi nút tương ứng trên T' , và pushup đường lên gốc. Độ phức tạp $\Theta(n + m \log n)$.

15.5.4 Chia để trị Top Tree trên đồ thị nối tiếp-song song tổng quát

Tiếp theo xét mở rộng chia để trị theo đỉnh của Top Tree. Bản thân chia để trị không phụ thuộc vào độ sâu, nên việc Top Tree có độ sâu tuyến tính không gây vấn đề.

Khác biệt nằm ở định nghĩa đồ thị con của một đồ thị con Top Tree. Nếu vẫn xóa toàn bộ cạnh nằm ngoài cây con của nút nông nhất, trong đồ thị nối tiếp-song song tổng quát có thể bỏ sót các đường đi đi qua phần ngoài cây con đó để nối hai điểm biên của cùng một cụm. Dạng đường bị bỏ sót là: đi từ một đỉnh trong cụm của r tới một điểm biên của r , qua phần nằm ngoài cây con của r để tới điểm biên còn lại, rồi quay vào cụm của r .

Vì vậy, với phần ngoài cây con của r , ta lấy đồ thị con gồm các đỉnh còn liên thông với cả hai điểm biên của r . Đồ thị con này vẫn là nối tiếp-song song tổng quát và có thể co thành một cụm có hai điểm biên chính là hai điểm biên của r . Nếu loại thông tin đang duy trì cho phép, cụm này có thể thay thế tương đương cho toàn bộ phần ngoài.

Trong quá trình chia để trị, giả sử đang xử lý đồ thị con T của Top Tree, chọn điểm chia r , và gọi cây con của r là T' . Khi đệ quy vào T' , cần thêm cụm thay thế toàn bộ các cạnh nằm ngoài cây con của r . Vì phần ngoài cây con của nút nông nhất trong T đã được thay bằng một cụm, khi tính cụm thay thế cho phần ngoài r chỉ cần xử lý một đồ thị có kích thước $\Theta(|T - T'|)$. Do đó độ phức tạp tổng vẫn là $\Theta(|V| \log |V|)$.

Ví dụ: Đi dạo công viên, tập huấn đội tuyển 2021. Cho đồ thị nối tiếp-song song tổng quát G có trọng số cạnh. Mỗi truy vấn cho một tập đỉnh, hỏi tổng khoảng cách ngắn nhất giữa mọi cặp đỉnh trong tập, lấy modulo 2013265921. Có $n, q \leq 10^5$, và tổng kích thước các tập truy vấn không vượt quá 10^5 .

Xử lý offline toàn bộ truy vấn. Dựng Top Tree T của G , rồi chia để trị trên T . Khi chọn điểm chia r , chia đồ thị con hiện tại thành cây con T' của r và $T - T' + \{r\}$. Gọi hai điểm biên của r là u, v . Trước hết tính trong cụm hiện tại khoảng cách ngắn nhất một nguồn từ u và từ v .

Xét một truy vấn. Nếu u hoặc v thuộc tập truy vấn thì đóng góp liên quan có thể tính trực tiếp. Với một điểm $x \in T'$ và một điểm $y \in T - T' + \{r\}$, đường ngắn nhất giữa chúng là đường ngắn hơn trong hai khả năng $x - u - y$ và $x - v - y$. Với mỗi điểm, tính hiệu khoảng cách tới u và tới v , rồi sắp xếp theo hiệu này. Khi duyệt x theo thứ tự tăng dần, tập các y mà đường $x - u - y$ tốt hơn là một tiền tố và tiền tố này chỉ giảm dần; có thể dùng hai con trỏ để tính tổng đóng góp.

Khi đệ quy, phần $T - T' + \{x\}$ chỉ cần thay bằng một cụm có hai điểm biên u, v , trong đó khoảng cách giữa u và v bằng khoảng cách ngắn nhất trong phần đó. Độ phức tạp cuối cùng là

$$\Theta(n \log^2 n + \sum k \log n),$$

với k là kích thước của từng tập truy vấn.

15.5.5 Xây dựng phân khối đồ thị nối tiếp-song song tổng quát

Cuối cùng, thử dùng Top Tree để dựng phân khối trên đồ thị nối tiếp-song song tổng quát. Thực hiện phân khối trên Top Tree của đồ thị; với mỗi khối của Top Tree, dùng định nghĩa đồ thị co của đồ thị con Top Tree để thu được một khối trên đồ thị gốc. Các khối có những tính chất sau:

- số khối là $\Theta(\sqrt{|V|})$;
- mỗi khối có số đỉnh và số cạnh đều là $\Theta(\sqrt{|V|})$;
- mỗi khối có thể co thành cụm ứng với nút nông nhất trong khối Top Tree; hai điểm biên của cụm này được gọi là điểm biên ngoài của khối;
- mỗi đỉnh của đồ thị gốc xuất hiện như một đỉnh không phải biên ngoài trong nhiều nhất một khối, còn mỗi cạnh xuất hiện đúng trong một khối;
- mỗi khối C giao với nhiều nhất hai khối khác tại các điểm không phải biên ngoài của C ; hai khối đó là các con của C trên Top Tree sau phân khối. Vì hai khối này cùng cây con của chúng có thể lập tức co thành một cụm, số giao điểm nhiều nhất là hai. Các giao điểm này được gọi là điểm biên trong của C . Do đó mỗi khối có nhiều nhất bốn điểm biên.

Như vậy ta thu được một phân khối cho đồ thị nối tiếp-song song tổng quát.

Ví dụ tự đề xuất. Cho đồ thị nối tiếp-song song tổng quát G gồm n đỉnh, cạnh có trọng số. Có q thao tác:

- 1 i c : đổi trọng số cạnh thứ i thành c ;
- 2 x d : hỏi có bao nhiêu đỉnh có khoảng cách ngắn nhất có trọng số tới x không vượt quá d .

Giới hạn $q \leq 3 \cdot 10^4$, $n \leq 10^5$.

Trước hết dựng Top Tree của G , rồi dựng phân khối đồ thị nối tiếp-song song tổng quát. Với truy vấn tại x , xét đường ngắn nhất từ x tới một đỉnh y . Đường này hoặc không đi qua điểm biên nào, hoặc đi qua ít nhất một điểm biên.

Nếu đường x, y không qua điểm biên, hai đỉnh nằm trong cùng khối; có thể chạy một lần đường đi ngắn nhất một nguồn trong khối của x . Nhưng ngay cả khi x, y ở cùng một khối, đường ngắn nhất vẫn có thể đi qua điểm biên, nên các đỉnh trong khối cũng phải được xét bằng cách bên dưới.

Nếu đường đi qua điểm biên, nó có thể tách thành ba đoạn: từ x tới một điểm biên u mà không qua điểm biên khác, từ u tới một điểm biên v , rồi từ v tới y mà không qua điểm biên khác. Nếu x là điểm biên thì đoạn đầu suy biến và $u = x$; nếu không, u chỉ có thể là một trong bốn điểm biên của khối chứa x .

Vì số điểm biên chỉ là $\Theta(\sqrt{n})$, ta có thể tính khoảng cách từ x tới mọi điểm biên bằng cách chạy đường đi ngắn nhất trên một đồ thị G' chỉ gồm các điểm biên, với các nguồn có khoảng cách khởi tạo từ bốn điểm biên của khối x . Đồ thị G' cần bảo đảm khoảng cách ngắn nhất giữa mọi cặp điểm biên trong G' đúng bằng khoảng cách trong G .

Sau khi biết khoảng cách tới từng điểm biên v , cần đếm các đỉnh y . Với mỗi khối, lưu lại đối với từng truy vấn còn có thể đi thêm bao xa sau khi tới bốn điểm biên của khối; đó là một tọa độ bốn chiều của truy vấn. Với mỗi đỉnh y trong khối, tính khoảng cách từ bốn điểm biên tới y ; đó là tọa độ bốn chiều của y . Đỉnh y không đóng góp khi và chỉ khi cả bốn tọa độ của y đều lớn hơn tọa độ tương ứng của truy vấn, nên có thể quy về đếm điểm bốn chiều offline.

Cách dựng G' là: với mỗi khối, tính đường đi ngắn nhất giữa mọi cặp điểm biên chỉ dùng cạnh trong khối, rồi nối các cặp đó trong G' . Nếu một đường ngắn nhất giữa hai điểm biên u, v trong G đi qua điểm biên khác w , thì chỉ cần khoảng cách u, w và w, v đã đúng trong G' , khoảng cách u, v cũng sẽ đúng. Vì mỗi khối có nhiều nhất bốn điểm biên, mỗi khối chỉ đóng góp $O(1)$ cạnh, nên G' có $\Theta(\sqrt{n})$ cạnh.

Khi sửa cạnh, xét khối chứa cạnh đó. Do trọng số trong khối thay đổi, cần xử lý ngay các truy vấn đã gom offline vào khối này, sau đó cập nhật các cạnh mà khối đóng góp cho G' . Nếu dùng chia để trị nhiều chiều để đếm bốn chiều, độ phức tạp là

$$\Theta((q\sqrt{n} + n) \log^3 n).$$

Nếu dùng bitset, độ phức tạp là

$$\Theta\left(\frac{nq}{w} + q\sqrt{n} \log n\right),$$

trong đó w là độ rộng từ máy. Lý thuyết thì cách thứ nhất tốt hơn, nhưng trong phạm vi dữ liệu mà máy tính hiện nay xử lý được, cách bitset thường chạy nhanh hơn.

15.6 Kết luận

Bài viết giới thiệu các ứng dụng của static Top Tree trên bài toán cây và đồ thị nối tiếp-song song tổng quát. Phần cuối đề xuất một thuật toán xây dựng phân khối đồ thị nối tiếp-song song tổng quát dựa trên Top Tree. Tác giả hy vọng nội dung này gợi mở thêm hướng nghiên cứu và giúp người đọc tiếp tục phát triển các ứng dụng mới.

15.7 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn thầy Li Shu, thầy Zhang Chao và thầy Shi Polei của Trường Ngoại ngữ Nam Kinh đã giảng dạy; cảm ơn cha mẹ đã quan tâm và hỗ trợ.

15.8 Tài liệu tham khảo

1. Renato F. Werneck. *Design and Analysis of Data Structures for Dynamic Trees*. Princeton University, 2006.

2. Robert E. Tarjan and Renato F. Werneck. Self-adjusting top trees. *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA 2005, Vancouver, British Columbia, Canada, January 23–25, 2005.
3. Zhao Yuyang. *Lý thuyết, cách dùng và hiện thực một số nội dung liên quan tới Top Tree*, 2019.
4. Zhou Xin. Bàn về một lớp thuật toán xây dựng phân khối trên cây và ứng dụng, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2021.
5. Wu Zuetong. Báo cáo ra đề *Công viên*, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2019.
6. Lin Haohan. Báo cáo ra đề *Đi dạo công viên*, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2021.

16 Bước đầu về hypercube và các thuật toán liên quan

Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông
Guan Yanru

Tóm tắt

Hypercube là một loại đồ thị vô hướng đặc biệt, có định nghĩa ngắn gọn và cấu trúc đẹp. Bài viết hệ thống nhiều cách định nghĩa và các tính chất của hypercube, khảo sát những thuật toán có bản chất liên quan tới hypercube, và thảo luận các ví dụ, mô hình gần gũi với thi tin học. Tác giả hy vọng bài viết gợi mở để người đọc tiếp tục học và nghiên cứu các bài toán trên hypercube.

16.1 Dẫn nhập

Hypercube là đồ thị vô hướng gồm 2^n đỉnh và $2^{n-1}n$ cạnh, có tính đối xứng cao. Trong toán học, việc nghiên cứu hypercube có thể truy ngược tới thập niên 1980. Mô hình này cũng từng xuất hiện trong tuyển chọn tỉnh Giang Tô năm 2013 và eJOI 2021, nhưng đến nay vẫn chưa thật sự quen thuộc với nhiều thí sinh.

Bài viết này tổng kết các tính chất của hypercube và khai thác giá trị ứng dụng của nó trong thi tin học. Nội dung gồm: các định nghĩa và đặc điểm cấu trúc; tính chất khi xem hypercube như đồ thị vô hướng; tính chất khi xem nó như một tập có thứ tự bán phần, bao gồm định lý Dilworth và đảo Mobius; các đồ thị con đặc biệt như chu trình và lưới; và cuối cùng là bài toán tồn tại, xây dựng Magic Labeling trên hypercube.

16.2 Định nghĩa

16.2.1 Quy ước ký hiệu

Ký hiệu K_n là đồ thị đầy đủ gồm n đỉnh; C_n là chu trình gồm n đỉnh và n cạnh; P_n là đường đi gồm n đỉnh. Với đồ thị vô hướng, $N(v)$ là tập hàng xóm của v , và

$$N(V) = \bigcup_{v \in V} N(v).$$

Với hai tập A, B , tích Descartes là

$$A \times B = \{(x, y) \mid x \in A, y \in B\}.$$

Với hai đồ thị đơn vô hướng $G_1 = (V_1, E_1)$ và $G_2 = (V_2, E_2)$, tích Descartes của hai đồ thị là $G_1 \times G_2 = (V, E)$, trong đó

$$V = V_1 \times V_2,$$

và

$$E = \{((x, y), (z, w)) \mid x = z, (y, w) \in E_2 \text{ hoặc } (x, z) \in E_1, y = w\}.$$

Nếu không xét dấu ngoặc trong bộ nhiều thành phần, tích Descartes có tính kết hợp.

16.2.2 Định nghĩa trực tiếp của hypercube

Với $n \in \mathbb{Z}_+$, định nghĩa đồ thị vô hướng $Q_n = (V, E)$ bởi

$$V = \{0, 1, \dots, 2^n - 1\},$$

$$E = \{(x, y) \mid \text{popcount}(x \oplus y) = 1, x, y \in V\}.$$

Nói cách khác, hai đỉnh có cạnh nối khi và chỉ khi biểu diễn nhị phân của chúng khác nhau đúng một bit. Đồ thị Q_n như vậy được gọi là hypercube bậc n .

Dễ thấy mọi đỉnh của Q_n đều có bậc n , nên

$$|E| = \frac{1}{2}n|V| = 2^{n-1}n.$$

Với $n = 1, 2, 3$, ta lần lượt có $Q_1 \simeq K_2$, $Q_2 \simeq C_4$, và Q_3 là khung của lập phương ba chiều. Trong phần sau, cạnh nối x với $x \oplus 2^k$ được gọi là cạnh có hướng k , có tổng cộng n hướng khác nhau.

16.2.3 Định nghĩa quy nạp

Dùng tích Descartes của đồ thị, ta có thể định nghĩa hypercube quy nạp.

Bổ đề. Đặt $Q_1 = K_2$, $Q_n = Q_{n-1} \times K_2$. Định nghĩa này tương đương với định nghĩa trực tiếp ở trên.

Chứng minh. Quy nạp theo n . Giả sử Q_{n-1} đã là hypercube bậc $n-1$, có thể gán nhãn từ 0 tới $2^{n-1} - 1$ sao cho cạnh thỏa điều kiện khác đúng một bit. Theo $Q_n = Q_{n-1} \times K_2$, Q_n tách thành hai phần G_0, G_1 , mỗi phần đẳng cấu với Q_{n-1} . Gán cho đỉnh trong G_0 nhãn cũ v , còn đỉnh tương ứng trong G_1 nhãn $v + 2^{n-1}$. Khi đó cạnh trong từng phần và cạnh nối hai phần đều thỏa điều kiện khác đúng một bit.

Khai triển kết luận trên ta được

$$Q_n = \underbrace{K_2 \times K_2 \times \dots \times K_2}_{n \text{ lần}}.$$

16.2.4 Các góc nhìn cấu trúc

Từ định nghĩa quy nạp, Q_n có thể được xem là hai bản sao của Q_{n-1} , nối nhau bởi các cạnh ghép cặp tương ứng. Thực ra, ta có thể chọn bất kỳ bit nào để phân lớp. Đặt

$$V_{i,k} = \{v \in V \mid \text{bit thứ } i \text{ của } v \text{ bằng } k\},$$

và $G_{i,k}$ là đồ thị con cảm sinh bởi $V_{i,k}$, với $0 \leq i < n$, $k \in \{0, 1\}$.

Tổng quát hơn,

$$Q_n = Q_{n-m} \times Q_m.$$

Ví dụ khi $m = 2$, có thể xem Q_n gồm bốn bản sao Q_{n-2} ; các cạnh giữa các bản sao tạo thành 2^{n-2} chu trình $Q_2 = C_4$.

Một góc nhìn khác là chia các đỉnh theo $\text{popcount}(v)$. Các đỉnh $0 \leq v < 2^n$ được chia thành $n+1$ tầng. Khi đó:

1. tầng i có $\binom{n}{i}$ đỉnh;
2. cạnh chỉ nối giữa hai tầng kề nhau;
3. mỗi đỉnh ở tầng i nối i cạnh xuống tầng $i - 1$ và $n - i$ cạnh lên tầng $i + 1$.

Trong bài viết, góc nhìn này được gọi là góc nhìn “hình cầu”, và S_i ký hiệu tập đỉnh tầng i .

16.3 Tính chất trên đồ thị vô hướng

16.3.1 Tính chất cơ bản

Đồ thị hai phía. Tô các đỉnh theo tính chẵn lẻ của popcount. Mỗi cạnh làm thay đổi đúng một bit, nên luôn nối một đỉnh đen với một đỉnh trắng. Vì vậy Q_n là đồ thị hai phía liên thông.

Đường kính. Với mọi $x, y \in V$,

$$\text{dist}(x, y) = \text{popcount}(x \oplus y),$$

nên đường kính của Q_n bằng n .

Tập độc lập lớn nhất. Lấy toàn bộ đỉnh có popcount chẵn, hoặc toàn bộ đỉnh có popcount lẻ, ta được một tập độc lập kích thước 2^{n-1} . Ngược lại, nếu chọn hơn 2^{n-1} đỉnh thì theo nguyên lý Dirichlet, tồn tại hai đỉnh được chọn giống nhau ở mọi bit trừ bit thấp nhất. Chúng kề nhau, mâu thuẫn với tập độc lập. Do đó kích thước tập độc lập lớn nhất là 2^{n-1} .

Phủ đỉnh nhỏ nhất. Lấy toàn bộ đỉnh có popcount chẵn hoặc lẻ, ta được phủ đỉnh kích thước 2^{n-1} . Mặt khác, Q_n có $n2^{n-1}$ cạnh và mỗi đỉnh có bậc n , nên cần ít nhất 2^{n-1} đỉnh để phủ hết cạnh. Vậy phủ đỉnh nhỏ nhất cũng có kích thước 2^{n-1} .

Matching hoàn hảo. Chọn bất kỳ một trong n hướng rồi tách Q_n thành hai tầng theo bit tương ứng; các cạnh giữa hai tầng tạo thành một matching hoàn hảo, tức ghép mỗi v với $v \oplus 2^i$. Ngoài ra, ở phần sau ta sẽ thấy Q_n có chu trình Hamilton, nên lấy các cặp đỉnh kề nhau xen kẽ trên chu trình Hamilton cũng tạo matching hoàn hảo. Vì vậy matching hoàn hảo của hypercube rất đa dạng.

Tự đẳng cấu. Hypercube có tính đối xứng tốt. Nếu đồng thời xor mọi nhãn đỉnh với một Δ , $0 \leq \Delta < 2^n$, đồ thị thu được vẫn là Q_n . Tương tự, với một hoán vị $p_0, p_1, \dots, p_{n-1}$ của các bit, ánh xạ

$$f(v) = \sum_{i=0}^{n-1} [\text{bit } p_i \text{ của } v \text{ bằng } 1] 2^i$$

là song ánh từ V tới V và cũng bảo toàn cấu trúc hypercube.

16.3.2 Ví dụ: JSOI 2013, *Hypercube*

Cho một đồ thị vô hướng N đỉnh, m cạnh. Cần kiểm tra nó có đẳng cấu với một hypercube hay không; nếu có, đưa ra một cách gán nhãn thỏa điều kiện hai đỉnh kề khác đúng một bit. Giới hạn $2 \leq N \leq 32768$.

Trước hết kiểm tra các điều kiện cơ bản: số đỉnh, số cạnh, bậc của mọi đỉnh và tính liên thông. Giả sử $N = 2^n$, $m = 2^{n-1}n$, và mọi đỉnh đều có bậc n . Nhờ tính tự đẳng cấu, có thể giả sử một đỉnh tùy ý mang nhãn 0, và theo một thứ tự tùy ý gán các hàng xóm của nó là

$$2^0, 2^1, \dots, 2^{n-1}.$$

Từ đỉnh 0, chạy BFS để lấy khoảng cách tới mọi đỉnh, tức thu được các tầng hình cầu. Nếu đồ thị hợp lệ, với mọi $v \in S_k$, $2 \leq k \leq \lfloor n/2 \rfloor$, nhân của v phải là siêu tập của nhân các đỉnh

$$u \in S_{k-1} \cap N(v),$$

và thực tế bằng phép OR của các nhân này. Làm lần lượt với $k = 2, 3, \dots, \lfloor n/2 \rfloor$ sẽ gán được nửa trên.

Phần còn lại đối xứng và có thể gán từ tầng $n - 2$ trở xuống. Vì thứ tự của $2^0, \dots, 2^{n-1}$ đã cố định, các hàng xóm của đỉnh $2^n - 1$, tức

$$2^n - 1 - 2^0, \dots, 2^n - 1 - 2^{n-1},$$

không còn được chọn tùy ý. Có thể từ mỗi đỉnh 2^i chạy BFS để tìm đỉnh có khoảng cách đúng n , rồi gán nó là $2^n - 1 - 2^i$. Nếu không tồn tại duy nhất một đỉnh như vậy, cấu trúc hình cầu không hợp lệ. Cuối cùng quét mọi cạnh để kiểm tra. Độ phức tạp $O(mn)$. Thực ra khi $k \geq \lfloor n/2 \rfloor$, vẫn có thể lấy phép OR nhân của $S_{k-1} \cap N(v)$, nên có thể gán từ trên xuống với độ phức tạp $O(m)$.

Một cách khác là đặt $\text{ans}[i]$ là đỉnh mang nhãn i . Giả sử $\text{ans}[0]$ và $\text{ans}[2^i]$ đã biết. Duyệt i tăng dần để xác định các $\text{ans}[i]$ còn lại. Khi $\text{popcount}(i) > 1$, chọn hai nhãn nhỏ hơn i , gọi là j_1, j_2 , khác i đúng một bit. Khi đó $\text{ans}[i]$ phải là hàng xóm chung chứa gán nhãn của $\text{ans}[j_1]$ và $\text{ans}[j_2]$. Trong hypercube, cặp này có đúng hai hàng xóm chung; một đã được gán trước đó, nên hàng xóm chưa gán là duy nhất. Nếu ở bước nào không tồn tại hoặc không duy nhất, đồ thị không phải hypercube. Cách này tốn $O(2^n n) = O(m)$.

Từ lập luận trên cũng suy ra hai kết luận tổng quát:

1. với $x, y \in V$, $x \neq y$, nếu $\text{popcount}(x \oplus y) = 2$ thì x, y có đúng hai hàng xóm chung; ngược lại chúng không có hàng xóm chung;
2. Q_n có $n!2^n$ cách gán nhãn khác nhau thỏa điều kiện hai đỉnh kề khác đúng một bit.

16.3.3 Ví dụ: Codeforces 1543E, *The Final Pursuit*

Tiếp nối ví dụ trước, cho Q_n , cần tô mỗi đỉnh bằng một trong n màu sao cho hàng xóm của mỗi đỉnh có màu đôi một khác nhau. In một phương án hoặc báo vô nghiệm. Giới hạn $1 \leq n \leq 16$, tổng kích thước không vượt 2^{16} .

Dùng các số $0, \dots, n - 1$ biểu diễn màu và giả sử nhãn của Q_n đã đúng định nghĩa. Vì mỗi đỉnh có bậc n , hàng xóm của nó phải nhận đủ n màu. Đếm hai lần cho thấy mỗi màu xuất hiện đúng $2^n/n$ lần. Do đó cần $n \mid 2^n$, tức $n = 2^k$. Nếu n không là lũy thừa của 2, báo vô nghiệm.

Khi $n = 2^k$, đặt

$$\text{col}(v) = \bigoplus_{\text{bit } i \text{ của } v \text{ bằng } 1} i.$$

Các hàng xóm của v là $v \oplus 2^i$, $0 \leq i < n$, nên màu của chúng là $\text{col}(v) \oplus i$. Khi i chạy từ 0 tới $2^k - 1$, các giá trị này lấy đủ mọi màu.

16.4 Tính chất trên tập thứ tự bán phần

16.4.1 Tập thứ tự bán phần

Với một quan hệ $A \subseteq V \times V$ trên tập V , viết

$$x \leq y \iff (x, y) \in A.$$

Quan hệ A là quan hệ thứ tự bán phần nếu thỏa:

1. phản xạ: $x \leq x$;
2. phản đối xứng: $x \leq y$ và $y \leq x$ suy ra $x = y$;
3. bắc cầu: $x \leq y$ và $y \leq z$ suy ra $x \leq z$.

Các ví dụ thường gặp là $(\mathbb{R}, \leq)$ và $(\mathbb{Z}_+, |)$.

Trên hypercube, ta định nghĩa $x \leq y$ khi tập bit 1 của x là tập con của tập bit 1 của y . Do đó có thể xem tập đỉnh của Q_n là họ tập

$$2^{\{0,1,\dots,n-1\}}.$$

Các cạnh của hypercube chính là sơ đồ Hasse của quan hệ thứ tự này; để được tập thứ tự bán phần đầy đủ, cần thêm toàn bộ cạnh trong bao đóng bắc cầu. Trong phần này, ta xét các bài toán trên

$$(2^{\{0,1,\dots,n-1\}}, \subseteq).$$

Góc nhìn hình cầu vẫn áp dụng và đóng vai trò quan trọng.

16.4.2 Định lý Dilworth

Với một quan hệ thứ tự bán phần (V, A) , một tập con $V' \subseteq V$ được gọi là phản dây nếu không tồn tại hai phần tử khác nhau $x, y \in V'$ sao cho $x \leq y$. Phản dây lớn nhất là phản dây có kích thước lớn nhất.

Một phân hoạch $V_1, V_2, \dots, V_k$ của V là phủ bằng dây nếu trong mỗi V_i , các phần tử có thể sắp theo thứ tự

$$d_1 \leq d_2 \leq \dots \leq d_i.$$

Phủ bằng dây nhỏ nhất là phủ có k nhỏ nhất. Định lý Dilworth nói rằng kích thước phản dây lớn nhất bằng số dây trong phủ bằng dây nhỏ nhất.

16.4.3 Định lý Dilworth trên hypercube

Trước hết xây dựng phủ bằng dây nhỏ nhất trên hypercube. Mỗi tầng trong góc nhìn hình cầu là một phản dây, nên phủ bằng dây cần ít nhất

$$|S_{\lfloor n/2 \rfloor}|$$

dây.

Bổ đề. Với $0 \leq i \leq n-1$, đồ thị hai phía tạo bởi S_i và S_{i+1} có matching hoàn hảo đối với phía ít đỉnh hơn.

Chứng minh. Trong đồ thị hai phía này, mỗi đỉnh ở phía S_i có bậc $n-i$, còn mỗi đỉnh ở phía S_{i+1} có bậc $i+1$. Đây là đồ thị hai phía chính quy theo từng phía và thỏa điều kiện Hall đối với phía ít đỉnh hơn, nên matching mong muốn tồn tại.

Bắt đầu từ tầng $\lfloor n/2 \rfloor$, cho mỗi phần tử tạo một dây riêng rồi mở rộng về phía tầng 0 bằng các matching trên. Tương tự, từ tầng $\lfloor n/2 \rfloor$ mở rộng xuống phía còn lại. Khi n lẻ, giữa hai tầng giữa cũng có matching hoàn hảo. Vì vậy ta dựng được phủ bằng dây kích thước $|S_{\lfloor n/2 \rfloor}|$. Mặt khác, trong phủ này, một phản dây giao mỗi dây nhiều nhất một phần tử, nên các tầng giữa đạt kích thước phản dây lớn nhất.

Định lý Sperner. Phản dây lớn nhất của $(Q_n, \subseteq)$ có kích thước không vượt

$$\binom{n}{\lfloor n/2 \rfloor}.$$

Chứng minh. Giả sử $S = \{v_1, \dots, v_m\}$ là một phản dây, trong đó mỗi v_i được xem như một tập con của $\{0, 1, \dots, n-1\}$. Đặt

$$F(v_i) = \{\pi \mid \{\pi_0, \pi_1, \dots, \pi_{|v_i|-1}\} = v_i\},$$

trong đó π là một hoán vị của $0, \dots, n-1$. Vì S là phản dây, các tập $F(v_i)$ đôi một rời nhau, nên

$$\sum_{i=1}^m |F(v_i)| \leq n!.$$

Mặt khác,

$$|F(v_i)| = (n - |v_i|)! |v_i|!,$$

và giá trị này nhỏ nhất khi $|v_i| = \lfloor n/2 \rfloor$ hoặc $\lceil n/2 \rceil$. Do đó

$$m \leq \frac{n!}{(n - \lfloor n/2 \rfloor)! \lfloor n/2 \rfloor!} = \binom{n}{\lfloor n/2 \rfloor}.$$

Đây là một chứng minh cổ điển của định lý Sperner.

16.4.4 Khái niệm đảo Mobius

Với tập thứ tự bán phần (V, A) , xét các hàm

$$f : V \times V \rightarrow \mathbb{R}$$

thỏa $f(x, y) \neq 0$ chỉ khi $x \leq y$. Có thể xem f như một ma trận $|V| \times |V|$.

Bổ đề. Nếu ma trận α thuộc lớp trên và khả nghịch, thì α^{-1} cũng thuộc lớp đó.

Chứng minh. Sắp các phần tử V theo một thứ tự topo, khi đó α có thể xem là ma trận tam giác trên. Nếu α khả nghịch thì các phần tử đường chéo chính đều khác 0, và α^{-1} cũng là tam giác trên. Với $x \not\leq y$, có thể chọn một thứ tự topo hợp lệ đặt y trước x , chẳng hạn thêm ràng buộc y trước x . Trong thứ tự này, vị trí (x, y) nằm dưới đường chéo, nên $\alpha^{-1}(x, y) = 0$. Kết luận không phụ thuộc vào thứ tự topo được chọn.

Gọi Z là ma trận đặc trưng của quan hệ thứ tự:

$$Z(x, y) = [x \leq y].$$

Theo bổ đề, $\mu = Z^{-1}$ tồn tại trong lớp hàm trên; đây chính là hàm Mobius của tập thứ tự bán phần.

16.4.5 Tính chất của ma trận Mobius

Từ $Z\mu = I$, với mọi $x, y \in V$ có

$$\sum_{x \leq z \leq y} \mu(z, y) = [x = y].$$

Suy ra $\mu(x, x) = 1$, và nếu $x \not\leq y$ thì $\mu(x, y) = 0$. Có thể diễn $\mu(x, y)$ theo thứ tự tăng dần của khoảng cách từ x tới y .

Ý nghĩa của μ là: với mọi hàm $f : V \rightarrow \mathbb{R}$, nếu

$$g(x) = \sum_{y \leq x} f(y),$$

thì

$$f(x) = \sum_{y \leq x} \mu(y, x) g(y).$$

Đôi khi tính trực tiếp f khó, nhưng tính g dễ hơn; khi đó đảo Mobius cho phép khôi phục f .

Ứng dụng quen thuộc là các hàm số học, tương ứng với tập thứ tự $(\mathbb{Z}_+, |)$. Nếu giới hạn V là tập các ước của

$$n = \prod_{i=1}^k p_i^{\alpha_i},$$

thì cấu trúc thứ tự này đẳng cấu với

$$P_{\alpha_1+1} \times P_{\alpha_2+1} \times \cdots \times P_{\alpha_k+1}.$$

Ở nghĩa này, số học có thể được nhìn như một trường hợp đặc biệt của tập thứ tự bán phần.

16.4.6 Đảo Mobius trên hypercube

Trên Q_n , nếu $A \subseteq B$, khoảng giữa A và B đẳng cấu với $Q_{n-|A|}$, nên

$$\mu(A, B) = \mu(\emptyset, B \setminus A).$$

Vì vậy chỉ cần xét $\mu(\emptyset, ?)$. Điền theo các tầng $i = 1, 2, \dots, n$. Do đối xứng, các giá trị trên cùng một tầng bằng nhau; ký hiệu là $\mu(\emptyset, S_i)$. Từ đẳng thức

$$\sum_{j=0}^i (-1)^j \binom{i}{j} = 0 \quad (i > 0),$$

có thể quy nạp được

$$\mu(\emptyset, S_i) = (-1)^i.$$

Tổng quát, nếu $A \subseteq B$, thì

$$\mu(A, B) = (-1)^{|B|-|A|}.$$

Kết luận này chính là bản chất của nguyên lý bao hàm–loại trừ thường dùng.

16.5 Các đồ thị con của hypercube

16.5.1 Chu trình

Bổ đề. Q_n có chu trình Hamilton.

Chứng minh. Chu trình Hamilton trên Q_n chính là mã Gray. Đặt G_n là dãy Gray dài 2^n , với $G_1 = \{0, 1\}$. Với $n \geq 2$, dựng

$$G_n = G_{n-1}(0), G_{n-1}(1), \dots, G_{n-1}(2^{n-1} - 1),$$

rồi nối tiếp với

$$G_{n-1}(2^{n-1} - 1) + 2^{n-1}, \dots, G_{n-1}(0) + 2^{n-1}.$$

Ví dụ $G_2 = \{0, 1, 3, 2\}$, $G_3 = \{0, 1, 3, 2, 6, 7, 5, 4\}$. Dãy thu được là một hoán vị và hai phần tử kế nhau, kể cả cặp cuối–đầu, luôn khác đúng một bit.

Từ đó, trong Q_n có thể tìm được đường đi đơn với mọi độ dài nhỏ hơn 2^n . Với chu trình đơn, do Q_n là đồ thị hai phía nên độ dài lẻ là không thể. Ngược lại, mọi độ dài chẵn đều làm được.

Bổ đề. Với mọi $4 \leq \ell \leq 2^n$, nếu ℓ chẵn thì trong Q_n tồn tại một chu trình đơn C_ℓ .

Chứng minh. Đặt $l = \ell/2$. Lấy l phần tử đầu của G_{n-1} , gọi là

$$d_0, d_1, \dots, d_{l-1}.$$

Khi đó

$$d_0, d_1, \dots, d_{l-1}, d_{l-1} + 2^{n-1}, \dots, d_1 + 2^{n-1}, d_0 + 2^{n-1}$$

là một chu trình đơn độ dài ℓ .

16.5.2 Ví dụ về chu trình Hamilton xen kẽ matching

Cho một matching hoàn hảo trên Q_n , gọi là các cạnh đỏ. Cần chọn thêm một matching hoàn hảo khác, gọi là cạnh xanh, sao cho cạnh đỏ và xanh xen kẽ tạo thành một chu trình Hamilton. Có thể chứng minh khi $n \geq 2$ luôn có nghiệm.

Thực ra có thể chứng minh mệnh đề mạnh hơn: các cặp đỏ không cần là cạnh của Q_n , chỉ cần là một cách ghép đôi 2^{n-1} cặp đỉnh. Gọi $\text{mat}[v]$ là đỉnh ghép với v . Chứng minh bằng quy nạp theo n .

Tách Q_n thành hai tầng G_0, G_1 , mỗi tầng là Q_{n-1} . Đặt

$$S_i = \{v \in G_i \mid \text{mat}[v] \in G_{i \oplus 1}\},$$

và T_i là tập các cặp đỏ nằm hoàn toàn trong G_i . Khi $|S_0| > 0$, ghép tùy ý các đỉnh trong S_0 thành cặp, kết hợp với T_0 , rồi áp dụng giả thiết quy nạp trong G_0 để thu được một chu trình xen kẽ. Thử tự các cặp trong S_0 trên chu trình đó quyết định cách ghép các đỉnh tương ứng trong S_1 ; áp dụng quy nạp cho G_1 , rồi cắt các cặp giả và thay bằng các cặp đỏ thật nối giữa S_0 và S_1 . Nếu ban đầu $S_0 = S_1 = \emptyset$, chọn lại hướng tách tầng theo một bit mà một cặp đỏ bất kỳ khác nhau để đưa về trường hợp trên.

16.5.3 Lưới

Ở đây “lưới” là đồ thị

$$P_{x_1} \times P_{x_2} \times \dots \times P_{x_k}.$$

Đặt $y_i = \lceil \log_2 x_i \rceil$. Nếu

$$n \geq \sum_{i=1}^k y_i,$$

có thể nhúng trực tiếp lưới vào Q_n bằng mã Gray: điểm $(\alpha_1, \dots, \alpha_k)$, với $0 \leq \alpha_i < x_i$, được ánh xạ thành chuỗi bit ghép nối

$$(G_{y_1}(\alpha_1))_2 (G_{y_2}(\alpha_2))_2 \dots (G_{y_k}(\alpha_k))_2.$$

Ví dụ: eJOI 2021, *Chép bài*. Cho n, m . Cần điền các số nguyên không âm đôi một khác nhau vào lưới $n \times m$, sao cho hai ô kề cạnh có hai số khác nhau đúng một bit trong nhị phân. Đồng thời, giá trị lớn nhất trên lưới phải nhỏ nhất. Giới hạn $1 \leq n, m \leq 2000$.

Cách phóng n, m lên lũy thừa của 2 gần nhất rồi dùng mã Gray là hợp lệ nhưng giá trị lớn nhất có thể không tối ưu. Thực ra, việc nhúng lưới không yêu cầu dãy Gray phải nối hợp lệ giữa phần tử cuối và đầu, nên có thể dựng dãy Gray độ dài tùy ý, chỉ cần các phần tử kề ở giữa hợp lệ.

Bổ đề. Với mọi $n \in \mathbb{Z}_+$, tồn tại một hoán vị p của $\{0, \dots, n-1\}$ sao cho

$$\text{popcount}(p_{i-1} \oplus p_i) = 1$$

với mọi $1 \leq i < n$.

Chứng minh. Viết

$$n = \sum_{i=1}^k 2^{d_i}, \quad 0 \leq d_1 < d_2 < \dots < d_k.$$

Ghép nối các khối Gray G_{d_i} , mỗi khối cộng thêm tổng kích thước các khối trước nó. Để điểm nối giữa hai khối hợp lệ, chọn khi i chẵn thì G_{d_i} bắt đầu bằng 0, kết thúc bằng 1, còn khi i lẻ thì ngược lại. Nhờ tính đối xứng của hypercube, luôn có thể chọn được dạng như vậy; cũng có thể lấy chu trình Gray đã dựng ở trên rồi quay vòng.

Sau đó cần kết hợp dãy p_n và p_m để điền vào bảng. Cách ghép nhị phân trực tiếp vẫn có thể chưa tối ưu. Thực tế, ta có thể chọn $\lceil \log_2 n \rceil$ bit nào dành cho dãy theo hàng và các bit còn lại dành cho dãy theo cột, không bắt buộc là hai đoạn tiền tố-hậu tố. Cách chọn tối ưu có thể tìm bằng DP hai chiều, hoặc tham lam từ bit cao nhất: mỗi lần chọn phương án còn lại có thứ tự từ điển nhỏ hơn.

Tính đúng đắn dựa trên tính độc lập của hai chiều. Với một cách điền hợp lệ $A(i, j)$, xét bốn ô của một hình vuông đơn vị:

$$A(i, j), A(i+1, j), A(i, j+1), A(i+1, j+1).$$

Hai ô chéo là hai hàng xóm chung của hai ô chéo còn lại trong hypercube, nên

$$A(i, j) \oplus A(i+1, j) = A(i, j+1) \oplus A(i+1, j+1).$$

Suy ra trên mỗi cặp hàng kề, giá trị xor giữa hai hàng là hằng theo cột; tương tự cho cột. Gọi các hằng này là H_i và L_j , đều là lũy thừa của 2. Nếu một bit k được dùng đồng thời cho một hàng và một cột, tức $H_i = L_j = 2^k$, thì bốn ô ở giao điểm sẽ cho

$$A(i, j) = A(i+1, j) \oplus 2^k = A(i, j+1) \oplus 2^k = A(i+1, j+1),$$

mâu thuẫn với yêu cầu các số đôi một khác nhau. Vì vậy hai chiều dùng các bit độc lập.

16.6 Magic Labeling

Phần này xét một dạng gán nhãn đặc biệt cho đỉnh hoặc cạnh, gọi là Magic Labeling. Ràng buộc trong bài viết mạnh hơn một số định nghĩa thông thường, nên đôi khi cũng được gọi là Super Magic Labeling.

16.6.1 Magic Labeling trên đỉnh

Với đồ thị vô hướng $G = (V, E)$, một ánh xạ

$$f : V \rightarrow \{0, 1, \dots, |V| - 1\}$$

là Magic Labeling trên đỉnh nếu f là song ánh và tồn tại Δ sao cho

$$\forall v \in V, \quad \sum_{(u,v) \in E} f(u) = \Delta.$$

Bổ đề. Nếu n lẻ, Q_n không có Magic Labeling trên đỉnh.

Chứng minh. Nếu tồn tại, thì

$$|V|\Delta = \sum_{v \in V} \deg(v)f(v),$$

nên

$$\Delta = \frac{1}{2}(2^n - 1)n.$$

Khi n lẻ, Δ không nguyên, mâu thuẫn.

Dễ thấy $n = 2$ có nghiệm. Xét $n \geq 4$ chẵn.

Bổ đề. Nếu $4 \mid n$, Q_n không có Magic Labeling trên đỉnh.

Chứng minh. Giả sử tồn tại. Đặt

$$F(i) = \sum_{v \in S_i} f(v).$$

Đếm hai lần trên cấu trúc hình cầu được

$$F(i-1)(n-i+1) + F(i+1)(i+1) = \Delta |S_i|$$

với $1 \leq i < n$. Thay $i = 1$ và $i = n-1$ cho thấy

$$F_0 = F_n \iff F_2 = F_{n-2}.$$

Lặp lại suy ra

$$F_0 = F_n \iff F_{2i} = F_{n-2i}.$$

Khi $4 \mid n$, điều này dẫn tới $F_{n/2} = F_{n/2}$ và buộc $f(0) = f(2^n - 1)$, mâu thuẫn với tính đơn ánh.

Định lý. Q_n có Magic Labeling trên đỉnh khi và chỉ khi

$$n \equiv 2 \pmod{4}.$$

Chứng minh phác thảo. Với ví dụ $n = 6$, cần dựng 6 vector nhị phân độ dài 6, độc lập tuyến tính, và ở mỗi tọa độ có đúng ba số 0, ba số 1. Gọi các vector này khi xem như số nhị phân là $v_0, \dots, v_5$, rồi đặt

$$f(v) = \bigoplus_{\text{bit } i \text{ của } v \text{ bằng } 1} v_i.$$

Do $v_0, \dots, v_5$ độc lập tuyến tính, các $f(v)$ đôi một khác nhau. Đồng thời mỗi tọa độ xuất hiện đúng $n/2$ lần trong tổng trên hàng xóm, nên tổng hàng xóm là hằng. Với mọi $n \geq 6$, $n \equiv 2 \pmod{4}$, có thể dựng các vector bằng ngẫu nhiên kết hợp kiểm tra cơ sở tuyến tính. Khi $4 \mid n$, yêu cầu mỗi tọa độ có đúng $n/2$ bit 1 làm xor của mọi vector bằng 0, nên chúng phụ thuộc tuyến tính, phù hợp với bổ đề vô nghiệm.

16.6.2 Magic Labeling trên cạnh

Với đồ thị vô hướng $G = (V, E)$, một ánh xạ

$$f : E \rightarrow \{0, 1, \dots, |E| - 1\}$$

là Magic Labeling trên cạnh nếu f là song ánh và tồn tại Δ sao cho

$$\forall v \in V, \quad \sum_{(u,v) \in E} f(u, v) = \Delta.$$

Dễ thấy Q_1 có nghiệm trên cạnh, còn Q_2 không có. Sau đây giả sử $n > 2$.

Bổ đề. Nếu n lẻ, Q_n không có Magic Labeling trên cạnh.

Chứng minh. Nếu tồn tại thì

$$2^n \Delta = m(m-1), \quad m = n2^{n-1},$$

suy ra

$$\Delta = \frac{n}{2}(n2^{n-1} - 1).$$

Khi n lẻ, Δ không nguyên.

Bổ đề. Nếu đồ thị hai phía $G = (V, E)$ là hợp của hai chu trình Hamilton, thì tồn tại Magic Labeling trên cạnh.

Chứng minh. Đặt $n = |V|$, khi đó n chẵn. Gọi hai chu trình Hamilton là C_0, C_1 , và giả sử C_0 đi qua các đỉnh $0, 1, \dots, n-1$. Trên C_0 , gán

$$f(2i, 2i+1) = i, \quad f(2i-1, 2i) = n-i$$

với chỉ số lấy modulo n . Khi đó tổng nhãn cạnh kề của đỉnh 0 là $n/2$; các đỉnh chẵn khác có tổng n , còn các đỉnh lẻ có tổng $n-1$.

Trên C_1 , cũng dựng tương tự từ đỉnh 0, nhưng giá trị x được thay bằng $2n-1-x$. Vì đồ thị hai phía không có chu trình lẻ, phần tăng thêm sẽ bù đúng các sai khác trên C_0 , và mọi đỉnh có cùng tổng

$$2(2n-1).$$

Bổ đề. Q_4 có Magic Labeling trên cạnh.

Chứng minh. Bản gốc đưa ra một cách tách Q_4 thành hai chu trình Hamilton. Áp dụng bổ đề trên là được một cách gán nhãn cạnh.

Bổ đề. Nếu Q_n với n chẵn có Magic Labeling trên cạnh, thì Q_{n+2} cũng có.

Chứng minh phác thảo. Dựa vào nhãn đỉnh modulo 4, tách Q_{n+2} thành bốn bản sao của Q_n , gọi là G_0, G_1, G_2, G_3 . Trong mỗi G_i , dùng lại cách gán nhãn của Q_n , nhưng với mỗi cạnh hướng j cộng thêm $d_{i,j}T$. Các hằng $d_{i,j}$ được chọn sao cho với mỗi j , bốn giá trị $d_{i,j}$ lấy đủ $\{0, 1, 2, 3\}$, và các tổng hàng $s_i = \sum_j d_{i,j}$ thỏa

$$s_0 - 1 = s_1 = s_2 = s_3 + 1.$$

Với $n = 4$, bài gốc đưa ra một ma trận d cụ thể; với n chẵn tổng quát, chỉ cần thêm luân phiên hai cột $[0, 1, 2, 3]^T$ và $[3, 2, 1, 0]^T$.

Chọn

$$T = (n+2)2^{n-1}$$

để bốn bản sao dùng các khoảng nhãn rời nhau; phần nhãn còn lại dành cho các chu trình C_4 nối giữa bốn bản sao. Ghép đầu-cuối các khoảng còn trống sẽ làm mọi tổng $x+y$ trên các cạnh nối giữa bản sao bằng nhau, hoàn tất bước quy nạp.

Từ các bổ đề trên:

Định lý. Với $n > 2$, Q_n có Magic Labeling trên cạnh khi và chỉ khi n chẵn.

16.7 Kết luận

Bài viết bắt đầu từ định nghĩa cơ bản của hypercube, trình bày các tính chất đẹp của nó khi xem như đồ thị vô hướng và khi xem như tập thứ tự bán phần, rồi phân tích một số ví dụ thi tin học liên quan mật thiết. Phần cuối khảo sát chi tiết bài toán xây dựng Magic Labeling trên hypercube, kết hợp các ý tưởng toán học và thuật toán, đồng thời cho thấy sự hữu ích của nhiều góc nhìn cấu trúc khác nhau.

Tác giả hy vọng bài viết giúp nhiều thí sinh biết tới hypercube như một mô hình toán học đặc sắc và tiếp tục nghiên cứu sâu hơn.

16.8 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng giao lưu và học tập; cảm ơn cha mẹ và gia đình đã quan tâm, ủng hộ trong nhiều năm; cảm ơn thầy Jin Jing của Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông đã hướng dẫn; cảm ơn Wang Youjia và Yang Ningyuan đã đọc bản thảo.

16.9 Tài liệu tham khảo

1. Frank Harary. A survey of the theory of hypercube graphs, 1988.
2. Oliver Bernardi. On the spanning trees of the hypercube and other products of graphs, 2018.
3. D. Lubell. A short proof of Sperner's lemma.
4. Youcef Saad and Martin H. Schultz. Topological properties of hypercubes. Research Report YALEU/DCS/RR-389, 1985.
5. W. D. Wallis, Edy T. Baskoro, Mirka Miller and Slamin. Edge-magic total labelings, 2000.
6. Petr Gregor and Petr Kovar. Distance magic labelings of hypercubes, 2013.
7. Curtis Greene. The Mobius function of a partially ordered set, 1982.
8. R. P. Dilworth. A decomposition theorem for partially ordered sets. *Annals of Mathematics*, Second Series, 51(1):161–166, 1950.
9. Peng Bo. Báo cáo lời giải *Matching tập con*, tập huấn đội tuyển năm 2021.

17 Bàn về một số phương pháp phân tích thừa số nguyên tố

Trường Trung học Ba Thục, Trùng Khánh
Luo Siyuan

Tóm tắt

Số học là một mảng lớn trong thi tin học; nhiều bài toán liên quan tới phân tích thừa số nguyên tố, và bản thân phân tích thừa số nguyên tố cũng có vị trí quan trọng trong học thuật. Bài viết giới thiệu ba thuật toán phân tích thừa số nguyên tố còn ít phổ biến trong phạm vi OI, nhưng bản thân thuật toán khá gọn: SQUFOF, CFRAC và sàng bậc hai.

17.1 Quy ước

Khi xét phân tích thừa số nguyên tố, ký hiệu N là số cần phân tích. Phân tích thừa số nguyên tố của N nghĩa là viết

$$N = \prod_{1 \leq i \leq k} p_i,$$

trong đó mọi p_i đều là số nguyên tố. Một số nguyên dương d được gọi là nhân tử không tầm thường của n khi và chỉ khi $d \mid n$ và

$$1 < d < n.$$

17.2 Kiến thức chuẩn bị

17.2.1 Kiểm tra nguyên tố Miller–Rabin

Trước hết nhắc lại ngắn gọn Miller–Rabin. Gọi số cần kiểm tra là n , giả sử n lẻ. Viết

$$n - 1 = 2^d r,$$

trong đó r lẻ và d là số nguyên dương. Chọn một cơ sở a , dùng lũy thừa nhanh tính

$$x = a^r \bmod n,$$

rồi cho x tự bình phương nhiều nhất d lần. Nếu n là số nguyên tố, cuối cùng x phải trở thành 1, vì theo định lý Fermat nhỏ,

$$a^{n-1} \equiv 1 \pmod{n}.$$

Khi x trở thành 1, thuật toán dừng.

Nếu ở một bước bình phương nào đó x trở thành 1, nhưng trước khi bình phương x không đồng dư với 1 hoặc $n - 1$, thì n chắc chắn hợp số. Lý do là nếu $2 \leq x \leq n - 2$ và

$$x^2 \equiv 1 \pmod{n},$$

thì $n \mid (x+1)(x-1)$. Nếu n nguyên tố, phải có $n \mid x-1$ hoặc $n \mid x+1$, mâu thuẫn với $x-1, x+1 \in [1, n-1]$.

Với một cơ sở a và hợp số n , nếu quá trình trên không xác định được n là hợp số, a được gọi là một *strong liar* của n . Tài liệu [3] chỉ ra rằng với hợp số n , số strong liar không quá $n/4$. Vì vậy nếu lặp k vòng với cơ sở ngẫu nhiên, xác suất sai không quá 4^{-k} . Cũng theo [3], với số nguyên dương nhỏ hơn

$$3\,825\,123\,056\,546\,413\,051 > 10^{18},$$

chọn chín số nguyên tố đầu tiên làm cơ sở là đủ để đúng chắc chắn; với số nhỏ hơn 2^{64} , chọn mười hai số nguyên tố đầu tiên là đủ. Miller–Rabin cần $O(k \log n)$ phép nhân modulo n , trong đó k là số vòng.

17.2.2 Thử chia

Nếu n không nguyên tố, nhân tử nguyên tố nhỏ nhất của n không vượt $\sqrt{n}$. Do đó có thể dùng sàng tuyến tính $O(\sqrt{n})$ để lấy mọi số nguyên tố không vượt $\sqrt{n}$, rồi khi phân tích chỉ cần thử chia bằng các số nguyên tố này. Theo định lý số nguyên tố [1], số nguyên tố không vượt x là $O(x/\log x)$, nên thời gian tiền xử lý là $O(\sqrt{n})$, và mỗi lần phân tích tốn

$$O\left(\frac{\sqrt{n}}{\log n}\right).$$

Ví dụ 1. Cho n số nguyên dương $a_1, \dots, a_n$, hỏi có tồn tại hai số a_i, a_j , $i \neq j$, không nguyên tố cùng nhau hay không. Giới hạn $1 \leq n \leq 10^5$, $1 \leq a_i \leq 10^9$, thời gian 3 giây.

Phân tích thừa số nguyên tố mọi a_i bằng thử chia, rồi kiểm tra có nhân tử nguyên tố nào xuất hiện trong hơn một số hay không. Nếu $m = \max a_i$, độ phức tạp là

$$O\left(n \frac{\sqrt{m}}{\log m}\right).$$

Vì số nguyên tố cần thử không nhiều, sau khi tiền xử lý thích hợp có thể dùng Barrett reduction để giảm hằng số của phép chia.

Trong một số bài toán, nhờ tính chất riêng của đề, có thể hạ ngưỡng thử chia để đạt độ phức tạp tốt hơn.

Ví dụ 2. Có T truy vấn; mỗi truy vấn cho x , cần tính $\mu(x)$. Giới hạn $T \leq 5000$, $1 \leq x \leq 10^{18}$, thời gian 2 giây.

Thử chia nhưng chỉ liệt kê các nhân tử nguyên tố không vượt $\sqrt[3]{x}$. Sau khi chia hết các nhân tử nhỏ, phần còn lại nếu khác 1 thì chỉ có thể là số nguyên tố, bình phương của số nguyên tố, hoặc tích của hai số nguyên tố. Hai trường hợp đầu có thể kiểm tra bằng Miller–Rabin và căn bậc hai nguyên; nếu đều không đúng thì phần còn lại là tích của hai nguyên tố. Chừng ấy thông tin đã đủ xác định giá trị của $\mu(x)$. Nếu $m = \max x$, thời gian là

$$O\left(T \frac{\sqrt[3]{m}}{\log m}\right).$$

17.2.3 Giới thiệu ngắn về liên phân số

Trước khi trình bày các thuật toán phân tích phía sau, cần nhắc tới liên phân số. Với mọi số thực x , tồn tại duy nhất một dãy hữu hạn hoặc vô hạn

$$[a_0; a_1, a_2, \dots]$$

sao cho $a_0 \in \mathbb{Z}$, $a_i \in \mathbb{N}^*$ với $i > 0$, và

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \dots}}}.$$

Dãy này được gọi là biểu diễn liên phân số thường của x . Trong phần sau, khi nói liên phân số mà không nói thêm, ta hiểu là liên phân số thường.

Để tính biểu diễn này, đặt $x_0 = x$, rồi lặp:

$$a_i = \lfloor x_i \rfloor, \quad x_{i+1} = \frac{1}{x_i - a_i}.$$

Nếu ở một bước $a_i = x_i$, tức x_i là số nguyên, thuật toán dừng.

Tính chất. Thuật toán dừng khi và chỉ khi x là số hữu tỉ.

Chứng minh. Nếu thuật toán dừng, liên phân số chỉ có hữu hạn hạng nên tính ngược lại được một số hữu tỉ. Ngược lại, giả sử $x = P/Q$, $Q > P$, $\gcd(P, Q) = 1$. Bước từ x_i sang x_{i+1} tương ứng với một bước của thuật toán Euclid trên cặp (P, Q) . Vì thuật toán Euclid chỉ có hữu hạn bước, liên phân số cũng hữu hạn.

Xét $\sqrt{n}$, với n không là số chính phương. Viết

$$x_i = \frac{\sqrt{n} + P_i}{Q_i},$$

trong đó $P_0 = 0$, $Q_0 = 1$. Khi đó

$$a_i = \left\lfloor \frac{\sqrt{n} + P_i}{Q_i} \right\rfloor,$$

và từ công thức nghịch đảo thu được truy hồi

$$P_{i+1} = a_i Q_i - P_i, \quad Q_{i+1} = \frac{n - P_{i+1}^2}{Q_i}. \quad (1)$$

Tính chất. Mọi P_i, Q_i đều là số nguyên.

Chứng minh phác thảo. Với $i = 0, 1$ điều này trực tiếp đúng. Nếu P_k, Q_k và P_{k-1}, Q_{k-1} đều nguyên, thì P_{k+1} nguyên. Hơn nữa

$$n - P_{k+1}^2 \equiv n - P_k^2 \equiv 0 \pmod{Q_k},$$

nên Q_{k+1} cũng nguyên.

Tính chất. Với mọi $i \geq 0$,

$$0 < Q_i < 2\sqrt{n}, \quad 0 \leq P_i < \sqrt{n}.$$

Chứng minh dùng quy nạp từ truy hồi (1). Từ quá trình tính liên phân số, khi $i \geq 1$ luôn có $x_i > 0, a_i \geq 1$. Nếu $Q_i > \sqrt{n}$ hoặc $Q_i < \sqrt{n}$, đều suy ra $P_{i+1} > 0$; sau đó từ

$$Q_{i+1} = \frac{n - P_{i+1}^2}{Q_i}$$

suy ra $Q_{i+1} > 0$ và $P_{i+1} < \sqrt{n}$. Cuối cùng,

$$Q_i = \frac{P_i + P_{i+1}}{a_i} < 2\sqrt{n}.$$

Do chỉ có hữu hạn cặp (P_i, Q_i) , liên phân số của $\sqrt{n}$ có chu kỳ. Thực ra, một số vô tỉ có liên phân số tuần hoàn khi và chỉ khi nó là nghiệm thực của một phương trình bậc hai hệ số nguyên; bài viết không chứng minh kết quả này.

Gọi

$$\frac{A_k}{B_k} = [a_0; a_1, \dots, a_k]$$

là hội tụ thứ k của liên phân số. Với $i \geq 2$,

$$A_i = a_i A_{i-1} + A_{i-2}, \quad B_i = a_i B_{i-1} + B_{i-2}. \quad (2)$$

Ngoài ra,

$$A_{i-1} B_i - A_i B_{i-1} = (-1)^i,$$

nên theo đẳng thức Bezout, $\gcd(A_i, B_i) = 1$ với mọi i .

Một tính chất nữa là với mọi $i \geq 2$,

$$x = \frac{A_{i-2} + A_{i-1} x_i}{B_{i-2} + B_{i-1} x_i}.$$

Nó xuất phát từ việc thay x_i bằng a_i trong liên phân số hữu hạn sẽ cho hội tụ tương ứng.

17.3 Square Form Factorization

Square Form Factorization, viết tắt SQUFOF, do Shanks đề xuất.

17.3.1 Ý tưởng thuật toán

SQUFOF dựa trên tính chất sau.

Tính chất. Với mọi $i \geq 1$,

$$A_{i-1}^2 - n B_{i-1}^2 = (-1)^i Q_i.$$

Chứng minh. Thay

$$\sqrt{n} = \frac{A_{i-2} + A_{i-1}x_i}{B_{i-2} + B_{i-1}x_i}, \quad x_i = \frac{\sqrt{n} + P_i}{Q_i},$$

rồi so sánh hệ số hữu tỉ và hệ số của $\sqrt{n}$, thu được

$$Q_i B_{i-2} + P_i B_{i-1} - A_{i-1} = 0,$$

$$Q_i A_{i-2} + P_i A_{i-1} - n B_{i-1} = 0.$$

Nhân phương trình đầu với A_{i-1} , phương trình sau với B_{i-1} , rồi lấy hiệu và dùng

$$A_{i-2} B_{i-1} - B_{i-2} A_{i-1} = (-1)^i,$$

suy ra kết luận.

Vì vậy

$$A_{i-1}^2 \equiv (-1)^i Q_i \pmod{n}.$$

Ý tưởng của SQUFOF là tìm một chỉ số chẵn i sao cho $Q_i = t^2$ là số chính phương. Khi đó

$$A_{i-1}^2 - t^2 \equiv 0 \pmod{n},$$

và với xác suất lớn,

$$\gcd(n, A_{i-1} \pm t)$$

là một nhân tử không tầm thường của n .

Quy trình cơ bản:

1. khởi tạo $P_0, Q_0, A_0, a_0, P_1, Q_1, A_1, a_1$;
2. dùng (1), (2) để sinh các hạng P, Q, A, a , cho tới khi gặp một vị trí chẵn i có Q_i là số chính phương;
3. trả về $\gcd(n, A_{i-1} \pm \sqrt{Q_i})$.

Nếu kết quả vẫn là nhân tử tầm thường, có thể chọn một số nguyên dương nhỏ k làm multiplier và lặp lại thuật toán trên $n' = kn$.

17.3.2 Tối ưu

Một số tối ưu hằng số:

- Khi n không quá lớn, có thể dùng Barrett reduction hoặc kỹ thuật tương tự để tối ưu việc tính A_i . Dù multiplier ban đầu là gì, A_i chỉ cần lấy modulo n , không phải modulo kn .
- Để kiểm tra x là số chính phương, chọn một môđun M , trước hết kiểm tra $x \bmod M$ có phải thặng dư bậc hai hay không; nếu có mới gọi hàm căn bậc hai.
- Khi tính P, Q, a , công thức trực tiếp dùng hai phép chia, một cho Q và một cho a . Có thể bỏ phép chia tính Q . Từ

$$Q_i Q_{i-1} = n - P_i^2, \quad Q_i Q_{i+1} = n - P_{i+1}^2,$$

lấy hiệu và dùng $P_{i+1} = a_i Q_i - P_i$ được

$$Q_{i+1} = Q_{i-1} + a_i(P_i - P_{i+1}). \quad (3)$$

- Mọi biến đều có thể cuộn, không cần mảng.

Tối ưu tránh tính A_k . Khi n^2 vượt quá phạm vi tính trực tiếp của kiểu máy nhưng n vẫn nằm trong phạm vi, P, Q, a vẫn tính bình thường, còn A khó tính. Có một quy trình tránh tính A_k :

1. khởi tạo $P_0, Q_0, a_0, P_1, Q_1, a_1$;
2. dùng (1), (3) sinh P, Q, a cho tới khi ở một vị trí chẵn $k, Q_k = w^2$;
3. bắt đầu từ

$$\frac{\sqrt{n} - P_k}{w},$$

đặt $P'_0 = -P_k, Q'_0 = w$, rồi dùng các truy hồi tương tự để sinh P', Q', a' , cho tới khi gặp $P'_i = P'_{i+1}$;

4. trả về $\gcd(Q'_i, n)$.

Tính chất. Nếu $P'_i = P'_{i+1}$, thì

$$n = Q'_i \left(Q'_{i+1} + \frac{(a'_i)^2 Q'_i}{4} \right).$$

Vì $P'_i + P'_{i+1} = a'_i Q'_i$, ta có $P'_{i+1} = a'_i Q'_i / 2$. Thay vào

$$n = (P'_{i+1})^2 + Q'_i Q'_{i+1}$$

là được. Do đó $\gcd(Q'_i, n)$ có khả năng cao là nhân tử của n ; không trả trực tiếp Q'_i vì hạng còn lại có thể không nguyên.

Trong quá trình sinh P', Q' , vẫn có

$$0 < Q'_i < 2\sqrt{n}, \quad 0 \leq P'_i < \sqrt{n},$$

nên cách hiện thực hai lần sinh dãy là như nhau. Số bước của lần sinh thứ hai xấp xỉ một nửa lần đầu; chứng minh dùng lý thuyết dạng bậc hai nhị nguyên và được dẫn trong [13].

17.3.3 Độ phức tạp và hiệu quả thực tế

Vì $Q_i < 2\sqrt{n}$, số chính phương trong khoảng này chiếm xấp xỉ

$$\Theta(n^{-1/4})$$

nếu xem Q_i gần như là các số ngẫu nhiên. Do đó sau khoảng $O(\sqrt[4]{n})$ hạng thường sẽ gặp một Q_i chính phương. Trên thực tế, SQUFOF có thể phân tích trong khoảng một giây các số ngẫu nhiên dưới 10^{30} tạo bởi tích của hai số nguyên tố cỡ 10^{15} .

17.4 Continued Fraction Method

CFRAC là một thuật toán phân tích thừa số nguyên tố khác dựa trên liên phân số của $\sqrt{n}$, do Morrison và Brillhart đề xuất.

17.4.1 Ý tưởng

CFRAC cũng dựa trên tính chất

$$A_{i-1}^2 - nB_{i-1}^2 = (-1)^i Q_i.$$

Khác với SQUFOF, CFRAC không chỉ “chờ” một Q_i là số chính phương, mà cố gắng ghép nhiều Q_i để tạo thành số chính phương. Nếu một tập chỉ số

$$U = \{i_1, i_2, \dots, i_k\}$$

thỏa

$$S = \prod_{i \in U} (-1)^i Q_i$$

là số chính phương, đặt

$$x = \sqrt{S} \bmod n, \quad y = \prod_{i \in U} A_{i-1} \bmod n.$$

Khi đó

$$x^2 - y^2 \equiv 0 \pmod{n},$$

và $\gcd(n, x \pm y)$ thường là một nhân tử không tầm thường.

Vấn đề là tìm một tập con có tích là số chính phương. Nếu phân tích được Q_i , ta biểu diễn phân tích của Q_i thành một vector trên $\mathbb{F}_2$: tọa độ thứ j là số mũ của số nguyên tố thứ j modulo 2. Cần tìm một tập vector có tổng bằng vector 0, tức giải một hệ tuyến tính trên $\mathbb{F}_2$.

Một số được gọi là B -smooth nếu mọi nhân tử nguyên tố của nó không vượt B . CFRAC chọn ngưỡng B , chỉ thử chia Q_i bằng các số nguyên tố $\leq B$, chỉ giữ các Q_i phân tích hết theo cách này, rồi dùng khử Gauss để tìm phụ thuộc tuyến tính. Khi Q_i có hệ số -1 , có thể xem -1 là một “số nguyên tố” bổ sung trong vector.

Một tối ưu quan trọng: nếu Q_i có nhân tử p , thì từ

$$A_{i-1}^2 - nB_{i-1}^2 \equiv 0 \pmod{p}$$

và $\gcd(A_{i-1}, B_{i-1}) = 1$, suy ra

$$\left(\frac{A_{i-1}}{B_{i-1}} \right)^2 \equiv n \pmod{p}.$$

Vậy n phải là thặng dư bậc hai modulo p . Có thể dùng ký hiệu Legendre để loại trước khoảng một nửa số nguyên tố vô ích.

CFRAC còn có tối ưu “large prime”. Sau khi thử chia, nếu phần còn lại là p , và có hai quan hệ cùng phần còn lại p ,

$$a_1^2 \equiv pq_1 \pmod{n}, \quad a_2^2 \equiv pq_2 \pmod{n},$$

trong đó q_1, q_2 là B -smooth đã biết phân tích, thì

$$\left(\frac{a_1 a_2}{p} \right)^2 \equiv q_1 q_2 \pmod{n}.$$

Như vậy ta thu được thêm một quan hệ đưa vào khử Gauss. Để không tốn quá nhiều thời gian và bộ nhớ, thường đặt thêm ngưỡng $B' \approx B^2$, chỉ lưu phần còn lại $p < B'$.

17.4.2 Độ phức tạp và hiệu quả thực tế

Trong thực tế, ma trận khử Gauss rất thưa và có thể nén bit, nên thường không phải nút thắt; phần lớn thời gian nằm ở thử chia. Bỏ tối ưu large prime, giả sử các Q_i gần như là số ngẫu nhiên trong $(0, 2\sqrt{n})$. Nếu tỉ lệ số B -smooth là $1/p(B)$, cần khoảng $p(B)\pi(B)$ hạng để có đủ vector, và mỗi hạng thử chia bằng khoảng $\pi(B)$ số nguyên tố. Tổng xấp xỉ

$$p(B)\pi(B)^2.$$

Dựa trên ước lượng số $x^{1/\alpha}$ -smooth không vượt x có tỉ lệ thô $\alpha^{-\alpha}$, có thể tối ưu B và thu được độ phức tạp á dưới dạng

$$O\left(\exp((1+o(1))\sqrt{2\log n \log \log n})\right).$$

Ngưỡng B tối ưu ở cỡ

$$\exp\left(\frac{\sqrt{\log n \log \log n}}{2\sqrt{2}}\right).$$

CFRAC cài đặt khá gọn và chạy nhanh; có thể phân tích số trong phạm vi `__int128` trong vài giây.

17.5 The Quadratic Sieve

Sàng bậc hai, hay QS, là thuật toán phân tích thừa số nguyên tố do Carl Pomerance phát minh.

17.5.1 Ý tưởng

Nếu chỉ cần sinh nhiều đồng dư dạng

$$x^2 \equiv y \pmod{n},$$

cách đơn giản nhất là bắt đầu từ $\lfloor \sqrt{n} \rfloor + 1$, liệt kê tăng dần x , rồi đặt $y = x^2 \bmod n$. Nếu thay phần sinh liên phân số của CFRAC bằng quá trình này, cấp số mũ của độ phức tạp vẫn tương tự; đây là phương pháp Kraitichik. Tuy nhiên nó kém CFRAC trong thực tế vì các số cần kiểm tra có kích thước khoảng $C\sqrt{n}$, lớn hơn đáng kể so với $2\sqrt{n}$, làm số smooth thưa hơn.

Điểm có thể tối ưu là: thay vì thử chia mọi số vô ích, ta trực tiếp tìm các giá trị có khả năng chia hết cho từng số nguyên tố. Với số dạng $x^2 - n$,

$$p \mid x^2 - n \iff x^2 \equiv n \pmod{p}.$$

Dùng thuật toán tính căn bậc hai modulo p , chẳng hạn Cipolla, có thể tìm $O(1)$ nghiệm x modulo p . Sau đó chỉ dùng p để sàng các vị trí hợp lệ modulo p . Theo phân tích của sàng Eratosthenes, quá trình này tốn

$$O(C \log \log C)$$

cho C giá trị.

Nếu áp dụng trực tiếp cho $x^2 - n$, ta đã có thuật toán mà phần phân tích nhân tử tốn $O(C \log \log C)$. Nhưng hiệu quả thực tế vẫn chưa tốt vì các số càng về sau càng lớn, khiến tỉ lệ smooth giảm. Vì vậy trong thực hành thường dùng nhiều đa thức bậc hai $f(x)$, mỗi đa thức chỉ xét một đoạn ngắn, để giữ $|f(x)|$ nhỏ.

Cụ thể, lấy

$$f(x) = (Ax + B)^2 - n,$$

và chọn số nguyên C sao cho

$$B^2 - n = AC.$$

Khi đó

$$f(x) = A(Ax^2 + 2Bx + C).$$

Nếu $A = p^2$ là số chính phương, thì

$$Ax^2 + 2Bx + C \equiv \left(\frac{Ax + B}{p}\right)^2 \pmod{n}.$$

Quy trình là chọn nhiều lần $A = p^2$, chọn B sao cho

$$B^2 \equiv n \pmod{A},$$

rồi đặt

$$C = \frac{B^2 - n}{A}.$$

Với mỗi x , số cần sàng là $Ax^2 + 2Bx + C$. Chọn A, B, C thích hợp làm giá trị tuyệt đối của biểu thức này nhỏ hơn; nếu nó âm, xem -1 là một nhân tử bổ sung. Tương tự CFRAC, nếu hai quan hệ sau khi phân tích còn cùng một phần dư lớn, có thể ghép chúng để thu thêm quan hệ cho khử Gauss.

17.5.2 Độ phức tạp và hiệu quả thực tế

Dùng kiểu phân tích tương tự CFRAC, phần phân tích nhân tử của sàng bậc hai có độ phức tạp

$$O\left(\exp((1+o(1))\sqrt{\log n \log \log n})\right),$$

với ngưỡng B ở cỡ

$$\exp\left(\frac{\sqrt{\log n \log \log n}}{2}\right).$$

Khi B lớn, có thể thay khử Gauss bằng các thuật toán dành cho ma trận thưa. Một số hiện thực QS bằng C có thể phân tích số khoảng 2^{200} trong vài giây.

17.6 Kết luận

Bài viết giới thiệu vài thuật toán phân tích thừa số nguyên tố còn ít được biết trong cộng đồng OI, đồng thời thể hiện một số hướng tiếp cận hiện đại cho bài toán phân tích: khai thác tính chất của liên phân số, phân tích smooth numbers rồi dùng khử tuyến tính để tìm đồng dư bình phương, và tổng quát hóa tư tưởng sàng trong số học. Tác giả hy vọng phần giới thiệu này gợi thêm ý tưởng cho các bài toán số học trong OI.

17.7 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn cha mẹ đã nuôi dưỡng; cảm ơn thầy Huang Xinjun đã quan tâm và hướng dẫn; cảm ơn Guo Yuhao đã đọc kiểm bản thảo.

17.8 Tài liệu tham khảo

1. Wikipedia. Prime number theorem.
2. Wikipedia. Barrett reduction.
3. Wikipedia. Miller–Rabin primality test.
4. OI Wiki. Continued fraction.
5. Wikipedia. Bezout’s identity.
6. Wikipedia. Shanks’s square forms factorization.
7. Wikipedia. Dickman function.
8. OI Wiki. Sieve.
9. michel-leonard. C factorization using quadratic sieve.
10. Zhong Ziqian. Quadratic Sieve.
11. Hans Riesel. *Prime Numbers and Computer Methods for Factorization*. Springer, 1994.
12. Carl Pomerance. A tale of two sieves. *Notices of the AMS*, 43(12):1473–1485, 1996.
13. Jason E. Gower and Samuel S. Wagstaff, Jr. Square form factorization. *Mathematics of Computation*, 77(261):551–588, 2008.
14. Michael A. Morrison and John Brillhart. A method of factoring and the factorization of F_7 . *Mathematics of Computation*, 29(129):183–205, 1975.

18 Một lớp cấu trúc dữ liệu xâu con cơ bản

Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc
Xu Tingqiang

Tóm tắt

Bài viết giới thiệu một cấu trúc dữ liệu xâu mới: cấu trúc xâu con cơ bản. Cấu trúc này mạnh ở chỗ có thể đồng thời xử lý thông tin tiền tố và hậu tố của xâu. Bài viết cũng trình bày một số ứng dụng của cấu trúc này trong thi tin học.

18.1 Mở đầu

Lý thuyết xâu đã phát triển lâu trong OI, từ đó xuất hiện nhiều cấu trúc hậu tố nâng cao như suffix array và suffix automaton. Các cấu trúc này phân tích sâu tính chất của xâu và giải hiệu quả nhiều bài toán. Tuy nhiên, phần lớn cấu trúc hậu tố chỉ có liên hệ một chiều: chúng xử lý tốt quan hệ hậu tố, nhưng khó đồng thời xử lý thông tin liên quan tới tiền tố.

Từ bài SDOI 2022 *Thống kê xâu con*, tác giả nghiên cứu một lớp cấu trúc có thể đồng thời xử lý tiền tố và hậu tố, rồi viết bài này. Vì cấu trúc này trước đó chưa xuất hiện, tác giả gọi nó là *cấu trúc xâu con cơ bản*. Bên cạnh đó, trên nền cấu trúc này, bài viết còn khảo sát một dạng phân rã chuỗi nặng nhẹ đặc biệt.

18.2 Quy ước và ký hiệu

Với xâu $s = s_1 s_2 \cdots s_n$, ký hiệu $s[l, r]$ là xâu con

$$s_l s_{l+1} \cdots s_r.$$

Kích thước bảng chữ cái mặc định là $O(1)$. Các kết quả liên quan tới suffix automaton và suffix tree có thể xem trong [1]; bài viết mặc định người đọc đã biết các kiến thức đó.

Số lần xuất hiện. Với xâu cố định s và một xâu con t , định nghĩa $\text{occ}(t)$ là số lần t xuất hiện trong s , tức số cặp (l, r) thỏa

$$s[l, r] = t.$$

Mỗi $s[l, r]$ như vậy được gọi là một vị trí xuất hiện của t .

Hai cây hậu tố. Đặt T^0 là cây parent của SAM trên xâu thuận, tức suffix tree của xâu đảo. Đặt T^1 là cây parent của SAM trên xâu đảo, tức suffix tree của xâu thuận. Khi nói một nút SAM, đôi khi ta cũng hiểu là nút tương ứng trên cây parent.

Với nút u của SAM xâu thuận, ký hiệu $s_0(u)$ là tập các xâu mà nút u biểu diễn. Với nút v của SAM xâu đảo, ký hiệu $s_1(v)$ tương tự.

Border. Một xâu t là border của s khi và chỉ khi t vừa là tiền tố vừa là hậu tố của s .

18.3 Cấu trúc xâu con cơ bản

18.3.1 Dẫn nhập

Trước hết xét bài toán sau.

Ví dụ mở đầu. Cho xâu s . Có q truy vấn, mỗi truy vấn cho một nút u trên cây parent T^0 của SAM xâu thuận và một nút v trên cây parent T^1 của SAM xâu đảo. Cần tìm giao

$$s_0(u) \cap s_1(v).$$

Giới hạn $n, q \leq 5 \cdot 10^5$.

Bài toán này buộc ta xét đồng thời hai cây parent của xâu thuận và xâu đảo, điều khá khó với suffix array hoặc SAM thông thường. Vì vậy cần một cách ghép hai SAM lại với nhau; đó chính là cấu trúc xâu con cơ bản.

18.3.2 Nội dung và tính chất

Xâu mở rộng. Với một xâu con t của s , định nghĩa $\text{ext}(t)$ là xâu con dài nhất t' của s sao cho t' chứa t và

$$\text{occ}(t) = \text{occ}(t').$$

Cần chứng minh định nghĩa này là tốt.

Bổ đề. $\text{ext}(t)$ tồn tại và duy nhất.

Chứng minh. Tồn tại là hiển nhiên vì t tự thỏa điều kiện. Để chứng minh duy nhất, giả sử có hai xâu t', t'' đều thỏa điều kiện. Vì chúng đều chứa t và có số lần xuất hiện bằng $\text{occ}(t)$, mỗi lần xuất hiện của t tương ứng duy nhất với một lần xuất hiện của t' , và cũng tương ứng duy nhất với một lần xuất hiện của t'' .

Xét một lần xuất hiện $s[l, r] = t$. Gọi các lần xuất hiện tương ứng của t', t'' là $s[l', r'], s[l'', r'']$. Khi đó

$$l', l'' \leq l \leq r \leq r', r''.$$

Đặt

$$L = \min(l', l''), \quad R = \max(r', r'').$$

Xâu $s[L, R]$ cũng xuất hiện kèm mỗi lần xuất hiện của t , nên

$$\text{occ}(s[L, R]) = \text{occ}(t).$$

Do $\text{ext}(t)$ dài nhất, buộc

$$R - L = r' - l' = r'' - l'',$$

tức $l' = l'', r' = r'',$ và $t' = t''$.

Từ chứng minh trên có hai hệ quả. Nếu $t' = \text{ext}(t)$, và một lần xuất hiện $s[l, r]$ của t tương ứng với $s[l', r']$ của t' , thì với mọi $l' \leq l'' \leq l, r \leq r'' \leq r'$, đều có

$$\text{ext}(s[l'', r'']) = t'.$$

Ngoài ra,

$$\text{ext}(\text{ext}(t)) = \text{ext}(t).$$

Lớp tương đương. Hai xâu con x, y được gọi là tương đương khi và chỉ khi

$$\text{ext}(x) = \text{ext}(y).$$

Nếu $\text{ext}(t) = t$, gọi t là đại diện của lớp tương đương chứa nó, ký hiệu $\text{rep}(a)$ với a là lớp đó. Mỗi lớp có đúng một đại diện, theo hệ quả vừa nêu.

Nếu hai xâu có cùng occ và một xâu chứa xâu còn lại, chúng thuộc cùng một lớp tương đương. Thật vậy, nếu t_2 chứa t_1 và $\text{occ}(t_1) = \text{occ}(t_2)$, thì $\text{ext}(t_2)$ cũng chứa t_1 và có cùng số lần xuất hiện, nên

$$\text{ext}(t_1) = \text{ext}(t_2).$$

Với một xâu con t , định nghĩa vị trí xuất hiện đầu tiên của nó là

$$s[\text{posl}(t), \text{posr}(t)].$$

Định lý hình học. Lập hệ tọa độ phẳng với hai trục l, r . Với cùng một lớp tương đương, tập các điểm

$$(\text{posl}, \text{posr})$$

của mọi xâu trong lớp tạo thành một lưới bậc thang có cạnh trên và cạnh trái thẳng hàng.

Chứng minh. Vị trí xuất hiện đầu tiên của t tương ứng với vị trí xuất hiện đầu tiên của $\text{ext}(t)$. Trong một lớp, xâu dài nhất là đại diện, nên điểm tương ứng là điểm trái-trên của tập. Theo hệ quả của bổ đề về ext , mọi điểm của lớp nằm phía phải-dưới điểm đại diện, và với hai điểm đã có, mọi điểm nguyên trong hình chữ nhật tạo bởi chúng cũng thuộc cùng lớp. Vì vậy tập điểm tạo thành một hình bậc thang.

Do đó mọi điểm nguyên $1 \leq l \leq r \leq |s|$ được chia thành các hình bậc thang không giao nhau. Mỗi lớp tương đương a ứng với $\text{occ}(\text{rep}(a))$ hình bậc thang giống nhau. Định nghĩa chu vi $\text{per}(a)$ của lớp a là tổng số hàng và số cột trong hình bậc thang của nó.

Vì tính trực quan của định lý này, ta cũng gọi một lớp tương đương là một *khối*. Sau đây là quan hệ giữa các khối và SAM.

Định lý. Với mỗi lớp tương đương, trong hình bậc thang tương ứng, mỗi hàng r cố định tương ứng với một nút của cây parent T^0 của SAM xâu thuận; mỗi cột l cố định tương ứng với một nút của cây parent T^1 của SAM xâu đảo. Hơn nữa, mọi hàng của mọi khối tương ứng một-một với các nút SAM xâu thuận, và mọi cột tương ứng một-một với các nút SAM xâu đảo. Tương ứng ở đây nghĩa là tập xâu được biểu diễn giống nhau.

Chứng minh. Chỉ cần xét hàng; cột đối xứng. Trong một hàng của một khối, r không đổi, l biến thiên liên tiếp và occ không đổi, nên các xâu này nằm trong cùng một nút SAM xâu thuận. Ngược lại, mọi xâu trong một nút SAM có quan hệ chứa nhau, cùng r và cùng occ , nên theo bổ đề trên chúng thuộc cùng một khối và cùng một hàng.

Suy ra tổng chu vi mọi khối là $O(n)$. Để hoàn tất cấu trúc, cần thêm các cạnh cây parent của hai SAM. Cạnh trong T^0 có thể xem như nối biên trên của một cột tới biên dưới của một cột trong khối khác; cạnh trong T^1 đối xứng, nối biên trái của một hàng tới biên phải của một hàng khác.

Từ góc nhìn khác, cấu trúc xâu con cơ bản là một đồ thị có hướng chứa mọi xâu con khác nhau của s , trong đó xâu con t nối tới các xâu $t + c$ và $c + t$, với c là một ký tự.

Bổ đề. Nếu $s[l, r_1]$ và $s[l, r_2]$ thuộc cùng một lớp tương đương, thì với mọi $1 \leq i < l$, hai xâu $s[i, r_1]$ và $s[i, r_2]$ cũng thuộc cùng một lớp.

Chứng minh. Giả sử $r_1 \leq r_2$. Do quan hệ chứa nhau,

$$\text{occ}(s[i, r_2]) \leq \text{occ}(s[i, r_1]).$$

Mặt khác, mỗi lần xuất hiện của $s[i, r_1]$ kéo theo một lần xuất hiện của $s[l, r_1]$, mà các lần xuất hiện của $s[l, r_1]$ và $s[l, r_2]$ tương ứng một-một. Do đó mỗi lần xuất hiện của $s[i, r_1]$ cũng mở rộng thành một lần xuất hiện của $s[i, r_2]$, nên hai số lần xuất hiện bằng nhau. Theo bổ đề về cùng occ và chứa nhau, chúng cùng lớp.

Định lý. Nếu hai nút SAM u, v nằm trong cùng một lớp tương đương, thì hai cây con của chúng trên cây parent, bỏ qua gốc cây con, có cấu trúc hoàn toàn giống nhau. Cụ thể, có thể song ánh các nút trong hai cây con sao cho hai cây con đẳng cấu khi xem như cây vô hướng, và các nút tương ứng chứa cùng số xâu, cùng occ. Hơn nữa, vị trí tương đối trong hình bậc thang của các nút tương ứng giống với vị trí tương đối của u, v .

Chứng minh phác thảo. Giả sử u, v nằm trên SAM xâu thuận. Gọi b_u, b_v là xâu dài nhất mà chúng chứa, và giả sử $\text{len}(b_u) \leq \text{len}(b_v)$. Vì u, v cùng khối và biên trái của hình bậc thang thẳng hàng,

$$\text{posl}(b_u) = \text{posl}(b_v).$$

Đặt $\delta = \text{len}(b_v) - \text{len}(b_u)$. Các lần xuất hiện của b_u là $s[l_i, r_i]$, còn của b_v là

$$s[l_i, r_i + \delta].$$

Theo bổ đề trước, với mọi $j < l_i$, các xâu $s[j, r_i]$ và $s[j, r_i + \delta]$ nằm trong cùng lớp và giữ nguyên vị trí tương đối. Các xâu trong cây con của u, v chính là các xâu dạng này. Từ đó thu được song ánh giữa các trie con; sau khi nén các nút bậc hai, được kết quả cho cây parent.

Một hệ quả là: các cạnh đi ra từ biên trên của một khối có thể chia thành một số nhóm; mỗi nhóm nối tới một đoạn liên tiếp và thẳng hàng trên biên dưới của một khối khác. Số nhóm chính là số con trong cây parent. Biên trái có kết luận đối xứng.

Nhờ hệ quả này, chuỗi trên automaton dễ nhận diện hơn. Với một nút của T^0 , nếu nút đó không tương ứng với biên trái của khối chứa nó, thì trên SAM nó có đúng một cạnh đi ra, tới cột ngay bên trái. Nếu nó nằm ở biên trái, mọi cạnh đi ra chính là các biên trái có thể đạt được bằng cạnh parent của SAM còn lại.

Đến đây, cấu trúc đã chứa đầy đủ thông tin của hai SAM. Về tổng thể, cấu trúc xâu con cơ bản gồm các lớp tương đương và các cạnh parent; lõi của nó là hình bậc thang của từng lớp. Cấu trúc automaton dạng DAG của SAM đã được thay bằng cây parent của SAM còn lại, vì DAG đó không có tính chất hình học thuận tiện.

18.3.3 Cách dựng

SAM có thể dựng trong $O(n)$; bài viết không chứng minh lại. Cấu trúc xâu con cơ bản cũng dựng được trong $O(n)$.

Trước hết dựng SAM của xâu thuận và xâu đảo. Với một lớp tương đương a , $\text{rep}(a)$ luôn là xâu dài nhất trong một nút SAM. Nhờ tính chất về cạnh đi ra, có thể nhận diện xâu dài nhất của một nút có phải đại diện hay không bằng cách kiểm tra cạnh automaton. Sau khi tìm mọi đại diện, cũng dựa trên cùng tính chất để xác định mỗi nút SAM thuộc lớp nào.

Để ghép các lớp tương đương từ hai SAM, chỉ cần tìm posl , posr của đại diện. Hình bậc thang trong mỗi khối được dựng bằng cách sắp các nút SAM theo r hoặc l ; thứ tự này tự nhiên trùng với thứ tự xuất hiện trong quá trình dựng SAM. Vì vậy tổng thời gian là $O(n)$. Cách cài đặt này ngắn gọn; hằng số có thể lớn, nhưng độ phức tạp tốt.

18.3.4 Ứng dụng

Quay lại ví dụ mở đầu. Sau khi dựng cấu trúc cây con cơ bản, nếu hai nút u, v không nằm trong cùng một khối, giao $s_0(u) \cap s_1(v)$ rỗng. Nếu cùng khối, trên hình bậc thang ta chỉ cần tìm giao của một đoạn ngang và một đoạn dọc, xử lý được trong $O(1)$. Tổng thời gian $O(n + q)$.

Ví dụ: Đánh trúng lặp lại. Cho cây s dài n . Ký tự thứ i có hai trọng số: trọng số trái wl_i và trọng số phải wr_i . Với một cây con $s[l, r]$, trọng số trái $vl(s[l, r])$ là tổng các wl tại đầu trái của mọi lần xuất hiện của nó; trọng số phải $vr(s[l, r])$ định nghĩa tương tự theo đầu phải. Độ lặp của cây con là

$$w(s[l, r]) = vl(s[l, r]) \cdot vr(s[l, r]).$$

Độ lặp của cả cây là tổng trên mọi cây con. Có q lần sửa một wl_i , sau mỗi lần hỏi tổng modulo 2^{64} .

Đáp án là một hàm tuyến tính cố định theo các wl_i , nên cần tính hệ số của từng wl_i . Sau khi dựng SAM, $vr(s[l, r])$ là tổng wr_i trên các tiền tố $s[1, i]$ trong cây con tương ứng; vl đối xứng. Dựng cấu trúc cây con cơ bản và xét từng khối: trong hình bậc thang, mỗi hàng có một giá trị vl , mỗi cột có một giá trị vr , cần tính tổng tích hàng-cột trên mọi ô của hình. Dùng tiền tố theo cột để tìm tổng vr ứng với mỗi vl , rồi quy đáp án thành tổ hợp tuyến tính của các wl_i qua một lần chuyển trên SAM cây đảo. Khi có hệ số, cập nhật điểm xử lý dễ dàng. Tổng thời gian $O(n + q)$.

Ví dụ: bài toán mẫu về đếm mẫu trong đoạn. Cho cây s dài n , m cây con của s làm mẫu. Mỗi truy vấn cho ql_i, qr_i , hỏi tổng số lần xuất hiện của mọi mẫu trong $s[ql_i, qr_i]$.

Số lần t_2 xuất hiện trong t_1 chính là số hậu tố của t_1 có tiền tố là t_2 . Dựng cấu trúc cây con cơ bản. Trước hết đánh dấu các mẫu bằng 1, rồi lấy tổng tiền tố dọc theo cây parent của SAM cây đảo, tương ứng hướng ngang trên hình bậc thang; tiếp tục lấy tổng tiền tố dọc theo cây parent SAM cây thuận, tương ứng hướng dọc. Cuối cùng truy vấn các điểm cần hỏi.

Số điểm cây con có thể tới $O(n^2)$, nên cần tối ưu. Trong mỗi hàng của hình bậc thang, loại tạm đóng góp của mẫu nằm ngay trên hàng đó, chỉ xét đóng góp vào cây con trừ gốc. Sau lần cộng tiền tố đầu, mọi điểm trong một hàng có cùng trọng số. Sau lần cộng tiền tố thứ hai, giá trị tại một điểm là tổng trọng số của các hàng trước nó, có thể xử lý bằng tiền tố trong khối và chuyển khối. Phần đã loại tạm có dạng nhiều đóng góp “hậu tố của một hàng cộng 1”, trong khối trở thành cộng 1 trên hình chữ nhật; xử lý offline bằng đếm điểm hai chiều. Tổng độ phức tạp

$$O(n + (m + q) \log n).$$

Ví dụ: Thống kê cây con. Cho cây s dài n . Đặt $T_0 = s$. Mỗi bước xóa ký tự đầu hoặc cuối của T_i để thu được T_{i+1} , sau $n - 1$ bước còn một ký tự. Có 2^{n-1} chuỗi thao tác. Trọng số của một chuỗi là

$$\prod_{i=1}^{n-1} \text{occ}(T_i).$$

Cần tính tổng trọng số mọi chuỗi thao tác modulo 998244353.

DP trực tiếp trên mặt phẳng (l, r) tốn $O(n^2)$. Đổi thứ tự DP: bắt đầu từ cây rỗng rồi liên tục thêm ký tự vào hai đầu cho tới s . Các vị trí khác nhau của cùng một cây con có cùng giá trị DP. Vì khi thêm ký tự, occ không tăng, ta duyệt các lớp tương đương theo occ giảm dần.

Trong một lớp, occ không đổi và chuyển DP diễn ra trên hình bậc thang. Để chuyển sang lớp tiếp theo, cần tính giá trị DP trên biên trên và biên trái của hình. Bài toán trở thành: trên một hình bậc thang,

cho các điểm khởi đầu trên đường bậc thang phải-dưới, cần tính tổng số đường đi tới từng điểm trên biên trên và biên trái. Hàm sinh có dạng gần như

$$\sum \frac{(\text{occ} \cdot x)^j}{(1 - \text{occ} \cdot x)^i},$$

với i, j đơn điệu và tổng kích thước không vượt per. Có thể dùng chia để trị FFT. Với lớp a , chi phí

$$O(\text{per}(a) \log^2 n),$$

nên tổng thời gian $O(n \log^2 n)$.

Ví dụ: truy vấn độ lặp theo tiền tố-hậu tố. Cho xâu s , mỗi ký tự có trọng số trái và phải như ví dụ đầu. Mỗi truy vấn cho l_1, r_1, l_2, r_2 . Xét tập các cặp (l, r) sao cho $s[l_1, r_1]$ là tiền tố của $s[l, r]$, $s[l_2, r_2]$ là hậu tố của $s[l, r]$, và độ dài $s[l, r]$ đủ lớn. Cần tính tổng $w(s[l, r])$ trên tập này modulo 2^{64} .

Các giá trị vl, vr được tính bằng DFS trên hai SAM. Trực giác là: các xâu có tiền tố $s[l_1, r_1]$ nằm trong một cây con của SAM xâu đảo, còn các xâu có hậu tố $s[l_2, r_2]$ nằm trong một cây con của SAM xâu thuận. Truy vấn là giao của hai cây con, trừ phần nằm trong cùng nút SAM nhưng độ dài chưa đủ. Phần cần trừ xử lý bằng đếm điểm hai chiều trong hình bậc thang. Phần giao hai cây con được biến đổi bằng thứ tự DFS và hiệu tiền tố, rồi dùng Mo hai lần offline nhờ tính chất tốt của các hình bậc thang. Độ phức tạp tổng là

$$O(n\sqrt{q} + q \log n),$$

không gian $O(n + q)$.

18.4 Phân rã chuỗi nặng nhẹ trên cấu trúc xâu con cơ bản

18.4.1 Nội dung

Phần này xét truy vấn trên một chuỗi của SAM. Mô tả cho cây parent T^0 ; phía còn lại đối xứng.

Với một nút không phải lá u , định nghĩa con nặng $\text{son}(u)$ là con có siz lớn nhất; nếu có nhiều con bằng nhau, chọn con có posr lớn nhất. Cạnh từ u tới $\text{son}(u)$ là cạnh nặng, các cạnh còn lại là cạnh nhẹ.

Định lý. Trong một nhóm cạnh như hệ quả về biên của khối đã nêu ở trên, các cạnh hoặc đồng thời là cạnh nặng, hoặc đồng thời là cạnh nhẹ.

Chứng minh. Theo định lý đẳng cấu cây con, các cây con tương ứng hoàn toàn giống nhau và vị trí tương đối của posr cũng giống nhau, nên lựa chọn con nặng của mọi nút tương ứng là giống nhau.

Hệ quả. Với mỗi lớp tương đương có occ vị trí xuất hiện khác nhau, có thể chọn một vị trí xuất hiện cho từng lớp sao cho với mọi nút không phải lá u , đường ngang của u trong hình bậc thang có biên trái trùng với biên phải của đường ngang ứng với $\text{son}(u)$.

Chọn đệ quy từ lá lên. Lá chỉ có một cách chọn vì $\text{occ} = 1$. Với nút trong, dựa vào vị trí đã chọn của con nặng, chọn vị trí nằm đúng bên phải nó.

Như vậy, sau khi gán lại tọa độ cho các điểm trong mỗi lớp, truy vấn một chuỗi trên T^0 được chuyển thành truy vấn $O(\log n)$ đường thẳng có cùng tọa độ r . Truy vấn chuỗi trên T^1 đối xứng thành $O(\log n)$ đường có cùng tọa độ l . Lưu ý hai cây dùng hai hệ tọa độ do phân rã nặng nhẹ khác nhau.

Nếu đánh số các chuỗi nặng theo posr của lá mà chúng chứa, thì trong mỗi lớp tương đương, các nút SAM nằm trên các chuỗi nặng có số hiệu tạo thành một đoạn liên tiếp. Điều này hiển nhiên vì posr chính là tọa độ r của các đường ngang.

18.4.2 Ứng dụng

Ví dụ: từ điển border cơ bản. Cho xâu s dài n . Mỗi truy vấn cho một xâu con của s , cần hỏi border ngắn nhất, border dài nhất và số border. n, q cùng bậc.

Với xâu truy vấn $s[l, r]$, các border là giao giữa tập hậu tố của $s[l, r]$ và tập tiền tố của $s[l, r]$. Tập hậu tố là một chuỗi từ nút tương ứng lên gốc trên T^0 , còn tập tiền tố là một chuỗi lên gốc trên T^1 . Vì cần giao kèm điều kiện độ dài, chỉ có $O(\log n)$ cặp chuỗi nặng cần truy vấn.

Mỗi truy vấn con có dạng: có bao nhiêu xâu độ dài không vượt L vừa nằm trên chuỗi nặng số x của T^0 , vừa nằm trên chuỗi nặng số y của T^1 . Với một nút của T^0 , theo định lý về đoạn liên tiếp, mọi xâu của nó nằm trên một đoạn liên tiếp các chuỗi nặng của T^1 ; trong mặt phẳng (số hiệu chuỗi nặng T^1 , độ dài), chúng là một đoạn xiên hệ số góc 1. Bài toán trở thành đếm điểm hai chiều với $O(q \log n)$ truy vấn và $O(n)$ cập nhật.

Cách này cho một phương pháp khác để lấy dãy border, nhưng tốn thêm một $\log n$. Đổi lại, nó định vị được các border trên SAM. Nếu thay truy vấn thành tổng occ của mọi border trong đoạn, cách suffix array cổ điển không còn trực tiếp dùng được, còn cấu trúc này vẫn thích hợp.

Ví dụ mạnh hơn. Cho xâu s , các dãy $f_1, \dots, f_n$ và $g_1, \dots, g_n$. Mỗi truy vấn cho một xâu con, cần tính tổng

$$f_{\text{occ}} \cdot g_{\text{len}}$$

trên mọi border của nó.

Như trên, trước hết quy về $O(q \log n)$ truy vấn giao của hai chuỗi nặng và điều kiện độ dài. Lúc này cần giữ đồng thời occ và len. Thay vì thống kê theo từng nút SAM, thống kê theo các đường chéo trong mỗi hình bậc thang. Trong một lớp a , mọi điểm trên cùng một đường $r - l$ có cùng occ và cùng độ dài. Số đường như vậy trong lớp không vượt $O(\text{per}(a))$, nên tổng là $O(n)$. Do đó bài toán vẫn quy về đếm điểm hai chiều với $O(q \log n)$ truy vấn và $O(n)$ cập nhật.

18.5 Kết luận

Bài viết giới thiệu cấu trúc xâu con cơ bản. So với suffix array và SAM thông thường, cấu trúc này mạnh hơn ở khả năng xử lý đồng thời các bài toán tiền tố và hậu tố. Có thể xem nó là phần mở rộng của SAM nén đối xứng: một điểm của SAM nén đối xứng được mở rộng thành một hình bậc thang chi tiết, nhờ đó chứa nhiều thông tin hơn và dễ thao tác hơn. Cấu trúc này dễ hiện thực, có độ phức tạp tốt và có triển vọng ứng dụng rộng trong thi tin học.

Dù vậy, lý thuyết xâu vẫn còn nhiều vấn đề mở. Tác giả hy vọng bài viết gợi hứng thú để người đọc tiếp tục nghiên cứu và hoàn thiện các lý thuyết liên quan.

18.6 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn thầy Ye Jinyi của Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc đã quan tâm và hướng dẫn; cảm ơn gia đình, bạn bè đã ủng hộ và khích lệ; cảm ơn Dai Jiangqi đã thảo luận và gợi ý; cảm ơn Du Yuhao, Chang Ruinian, Ye Zichuan và Wu Hang đã đọc bản thảo.

18.7 Tài liệu tham khảo

1. OI Wiki contributors. Suffix automaton, OI Wiki, 2022.

2. Trao đổi riêng với Dai Jiangqi, 2022.
3. Xu Yixuan. Bàn về suffix automaton nén, tập luận văn đội tuyển quốc gia Trung Quốc IOI 2020.

19 Xem xét lại một vài bài toán số học cổ điển

Trường Yali, Trường Sa
Zheng Xuanye

Tóm tắt

Bài viết thảo luận ba bài toán số học lâu đời: kiểm tra tính nguyên tố, phân tích số nguyên và logarit rời rạc. Nội dung điểm lại một số thuật toán đã phổ biến trong OI, nêu các tính chất đặc biệt của chúng, đồng thời giới thiệu vài thuật toán tốt nhưng chưa được dùng rộng rãi trong OI.

19.1 Tổng quan

19.1.1 Lời nói đầu

Kiểm tra tính nguyên tố, phân tích số nguyên và logarit rời rạc đều là các bài toán số học rất kinh điển. Tuy vậy, trong môi trường OI hiện nay, nhiều thí sinh vẫn biết khá ít về các thuật toán liên quan; ngay cả một số thí sinh trình độ cao cũng gặp tình trạng này. Khi tra cứu tài liệu, tác giả còn thường gặp những bài viết mâu thuẫn với nhau hoặc có sai sót thực tế. Vì mong muốn thúc đẩy sự phát triển của OI, bài viết này giới thiệu một số thuật toán cho ba bài toán trên và đưa ra vài gợi ý sử dụng.

19.1.2 Định nghĩa cơ bản

Một ước không tầm thường của số nguyên n là một số d sao cho

$$d \mid n, \quad 1 < d < n.$$

Số nguyên tố là số nguyên dương không có ước không tầm thường; hợp số là số nguyên dương có ước không tầm thường. Quy ước riêng 1 không phải số nguyên tố cũng không phải hợp số.

Smooth-number là số có các thừa số nguyên tố đều nhỏ. Nếu mọi thừa số nguyên tố của một số không vượt quá B , ta gọi nó là B -smooth.

Ký hiệu $\tilde{O}(f(n))$ nghĩa là tồn tại hằng số k để độ phức tạp là

$$O(f(n) \log^k f(n)).$$

Ký hiệu tiệm cận

$$L_n[\alpha, c] = \exp((c + o(1))(\ln n)^\alpha (\ln \ln n)^{1-\alpha}),$$

với $0 \leq \alpha \leq 1$, $c > 0$, thường dùng trong phân tích các thuật toán số học. Ta có $L_n[0, c] = (\ln n)^{c+o(1)}$, tức đa thức theo $\ln n$, còn $L_n[1, c] = n^{c+o(1)}$, tức mũ theo $\ln n$. Vì vậy khi $\alpha \in (0, 1)$, đây là mức dưới mũ theo độ dài đầu vào.

19.2 Bài toán kiểm tra tính nguyên tố

19.2.1 Mô tả bài toán

Bài toán kiểm tra tính nguyên tố: nhập một số nguyên dương p , xác định p có phải số nguyên tố hay không.

19.2.2 Thuật toán Miller–Rabin

Thuật toán Miller–Rabin xét xem một số có thỏa các tính chất cần có của số nguyên tố hay không. Trước hết nhìn vào phần đảo của định lý nhỏ Fermat: với số cần kiểm tra p và một số nguyên a thỏa $1 \leq a < p$, nếu

$$a^{p-1} \not\equiv 1 \pmod{p},$$

thì p không phải số nguyên tố. Bước này gọi là phép thử Fermat.

Điểm yếu của phép thử Fermat là chiều ngược lại không đúng: nếu $a^{p-1} \equiv 1 \pmod{p}$, ta chưa thể kết luận p nguyên tố. Tuy nhiên, nếu p thật sự là số nguyên tố, thì khi đó phải có

$$a^{(p-1)/2} \equiv 1 \pmod{p} \quad \text{hoặc} \quad a^{(p-1)/2} \equiv p-1 \pmod{p}.$$

Vì thế ta có thể tiếp tục kiểm tra $a^{(p-1)/2} \pmod{p}$. Nếu giá trị này không phải 1 hoặc -1 , có thể kết luận p là hợp số. Nếu $(p-1)/2$ còn chẵn và giá trị vừa xét là 1, ta lại thử tiếp $a^{(p-1)/4} \pmod{p}$. Tổng quát hóa quá trình này thu được một phép thử Miller–Rabin với cơ số a .

Một lần thử có thể mô tả như sau.

1. Nếu $p = 2$, trả lời p nguyên tố; nếu p là số chẵn khác 2 hoặc $p = 1$, trả lời p không nguyên tố.
2. Đặt $k = p - 1$.
3. Nếu $a^k \not\equiv 1 \pmod{p}$, trả lời p không nguyên tố.
4. Tính $r = a^{k/2} \pmod{p}$.
5. Nếu $r \not\equiv 1 \pmod{p}$ và $r \not\equiv -1 \pmod{p}$, trả lời p không nguyên tố.
6. Nếu $r \equiv 1 \pmod{p}$ và $k/2$ chẵn, đặt $k \leftarrow k/2$ rồi quay lại bước tính r .
7. Nếu không phát hiện mâu thuẫn, phép thử trả lời p có thể là số nguyên tố.

Khi hiện thực, cần tính các lũy thừa

$$a^{p-1}, a^{(p-1)/2}, a^{(p-1)/4}, \dots$$

Thay vì gọi lũy thừa nhanh nhiều lần, ta phân tích

$$p-1 = 2^\ell t$$

với t lẻ, tính a^t một lần, rồi bình phương dần để có $a^{2t}, a^{4t}, a^{8t}, \dots$. Nhờ đó một phép thử tốn $O(\log p)$ phép nhân và lấy modulo trên số nguyên.

19.2.3 Tính chất của thuật toán

Miller–Rabin là thuật toán xác suất một phía. Khi thuật toán trả lời p không nguyên tố thì kết luận chắc chắn đúng; khi nó trả lời p có thể nguyên tố thì vẫn có khả năng p là hợp số.

Cách cải tiến đơn giản và hiệu quả là chọn nhiều cơ số, chạy Miller–Rabin cho từng cơ số, và chỉ khi tất cả đều trả lời có thể nguyên tố mới xem p là nguyên tố. Hai kết quả sau làm cho cách làm này đáng tin cậy.

Định lý 1. Giả sử giả thuyết Riemann mở rộng (ERH) đúng. Khi đó thuật toán kiểm tra Miller–Rabin với mọi cơ số nguyên $a \in [1, 2 \ln^2 p]$ là đúng.

Định lý 2. Nếu p là hợp số, thì trong khoảng $[1, p)$ có ít nhất

$$\frac{3(p-1)}{4}$$

số có thể làm chứng cứ Miller–Rabin rằng p là hợp số, tức khi dùng chúng làm cơ sở, phép thử sẽ trả lời không nguyên tố.

Chứng minh hai định lý này khá phức tạp nên bài viết lược bỏ. ERH vẫn là giả thuyết chưa được chứng minh, nhưng đa số nhà toán học tin nó có khả năng đúng. Dưới giả định đó, định lý 1 biến Miller–Rabin thành thuật toán kiểm tra tính nguyên tố xác định trong thời gian đa thức.

Trong thực tế, phiên bản phổ biến hơn là chọn ngẫu nhiên vài cơ sở. Theo định lý 2, nếu chọn ngẫu nhiên k cơ sở độc lập, xác suất đúng ít nhất là

$$1 - 4^{-k}.$$

Trong OI thường không cần xử lý số cực lớn. Khi đó có thể dùng các bộ cơ sở cố định để tăng tốc. Ví dụ, với các cơ sở 2, 3, 7, 61, 24251, trong phạm vi 10^{15} chỉ có số 46856248255981 bị kiểm tra sai. Có thể tìm thêm các bộ cơ sở có tính chất khác trên mạng.

19.2.4 Các thuật toán khác

Solovay–Strassen ra đời sớm hơn Miller–Rabin và có ý nghĩa lịch sử nhất định. Nó cũng là thuật toán xác suất một phía, mỗi lần thử có xác suất đúng ít nhất 50%. Khác biệt là thuật toán phải tính ký hiệu Jacobi, nên hiệu quả kém hơn Miller–Rabin đã tối ưu và hiện gần như bị thay thế.

AKS là thuật toán đầu tiên kiểm tra tính nguyên tố trong thời gian đa thức, xác định và không dựa trên giả thuyết chưa chứng minh. Ở đây “đa thức” nghĩa là đa thức theo độ dài đầu vào, tức theo $\ln p$. AKS có ý nghĩa lý thuyết rất lớn, nhưng các bước phức tạp và hiệu suất thực tế thấp nên hiếm khi được dùng.

19.3 Bài toán phân tích số nguyên

19.3.1 Mô tả bài toán

Bài toán phân tích số nguyên: nhập một hợp số n , tìm một ước không tầm thường của nó.

Các thuật toán phân tích số có thể chia thành hai loại: thuật toán cho trường hợp đặc biệt và thuật toán tổng quát. Bài viết chọn mỗi loại một đại diện thực dụng để giới thiệu.

19.3.2 Thuật toán Pollard $p-1$

Pollard $p-1$ xuất phát từ ý tưởng về smooth-number. Nếu n có một thừa số nguyên tố không lớn $p \leq B$, ta muốn xây dựng một số chia hết cho p , rồi lấy gcd với n để tách được nhân tử.

Định lý nhỏ Fermat lại hữu ích: với $a \not\equiv 0 \pmod{p}$, số

$$a^{p-1} - 1$$

chia hết cho p . Vấn đề là ta không biết p . Nếu chọn một số M sao cho $(p-1) \mid M$, thì $a^M - 1$ cũng dùng được. Vì $p-1 < B$, có thể lấy

$$M = \prod_{\text{prime } q < B} q^{\lfloor \log_q B \rfloor},$$

khi đó chắc chắn có $(p-1) \mid M$.

Quy trình Pollard $p - 1$ rất đơn giản: chọn a, B , tính

$$\gcd(a^M - 1, n).$$

Thực tế thường lấy $a = 2$. Điểm cần cân nhắc là chọn B : quá lớn thì chậm, quá nhỏ thì điều kiện $p \leq B$ dễ không thỏa. Có thể tăng dần B để nâng xác suất thành công.

Thuật toán này không phải lúc nào cũng phân tích được. Trong vài trường hợp, chẳng hạn n là tích của hai số nguyên tố lớn, nó gần như vô dụng. Đây là nhược điểm của thuật toán thiết kế cho trường hợp đặc biệt. Bù lại, nếu điều kiện thuận lợi, nó rất nhanh và có thể kết hợp với thuật toán tổng quát.

19.3.3 Thuật toán Pollard Rho

Pollard Rho đã khá phổ biến trong OI, nên bài viết chỉ nhắc lại quy trình và một số tính chất quan trọng. Với phiên bản dùng nhân gộp theo lô, thuật toán có thể mô tả như sau.

1. Chọn hằng số c cho hàm giả ngẫu nhiên

$$f(x) = (x^2 + c) \bmod n,$$

đồng thời chọn một hằng số dương B .

2. Khởi tạo s, t bằng cùng một số ngẫu nhiên trong $[0, n)$. Đặt độ dài nhân đôi $\text{len} = 2$, biến vòng lặp $i = 0$, và tích gộp $q = 1$.

3. Cập nhật $t \leftarrow f(t)$, rồi

$$q \leftarrow q \cdot |t - s| \bmod n.$$

4. Nếu $i \equiv 0 \pmod{B}$, tính $\gcd(q, n)$. Nếu kết quả khác 1, trả về kết quả đó.

5. Tăng i . Nếu $i = \text{len}$, đặt $\text{len} \leftarrow 2 \text{len}$, $s \leftarrow t$, $i \leftarrow 0$.

6. Lặp lại quá trình.

Pollard Rho có thể trả về n ; khi đó chưa tìm được ước không tầm thường và cần chạy lại. Nếu xử lý đúng, trường hợp này thường không ảnh hưởng lớn tới hiệu suất.

Chuỗi giả ngẫu nhiên trong thuật toán có một hậu tố dạng vòng, nên hình dạng giống chữ ρ . Hàm giả ngẫu nhiên không được chọn tùy tiện. Nếu

$$t - s \equiv 0 \pmod{p}$$

với một thừa số nguyên tố không tầm thường $p \mid n$, thì cũng có

$$f(t) - f(s) \equiv 0 \pmod{p}.$$

Vì vậy trên vòng của hình ρ , nếu có hai số cách nhau d bước giúp tìm được nhân tử p hoặc bội của p , thì mọi cặp cách nhau d bước cũng có khả năng tương tự. Tác giả nhấn mạnh điểm này vì nhiều người dùng Pollard Rho không thật sự nắm được nguyên lý này.

Thời gian chạy của Pollard Rho không ổn định và dao động khá lớn. Nếu hàm $f(x) = (x^2 + c) \bmod n$ đủ đều và tham số $B = \Theta(\log n)$, kỳ vọng thời gian là $O(p^{1/2})$, trong đó p là thừa số nguyên tố nhỏ nhất của n ; vì thế không vượt quá $O(n^{1/4})$. Việc hàm giả ngẫu nhiên trên có đủ đều hay không vẫn là một vấn đề mở, nhưng trong đa số trường hợp Pollard Rho hoạt động tốt và trở thành thuật toán tổng quát được dùng rộng rãi.

19.3.4 Các thuật toán khác

General number field sieve (GNFS) là thuật toán truyền thống có độ phức tạp lý thuyết tốt nhất hiện nay cho phân tích số nguyên, với độ phức tạp

$$L_n\left[\frac{1}{3}, \sqrt[3]{\frac{64}{9}}\right].$$

Do hiện thực quá phức tạp, gần như không thể dùng trong OI.

Shor là thuật toán lượng tử có độ phức tạp đa thức, có thể tối ưu đến $\tilde{O}(\log^2 n)$. Tuy nhiên máy tính lượng tử chưa đủ phát triển; theo bài viết, số lớn nhất từng được phân tích bằng thuật toán này mới là 21.

19.4 Logarit rời rạc

19.4.1 Mô tả bài toán

Trong một nhóm tổng quát, bài toán logarit rời rạc là: cho nhóm G và một phần tử sinh g , nhập một phần tử $b \in G$, tìm số nguyên x sao cho

$$g^x = b,$$

cũng viết $x = \log_g b$. Ký hiệu kích thước nhóm là $|G| = n$.

Một trường hợp đặc biệt quan trọng là logarit rời rạc trong nhóm nhân $\mathbb{Z}_p^\times$, với p nguyên tố. Khi đó g là một căn nguyên thủy modulo p . Vì hầu hết bài toán logarit rời rạc trong OI thuộc dạng này và nhóm nhân số nguyên cũng quen thuộc hơn, các phần sau chủ yếu nhìn từ $\mathbb{Z}_p^\times$. Nếu không nói riêng, thuật toán vẫn có thể làm việc trên nhóm tổng quát.

19.4.2 Thuật toán BSGS

BSGS là thuật toán logarit rời rạc phổ biến nhất trong OI. Quy trình:

1. Chọn ngưỡng B .
2. Đưa $g^0, g^1, g^2, \dots, g^{B-1}$ vào bảng băm.
3. Duyệt $i = 0, 1, 2, \dots$, tra trong bảng băm giá trị $b \cdot g^{-iB}$. Nếu tìm được g^c , trả lời $x = iB + c$.

Khi $B = \Theta(\sqrt{n})$, thuật toán đạt $O(\sqrt{n})$ thời gian.

Nhờ tính tổng quát, quy trình đơn giản và hiệu suất ổn định, BSGS rất được ưa chuộng. Nhược điểm là độ phức tạp $\sqrt{n}$ vẫn chưa đủ tốt khi phạm vi lớn.

19.4.3 Pollard Rho cho logarit rời rạc

Cần phân biệt thuật toán này với Pollard Rho dùng cho phân tích số nguyên. Chúng cùng tên nhưng giải quyết hai bài toán khác nhau.

Nếu tìm được bốn số $\alpha_1, \beta_1, \alpha_2, \beta_2$ sao cho

$$g^{\alpha_1} b^{\beta_1} = g^{\alpha_2} b^{\beta_2}$$

và $\beta_1 \not\equiv \beta_2 \pmod{n}$, do $b = g^x$, ta có

$$\alpha_1 - \alpha_2 \equiv x(\beta_2 - \beta_1) \pmod{n},$$

từ đó giải x bằng thuật toán Euclid mở rộng nếu điều kiện cho phép.

Tinh thần của Pollard Rho logarit rời rạc là tạo va chạm bằng một dãy giả ngẫu nhiên. Ta sinh các bộ ba (v_i, α_i, β_i) thỏa

$$v_i = g^{\alpha_i} b^{\beta_i}.$$

Nếu dãy đủ ngẫu nhiên, kỳ vọng sau $O(\sqrt{n})$ phần tử sẽ có hai giá trị $v_i = v_j$, khi đó kiểm tra phương trình trên để giải đáp án.

Một quy tắc sinh thường dùng:

1. Chia nhóm G thành ba phần xấp xỉ bằng nhau S_1, S_2, S_3 .
2. Nếu $v \in S_1$, chuyển sang $(vg, \alpha + 1, \beta)$.
3. Nếu $v \in S_2$, chuyển sang $(vb, \alpha, \beta + 1)$.
4. Nếu $v \in S_3$, chuyển sang $(v^2, 2\alpha, 2\beta)$.

Khi chia tập cần bảo đảm $1 \notin S_3$, nếu không thuật toán có thể rơi vào vòng lặp chết. Tác giả không tìm thấy tài liệu đánh giá chặt mức ngẫu nhiên của hàm này, nhưng thực tế nó thường tạo va chạm nhanh. Hiện thực không khó và hiệu suất có thể sánh với BSGS.

Dù vậy, thuật toán có một vấn đề nghiêm trọng: khi giải

$$\alpha_1 - \alpha_2 \equiv x(\beta_2 - \beta_1) \pmod{n},$$

có thể xảy ra $\gcd(\beta_2 - \beta_1, n) \neq 1$, khiến không có nghiệm duy nhất. Nếu n là số nguyên tố, tình huống này hiếm và thường chỉ cần đổi seed chạy lại. Nhưng trong OI, ta hay gặp $n = \varphi(p) = p - 1$, chắc chắn không nguyên tố; khi đó chạy lại nhiều lần cũng không chắc giải quyết được.

Tác giả từng thử với các modulo quen thuộc 998244353 và $10^9 + 7$, nhận thấy Pollard Rho đôi khi chạy lại hàng nghìn lần vẫn không tìm được nghiệm; nếu may mắn kết thúc thì thời gian cũng có thể lớn hơn BSGS nhiều bậc. Vì vậy không khuyến nghị dùng Pollard Rho trong tình huống này.

19.4.4 Thuật toán Pohlig–Hellman

Pohlig–Hellman khai thác trường hợp kích thước nhóm là smooth-number. Trước hết xét $n = q^k$, với q là số nguyên tố nhỏ, và tìm đáp án x theo các chữ số cơ số q từ thấp lên cao.

Quy trình:

1. Khởi tạo $x_0 = 0$, tính $t = g^{q^{k-1}}$.
2. Với $i = 0, 1, \dots, k - 1$:

(a) Tính

$$v = (b \cdot g^{-x_i})^{q^{k-i-1}}.$$

(b) Tìm d_i sao cho $t^{d_i} = v$, $0 \leq d_i < q$. Bước này có thể làm bằng duyệt trực tiếp nếu q nhỏ.

(c) Đặt $x_{i+1} = x_i + q^i d_i$.

3. Trả về x_k .

Ở vòng thứ i , x_i chính là phần gồm i chữ số thấp của x trong cơ số q . Thuật toán loại bỏ phần đã biết, triệt tiêu các chữ số cao, rồi tìm riêng chữ số kế tiếp.

Với n không phải lũy thừa nguyên tố, phân tích

$$n = \prod_i p_i^{e_i}.$$

Với từng $p_i^{e_i}$, giải $x \bmod p_i^{e_i}$ bằng cách đặt

$$g' = g^{n/p_i^{e_i}}, \quad b' = b^{n/p_i^{e_i}},$$

rồi chạy quy trình trên. Cuối cùng dùng định lý phần dư Trung Quốc để ghép nghiệm.

Thực chất bước tìm d_i là một bài logarit rời rạc trong nhóm bậc q . Nếu thay duyệt bằng BSGS hoặc thuật toán khác, có thể tối ưu thêm. Khi

$$n = \prod_i p_i^{e_i},$$

phiên bản dùng BSGS có độ phức tạp xấp xỉ

$$O\left(\sum_i e_i (\sqrt{p_i} + \log n)\right).$$

Do đó với n smooth hoặc có nhiều thừa số nhỏ, thuật toán rất lợi. Ví dụ trong $\mathbb{Z}_{998244353}^\times$,

$$998244352 = 2^{23} \cdot 7 \cdot 17.$$

Ngược lại, nếu n là số nguyên tố hoặc có một thừa số nguyên tố rất lớn, Pohlig–Hellman gần như không giúp ích.

19.4.5 Index calculus

Trong phần này xét logarit rời rạc trên $\mathbb{Z}_p^\times$.

Ý tưởng. Index calculus cố gắng quy toàn bộ vấn đề về smooth-number. Với một ngưỡng B , thuật toán chọn ngẫu nhiên l cho tới khi $b \cdot g^l$ là B -smooth. Khi đó bài toán $\log_g b$ được đưa về bài toán biết logarit của các số nguyên tố không vượt B .

Để tìm các logarit nhỏ này, thuật toán lại chọn ngẫu nhiên l sao cho g^l là B -smooth. Phân tích g^l thành thừa số nguyên tố sẽ tạo một phương trình tuyến tính theo các ẩn $\log_g p_i$. Gom đủ phương trình, ta dùng khử Gauss để giải toàn bộ $\log_g p_i$.

Quy trình.

1. Chọn ngưỡng B .
2. Chọn ngẫu nhiên l cho tới khi g^l là B -smooth. Nếu

$$g^l = \prod_i p_i^{e_i},$$

thu được phương trình

$$\sum_i e_i \log_g p_i = l.$$

3. Lặp lại cho đến khi đủ phương trình để giải mọi $\log_g p_i$ với $p_i \leq B$.
4. Để tính $\log_g b$, chọn ngẫu nhiên l cho tới khi $b \cdot g^l$ là B -smooth. Nếu

$$b \cdot g^l = \prod_i q_i^{c_i},$$

thì

$$\log_g b = \sum_i c_i \log_g q_i - l.$$

Độ phức tạp. Gọi $k = O(B/\ln B)$ là số lượng số nguyên tố không vượt B . Điểm then chốt là số lần cần thử ngẫu nhiên để gặp một B -smooth. Do l được chọn đều, g^l và $b \cdot g^l$ cũng phân bố đều trên $[1, p-1]$. Theo phân tích trong tài liệu tham khảo, xác suất một số là B -smooth xấp xỉ u^{-u} , với

$$u = \frac{\ln p}{\ln B}.$$

Chỉ cần khoảng $O(k)$ phương trình để giải hết các ẩn, nên độ phức tạp toàn cục có dạng

$$\tilde{O}(ku^u + k^3).$$

Khi chọn

$$u \approx \sqrt{\frac{2 \ln p}{3 \ln \ln p}}, \quad B = L_p[1/2, 2^{-1/2}],$$

thu được độ phức tạp

$$L_p[1/2, 3/\sqrt{2}].$$

Có thể tăng tốc bước nhận diện smooth-number để tối ưu thành

$$L_p[1/2, \sqrt{2}],$$

nhưng bài viết không trình bày chi tiết.

Ưu và nhược điểm. Nhờ lợi thế độ phức tạp, Index calculus xử lý được phạm vi logarit rời rạc lớn hơn nhiều thuật toán thường gặp. Ví dụ tác giả ghi nhận một hiện thực chưa tối ưu hằng số có thể giải khoảng 200 truy vấn cỡ $3 \cdot 10^{18}$ trong khoảng 6 giây trên LOJ 6542. Nhược điểm là hiện thực rườm rà hơn, nên khi dùng trong OI cần cân nhắc chi phí cài đặt.

19.4.6 Một số biến thể của logarit rời rạc

Nhiều truy vấn. Khi cần giải nhiều truy vấn trong cùng một nhóm, có thể điều chỉnh thuật toán để đổi thời gian tiền xử lý lấy thời gian trả lời. Giả sử trong $\mathbb{Z}_p^\times$ có T truy vấn logarit rời rạc.

Với BSGS, nếu chọn ngưỡng B , ta tiền xử lý $O(B)$ và trả lời một truy vấn trong $O(n/B)$. Chọn $B = \Theta(\sqrt{Tn})$ cho tổng độ phức tạp

$$O(\sqrt{Tn}),$$

tốt hơn $O(T\sqrt{n})$ của cách chạy độc lập.

Pollard Rho kém linh hoạt, khó sửa để tận dụng nhiều truy vấn. Pohlig–Hellman thực chất chuyển bài toán gốc thành nhiều bài logarit rời rạc nhỏ hơn; trong các bài nhỏ đó có thể áp dụng tối ưu nhiều truy vấn tương ứng. Index calculus cũng có thể tăng B trong phạm vi hợp lý để giảm số lần thử smooth-number khi trả lời từng truy vấn, đổi lại tiền xử lý chậm hơn.

Bài viết còn nêu một thuật toán khá thực dụng trong OI. Xét phép chia có dư

$$p = qb + r, \quad 0 \leq r < b.$$

Từ đó suy ra

$$\log_g b \equiv \log_g(-1) + \log_g r - \log_g q \pmod{n}.$$

Mặt khác,

$$p = (q+1)b + (r-b),$$

nên

$$\log_g b \equiv \log_g(b-r) - \log_g(q+1) \pmod{n}.$$

Nếu $b > \sqrt{p}$, thì $q + 1 \leq \sqrt{p} + 1$. Nếu đã tiền xử lý bằng logarit cho các số không vượt $\sqrt{p} + 1$, bài toán $\log_g b$ được quy về $\log_g r$ hoặc $\log_g(b - r)$, với đối số nhỏ nhất không vượt $b/2$. Vì vậy chỉ cần $O(\log b)$ bước đệ quy cho một truy vấn.

Để xây bảng logarit đến $\sqrt{p} + 1$, dùng sàng tuyến tính cho thấy chỉ cần biết logarit của các số nguyên tố trong phạm vi đó. Nếu dùng BSGS để tìm các giá trị này, tiền xử lý khoảng $O(p^{3/4} / \ln p)$. Với modulo cỡ 10^9 thường gặp trong OI, cách này tiền xử lý được khá nhanh và cho thời gian truy vấn $O(\log p)$.

Thông điệp của ví dụ này là không nên ràng buộc vào một thuật toán duy nhất. Cần chọn hoặc kết hợp phương pháp theo dữ liệu cụ thể.

Cơ số không phải phần tử sinh. Cho tới đây ta chỉ xét cơ số cố định là phần tử sinh g . Đôi khi bài toán cho a, b và cần tìm x sao cho

$$a^x = b.$$

Nếu biết một phần tử sinh g của nhóm, ta có thể tính riêng $\log_g a$ và $\log_g b$, rồi giải phương trình tuyến tính

$$(\log_g a)x \equiv \log_g b \pmod{n}$$

bằng Euclid mở rộng. Nhờ vậy, bài toán lại quay về dạng quen thuộc. Nếu có nhiều truy vấn với cơ số khác nhau, phép biến đổi này cũng đưa chúng về dạng cùng cơ số để áp dụng tối ưu nhiều truy vấn.

Khi không biết phần tử sinh, một số thuật toán như BSGS hoặc Pollard Rho vẫn có thể tìm nghiệm hoặc báo vô nghiệm, nhưng ngoài ra tác giả chưa thấy cách xử lý tốt. Vì thế nên cố gắng chuyển bài toán về một phần tử sinh.

19.5 Ý nghĩa của các bài toán

19.5.1 Lý thuyết tính toán

AKS cho thấy kiểm tra tính nguyên tố có thuật toán xác định thời gian đa thức. Ngược lại, phân tích số nguyên và logarit rời rạc đến nay vẫn chưa có thuật toán truyền thống thời gian đa thức.

Dù vậy, cũng chưa chứng minh được hai bài toán này thuộc lớp NPC. Hiện tại người ta cho rằng chúng rất có thể là các bài toán NP-intermediate: thuộc NP nhưng không thuộc P và cũng không NPC. Điều này hàm ý việc cải thiện độ phức tạp cho chúng có thể cần những phương pháp hoàn toàn mới và rất khó.

Trên máy tính lượng tử, cả hai bài toán lại có thuật toán thời gian đa thức. Điều đó cho thấy thuật toán lượng tử có thể là con đường hiệu quả trong tương lai.

19.5.2 Liên hệ với mật mã học

Nhiều giao thức và thuật toán mật mã hiện đại dựa trên độ khó của phân tích số nguyên và logarit rời rạc, chẳng hạn RSA, trao đổi khóa Diffie–Hellman và ElGamal. Vì thế độ an toàn của các hệ này liên quan trực tiếp đến hiệu quả giải hai bài toán trên.

Với logarit rời rạc, do nhóm nhân $\mathbb{Z}_p^\times$ đã được nghiên cứu sâu và có thuật toán dưới mũ khá nhanh, mật mã hiện đại thường dùng các nhóm đường cong elliptic được xây dựng cẩn thận. Trên nhóm tổng quát, thuật toán tốt nhất đã biết trong bối cảnh bài viết là BSGS; thực tế cũng thường dùng Pollard Rho để giải logarit rời rạc trên nhóm tổng quát.

19.6 Tổng kết

Bài viết thảo luận ba bài toán số học cổ điển: kiểm tra tính nguyên tố, phân tích số nguyên và logarit rời rạc. Đồng thời, bài viết giới thiệu một số thuật toán liên quan và nêu ngắn gọn khả năng liên hệ của chúng với OI, trong đó có những thuật toán chưa phổ biến trong OI. Kiến thức khoa học máy tính rất rộng; tác giả hy vọng nội dung này có thể bổ sung thêm sức sống cho OI và thu hút nhiều người đọc có năng lực tiếp tục nghiên cứu, phát triển các lý thuyết liên quan.

20 Bàn về ứng dụng tính lồi của hàm số trong OI

Trường THPT số 2 trực thuộc Đại học Sư phạm Hoa Đông
Guo Yuchong

Tóm tắt

Khai thác tính chất đặc biệt của hàm số để tối ưu thuật toán là một kỹ thuật phổ biến và quan trọng trong OI. Tính lồi của hàm số là một trong những tính chất đẹp và mạnh. Bài viết xoay quanh chủ đề này, khảo sát một số phương pháp dùng tính lồi để tối ưu thuật toán trong OI.

20.1 Kiến thức chuẩn bị

20.1.1 Định nghĩa tính lồi

Trong bài viết, tính lồi được chia thành *lồi trên* và *lồi dưới*. Với hàm $f(x)$ có miền xác định $[l, r]$, f là lồi trên khi và chỉ khi

$$\forall x_1, x_2 \in [l, r], \quad f\left(\frac{x_1 + x_2}{2}\right) \geq \frac{f(x_1) + f(x_2)}{2}.$$

Tương tự, f là lồi dưới khi và chỉ khi

$$\forall x_1, x_2 \in [l, r], \quad f\left(\frac{x_1 + x_2}{2}\right) \leq \frac{f(x_1) + f(x_2)}{2}.$$

20.1.2 Bao lồi và vỏ lồi

Với một tập điểm S trên mặt phẳng, chứa ít nhất ba điểm không thẳng hàng, bao lồi là đa giác lồi nhỏ nhất chứa toàn bộ các điểm trong S . Tương tự hàm lồi, vỏ lồi cũng có vỏ trên và vỏ dưới. Gọi a là điểm trên bao lồi có hoành độ nhỏ nhất, nếu có nhiều điểm thì lấy tung độ nhỏ nhất; gọi b là điểm có hoành độ lớn nhất, nếu có nhiều điểm thì lấy tung độ lớn nhất. Đường thẳng qua a, b chia bao lồi thành hai phần: phần phía trên là vỏ lồi trên, phần phía dưới là vỏ lồi dưới. Bỏ đi các đoạn thẳng đứng, vỏ trên là một hàm lồi trên và vỏ dưới là một hàm lồi dưới.

20.1.3 Tích chập (min, +) và (max, +)

Cho $f(x)$ có miền $[l_1, r_1]$, $g(x)$ có miền $[l_2, r_2]$. Kết quả tích chập (min, +) là $u(x)$, kết quả tích chập (max, +) là $v(x)$. Chúng có miền xác định $[l_1 + l_2, r_1 + r_2]$, và với mọi $i \in [l_1 + l_2, r_1 + r_2]$,

$$u(i) = \min_{\substack{j \in [l_1, r_1] \\ i-j \in [l_2, r_2]}} \{f(j) + g(i-j)\},$$

$$v(i) = \max_{\substack{j \in [l_1, r_1] \\ i-j \in [l_2, r_2]}} \{f(j) + g(i-j)\}.$$

20.1.4 Bất đẳng thức tứ giác và bài toán chia dãy

Ma trận A thỏa bất đẳng thức tứ giác khi và chỉ khi

$$\forall i < j \leq k < l, \quad A_{i,k} + A_{j,l} \leq A_{i,l} + A_{k,j}.$$

Điều này tương đương với

$$\forall i < j, \quad A_{i,j} + A_{i+1,j+1} \leq A_{i+1,j} + A_{i,j+1}.$$

Bài toán chia dãy: cho n, m và ma trận A , tìm dãy tăng p độ dài m , thỏa $p_0 = 0, p_m = n$, sao cho

$$\sum_{i=1}^m A_{p_{i-1}, p_i}$$

nhỏ nhất.

Nếu A thỏa bất đẳng thức tứ giác, thì đáp án theo m là một hàm lồi dưới. Gọi $f(m_0)$ là đáp án khi $m = m_0$, và $p(m_0)$ là một nghiệm tối ưu tương ứng. Cần chứng minh

$$2f(m_0) \leq f(m_0 - d) + f(m_0 + d).$$

Đặt $p_1 = p(m_0 - d), p_2 = p(m_0 + d)$. Theo nguyên lý Dirichlet, tồn tại

$$i \in [0, m_0 - d), \quad p_{1,i} < p_{2,i+d} < p_{2,i+d+1} \leq p_{1,i+1}.$$

Từ hai nghiệm này ghép được hai nghiệm cho $m = m_0$. Bất đẳng thức tứ giác cho tổng trọng số của hai nghiệm mới không vượt $f(m_0 - d) + f(m_0 + d)$, nên kết luận được chứng minh.

20.2 Tối ưu theo độ dốc

20.2.1 Giới thiệu

Tối ưu theo độ dốc là kỹ thuật cổ điển dùng tính lồi để tối ưu quy hoạch động. Ý tưởng chính là duy trì vỏ lồi trên hoặc vỏ lồi dưới của một tập điểm trong mặt phẳng, rồi tìm tiếp tuyến có độ dốc cho trước để xác định quyết định tối ưu. Điểm mấu chốt khi áp dụng là biến đổi chuyển trạng thái về bài toán tìm đường thẳng có một độ dốc cố định và có tung độ gốc nhỏ nhất hoặc lớn nhất.

20.2.2 Ví dụ 1: chia dãy với tổng bình phương

Cho dãy a độ dài n và tham số m . Trọng số đoạn $[l, r]$ là

$$\left(m - \sum_{i=l}^r a_i\right)^2.$$

Cần chia dãy thành một số đoạn để tổng trọng số nhỏ nhất.

Gọi dp_i là đáp án cho i phần tử đầu, $s_i = \sum_{j=1}^i a_j$. Có

$$dp_i = \min_{j=0}^{i-1} \{dp_j + (m - s_i + s_j)^2\}.$$

Biến đổi:

$$dp_j + (m - s_i + s_j)^2 = (m - s_i)^2 + dp_j + s_j^2 + 2(m - s_i)s_j.$$

Với mỗi j , dựng điểm $(s_j, dp_j + s_j^2)$. Khi chuyển trạng thái ở i , cần tìm điểm làm cho đường thẳng có độ dốc $-2(m - s_i)$ đạt tung độ gốc nhỏ nhất. Điểm tối ưu nằm trên vỏ lồi dưới.

Nếu a_i không âm, các điểm được thêm có hoành độ không giảm và độ dốc truy vấn cũng không giảm, nên dùng hàng đợi duy trì vỏ lồi trong $O(n)$. Nếu a_i có thể âm, s_i không đơn điệu; khi đó dùng cây cân bằng để thêm điểm, xóa các điểm không còn thuộc vỏ, và nhị phân tìm tiếp điểm, đạt $O(n \log n)$.

20.3 WQS binary search

20.3.1 Giới thiệu

WQS binary search là phương pháp tối ưu rộng, thường kết hợp với quy hoạch động. Nó áp dụng khi đáp án theo một tham số là hàm lồi, và ta chỉ cần một điểm của hàm. Ý tưởng là nhị phân độ dốc k , dùng đường thẳng độ dốc k làm tiếp tuyến, rồi tìm k sao cho tiếp tuyến đi qua điểm cần hỏi. Khi dùng cho quy hoạch động, cách này thường giảm được một chiều trạng thái. Tính lồi có thể chứng minh trực tiếp, hoặc qua mô hình luồng chi phí: chi phí nhỏ nhất theo lưu lượng là lồi dưới, chi phí lớn nhất theo lưu lượng là lồi trên.

20.3.2 Ví dụ 3: chọn các số không kề nhau

Cho dãy a , cần chọn đúng m số đôi một không kề nhau sao cho tổng lớn nhất. Có thể chứng minh đáp án theo m là lồi trên bằng mô hình luồng chi phí: chia vị trí theo parity thành đồ thị hai phía, dùng lưu lượng đúng bằng số phần tử được chọn.

Sau khi nhị phân độ dốc k , bài toán trở thành chọn số lượng tùy ý các số đôi một không kề nhau trong dãy $a_i - k$ để tổng lớn nhất. Đặt dp_i là đáp án cho i phần tử đầu:

$$dp_i = \max\{dp_{i-1}, dp_{i-2} + a_i\}.$$

Đồng thời lưu z_i , số phần tử được chọn nhiều nhất trong nghiệm tối ưu, để biết hoành độ tiếp điểm. Một lần kiểm tra tốn $O(n)$, tổng độ phức tạp $O(n \log V)$.

20.3.3 Ví dụ 4: chọn các đoạn rời nhau

Cho dãy a , cần chọn đúng m đoạn con đôi một không giao nhau sao cho tổng lớn nhất. Đáp án theo m cũng là lồi trên. Sau khi nhị phân k , mỗi đoạn $[l, r]$ có trọng số

$$-k + \sum_{i=l}^r a_i.$$

Với s_i là tổng tiền tố:

$$dp_i = \max\left\{dp_{i-1}, \max_{j=0}^{i-1}(dp_j - k + s_i - s_j)\right\}.$$

Duy trì giá trị tốt nhất của $dp_j - s_j$, đồng thời lưu số đoạn nhiều nhất trong nghiệm tối ưu. Tổng độ phức tạp $O(n \log V)$.

20.4 Duy trì hàm lồi

20.4.1 Giới thiệu

Một số thao tác khó làm nhanh trên hàm tổng quát lại có thuật toán hiệu quả trên hàm lồi. Chẳng hạn tích chập ($\min, +$) giữa hai hàm tổng quát thường là $O(n^2)$, nhưng với hai hàm lồi dưới có thể gộp các đoạn theo thứ tự độ dốc tăng dần để đạt $O(n)$. Bài toán thực tế thường cần cấu trúc dữ liệu để duy trì độ dốc từng đoạn, hỗ trợ sửa, truy vấn, lazy tag hoặc thao tác theo đoạn.

20.4.2 Ví dụ 5: chọn dãy nhị phân để chặn tổng đoạn lớn nhất

Cho hai dãy a, b độ dài n . Chọn dãy nhị phân S có đúng m phần tử bằng 1, đặt

$$c_i = S_i a_i + (1 - S_i) b_i.$$

Cần làm cho tổng đoạn con lớn nhất của c , cho phép đoạn rỗng, là nhỏ nhất.

Tổng đoạn con lớn nhất của một dãy có thể tính bằng biến now : ban đầu $now = 0$; với mỗi phần tử x , cập nhật

$$now \leftarrow \max\{now + x, 0\},$$

và lấy giá trị lớn nhất của now . Nhị phân đáp án mid , yêu cầu mọi thời điểm $now \leq mid$. Gọi $dp_{i,j}$ là giá trị now nhỏ nhất sau khi xét i phần tử đầu và đã chọn j số 1:

$$dp_{i,j} = \max\{\min(dp_{i-1,j-1} + a_i, dp_{i-1,j} + b_i), 0\}.$$

Biến đổi:

$$dp_{i,j} = \max\{\min(dp_{i-1,j-1} + a_i - b_i, dp_{i-1,j}) + b_i, 0\}.$$

Xem dp_i là hàm theo j , có thể quy nạp rằng hàm này lồi dưới. Mỗi bước gồm: chèn một đoạn độ dài ngang 1, độ dốc $a_i - b_i$; cộng toàn cục b_i ; nâng phần âm lên 0; và xóa phần vượt mid . Duy trì các độ dốc quanh đoạn có độ dốc 0 bằng hai tập và lazy tag, tổng

$$O(n \log n \log V).$$

20.4.3 Ví dụ 6: chọn điểm trên cây để tối đa hóa chu trình

Cho cây n đỉnh có trọng số cạnh. Chọn đúng m đỉnh phân biệt $a_1, \dots, a_m$ để tối đa hóa

$$\sum_{i=1}^m \text{dis}(a_i, a_{i+1}), \quad a_{m+1} = a_1.$$

Với cạnh (u, v, w) , nếu bỏ cạnh tạo hai thành phần chứa cnt_1, cnt_2 đỉnh được chọn, cạnh đó được đi qua nhiều nhất $2 \min(cnt_1, cnt_2)$ lần, và cận này đạt được. Vì vậy dùng DP trên cây; gộp các con bằng tích chập $(\max, +)$, sau đó cộng

$$2 \min(i, m - i) w_u$$

cho cạnh nối với cha.

Xem dp_u là hàm theo i , nó lồi trên. Gộp con tương ứng với gộp các đoạn theo độ dốc giảm dần; chọn thêm chính u là chèn đoạn độ dốc 0; cộng cạnh tương ứng tăng độ dốc một tiền tố và giảm độ dốc một hậu tố. Dùng hai heap quanh mốc $m/2$, kèm lazy tag và gộp heuristic, đạt $O(n \log^2 n)$.

20.4.4 Ví dụ 7: đường đi đơn điệu gần các điểm nhất

Cho n điểm (x_i, y_i) , $0 \leq x_i \leq y_i$. Cần chọn đường đi vô hạn từ $(0, 0)$, mỗi bước sang phải hoặc lên trên, sao cho tổng khoảng cách Chebyshev từ các điểm đến đường đi nhỏ nhất. Điểm trên đường đi gần (x, y) nhất nằm trên đường $x_0 + y_0 = x + y$, nên xử lý các điểm theo thứ tự tăng của $x_i + y_i$.

Gọi $dp_{i,j}$ là đáp án khi đi tới đường $x + y = i$, tại điểm có hoành độ j , và

$$w_{i,j} = \sum_{x_k + y_k = i} |j - x_k|.$$

Chuyển trạng thái:

$$dp_{i,j} = \min\{dp_{i-1,j}, dp_{i-1,j-1}\} + w_{i,j}.$$

Hàm theo j là lồi dưới. Bước chuyển chèn một đoạn độ dốc 0; mỗi điểm trên đường hiện tại làm giảm độ dốc của một tiền tố và tăng độ dốc của một hậu tố. Duy trì hai heap quanh đoạn độ dốc 0 và cập nhật đáp án khi một điểm nằm ngoài khoảng trung vị hiện tại. Tổng độ phức tạp $O(n \log n)$.

20.5 Ứng dụng tổng hợp của tối ưu lồi

20.5.1 Ví dụ 8: sắp xếp dãy bằng ít giá trị khác nhau

Cho dãy a độ dài n . Cần tìm dãy b có nhiều nhất m giá trị khác nhau, sao cho

$$\sum_{i=1}^n |a_i - b_i|$$

nhỏ nhất. Giả sử a đã được sắp không giảm. Khi một đoạn liên tiếp của b bằng nhau, giá trị tối ưu là trung vị của đoạn đó, nên bài toán trở thành chia dãy. Ma trận chi phí $w_{i,j}$ thỏa bất đẳng thức tứ giác, do đó đáp án theo số đoạn là lồi dưới.

Dùng WQS nhị phân k :

$$dp_i = \min_{j=0}^{i-1} \{dp_j + w_{j,i} - k\}.$$

Có thể dùng tính đơn điệu quyết định để mỗi vòng chạy $O(n)$ hoặc $O(n \log n)$. Bài viết còn nêu một cách $O(n)$ mỗi vòng: đưa việc chọn đại diện của đoạn vào chính DP, dùng hai trạng thái $dp_{i,0/1}$, rồi tối ưu các chuyển trạng thái bằng độ dốc. Tổng đạt $O(n \log V)$.

20.5.2 Ví dụ 9: matching lớn nhất theo kích thước trên cây

Cho cây có trọng số cạnh. Với mỗi i , cần tìm matching trọng số lớn nhất có đúng i cạnh. Tính lồi trên của hàm đáp án được chứng minh bằng mô hình luồng chi phí, chia đỉnh theo độ sâu chẵn lẻ. Trên một đường thẳng, dùng chia để trị đoạn; mỗi đoạn lưu ma trận 2×2 , hai chiều biểu thị cạnh hai đầu có được chọn hay không, mỗi ô là một hàm lồi trên theo số cạnh. Hai nửa gộp tuyến tính theo độ dài đoạn, tổng $O(n \log n)$.

Với cây tổng quát, dùng phân rã nặng nhẹ. Tổng kích thước cây con của các đỉnh nhẹ không vượt $n \log n$. Gộp các con nhẹ bằng chia để trị, rồi xử lý từng chuỗi nặng như trường hợp đường thẳng. Tổng độ phức tạp $O(n \log^2 n)$, có thể tối ưu đến $O(n \log n)$ bằng kỹ thuật cây nhị phân cân bằng toàn cục.

20.5.3 Ví dụ 10: truy vấn nhiều đoạn con rời nhau

Cho dãy a , mỗi truy vấn (l, r, c) yêu cầu trong $[l, r]$ chọn đúng c đoạn con đôi một không giao nhau sao cho tổng lớn nhất. Đáp án theo c là lồi trên. Dựng cây đoạn; mỗi nút lưu ma trận 2×2 , hai chiều cho biết hai đầu đoạn có được chọn hay không, mỗi ô là một hàm lồi trên theo số đoạn chọn. Hai con được gộp bằng tính lồi, tiền xử lý $O(n \log n)$.

Với truy vấn, tách $[l, r]$ thành $O(\log n)$ nút. Dùng WQS: với độ dốc k , trong mỗi nút lấy tiếp điểm của từng ô, biến ma trận hàm thành ma trận số rồi gộp. Độ phức tạp ban đầu là

$$O(n \log n + m \log^2 n \log V).$$

Thay nhị phân từng truy vấn bằng nhị phân song song: ở mỗi lớp chỉ xét các nút cây đoạn xuất hiện trong các truy vấn của lớp đó và cắt bỏ phần hàm không cần. Tổng độ phức tạp trở thành

$$O((n + m) \log n \log V).$$

20.6 Tổng kết

Bài viết giới thiệu một số phương pháp tối ưu dựa trên tính lồi, đồng thời dùng mười ví dụ để minh họa cụ thể cách chúng xuất hiện trong các bài OI. Các kỹ thuật này có phạm vi ứng dụng rộng và sức mạnh lớn: nhiều khi chúng giảm đáng kể độ phức tạp thời gian hoặc bộ nhớ, làm chương trình hiệu quả hơn.

Những nội dung được nhắc tới vẫn chỉ là một phần nhỏ của tối ưu lỗi. Tác giả hy vọng bài viết có thể gợi mở, khơi dậy hứng thú để người đọc tiếp tục nghiên cứu sâu hơn về lĩnh vực này.

20.7 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu; cảm ơn các thầy Jin Jing và Wu Shenguang đã hướng dẫn; cảm ơn cha mẹ đã nuôi dưỡng, dạy bảo; cảm ơn tất cả bạn học và thầy cô đã giúp đỡ.